

解码开源：开源生态 60 问

庄表伟 著

汉 魂 公 司	编 辑：李永涛	印 制：王郁
	书 号：70443	活 号：26-1197
	开 本：168*238	版 芯：39 行×35 字
	页 数：18+238	校 次：三校
	工 艺：GLJ	操作员：WJW
	质检员：	日 期：

人 民 邮 电 出 版 社

北 京

内 容 提 要

本书追溯开源从激进哲学理念演变为全球协作运动的完整发展历程，深入剖析全球协作、透明迭代、零门槛复用、事实标准、商业模式再定义、长尾创新民主化六大核心创新机制，阐释开源如何深度重塑人工智能、区块链、物联网等前沿科技的发展版图；直面开源“黄金二十年”后面临的地缘政治冲击、资本逻辑博弈与社区内部可持续性危机等现实挑战，提出构建“多元共治”生态新格局的未来发展路径。

本书系统论述开源本质理念、生态角色与治理模式、商业变现路径、社区运营与贡献者激励、开源政策等核心议题，以问答式结构搭建认知体系，跳出“免费代码”的浅层认知，穿透开源生态的复杂运行逻辑，兼具历史纵深、现实深度与前瞻视野。

本书面向开源领域的探索者、建设者与思考者，适用于开发者、产品经理、企业管理者、政策研究者等各类关注开源发展的读者，能够帮助读者建立全面、专业、系统的开源生态认知框架。

前言

在开源的世界里，我们为何依然需要一份地图

开源技术，已然成为数字世界里的空气与水。当我们启动手机时，背后是开源的 Android 或 Linux 内核在运行；当我们惊叹于人工智能（Artificial Intelligence, AI）的奇迹时，背后是 PyTorch 与 TensorFlow 等开源框架在驱动；当我们享受云计算的便捷时，背后是 Kubernetes 在无声地编排。开源，已经从少数技术极客的理想，演变为支撑全球数字经济运行的底层基础设施，并且无处不在。

然而，一个深刻的悖论也随之浮现：尽管我们每天都在使用开源，但我们真的理解它吗？当我们的认知还停留在“免费代码”的浅层时，我们是否错过了其背后复杂的生态系统、独特的演化逻辑，以及未来正面临的严峻挑战？我们是否意识到，这个看似自由流动的世界，正被地缘政治的裂痕、资本逻辑的博弈以及社区内部的可持续性危机所撕扯？

这正是本书的写作初衷。它并非一本简单的技术手册，也不是一本布道式的宣言。它是一份为所有开源世界的探索者、建设者与思考者绘制的深度导航地图，旨在穿透迷雾、解答困惑，帮助读者系统理解我们身处其中却又常常感到困惑的开源世界。

一、超越代码：一部从理念到生态的演进史诗

要真正理解开源，第一步便是追溯其源头，剖析其如何从一种激进的哲学理念，演变为一场务实的全球协作运动。这段波澜壮阔的历程，不仅塑造了今天的技术格局，更预示着未来的演化方向。下面将为你揭示开源的“前世今生”。

（1）追本溯源：从“自由”到“开放”的转变。

开源的故事，始于一场关于“自由”的抗争。20 世纪 80 年代，理查德·斯托曼（Richard Stallman）因无法修改打印机驱动而感受到自由被剥夺，发起了“自由软件”（Free Software）运动。他提出的“四大自由”（自由运行、研究、分发、修改）强调软件应赋予用户的权利，这是一种带有强烈道德哲学色彩的理想。然而，“自由”（Free）一词在英文中的双重含义——自由与免费，以及其对商业模式的挑战，使其在商业世界中步履维艰。

历史的转折点出现在 1998 年。为了摆脱“自由软件”激进的政治色彩和商业困境，克里斯汀·彼得森（Christine Peterson）提出了“Open Source”（开源）一词。这一概念更加务实，强调开放源代码在技术协作、质量提升和商业价值上的优势。它不再将“自由”本身作为最终目的，而是将“开放”作为实现“更好软件”的手段。尽管理念有所调整，但“开源”与“自由软件”在开放源代码、社区协作和对抗封闭垄断的核心追求上一脉相承，共同构成了我们今天所知的开源软件运动。

（2）里程碑的印记：塑造开源世界的关键事件。

开源的演进由一系列关键事件所标记，它们共同奠定了这个世界的基石。

首先是 Linux 的诞生。1991 年，芬兰学生林纳斯·托瓦兹（Linus Torvalds）出于个人兴趣开发了一个操作系统内核。起初，他采用了一种限制商业用途的许可证。然而，随着社区的发展和 GNU 项目的影响，托瓦兹逐渐意识到 GPL 更能保护 Linux 的开放性，并于 1992 年最终决定将内核许可证改为 GPLv2。这一决定性转变，通过开放协作的“集市模式”，最终创造出改变世界的操作系统内核，成为开源力量的最佳证明。

其次是“大教堂与集市”的启示。1997 年，埃里克·雷蒙德（Eric S.Raymond）发表了这篇影响深远的文章。他系统性地总结了 Linux 内核开发的“集市模式”相对于传统封闭“大教堂模式”的优越性，并提出了“礼物文化”这一核心概念，深刻解释了开发者无偿贡献的内在动机。这篇文章为开源协作提供了坚实的理论基石，被誉为“开源圣经”。

最后是 Netscape Navigator 浏览器的开源。在与微软 Internet Explorer 的“浏览器大战”中，处于下风的 Netscape 公司于 1998 年做出了一个革命性的决定。这一决策的直接催化剂，竟是公司员工弗兰克·海克（Frank Hecker）在听完雷蒙德的“大教堂与集市”演讲后，撰写的一份建议公司开源浏览器代码的内部白皮书。最终，Netscape 公司将 Netscape Navigator 浏览器的源代码开源，这成为大型商业公司首次将核心产品开源的标志性事件，标志着商业世界开始正视并接纳开源技术的价值与力量。

（3）生态的雏形：一个复杂的社会经济系统。

随着开源的演进，一个远比代码本身复杂的生态系统逐渐形成。它早已不是开发者的独角戏，而是一个多元角色共存的社会经济系统。这个生态的构成是分层且动态演进的，其核心是项目发起者、核心维护者、贡献者与广大的用户，他们是生态活力的直接源泉。随着项目的成熟，生态会扩展出更多专业化角色，如负责社区健康度的社区经理、指导新人的导师以及保障代码质量的安全审计员。

在此基础上，生态进一步演化，吸纳了组织层面的参与者。企业从用户和贡献者，成长为生态的投资者与塑造者；开源基金会（如 Linux 基金会、Apache 软件基金会）则扮演着“公共管家”的角色，提供中立的法律与治理框架；此外，还包括提供资金与资源的赞助商以及构建产业生态的生态合作方。最终，这个系统延展至更广阔的社会层面，教育者、学术研究者与政策制定者共同构筑了开源的文化、制度与知识体系。理解这些层次分明的角色及其互动关系，是理解开源世界如何运转的关键。当我们理解了开源的起源、理念嬗变与生态构成后，更重要的是审视它在当今世界所扮演的创新引擎角色，及其带来的深远影响。

二、创新的引擎：开源如何重塑科技版图

开源不仅是一种软件开发模式，更是当代技术创新的核心驱动力。它通过一套独特的机制，在全球范围内加速知识的创造、传播与应用，并以前所未有的深度，重塑了 AI、区块链和物联网等前沿科技的版图。

（1）剖析创新机制。

开源之所以能成为创新的引擎，源于其内建的六大核心机制。

- 全球协作。开源打破了地理与组织的边界，汇聚全球智慧。如同 Linux 内核的开发模式，成千上万的开发者在异步协作中，共同构建和维护着人类历史上最复杂的软件系统之一。
- 透明迭代。所有代码、讨论和决策过程都公开可见，这使得技术演进在持续的同行评审中得以加速。任何一个想法或补丁，都能迅速获得全球范围的反馈，从而快速迭代、持续优化。
- 零门槛复用。开源极大地避免了“重复造轮子”的浪费。无论是基础库、开发框架还是预训练模型（如 Hugging Face 上托管的海量 AI 模型），开发者都可以直接取用，站在前人的肩膀上进行创新。
- 事实标准。当一个开源项目被广泛采用时，它便自然而然地成为行业的事实标准。Kubernetes 正是通过这种方式一统容器编排的“江湖”，为整个云原生生态提供了稳定、统一的技术底座。
- 商业模式再定义。开源通过将核心技术开放，催生了以增值服务、订阅和云服务为核心的新商业模式。从红帽（Red Hat）被 IBM 以 340 亿美元收购，到 MongoDB 的 Atlas 云服务收入占比持续攀升，都证明了开源与商业可以共生。
- 长尾创新民主化。开源极大地降低了技术门槛和创新成本，使得个人、高校乃至新兴市场的开发者都能参与到前沿创新中。成本极低的树莓派（Raspberry Pi）教育套件，就催生了全球数千个物联网项目，激发了无数草根创新。

（2）赋能前沿科技。

创新机制在前沿科技领域展现出巨大的颠覆性力量。

在 AI 领域，开源是 AI 技术民主化的基石。从 TensorFlow、PyTorch 等深度学习框架，到 Llama、Qwen（千问）等开源大模型，开源为 AI 研究者和开发者提供了强大的工具和基础模型，使得顶尖的 AI 技术不再是少数巨头的专利。

在区块链（Blockchain）领域，“代码即法律”的理念与开源精神天然契合。区块链的信任与透明，正是建立在开源代码可被公开审计的基础之上。正是开源，催生了去中心化金融（DeFi）、去中心化自治组织（DAO）等革命性的应用，构建了一个全新的数字经济范式。

在物联网（Internet of Things, IoT）领域，开源硬件（如 Arduino、ESP32）与开源软件（如 Zephyr OS）的结合，极大地降低了物联网应用的开发成本与门槛。从智能家居到工业自动化，开源让数以万计的开发者能够快速构建原型、实现创意，加速了万物互联时代的到来。

（3）张力与博弈：知识产权的永恒议题。

开源的崛起，也引发了其与传统知识产权体系之间的深刻张力。然而，开源并非否定知识产权，而是对其进行了一场深刻的哲学重构。它巧妙地利用了版权法这一传统工具，但彻底颠覆了其应用逻辑：以 GPL 为例，它利用版权法赋予的排他性

权利，反向构建起一种强制共享的法律框架。这种“传染性”条款，正是开源世界“以子之矛，陷子之盾”的智慧体现。

这场重构的深远意义在于，它将知识产权的博弈焦点从“如何保护”转向了“如何治理”，从追求“垄断”转向了促进“协作”。这种模式重构了创新的激励机制，但也带来了商业模式与法律合规的持续挑战，使得开源与知识产权的博弈成为一个永恒的议题。开源在取得巨大成功、成为数字基础设施的同时，其生态系统内部也正面临着前所未有的结构性挑战，这使得我们必须以更审慎的目光展望其未来。

三、未来的十字路口：在撕裂与失衡中探寻可持续之路

开源的“黄金二十年”或许已经结束。曾经那个以全球化协作为理想的乌托邦，正遭遇地缘政治、资本逻辑和社区内部可持续性危机的严峻挑战。开源世界正站在一个未来的十字路口。本书的价值不仅在于回顾历史，更在于直面这些撕裂与失衡，探寻一条通往可持续未来的路径。

（1）外部撕扯：从全球协作到阵营化。

技术民族主义的兴起，正在侵蚀开源赖以生存的信任基础。部分国家开始将开源视为可以控制和利用的战略资源，而非全球共享的数字公共产品。GitHub 限制特定国家访问等事件，如同在开放的数字世界里竖起了“栅栏与鸿沟”，曾经的全球协作理想正面临“阵营化”的危险。开源的“全球性”正在被一次次现实冲击撕扯出裂缝。

（2）内部困境：公地悲剧由谁来买单。

开源社区内部的可持续性危机同样触目惊心。白鲸开源创始人郭炜曾发出灵魂拷问：“我们热爱开源！但是谁来爱我们？”这句话精准地道出了开源世界的“公地悲剧”：少数核心维护者用爱发电，承担着巨大的开发与维护压力，而海量的商业公司和个人用户则心安理得地“搭便车”，享受着开源带来的便利却吝于回馈。

这种严重失衡的付出与回报，正让越来越多的维护者感到倦怠与失望。如果贡献者的付出无法获得合理回报，那么这种依赖奉献精神创新模式终将不可持续。

（3）寻找出路：“多元共治”的未来图景。

应对内外部的严峻挑战，开源生态迫切需要构建一个“多元共治”（Multi-stakeholder Governance）的新格局。这意味着每个参与方都必须承担起与自身获益相匹配的责任，而不是将所有压力都推给少数志愿者。

- 政府应将关键开源项目视为“数字公共基础设施”，像维护道路和桥梁一样为其提供稳定的公共资金支持，并为其争取国际合作的缓冲地带，避免其成为地缘政治的牺牲品。
- 企业必须完成从“开源消费者”到“开源投资者”的角色升级。这不仅意味着提供资金赞助，更需要通过投入人力资源、开放内部工具、将改进回馈给上游社区等方式，系统性地反哺生态。
- 基金会应继续扮演“公共管家”的角色，提供中立的治理与法律框架，确保项目不被单一商业利益或政治力量所绑架，维护其公共性和长期发展的稳定性。

- 核心贡献者需要学会“设计制度”而非做“超级英雄”，通过建立清晰的治理规则和贡献路径，让更多人参与进来。而广大的普通用户应培养“微贡献”的习惯，无论是提交一份高质量的漏洞（Bug）报告、改进一份文档，还是为项目提供小额捐赠，都是为生态注入养分的宝贵行动。

只有构建起一个人人尽责、利益共享的多元共治体系，开源才能在撕裂与失衡中找到那条通往可持续未来的道路。

四、写作初心：一份为探索者、建设者与思考者准备的指南

我们正处在一个开源世界经历深刻变革的时代，机遇与挑战并存，理想与现实交织。在这样一个复杂而动态的环境中，我们比以往任何时候都更需要一个系统性的认知框架，而不仅仅是零散的知识点。

本书的独特价值正在于此。本书采用问答式的结构，并非为了提供唯一的标准答案，而是引导读者进行一场结构化的深度思考。它将那些看似孤立的问题——许可证的选择、社区的治理、商业模式的探索、全球化的挑战等有机地联系起来，最终展现出一幅完整的开源生态全景图。

在此，我向所有开源世界的参与者发出诚挚的邀请：阅读本书并开始你自己的思考。无论你是开发者、产品经理、企业管理者，还是政策研究者，都希望你能通过阅读本书，找到自己关心问题的答案，更能建立起对整个生态的宏观理解与深刻洞察。让我们共同参与到这场伟大的事业中，构建一个更加开放、包容、可持续的开源未来。

庄表伟
2026 年 5 月

目录

第 1 篇 开源的定义、本质与理念	1
第 1 问 开源软件与自由软件有何区别	1
1.1 自由软件与自由软件运动的由来	1
1.2 如何定义“开源”	2
1.3 从自由软件到开源软件的不变与变	3
第 2 问 开源运动是如何起源的	4
2.1 Linux 内核的诞生	4
2.2 《大教堂与集市》: 最早的观察与总结	5
2.3 浏览器大战与 Netscape Navigator 的开源	6
2.4 “Open Source”一词的提出与 OSI 的成立	6
2.5 从互联网泡沫到 Linux 操作系统公司上市	7
第 3 问 开源软件目前的发展状况如何	7
3.1 开源在全球范围内的成功	8
3.2 全球开源进入成熟化平台期, AI 开源成为新增长极	9
3.3 开源在中国的发展状况	11
3.4 中国开源的挑战与不足	15
第 4 问 为什么开源软件比专有软件更有吸引力	16
4.1 成本优势	17
4.2 灵活定制	17
4.3 安全可靠	17
4.4 社区创新	18
4.5 有利于人才招聘	18
第 5 问 开源的核心价值观是什么	18
5.1 开源项目的本质	19
5.2 开源生态的本质	20
5.3 开源从何而来	20
5.4 开源的价值	20
5.5 开源的核心价值观	21
第 6 问 开源软件的许可证有哪些类型? 它们各自的特点是什么	21
6.1 许可证及许可证的各种类型	22
6.2 开源许可证的重要性	22
6.3 开源许可证的分类概览	23

6.4 常见开源许可证的特征	24
第 7 问 选择开源许可证时需要考虑哪些因素	25
7.1 企业使用开源软件	25
7.2 企业参与外部开源社区	26
7.3 企业对外开源自身软件	27
第 8 问 开源如何影响软件开发的方式和文化	28
8.1 从“大教堂”到“集市”：软件开发范式的转变	28
8.2 开源社区的实践原则：快速迭代与用户反馈	29
8.3 分布式开发与精英治理：多元化的贡献者结构	29
8.4 林纳斯法则：质量保障的独特逻辑	30
8.5 在线包管理工具：生态系统与再利用的繁荣	30
8.6 社交化编程与自动化工具：从协作到文化升级	30
8.7 开放文化渗透企业：从开源到内源	31
第 9 问 开源人有哪些特点？如何与他们做朋友	31
9.1 开源人的特点	31
9.2 与开源人做朋友	32

第 2 篇 开源生态系统 36

第 10 问 开源生态系统包括哪些关键角色	36
10.1 从“个人”与“组织”维度分类	36
10.2 从“教育、学术与政策”视角扩展出的角色	38
10.3 角色之间的交互与多重身份	40
第 11 问 开源基金会的角色定位是什么	40
11.1 开源基金会的角色定位	40
11.2 开源基金会的主要作用	42
第 12 问 如何理解开源项目的治理模式	43
12.1 开源项目治理模式的主要特征	43
12.2 主流治理模式的分类与特点	44
12.3 选择开源项目治理模式要考量的关键因素	45
第 13 问 开源项目的生命周期是怎样的	46
13.1 开源项目生命周期	47
13.2 阶段化策略	49
第 14 问 什么是开源项目的健康指标	50
14.1 开源项目健康度评估的主流框架	50
14.2 深入思考开源指标体系	55
第 15 问 开源标准化在开源生态系统中的价值是什么	56
15.1 开源标准化的基本概念与发展历程	56

	15.2 标准化在开源生态系统中的核心价值	58
	15.3 标准化缺失导致开源项目失败的反面案例	59
	15.4 标准化面临的挑战与未来发展	60
第 16 问	开源项目如何实现全球化？面临哪些挑战	61
	16.1 开源全球化的 4 层境界	61
	16.2 开源全球化的未来趋势	65
第 17 问	开源生态与开源供应链是什么关系	66
	17.1 开源生态与开源供应链的结构关系	66
	17.2 开源生态治理与开源供应链治理的区别	67
第 18 问	如何理解和防范开源生态风险与开源供应链风险	68
	18.1 开源生态风险与开源供应链风险的区别	68
	18.2 对开源供应链风险的理解	69
	18.3 开源供应链的本质	70
	18.4 企业内部的开源治理	70
第 3 篇 开源商业模式		72
第 19 问	企业如何通过开源降低成本并提升效率	72
	19.1 企业使用开源能够降低成本	72
	19.2 企业使用开源能够提升效率	73
	19.3 企业使用开源的最佳实践	74
第 20 问	开源软件如何实现盈利	74
	20.1 开源软件实现盈利的主流模式	75
	20.2 各种模式的优缺点与适用场景分析	76
	20.3 开源商业的混合模式	77
	20.4 各类混合模式的商业动机与技术考量	78
	20.5 典型企业的混合策略案例分析	78
第 21 问	开源商业模式的本质是什么	81
	21.1 一个简单的开源经济学模型	81
	21.2 影响开源商业模式整体利润的因素	82
	21.3 一种打击竞争对手的开源商业模式	83
第 22 问	服务与支持如何成为开源软件的收入来源	84
	22.1 开源软件服务与支持的具体形式	84
	22.2 开源软件服务与支持的收入情况	85
第 23 问	如何建立企业开源合规治理框架	86
	23.1 企业开源面临的合规性挑战	86
	23.2 企业开源合规治理框架	88
第 24 问	开源商业化有哪些成功的案例	91
	24.1 如何选择开源商业化的成功案例	91

	24.2 红帽：开源商业化的范式革新	92
	24.3 谷歌：开源生态的流量霸权	93
	24.4 PingCAP：开源数据库的全球化突围	94
	24.5 华为：从封闭到开放的生态转型	95
第 25 问	如何评估开源商业模式的可行性	96
	25.1 市场需求：产品是否能够满足特定的市场需求，目标用户 是否有足够的付费意愿	96
	25.2 技术架构：发展阶段与成熟度	97
	25.3 社区状况：活跃度与贡献者画像	97
	25.4 竞争格局：竞争态势与团队能力	98
	25.5 知识产权：保护策略与合规风险	99
第 26 问	企业应如何制定开源战略	100
	26.1 评估经营状况	100
	26.2 明确战略目标	100
	26.3 搭建组织与制定政策	101
	26.4 具体执行与度量	102
第 27 问	企业在开源中的常见误区有哪些	102
	27.1 从核心维度看企业开源误区	102
	27.2 时间维度下的误区演进	103
第 28 问	如何在企业中推广开源文化	104
	28.1 为何要在企业中推广开源文化	104
	28.2 推广路径与实践策略	104
	28.3 关键挑战与破解之道	106
第 29 问	什么是内源，为何在企业中推广内源	106
	29.1 内源的定义与核心特征	106
	29.2 企业推广内源的核心动因	107
	29.3 如何解答那些对于内源的质疑	108
第 30 问	如何在企业中建立开源项目办公室	109
	30.1 开源项目办公室的定义	109
	30.2 开源项目办公室的价值	110
	30.3 开源项目办公室的成长路径	110
	30.4 开始前的准备工作	111
	30.5 如何搭建开源项目办公室团队	113
	30.6 建立开源项目办公室的常见挑战与解决方案	113
第 31 问	如何从开源社区招募人才	114
	31.1 对开源人才的渴求程度与原因	114
	31.2 战略定位：从短期挖人到长期共建	115
	31.3 如何精准识别开源人才	115

第 32 问	如何通过开源提升企业的技术品牌	117
32.1	技术品牌建设的 4 个阶段	118
32.2	核心实施策略	118
32.3	效果评估指标	119
32.4	针对中小企业的建议	119
32.5	典型案例参考	120
第 4 篇	开源社区	122
第 33 问	开源社区的结构与运作是怎样的	122
33.1	社区的基本成员	122
33.2	社区成员的来源	123
33.3	维护者与核心开发者的工作	124
33.4	其他非技术性工作	124
33.5	社区的组织架构与决策机制	124
第 34 问	开源社区如何吸引和留住贡献者	124
34.1	吸引贡献者的策略	125
34.2	留住贡献者的策略	125
34.3	社区治理和工具支持	126
34.4	未来挑战和特征	127
第 35 问	如何开始为一个开源项目做贡献	127
35.1	参与开源的意义	127
35.2	贡献的多种形式	128
35.3	需要的基本技能	128
35.4	寻找适合新手的开源项目	129
35.5	第一次贡献的完整流程	130
35.6	其他帮助新手入门的资源	131
35.7	开始行动	131
第 36 问	从贡献者到维护者的成长路径是什么	131
36.1	成长的典型阶段	132
36.2	需要掌握的核心能力	133
36.3	如何获得晋升与建立信任	133
36.4	加速成长的实用策略	134
36.5	可能遇到的挑战及应对	134
第 37 问	开源社区如何管理冲突和分歧	135
37.1	冲突管理的总体思路	135
37.2	建立清晰的行为准则和社区规范	135
37.3	构建结构化的沟通与决策流程	136
37.4	实施协作式冲突解决机制	136

	37.5 利用技术与工具辅助管理	136
	37.6 坚持公开透明与持续反思	137
	37.7 应对极端冲突的策略	137
第 38 问	开源社区的包容性和多样性为何重要	138
	38.1 多元与包容的重要性	139
	38.2 需要解决的现实问题	140
	38.3 提升多元与包容性的改进方案	141
第 39 问	开源社区如何评估贡献者的价值	142
	39.1 技术贡献	142
	39.2 非技术性贡献	142
	39.3 行为品质	143
	39.4 商业价值	143
	39.5 量化指标的尝试	144
第 40 问	开源项目如何保障其长期可持续发展	144
	40.1 经济可持续性	145
	40.2 治理可持续性	145
	40.3 技术可持续性	146
	40.4 项目可持续性与生态可持续性	147
第 41 问	开源社区里的规则与潜规则有哪些	147
	41.1 社区、潜规则与规则	147
	41.2 开源伦理的演进	149
	41.3 社会之礼与社区之礼	150
	41.4 开源社区里的“成文法”与自动化规则	151
	41.5 社区中的潜规则	152
	41.6 社区运营的艺术	153
第 5 篇	开源创新与未来	155
第 42 问	开源如何驱动技术创新	155
	42.1 开源驱动创新的六大核心机制	155
	42.2 开源驱动创新的实际成效	156
	42.3 开源创新影响的量化评估	157
	42.4 开源在中国语境下的机遇与挑战	158
	42.5 开源创新面临的挑战与未来展望	159
第 43 问	开源如何影响 AI 领域的发展	159
	43.1 开源在 AI 发展中扮演的角色	160
	43.2 开源组件——奠定 AI 创新与普及的基石	160
	43.3 2025 年开源 AI 项目的技术版图	161

	43.4 开源 AI 的伦理、安全与治理挑战	162
第 44 问	开源如何影响区块链领域的发展	163
	44.1 开源对区块链发展的六大核心影响	163
	44.2 在开源推动下诞生的新技术及跨领域影响	164
	44.3 区块链对开源社区的反向推动	165
	44.4 挑战与趋势	165
第 45 问	开源如何影响物联网领域的发展	166
	45.1 降低成本与开发门槛	167
	45.2 加速创新与迭代	167
	45.3 推动互操作性与标准化	168
	45.4 增强安全性与透明度	169
	45.5 构建繁荣生态与商业模式	169
	45.6 面临的挑战、风险与展望	171
第 46 问	开源与知识产权之间，存在何种张力	171
	46.1 知识产权遭遇的历次挑战	171
	46.2 开源与知识产权的基本关系	173
	46.3 不同的角色如何认识与应对这样的张力	173
第 47 问	开源如何推动数字化转型	175
	47.1 什么是数字化转型	175
	47.2 企业与政府的数字化转型	176
	47.3 开源在数字化转型中的作用	176
第 48 问	开源如何推动高校科研创新	178
	48.1 从开源到开放科学	178
	48.2 在高校开展开源教育的价值	179
	48.3 推动高校的科研成果开源	180
	48.4 展望未来的高校科研与开放科学	181
第 49 问	开源社区的未來会怎么发展	181
	49.1 从代码社交到意图社交	182
	49.2 项目的生命周期缩短，而社区的流动性加快	182
	49.3 Agent 社区与 AI 进化乐园	183
	49.4 从开源到开放资源，再到开放智能	183
第 50 问	开源技术的未来趋势是什么	183
	50.1 被 AI 全面增强的开源	184
	50.2 AI 对未来基础设施提出的挑战	185
	50.3 开源安全与治理成为“头等大事”	186
第 51 问	如何构建开放、包容、可持续的开源未来	188
	51.1 开源生态正在遭遇哪些结构性挑战	188
	51.2 在撕裂与不平衡中重建开源的“多元共治”	190
	51.3 在不完美的世界里，坚持一种“更好的开放”	192

第 52 问	开源创新的边界在哪里	194
52.1	创新的多重场域与开源的真实位置	194
52.2	科研、企业与开源的分工与互补	194
52.3	开放协作与有组织创新的张力	195
52.4	国家视角下的开源创新：期待与纠偏	196
52.5	开源创新的几条关键边界	196
第 6 篇	政府与开源政策	198
第 53 问	政府如何在开源项目中发挥积极作用	198
53.1	政府介入开源的现实背景与必要性	198
53.2	政府在开源中的多重角色定位	199
53.3	支持开源项目的关键方法与工具	200
53.4	“积极”与“克制”的边界	201
第 54 问	如何制定切实可行的开源公共政策	202
54.1	开源公共政策的内涵与定位	202
54.2	制定开源公共政策的目标是什么	202
54.3	制定开源政策的基本原则	203
54.4	开源政策的主要工具箱	204
54.5	自上而下与自下而上的结合	205
第 55 问	制定开源政策时需要避免哪些典型误区	206
55.1	基于错误“供应链想象”的开源断供恐慌	206
55.2	错误的科技创新想象：把开源当成万能的 “突破秘籍”	207
55.3	把补贴当主角：试图通过“涓滴效应”扶植开源生态	208
55.4	用行政与 KPI 逻辑替代社区逻辑	208
55.5	忽视“数字公共产品”本质与制度配套的狭隘视角	209
第 56 问	政府与企业如何通过采购策略与产业政策共建开源生态	210
56.1	从“项目采购”到“生态共建”的政策逻辑	210
56.2	自主可控：关键在于保障开源供应链安全	211
56.3	产业政策：把开源当作“基础设施投资”，而不仅是 “应用补贴”	211
第 57 问	政府与开源基金会如何通过中立平台实现多方共治	212
57.1	案例研究：欧盟与 Eclipse 基金会的协作关系	212
57.2	案例研究：德国与主权科技基金的协作关系	214
57.3	中国特色的实践	215
第 58 问	政府与教育机构：如何以开源为抓手培养人才与推动科研	216
58.1	开源是新型人才培养方式	216

58.2	开源教育相关案例介绍	217
58.3	受到开源启发的开放科学	218
58.4	开放科学的相关案例介绍	219
58.5	政府如何推动基于开源的产学研协同	220
第 59 问	如何打造能够吸引全球开源人才的城市	221
59.1	人才的交叠性质：头部人才为何反复涌入少数头部 城市	221
59.2	“吸引”的不同层次：从短期停留到长期定居的分层 目标	222
59.3	人才最重视的机会与基础设施：开源导向人才的“核心 需求画像”	223
59.4	城市人才战略：从愿景、定位到实施	224
59.5	度量与保障体系：把战略固化为治理能力	225
第 60 问	如何通过开源实现科技外交	226
60.1	科技外交	226
60.2	开源作为科技外交的可能性与特殊性	226
60.3	中国的科技外交策略	227
60.4	中国通过开源实现科技外交的区域策略	229
后记	AI 时代来临之后，正在剧变中的开源生态	232
附录	知名开源基金会介绍	236

第 1 篇 开源的定义、本质与理念

本篇将深入探讨开源的定义、本质与理念。首先，从开源的确切定义入手，并将其与自由软件进行对比，明确两者之间的联系和区别。此外，我们也将追溯开源运动的历史起源、分析其发展现状，以及为什么越来越多的开发者和用户青睐开源软件。然后，我们将探究开源的核心价值观，如开放、协作、分享、共赢等，并介绍不同类型的开源许可证及其特点，帮助读者理解如何选择适合自己项目的许可证。最后，我们将讨论开源对软件开发方式和文化的影响，以及如何与充满热情的开源社区成员建立良好的关系。

通过阅读本篇，读者能够对开源的概念形成更加清晰的认识，并了解其背后的理念和价值观。

第 1 问 开源软件与自由软件有何区别

1.1 自由软件与自由软件运动的由来

自由软件运动可以追溯到 20 世纪 80 年代早期，这场运动由理查德·斯托曼发起，其核心是对用户自由的追求。但是，斯托曼当时所说的用户和我们现在所理解的用户大不相同。

在麻省理工学院（Massachusetts Institute of Technology, MIT）人工智能实验室工作期间，斯托曼因一台打印机的驱动程序无法修改而受挫，这一事件使他意识到，软件的闭源特性和限制用户修改的行为是对（像他这样的用户的）自由的剥夺。在他看来，软件应该是开放的，允许用户自由地使用、复制、研究、修改和分发。因此，1983 年，斯托曼发起了“GNU 计划”，旨在创建一个完全自由的操作系统。

GNU 计划中，GNU 是 GNU's Not Unix 的递归缩写，这也是那个时代人们喜欢玩的一种文字游戏。英文中“Free”既有自由又有免费的含义，笔者认为这也是斯托曼特别想要达到的一个效果。

1985 年，斯托曼发起成立了自由软件基金会（Free Software Foundation）。1986 年，在基金会的出版刊物 *GNU's Bulletin* 上，斯托曼第一次发布了自由软件的定义，在这个定义中，他描述了软件的四大自由。

- 你可以按照自己的需求和目的自由运行这个程序（自由 0）。
- 你可以研究程序的工作原理，并进行修改，使它能够按照你的需求进行计

算（自由 1）。要实现这一点，你需要能够访问程序的源代码。

- 你可以自由地分发程序的副本，这样就可以帮助邻居或朋友（自由 2）。
- 你可以将修改后的版本分发给其他人，让他们也能使用改进后的程序（自由 3）。这样做可以让整个社区都有机会从你的修改中受益。获取源代码是实现这一点的先决条件。

在自由软件运动中，GNU 计划不仅开发了大量基础软件，还创建了 GNU 通用公共许可证（GNU General Public License, GPL）。这是一个革命性的许可证，旨在确保软件的自由性和开放性。著名的 Linux 内核项目就采用了 GPLv2 这样的许可证，并获得了巨大的成功。

随着时间的推移，人们逐渐意识到自由软件运动带来了一些“困扰”。首先，尽管斯托曼一直强调自由软件中的“Free”指的是 Free Speech（自由表达）而非 Free Beer（免费啤酒），但实际上，基于自由软件营利的难度仍然较大，而且自由软件运动往往蕴含着强烈的“政治/道德”批判色彩，这让许多务实、只想专注于编码的程序员感到“有些过于激进”。正是在这种背景下，商业开源与务实编码相遇，从而催生了开源的概念。

1.2 如何定义“开源”

可以说开源软件如今已经席卷全球，甚至有更加夸张的说法是“软件吞噬世界，开源吞噬软件”。在 IT 领域中，从硬件驱动、操作系统，到云计算和大数据，从网站开发到目前最热门的 AI，从 IT 自媒体到 CCTV 的新闻报道，再到企业战略、国家政策，甚至国际政治，我们都能找到关于开源的各种观点和论述。

开源最简单的理解就是：开放源代码，尤其是软件领域的开放源代码。正如自由软件所强调的四大自由，软件要能够被任何人查看、使用、修改和分发，就必须开放源代码。

1998 年，美国开源促进会（Open Source Initiative, OSI）成立，其发布的“The Open Source Definition”（OSD，开源定义）中开篇第一句就明确表示：“开源不仅仅意味着可以访问源代码，开源软件的分发条款必须符合以下标准”。OSD 中提出了 10 条标准，其定义逻辑是：如果符合 10 条标准的许可证（license）被一款软件用作分发时的许可条款，那么这款软件就可以算是开源软件。因此，OSD 本质上只是开源软件许可证的 10 条标准，而不是开源的定义。由于开源不断发展，从开源软件发展到开源硬件、开放数据、AI 模型、开放文档等，故再用定义开源软件分发协议的方式来间接定义开源已经不合适了。

中国电子工业标准化技术协会（中电标）在 2023 年正式发布的团体标准《信息技术 开源 术语与综述》（T/CESA 1269—2023）中对开源的相关定义如图 1-1 所示。

在参与《信息技术 开源 术语与综述》标准制定的过程中，笔者未关注到数字公共产品标准（Digital Public Goods Standard）在 GitHub 上有一个开源标准项目 DPG-Standard。从 2020 年 9 月开始，在联合国指导（认可）下成立的数字公共产

品联盟（Digital Public Goods Alliance）致力于促进符合标准和最佳实践的开源解决方案的发现、开发、使用和投资，这些解决方案与实现可持续发展目标相关。

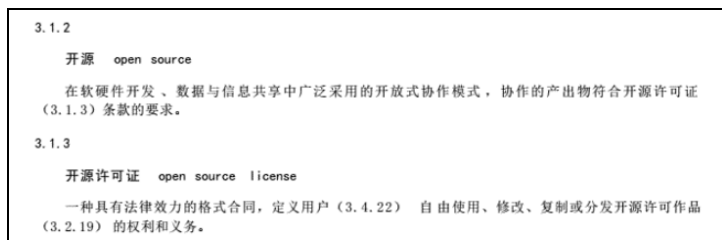


图 1-1 开源的相关定义

所以，如果能够继续迭代开源的定义，复用数字公共产品的定义，开源的定义可以表述为：以开放式协作模式协作生产数字公共产品。（开放式协作模式和数字公共产品这两个概念详见第 5 问。）

1.3 从自由软件到开源软件的不变与变

开源软件席卷全球，令人遗憾的是，“自由软件”这一概念似乎逐渐被人们遗忘了。

有一种极简叙事的风格大概是这样的：20 世纪 80 年代，自由软件开始出现，到了 1998 年开源这个词被提出，之后自由软件逐渐没落，不再被人提及。真的这样吗？

作为一个客观事实，自由软件、GNU 运动、四大自由之类的说法，的确被人越来越少提及，背后的原因是什么呢？从自由软件到开源软件，哪些始终未变，哪些又发生了变化呢？

说到不变的部分，首先是源代码。开放源代码始终是自由软件与开源软件（Free/Libre and Open Source Software, FLOSS）不变的追求。因为很多软件，既属于自由软件，也属于开源软件。同为自由软件，有些愿意自称 Free，有些愿意自称 Libre。所以，可以认为，FLOSS 这个词本身就是一个寻求最大公约数的努力。坚持源代码必须开放，让用户能够自由访问、查看、研究，甚至修改软件的源代码。这种开放性是自由软件和开源软件的共同基础。

其次是社区，无论是自由软件社区，还是开源社区，都非常强调“开放式协作”，两者都鼓励通过社区协作来推动软件的开发和改进。更进一步地，通过开放式协作来推动各种各样的创新。

最后一点是，它们共同的“敌人”都是专有软件，以及由此而来的封闭与垄断。

至于变了的部分，其实也很明显，首先是用户的范围变了。自由软件的用户是重视自由、隐私和控制权的有开发能力的技术极客，而开源软件的用户则大大扩展，包括开发者、个人用户、企业用户等。

其次是理念和价值观变了。自由软件运动是一种带有伦理和社会责任感的运

动，追求的是更多的自由；而开源则更强调开发模式的优势，如更高效的开发过程、快速的迭代和高质量的代码，关注的是实用性和商业价值，最终实现的是更好的软件。前者（自由）本身就是目的，而后者（开源）只是一种手段。

“对商业更加友好”，或者按照自由软件运动的支持者的说法“对商业利益的妥协”，引发了开源软件与自由软件的更多差异，包括更加宽松的许可证、更多商业力量参与的社区与市场推广以及更高的市场接受度。

总体而言，开源运动不再像自由软件运动那样只是一小群技术极客的努力，而是商业、投资、教育等更多相关利益方都积极参与进来的一场运动。在很多国家，推动开源创新，发展开源生态甚至上升为一种国家战略。这也可以回答我们在最开始的一个疑问：“为何开源正变得越来越重要，而自由软件却越来越少被人提及？”

第 2 问 开源运动是如何起源的

2.1 Linux 内核的诞生

Linux 内核的诞生始于 1991 年，源于芬兰学生林纳斯·托瓦兹（Linus Torvalds）的一次个人项目。当时，托瓦兹对计算机系统充满兴趣，受安德鲁·斯图尔特·特南鲍姆（Andrew Stuart “Andy” Tanenbaum）开发的教育用类 UNIX 操作系统——MINIX 操作系统的启发，他着手设计一个类似 UNIX 的内核系统。

最初，托瓦兹只是因个人使用想开发一个能够在 Intel 386 架构上运行的简单操作系统，用以代替局限性较大的 MINIX 操作系统。但后来他并不满足于 MINIX 操作系统的功能，希望通过自己的系统获得更高的自由和可定制性。在设计过程中，他决定将内核代码公开，并邀请其他程序员参与开发和改进。

1991 年 8 月 25 日，托瓦兹在一个网络新闻组（comp.os.minix）上发布了一条公告，说明自己正在编写一个“仅供兴趣使用”的操作系统内核，并开放源代码让大家使用和测试。他在邮件中谦虚地称这个项目“只是一个爱好，不会像 GNU 或大型商业系统那样专业”，但这条公告吸引了许多程序员的关注，大家开始纷纷参与其中。

Linux 内核的开发过程也受到 GNU 项目的影响和帮助，特别是在工具链方面。GNU 项目提供的编译器、工具、库等，为 Linux 系统构建了完整的工具环境，这也使 Linux 能够发展为一个功能齐全的操作系统。

托瓦兹对 Linux 的开放性、鼓励贡献和协作的态度，使得这个项目在全球迅速发展，并得到了大量的程序员、爱好者甚至企业的支持。开发者们不但贡献了代码，还改进了性能、增加了功能，使 Linux 在稳定性和可靠性方面逐渐成为企业级服务器系统的首选。

托瓦兹在 1991 年发布 Linux 内核时，采用了一种更为宽松的许可证，允许用户自由修改和分发代码，但限制了商业用途。然而，随着 Linux 社区的发展和 GNU 项目的影响，托瓦兹意识到 GPLv2 可以更好地保护 Linux 的开源性质，同时促进社区协作。（GPL 相关的内容详见第 6 问。）

在 1992 年，托瓦兹最终决定将 Linux 内核的许可证更改为 GPLv2。此举不仅确保了 Linux 内核代码的自由使用、修改和分发，也使得所有对内核的改动都必须保持开放，从而进一步推动了 Linux 社区的发展和内核的完善。

2.2 《大教堂与集市》：最早观察与总结

“The Cathedral & the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary” 是埃里克·雷蒙德根据他对 Linux 内核开发过程的观察和管理开源项目 Fetchmail 的经验撰写的一篇关于软件工程方法的文章，后来成为一本书。这篇文章探讨了自上而下设计与自下而上设计之间的斗争。这篇文章于 1997 年 5 月 27 日由雷蒙德在德国维尔茨堡举行的 Linux 大会上首次发表，并于 1999 年作为同名书籍出版，中文版书名为《大教堂与集市》。

可以观察到，在 1997 年（开源诞生前一年）与 1999 年（开源诞生后一年）这两个时间点，《大教堂与集市》这本书可以算是既催生了开源，又成为对开源最为经典的总结，被称为“开源圣经”是毫不过誉的。

（1）软件工程领域的洞见。

早在 20 世纪末，雷蒙德就在《大教堂与集市》一书中提出了两种软件开发模式：封闭的“大教堂模式”（传统瀑布模式）和开放的“集市模式”（类似敏捷开发模式）。集市模式强调“早发布，经常发布”，倾听用户反馈和模块化设计，帮助开发者快速响应需求和修复问题，提升软件质量。雷蒙德的“林纳斯法则”提出“只要有足够多的人查看，所有的漏洞都会变得显而易见。”说明在集市模式下，广泛参与的开发者能有效发现并解决问题。《大教堂与集市》挑战了弗雷德里克·布鲁克斯（Frederick P. Brooks, Jr.）在《人月神话》中提出的观点，认为在适当的开发工具和社区管理下，人多不仅不会增加负担，反而提升了效率与创新能力。（开源对软件开发的影响详见第 8 问。）

（2）礼物文化影响深远。

在《大教堂与集市》的第 3 章“开垦心智层”中，雷蒙德系统地提出了“礼物文化”的概念，用以解释社区中开发者们志愿奉献的行为，即为什么程序员愿意参与开源项目并免费贡献代码。雷蒙德认为，这种动机不仅仅是技术兴趣或志愿奉献，而是一种通过协作获得成就感、提升技能和在社区中建立声誉的激励机制。他将其称为：集市模式的社会动力学。

换句话说就是：谁在社区奉献（代码、帮助他人、回答问题等）越多，谁的社区地位就越高。“礼物文化”这一来自人类学领域的概念，有效地解释甚至强化了很多开发者的社区行为，这在很长一段时间里成为最重要的开源社区共识：你在社区贡献，我们献上崇拜，其他的则不必深谈，“Just for fun”就好。

（3）对开源商业模式的早期探索。

在提出了礼物文化之后，雷蒙德也意识到，大多数开源开发者实际上生活在混合的经济环境下（也要养家糊口），所以他也思考了很多开源软件应该以何种商业模式赚钱的问题。一方面，我们必须承认雷蒙德具备深刻的洞见，他提出了多种不

同的开源收费策略，以及使用开源作为战略武器的概念。另一方面，我们也要意识到，在开源软件全面吞噬世界之后的今天，任何一款软件在技术架构、开源成分比例、开发成本等方面，都已经发生了重大变化，雷蒙德对未来（也就是我们现在）的很多推测也已经失效了，我们必须基于现在的实际情况，回答开源商业化的问题。（开源商业模式的相关内容详见第3篇。）

2.3 浏览器大战与 Netscape Navigator 的开源

20 世纪 90 年代中期，互联网的兴起催生了浏览器的激烈竞争。这个竞争被称为“浏览器大战”，最著名的两个参战者是 Netscape Navigator 和 Internet Explorer。这一时期的浏览器竞争不仅塑造了互联网的早期面貌，还对开源运动产生了深远的影响。

1994 年，马克·安德森（Marc Andreessen）等人创建了 Netscape 公司，推出了 Netscape Navigator，这是第一款广泛使用的图形化网页浏览器。Netscape Navigator 为用户提供了一个直观、易用的界面，推动了万维网的普及。1995 年，Netscape Navigator 已经成为市场上最受欢迎的浏览器，占据了大约 80% 的市场份额。

与此同时，Microsoft 公司也开始关注浏览器市场。1995 年，Microsoft 公司推出了自己的浏览器 Internet Explorer（IE），并将其捆绑在 Windows 操作系统中。这一策略使得 Internet Explorer 迅速获得了大量用户，并与 Netscape Navigator 形成了激烈竞争。随着时间的推移，Microsoft 公司凭借其操作系统的市场份额和捆绑策略，逐渐占据了浏览器市场的主导地位。

1998 年，Netscape 公司意识到自己在浏览器市场的失利，尤其是在与 Microsoft 公司的浏览器竞争中落于下风。无巧不成书，Netscape 公司的员工弗兰克·海克正好有机会听到了雷蒙德的“大教堂与集市”演讲，并起草了一份供公司管理层参考的白皮书，建议公司将 Netscape Navigator 的源代码开源。这标志着 Netscape Navigator 从闭源商业软件转变为开源软件，成为开源软件历史上的一座里程碑。

1998 年 2 月初，雷蒙德受邀飞往山景城，开始协助 Netscape 公司策划 Netscape Navigator 开源相关的一系列行动。Netscape 公司的开源决定不仅对浏览器市场产生了深远影响，还对整个软件产业产生了革命性的推动作用。它是商业公司向开源转型的一个重要案例，也加速了开源运动的发展。

2.4 “Open Source”一词的提出与 OSI 的成立

一群经验丰富的高手，他们不仅仅是在讨论如何帮助 Netscape 公司开源自己的浏览器，更想要发起一场更大规模的“运动”，这个运动要想获得成功，要先起一个好名字。在 1998 年 2 月 5 日的会议上，为了避免“自由（免费）软件”引发的商业误解，克里斯汀·彼得森首次提出“Open Source”（开源）这一术语，强调技术的开放性和协作性，而非哲学性的自由。雷蒙德和布鲁斯·佩伦斯（Bruce Perens）等人迅速接受并推广了这一术语。随后，蒂姆·奥莱利（Tim O'Reilly）

也为这一术语的传播提供了平台支持。

行动力超强的一群人迅速注册了 `opensource.org` 这个域名，并于 1998 年 2 月 8 日，由雷蒙德与佩伦斯共同成立了开源促进会，致力于推广开源理念并维护开源软件的标准。OSI 的核心任务是制定并维护开源定义，为开源软件的合法性和可持续发展提供明确指导。另外一个冷知识是：OSD 是由之前佩伦斯撰写的“Debian Free Software Guidelines”（Debian 自由软件指导）改写而来的。

同年 4 月，奥莱利举办的“自由软件峰会”更名为“开源峰会”，正式确立了“开源”作为新的主流术语。

开源运动迅速扩展到技术、商业和教育等领域，推动了 Linux、Apache、MySQL 等项目的发展，并促成了全球性的开源生态系统。

2.5 从互联网泡沫到 Linux 操作系统公司上市

开源运动与互联网的发展息息相关。互联网为开源项目提供了协作和传播的平台，加速了代码共享和社区协作，使开源从个人爱好演变为全球现象。20 世纪 90 年代，Linux Kernel 和 Apache HTTP Server 等开源项目通过互联网迅速壮大，成为互联网基础设施的重要组成部分。

1995 年以后，互联网泡沫也使大量资本涌入互联网行业，推动了开源的商业化。企业纷纷采用开源软件（如 Linux、MySQL）替代昂贵的专有软件，以降低成本和快速创新。在此背景下，许多专注于开源软件服务的公司成立，如红帽和 VA Linux。

1999 年是开源商业化的里程碑：红帽于同年 8 月成功上市，其订阅服务模式结合了开源的灵活性与企业需求，首日股价上涨约 272%。同年 12 月，VA Linux 上市，首日股价上涨近 700%，创下当时的纳斯达克纪录。这些事件证明了市场对开源模式的信心，同时也反映了互联网泡沫中的投资热潮。

2000 年互联网泡沫破裂后，许多互联网公司倒闭，但开源软件因其低成本、高效性和社区驱动的特性成为幸存的关键技术之一。IBM 等大型企业开始支持开源，推动开源社区进一步壮大，Git 等协作工具的普及使开源开发更为高效。

从互联网泡沫到 Linux 公司上市，开源运动与互联网形成了相互促进的关系。互联网为开源提供了平台，开源软件则以其灵活、高效的特点推动了互联网经济的发展，最终奠定了现代技术产业的基础。

第 3 问 开源软件目前的发展状况如何

简单概要的回答是：开源在当今世界已取得了举世瞩目的成就，但其发展轨迹正呈现出一些新的态势。从全球范围来看，开源无疑是成功的，但整体增长曲线已步入平台期。在中国，开源虽在近年来取得了长足进步，然而因起步相对较晚，且面临国际竞争与地缘政治等诸多挑战，要迈向更高层次也绝非易事。

3.1 开源在全球范围内的成功

一、技术领域全面崛起

在技术层面，开源的成功令人瞩目。Linux 操作系统堪称开源领域的璀璨明珠，依据 Top 500 超级计算机榜单，排名前 500 的超级计算机几乎清一色运行 Linux 操作系统。在服务器市场，Linux Server 的占有率亦颇为可观，尤其在 Web 服务器与云计算领域表现卓越。据市场研究机构估算，其在全球服务器市场的占有率处于 70%~80%。尽管在桌面领域长期处于较低水平，仅在 2%~3%徘徊，但 2024 年 7 月 StatCounter 发布的数据显示，Linux Desktop 取得了历史性突破，首次逼近 4.5%。而在编程语言、开发工具、数据库、中间件、云计算等众多领域，开源技术已然占据主导，呈现出一统天下的格局。在 AI 领域，众多开源技术正蓬勃发展，展现出无限的活力与潜力。各个技术领域中的著名开源项目如表 3-1 所示。

表 3-1 各个技术领域中的著名开源项目

技术领域	著名开源项目
操作系统	Linux、FreeBSD、Ubuntu、Android、openEuler、OpenHarmony
编程语言	Python、Ruby、Rust、Go、Perl、Swift、MoonBit、Cangjie
开发工具	Git、Vim、Emacs、GCC、LLVM、Visual Studio Code、Eclipse、Jenkins
数据库	MySQL、PostgreSQL、SQLite、MongoDB、Redis
中间件	Apache HTTP Server、Nginx、Apache Kafka、RabbitMQ、Apache Tomcat、ZooKeeper
云计算	OpenStack、Docker、Kubernetes、Prometheus
AI	TensorFlow、PyTorch、OpenCV、Llama、Qwen

二、商业领域硕果累累

开源在商业方面同样成绩斐然。早期，红帽和 VA Linux 在 IPO 阶段便创造了惊人的商业奇迹。1999 年 8 月 11 日，红帽公司股票于纳斯达克挂牌上市，当日股价飙升 272%。1999 年 12 月 9 日，其股价更是上涨至每股 286 美元。同一日，VA Linux 股票以 30 美元首次公开发行，上市当日股价上涨近 700%，创下前所未有的纪录，最高价达 320 美元，最终以 239 美元收盘。此后，开源企业斩获巨大商业成功的案例不断涌现，如 2008 年 Oracle 公司以 10 亿美元收购 MySQL，2009 年 VMware 公司以 4.2 亿美元收购 Spring Source，2018 年 Microsoft 公司以 75 亿美元收购 GitHub，2019 年 IBM 公司以 340 亿美元收购红帽等。

三、开源基金会蓬勃发展

众多开源基金会也在蓬勃发展，其中，Linux 基金会（Linux Foundation）和 Apache 软件基金会（Apache Software Foundation）极具代表性。ProPublica 数据显示，2022 年 Linux 基金会收入达 1.77 亿美元，旗下拥有 30 多个子基金会，管理着 600 多个开源项目；Apache 软件基金会在 2023 年的收入达 231 万美元，管理着近

300 个开源项目，这些开源项目作为全球数字基础设施的关键组成部分，正发挥着不可估量的巨大作用，有力地推动着全球数字化进程。两大开源基金会最近 10 年的收入情况如图 3-1 所示。

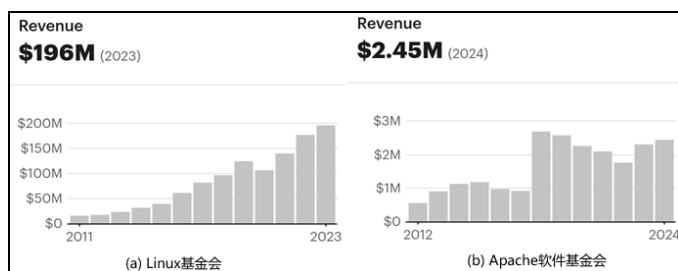


图 3-1 两大开源基金会最近 10 年的收入情况（来源：ProPublica）

3.2 全球开源进入成熟化平台期，AI 开源成为新增长极

近年来，全球开源软件的发展已经不再表现为早期那种单一的高速扩张，而是呈现出“总量继续增长、增速趋于稳定、增长结构重心转移”的新特征。从整体规模看，开源并未停止增长。GitHub《Octoverse 2025》显示，2025 年 GitHub 平台开发者总量已超过 1.8 亿，全年新增开发者数量超过 3600 万，开发者平均每秒新增超过 1 人；同时，全年新增仓库数超过 1.21 亿，公共及开源项目贡献量达到 11.2 亿，同比增长 13%。这说明全球开源仍保持扩张态势，并未出现绝对意义上的停滞。全球新增活跃开发者变化趋势如图 3-2 所示。

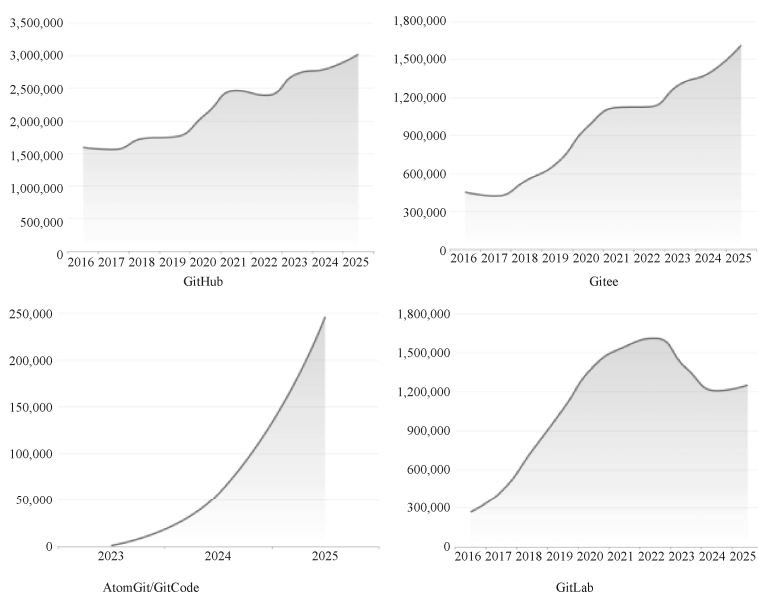


图 3-2 2016~2025 年全球新增活跃开发者变化趋势（来源：OpenDigger 平台数据整理）

但是，如果从高质量协作网络和开发者互动强度来看，全球开源确实进入了成熟化平台期。《2025 中国开源年度报告》基于 OpenDigger/OpenRank 数据指出，GitHub 平台近 10 年活跃开发者累计达到 2493 万名，国内 Gitee/AtomGit/GitCode 等平台活跃开发者累计接近 1000 万名；2025 年 GitHub 新增活跃开发者约 301 万名，较 2024 年增长 7.73%，增速明显低于早期高速增长阶段。更重要的是，全球开源开发者 OpenRank 贡献度在 2016~2024 年由 747 万提升至 2052 万，但 2025 年回落至约 1926 万；OpenRank 影响力在 2024 年后也出现回落。这表明开源的“参与人数”和“仓库数量”仍在增长，但跨项目、跨组织的深度协作密度正在进入调整期。全球开源开发者贡献度与影响力变化如图 3-3 所示。

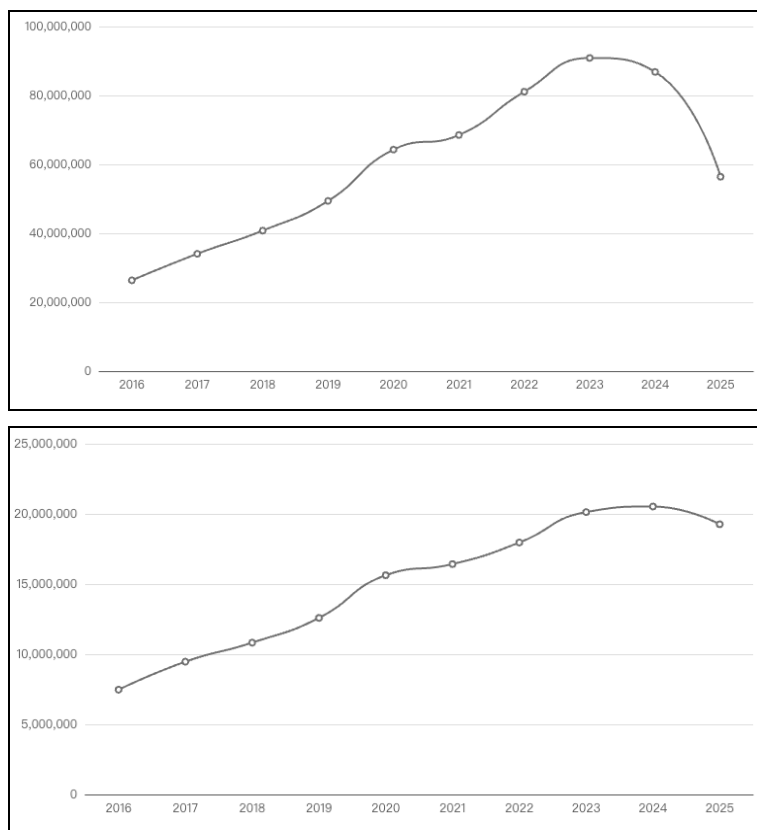


图 3-3 2016~2025 年全球开源开发者 OpenRank 贡献度变化与影响力变化 (GitHub 平台) (来源: OpenDigger 平台数据整理)

造成这一变化的原因是多方面的。

其一，开源软件的普及率已接近饱和。Black Duck《2025 Open Source Security and Risk Analysis》显示，在其审计的商业代码库中，97%包含开源代码，70%的扫描代码来源于开源；OpenLogic《2026 State of Open Source Report》也指出，过去一年 98%的组织增加或维持了开源软件使用量。这意味着开源已经从“是否采

用”的选择问题，转变为“如何治理、如何维护、如何安全使用”的管理问题。（开源供应链相关的内容详见第 17 问和第 18 问。）

其二，生成式 AI 正在改变开源协作方式。一方面，AI 编程助手和自动化工具提高了个人开发效率，降低了原本依赖社区协作完成的部分任务量；另一方面，AI 也推动了开源生态的新一轮扩张。GitHub 数据显示，2025 年已有超过 110 万个公共仓库使用 LLM SDK，过去一年新增此类项目近 69.4 万个，同比增长 178%；AI 相关项目成为 GitHub 开源增长最快的类别之一。也就是说，传统代码协作增长趋缓，并不意味着开源衰退，而是意味着开源的核心载体正在从“代码仓库”扩展到“模型、数据集、工具链、智能体和推理基础设施”。

其三，开源风险约束明显增强。Black Duck 的审计数据显示，86%的商业代码库包含存在漏洞的开源组件，81%包含高危或严重漏洞，56%存在许可证冲突，91%包含非最新版本的开源组件，90%包含落后当前版本超过 10 个版本的组件。随着开源软件成为数字基础设施，基于软件物料清单（Software Bill of Materials, SBOM）进行供应链安全管理、许可证合规治理、依赖治理、长期维护和漏洞响应已经成为全球开源发展的主要约束条件。

因此，全球开源当前的发展状况可以概括为采用率已接近饱和，传统协作增长进入成熟平台期；但 AI、模型平台、云原生、开发者工具和基础软件仍在创造新的增长曲线。未来全球开源竞争的关键，不再只是项目数量和代码规模，而是生态治理能力、全球开发者吸引力、供应链安全能力、模型开放程度和长期可持续维护能力。

3.3 开源在中国的发展状况

开源在中国的发展有几个关键的阶段：学术推动期、操作系统技术推动期、互联网企业推动期和国家战略推动期。早在 1985 年，陈钟教授作为杨芙清院士的硕士和博士研究生参与了 AT&T UNIX 操作系统油印纸质版源代码的阅读和分析，成为国内首批 UNIX 操作系统内核研究与开发的骨干之一。1992 年，AT&T 贝尔实验室 USL/USG 与中国合作，美方将最新开发的 UNIX SVR 4.2 版本源代码向中方开放，中方基于此推出了 UNIX SVR 4.2 中文版。因此，可以将 1992 年认定为“中国开源发展的元年”。

在 20 世纪 90 年代末至 21 世纪初，中国的开源活动主要围绕 Linux 操作系统展开。这一时期，国内的技术社区和高校开始关注并推广 Linux，培养了一批早期的开源爱好者和开发者。1999 年，国产操作系统迎来了第一次真正的爆发，以 Xteam Linux、蓝点 Linux、红旗 Linux、中软 Linux 为代表的众多国产操作系统走上了历史舞台。

随着互联网、云计算、移动互联网等一系列技术热潮的兴起，中国也诞生了不少著名的大公司（如阿里巴巴、华为、百度、腾讯、美团、小米等）的开源项目，以及著名的开源企业（如 PingCAP、Kyligence、DAOCloud、飞致云、白鲸开源、涛思数据等）。

2020 年 6 月，开放原子开源基金会正式成立，成为中国首个在民政部注册的致力于开源产业公益事业的非营利性独立法人机构。同年 9 月，华为将其智能终端操作系统基础能力相关代码捐赠给该基金会，项目被命名为 OpenHarmony。这标志着中国在开源操作系统领域迈出了重要一步，促进了国内开源生态的进一步繁荣。

《中华人民共和国国民经济和社会发展第十四个五年规划和 2035 年远景目标纲要》提到“支持数字技术开源社区等创新联合体发展，完善开源知识产权和法律体系，鼓励企业开放软件源代码、硬件设计和应用服务。”可以看出国家在战略层面对开源的肯定和支持。

2021 年至今，全国各级政府和机构（如国务院、地方政府、科技园区管委会等）陆续发布了关于开源软件和数字经济创新的政策。整体趋势从早期的国家层面指导政策（如“十四五”规划）逐步延伸到地方政府的具体行动计划，聚焦于促进开源软件生态建设、数字经济发展、AI 创新等领域。

在国家层面，强调知识产权保护与运用，推动数字经济、AI、基础技术等重点领域发展。明确支持开源技术在数字经济和社会发展中的重要地位。在地方层面，各地都提出要构建开源平台、开展试点示范工程、完善开源治理生态，加快推进本地软件名城建设，推动相关技术和服务输出。开源政策兼顾产业创新与社会经济发展，展现出较强的系统性。

2024 年以后，中国开源已经从早期的技术引入、社区学习和企业跟随阶段，进入“政策支持、平台建设、企业主导、基础软件突破、AI 开源加速”的体系化发展阶段。根据《2025 中国开源年度报告》，截至 2025 年底，中国在 GitHub 平台的活跃开发者数量超过 210 万，规模位居全球第三；如果加上 Gitee/AtomGit/GitCode 等国内平台，整体活跃开发者数量预计超过 350 万，位居全球第二。开放原子开源基金会发布的《中国开源发展深度报告（2024）》也指出，2024 年中国活跃开源开发者数量已超过 220 万，北京、上海活跃开源项目均超过 50 万个，深圳、杭州均超过 20 万个，国内开源创新潜能持续释放。

从平台看，中国开源不再只依赖 GitHub。Gitee、AtomGit、GitCode 等国内平台在政企、高校、国产基础软件、合规要求和本土开发者服务方面发挥了越来越重要的作用。《2025 中国开源年度报告》显示，国内 Gitee/AtomGit/GitCode 等平台活跃开发者数量累计接近 1000 万，活跃仓库规模超过 2600 万；其中 Gitee 活跃仓库规模在 2025 年达到约 438.9 万，AtomGit/GitCode 作为新兴平台，在 2025 年活跃仓库规模超过 60 万，呈现出早期快速扩张特征。

从企业看，中国头部科技企业的开源影响力显著增强。2025 年全球开源企业影响力榜中，华为以 89,368.78 的 OpenRank 位居全球第二，是唯一进入全球前 5 名的非美国企业；阿里巴巴位居全球第十。国内榜单中，华为、阿里巴巴、蚂蚁集团、字节跳动、百度、PingCAP、乐鑫、腾讯等企业共同构成了中国开源企业矩阵，如表 3-2 所示，覆盖操作系统、AI 框架、云计算、数据库、物联网、向量数据库和开发工具等方向。

表 3-2 2025 年中国开源企业影响力 Top 15

排名	企业	OpenRank（含同比变化）	活跃仓库规模	活跃开发者规模
1	华为	89,368.78（+12,607.45）	8,273	12,716
2	阿里巴巴	26,349.25（-5,670.27）	2,797	14,595
3	蚂蚁集团	18,679.05（+4,002.61）	768	10,726
4	字节跳动	16,625.76（+6,703.16）	458	12,398
5	百度	13,651.87（-5,339.68）	152	5,841
6	PingCAP	7,917.36（+1,878.88）	99	845
7	乐鑫	6,141.94（+1,226.29）	171	4,293
8	腾讯	5,717.46（+1,492.99）	254	4,629
9	飞致云	4,945.60（+2,065.73）	65	2,562
10	道客	4,558.18（-8,306.29）	61	1,929
11	飞轮数据	4,099.92（+873.02）	11	756
12	LobeHub	4,037.59（+231.96）	26	1,865
13	旷睿信息	3,561.51（-431.39）	55	1,502
14	开放麒麟	3,180.90（-922.31）	1,283	773
15	镜舟	2,770.29（-762.91）	19	762

从项目看，中国开源的重心正在从应用层项目向基础软件和 AI 原生基础设施上移。2025 年中国开源项目影响力榜中，OpenHarmony 位列第一，OpenRank 达 60,089.18；openEuler 位列第二，OpenRank 达 16,257.57，详表 3-3。这表明中国开源已经形成以操作系统为底座、AI 与大模型生态为先锋、数据库和中间件为支撑的立体化格局。

表 3-3 2025 年中国开源项目影响力 Top 15

排名	项目	OpenRank	开发者规模	发起组织
1	OpenHarmony	60,089.18	6,487	开放原子开源基金会
2	openEuler	16,257.57	3,285	开放原子开源基金会

续表

排名	项目	OpenRank	开发者规模	发起组织
3	MindSpore	8,064.37	1,211	华为
4	PaddlePaddle	7,305.94	3,168	百度
5	Apache Doris	4,031.07	747	阿帕奇软件基金会
6	ModelScope	3,681.64	3,732	阿里巴巴
7	VolcEngine	3,638.97	3,686	字节跳动
8	Ant Design	3,288.48	3,288	蚂蚁集团
9	openKylin	3,180.90	773	开放原子开源基金会
10	TiDB	3,162.02	498	平凯星辰
11	Anolis OS	2,996.83	514	开放原子开源基金会
12	verl	2,785.19	2,875	字节跳动
13	StarRocks	2,770.29	762	镜舟
14	Milvus	2,640.81	986	Linux 人工智能与数据基金会
15	1Panel	2,304.38	1,369	飞致云

尤其值得注意的是，中国在开源 AI 与开源模型生态中的地位明显上升。Hugging Face 2026 年春季报告显示，截至 2025 年，Hugging Face 已增长至 1300 万用户，拥有超过 200 万个公共模型和超过 50 万个公共数据集；过去一年，中国模型在 Hugging Face 下载量中占据 41% 的最大单一份额，并在月度下载和总体下载上超过美国。Qwen 家族衍生模型超过 11.3 万个，若按 Qwen 标签统计则超过 20 万个；DeepSeek、Qwen、GLM、Kimi 等中国模型正在成为全球开源 AI 生态的重要力量。

从国家和地方政策看，中国开源已经从技术社区议题上升为数字经济、基础软件、AI、产业安全和人才培养的重要组成部分。开放原子开源基金会、开源社、木兰开源社区、各地开源创新中心、高校开源课程和开源实践活动共同推动了开源生态建设。开放原子相关报告指出，开源操作系统、AI 和数据库是中国开源表现突出的三大领域，开源鸿蒙、openEuler、openKylin、OpenAnolis、DeepSeek、千问、TiDB、openGauss、OceanBase 等项目在生态构建和技术影响力方面持续增强。

综合来看，中国开源已经不再只是全球开源生态的跟随者和使用者，而是在操

作系统、AI 模型、数据库、云原生、物联网和开发工具等方向形成了若干具有国际影响力的项目和企业。中国开源的阶段性特征可以概括为：开发者规模全球领先，基础软件项目开始突破，AI 开源形成全球影响，企业和基金会成为生态核心力量，但国际化协作深度和全球社区治理能力仍需提升。

3.4 中国开源的挑战与不足

中国开源在快速发展的同时，仍面临多重结构性挑战。

第一，顶级项目数量和全球生态吸引力仍然不足。2025 年全球开源项目影响力 Top 100 中，美国项目有 49 个，中国项目有 12 个，虽然中国已位居第二，但与美国仍有明显差距。

第二，核心技术原创能力仍需加强。中国已经在 OpenHarmony、openEuler、PaddlePaddle、MindSpore、ModelScope、DeepSeek、Qwen、TiDB、Milvus 等方向取得突破，但在全球开源底层生态中，美国仍然在 AI 框架、编译器、云原生、开发者工具链、芯片软件生态等关键领域占据主导地位。《2025 中国开源年度报告》也指出，中美技术话语权呈现结构性分化：中国在 AI 大模型、操作系统、分布式数据库等方向快速追赶，部分领域实现并跑甚至局部领跑，但美国在 PyTorch、Kubernetes、编译器等底层领域仍具优势。

第三，开源人才体系仍存在“使用者多、维护者少、国际化开源维护者少”的问题。2025 年开源社问卷显示，受访者中“使用者”和“贡献者”比例较高，但维护者、生态运营者等深度角色仍然有限；在高校开源实践方面，超过六成受访者没有参加过相关活动，说明开源教育和实践活动的覆盖面仍有较大提升空间。开源生态真正需要的不只是会使用开源软件的开发者，而是能够长期维护项目、制定路线图、进行代码评审、处理安全漏洞、组织社区协作、参与国际治理的核心贡献者。

第四，开源供应链安全和合规风险更加突出。随着开源被广泛用于金融、电信、政务、工业、汽车、医疗和 AI 等关键场景，依赖漏洞、传递依赖、许可证冲突、组件停更、恶意包投毒等问题会直接影响企业和公共部门的安全韧性。Black Duck 的数据表明，64%的开源组件是传递依赖，56%的审计代码库存在许可证冲突，91%的代码库包含过时开源组件。这意味着企业不能再把开源治理视为研发团队的自发行为，而需要建立 SBOM、SCA、漏洞响应、许可证审查、依赖升级和安全例外审批等制度化流程。

第五，AI 开源带来了新的定义、治理和安全挑战。模型开源并不等同于传统软件开源，开放权重、开放推理代码、开放训练代码、开放数据集、开放评测体系之间存在明显差异。OSI 已发布 Open Source AI Definition 1.0，强调 AI 系统应当赋予使用、研究、修改和分享的自由，并指出传统软件许可证视角不足以完全保障 AI 系统的开放性；同时，OSI 也明确提出要警惕“openwashing”，即企业将并不符合开源自由原则的 AI 系统包装为“开源”。中国大模型在全球影响力提升的同时，也必须面对训练数据可追溯、模型许可证清晰度、衍生模型合规、安全评测、

滥用防护和国际社区信任等问题。

第六，商业化和社区可持续性仍需平衡。开源项目不能只依赖企业短期投入或开发者热情，还需要稳定的商业闭环、基金会治理、维护者激励、社区中立性和长期路线图。OpenLogic《2026 State of Open Source Report》指出，随着开源采用成熟，组织关注点正在从“是否使用开源”转向“如何可持续地管理开源”；近半数受访组织把 50% 以上时间用于维护和修复缺陷，20% 的组织没有正式 CVE 处理流程。这一趋势对中国开源同样适用：没有持续维护能力的开源项目，很难从“开源发布”走向“生态成功”。

综上所述，中国开源产业接下来的重点，不应只是继续扩大项目数量和开发者规模，而应转向高质量发展：提升原创技术能力，孵化国际化顶级项目，培养核心维护者和社区治理人才，完善供应链安全和许可证合规体系，推动 AI 开源的透明化与标准化，并通过基金会、企业、政府、高校和社区之间的协同机制，形成可持续、可信任、可国际协作的开源生态。只有这样，中国开源才能真正从“使用开源、参与开源”进一步走向“贡献开源、治理开源、引领开源”。

第 4 问 为什么开源软件比专有软件更有吸引力

在开源软件的早期发展阶段，以微软公司为代表的“反开源”势力，曾经散布了很多攻击性的言论，试图通过传播 FUD（Fear, Uncertainty, Doubt，即恐惧、不确定性和怀疑）来影响公众对开源软件的看法。2001 年，微软 CEO 史蒂夫·鲍尔默（Steve Ballmer）甚至直接称 Linux 是“癌症”。这些言论主要集中在以下几个方面。

- 安全性质疑：开源软件因为源代码是公开的，所以更容易被黑客攻击，存在严重的安全漏洞。
- 法律与知识产权风险：GNU 通用公共许可证等许可证可能会“传染”企业的代码，迫使企业公开所有源代码，破坏商业机密。
- 质疑社区的技术能力与管理能力：社区的开发者都是草根、业余爱好者，缺乏管理，一团混乱，不可能实现长期有效管理。
- 开源经济模型不可持续：开源软件是免费的，不可能支持开发者长期开发。那些爱好者没有稳定收入来源，开源软件项目无法持续。
- 总拥有成本上升：虽然开源软件看起来不要钱，但是需要专门雇人维护它，结果会花更多的钱。

虽然这些攻击性的言论曾经对开源软件的发展带来了一定阻力，但事实证明，开源软件不仅在安全性、技术能力、经济模型上表现出色，还成为推动全球技术创新的重要力量。即便是微软，也在近些年转变了立场，成为开源的积极支持者。甚至到了 2014 年，微软新任 CEO 萨提亚·纳德拉（Satya Nadella）公然宣称“微软爱 Linux”！所以，我们确实有必要回答这个问题：为什么开源软件比专有软件更有吸引力？

4.1 成本优势

开源软件通常是免费的，这为企业和个人节省了巨大的许可费用。同时，用户无须为更新、扩展或长期使用支付额外费用。因此，越是被广泛采纳的开源软件，越是能够在市场上找到足够丰富的公开文档、使用指南，也能够以更低廉的价格招募开发人员与维护人员，这种成本优势对于初创企业、小型团队和预算有限的个人尤其具有吸引力。此外，即使在涉及需要专人维护的情况下，事实证明，长期使用开源软件的总拥有成本（Total Cost of Ownership, TCO）通常仍低于专有软件，因为用户可以避免高昂的专有系统维护和升级费用。

相反，如果依赖于特定厂商的专有软件，在软件供应商涨价时，企业根本无力反抗，这种“单一厂商锁定”的风险，才是更大的成本风险。

4.2 灵活定制

不存在任何一款软件可以完全适配所有用户的全部需求，其功能供给要么冗余过度，要么有所不足，要么设计简陋，要么体验不佳。此外，绝大多数软件都存在程序漏洞，也时常为用户带来困扰。对于使用专有软件的用户而言，遇到此类问题时，只能向软件厂商寻求支持，等待厂商版本升级或补丁修复。若用户提出的功能需求与软件厂商既定的产品规划偏差较大，厂商完全有理由拒绝进行针对性开发。

采用开源软件虽仍存在各类缺陷与不足，但用户可自行对软件进行修改。经自行修改后的软件，仅在组织内部使用不存在合规问题。若适用授权条款较为宽松的许可证，下游厂商甚至可直接销售经其修改后的软件版本，并获取相应收益。（许可证相关的内容详见第6问和第7问。）

4.3 安全可靠

开源软件究竟更不可靠、更易被攻击利用，还是更安全可靠？这一问题已存在长期争论。雷蒙德在《大教堂与集市》中提出林纳斯法则：“只要有足够多的人查看，所有的漏洞都会变得显而易见。”该观点显然明确支持“开源更安全”的立场。

站在中间立场，我们会发现：争论的核心在于“透明性”与“攻击面”之间的权衡。有多少眼睛在审视代码，以及有多少眼睛在尝试寻找漏洞。还有一个问题也需要分辨：在尝试寻找漏洞的人中，多少人是善意的安全人员，多少人是恶意的黑客。

所以，更加持平的意见是：不应该泛泛地讨论所有的开源软件是否更安全。有些开源软件，因为用户众多、眼睛够多，的确比相对应的专有软件更安全。

还需要指出的是，开源软件生态中存在着一套较为成熟的安全漏洞通知与修复机制，作为开源软件的用户，应该非常关注自己使用了哪些开源软件，并及时获取

相关的安全咨询，这才是对自己更加负责任的做法。（开源生态风险和开源供应链风险相关的内容详见第 18 问。）

4.4 社区创新

在《大教堂与集市》一书的前言中，雷蒙德说道：“自由不是一个抽象的商业概念。任何行业的成功几乎都直接和这个行业供应商及客户所享有的自由度相关……所以该‘开源软件’行业在产品 and 客户价值方面的创新速度，是人类前所未见。”

因为代码开源、社区开放，人们在社区里围绕代码进行开放式协作，所以软件这个行业才会在最近几十年里，不断地诞生令人难以置信的创新。选用开源软件，就是选择一个发展更快的生态，就是选择跟随更有活力的技术趋势。像 Kubernetes 和 TensorFlow 这样的开源项目就是开源社区创新的典范，它们不仅引领技术发展，还成为行业标准。

更进一步，用户不仅可以坐等社区创新，还可以参与其中，成为社区创新的原动力，为上游发展做贡献，推动技术发展更符合自己的需要，这就是更吸引人的部分了。

4.5 有利于人员招聘

开源软件为企业吸引并留住顶尖技术人才提供了重要渠道，只采用“专有软件<部分采用开源软件<在企业内部对开源软件进行改进<鼓励员工将开源软件的内部改进反馈到上游<鼓励员工积极参与开源社区的各项工作<以企业的名义对外开源并因此成为技术领导者”这一模式。以上的小于号（<）代表企业对于高手（顶尖人才）吸引力不断提升的阶段。

在开源社区有影响力的企业越开放，越能够吸引顶尖人才的加盟，这已经是被反复验证的现实了。企业参与开源项目不仅可以提升品牌，还能通过与社区互动发现并培养潜在人才。这就是开发者关系（DevRel）这一专门领域的话题了。（推荐阅读《开发者关系：方法与实践》。）

总之，开源软件之所以比专有软件更有吸引力，归根结底在于其开放性所带来的生态优势。这种优势不仅体现在技术和成本上，还反映了社会合作和创新的力量。随着时代的进步，开源已经成为推动技术变革的重要引擎，也让企业和开发者在数字化浪潮中占据主动地位。

第 5 问 开源的核心价值观是什么

要谈开源的核心价值观，就要先理解开源的本质。

开源的本质需要从两个层面来理解：第一层面是一个开源项目，以及围绕这个

开源项目形成的开源社区，在这个社区的努力下，共同创造出开源产品；第二层面是由无数开源项目所组成的开源生态，以及围绕开源生态的各种利益相关者共同构建的开源世界。

5.1 开源项目的本质

开源项目的本质是一群人以开放式协作的方式，组成一个开源社区，共同创造数字公共产品。开放式协作不仅是开源软件的核心特质，也广泛应用于互联网论坛、邮件列表、在线社区，以及像比特币、TEDx 和维基百科等多种企业和平台项目。这种协作模式强调经济价值的产品、开放的贡献和消费渠道、互动和交流及有目的但松散协调的工作方式。在这样的协作环境中，任何人都可以自由地作出贡献和共享成果，这些成果是由一群分布式协同的互动参与者共同创造的。用户生成内容的商业网站也常常采用这种开放式协作模式，鼓励人们自由地交流和分享。

开放式协作具有以下特点。

- 开放参与：任何人或组织都有机会参与到社区协作中来。
- 规则公开：协作相关的规则尽可能追求公开透明，并且随着参与人数的增加，规则还可以被公开讨论并修改。
- 过程公开：社区协作的中间讨论过程、最终结论及社区发布的产品，都是公开可查阅的。
- 基于功绩的协作：协作群体内的决策权威和地位是基于各自的功绩而非外部强加的任命。
- 借助数字公共服务平台开展协作：GitHub、Gitee、维基百科、Hugging Face 等都是这样的协作平台。

按照数字公共产品联盟的定义，数字公共产品包括开源软件、开放数据、开放 AI 模型、开放标准和开放内容。在数字公共产品的标准定义中，还详细列出了以下 9 条指标和要求。

- 与可持续发展目标相关。
- 使用开放许可证。
- 清晰的所有权。
- 平台独立性。
- 完善的文档支持。
- 数据提取机制。
- 遵守隐私与相关法律。
- 符合标准与最佳实践。
- 采用“无害化设计”（包括数据隐私、安全、防骚扰等）。

由此，可以将数字公共产品理解为：以数字形式凝聚的人类知识，这些知识可以以多种方式节约人类的时间。

5.2 开源生态的本质

开源生态的本质就是由各种各样许可证或者开源世界的规则界定的、由全部数字公共产品组成的数字知识公地，所有参与这个开源生态的主体，基于共同遵守的规则，形成开放利益共同体。

- 数字知识公地与“公地悲剧^①”中所说的物理世界的公地不同，只要数字世界依然存在，只要复制成本始终接近于 0，数字世界的公地就不会减少，只会越来越繁荣。
- 开源世界的规则就是约束力大小不等的各种共识，共识内容、共识者的数量、共识被执行的可能性共同决定了这个数字知识公地的使用成本。
- 开放利益共同体就是各种主体基于各自不同的理由、价值判断，都认为向这个数字知识公地作出贡献，利大于弊。哪怕大家对于利弊的定义和理解，都完全不同。

5.3 开源从何而来

开源之所以能够发生，是因为人类有助人为乐的本能。助人为乐，并不是有人（人人）以助人为乐，而是任何人在助人之后都能够感受到快乐。这样的快乐有 3 种来源：自我肯定、受助者的感谢和他人（社会）的赞扬。

一个人行善举的原因，在于助人的收获大于付出，其中收获包括感受到的各种（当下的、长期的）快乐，而付出包括（当下的）成本与（长期的、潜在的）损失。

在开源领域，因为数字产品的复制成本接近 0，付出者自身并无损失，因此当下的付出几乎为 0。互联网的传播效应，使助人者收获的快乐也被放大了。因此，在自由/开源软件诞生的早期，有一技之长的开发者会非常乐于分享自己的智力成果。

至于开源为何从个人参与引发企业参与，为何开源这一个人英雄主义的现象进化为广泛存在的商业现象，可以进一步阅读第 3 篇。

5.4 开源的价值

开源的价值可以归纳为以下几点。

- 作为数字化知识的价值：由于开源公开可获取的特征，能够减少重复思考、重复劳动，从而为全人类节约大量的时间。
- 作为数字资产的价值：代码作为数据、代码的提交过程作为数据、整个社区的生产协作过程作为数据，都具有“资产”性质，这些数字资产，有可能被以不同的方式“挖掘”，从而产生层出不穷的价值。

^① 1968 年，加勒特·哈丁（Garrett Hardin）在期刊《科学》上发表以此概念为名之论述文章后，该概念被广泛地称为“公地悲剧”。

- 作为协作社区的价值：人与人在社区中协作，知识在社区中传递，同时被沉淀下来。社区的价值就体现为教育与转化的作用。

5.5 开源的核心价值观

开源的核心价值观可以归纳为以下几点。

- 分享。当一个软件工程师开发出一款优秀的软件，他不会吝于分享，也不会怯于分享。他乐于分享，是因为他相信：这个软件可能会对别人也有帮助，更会有人帮助他，一起做出更好的软件。赠人玫瑰，手留余香。我们都相信：乐于分享是一切善举的开端。
- 开放。开放的重要性体现在多个方面。开放源代码可以让智慧跨越时间和空间在一起交融。建立公开透明的社区，能够吸引更多的朋友加入，也能够帮助新来者理解并认同社区规则，还能够提升监督水平以促进社区运行的程序正义。开放包括欢迎一切可能性，欢迎来自世界任何一个角落的使用者、参与者和贡献者。海纳百川，有容乃大。我们都相信：公开透明是一切良好协作的基石。
- 平等。欢迎任何人的任何贡献，以统一的标准平等地评审每一次代码或文档的提交，且仅评审代码或文档本身的质量与价值，而不以贡献者的年龄、种族、性别、学历或职位等作为判断标准。人皆生而平等。我们都相信：平等是社区健康的保障。
- 协作。开源社区的协作，正是从接纳点滴贡献开始的。一个开放的社区，崇尚开放式的协作，这种协作不会在整个群体达成所有共识之后再开始，而是欢迎来自每个人的一点一滴的改进。不积小流，无以成江海。我们都相信：开放式协作，逐步凝聚共识是社区繁荣的秘诀。
- 创造美好世界。每一位投身开源的朋友，都或多或少是理想主义者。我们相信：这个并不完美的世界，理应变得更好。我们也相信：通过自己掌握的技术，借助开源的方法，能够把这个世界变得更好。我们还相信：开源的精神内涵，应该被推广到更多的领域。我们更相信：创造更加美好的世界是开源的终极追求。

因此，可以总结如下：开源倡导通过开放、平等、协作与共享的方式，创建数字知识公地，惠及全球，并推动知识积累、创新与人类进步。这不仅是开源运动的基石，也是未来全球数字化发展的重要支柱。

第6问 开源软件的许可证有哪些类型？它们各自的特点是什么

开源软件的崛起与普及，不仅在于源代码的公开与可自由获取，也得益于相对

完善的法律与许可框架。这些开源许可证为软件的使用、复制、修改与再发布提供了法律保障和行为准则。

6.1 许可证及许可证的各种类型

一、许可证

许可证是一种法律文件，通常由版权或其他知识产权的所有者授予他人使用其作品的权利。它规定了使用、分发和复制受保护内容的条件和条款。

许可证可以适用于各种类型的创作，包括软件代码、文档、音乐、图像等。

许可证的主要目的是平衡创作者或权利所有者的利益与使用者的需求，确保创作者能够从他们的工作中受益，同时允许他人在特定条件下合法地利用这些作品。许可证通常包含以下几方面内容。

- (1) 授权范围：明确用户可以做什么和不可以做什么。
- (2) 分发条款：规定如何分享、复制或修改受保护的作品。
- (3) 许可限制：可能包括商业使用限制、源代码披露要求等。
- (4) 法律责任：定义在违反许可证条款时的责任和后果。

二、软件许可证

软件许可证是许可证在软件领域的实例，用于规定用户如何安装、复制、修改和分发特定软件产品。

软件许可证通常分为以下几种类型。

(1) 开源软件许可证 (Open Source License)：这类许可证允许用户自由使用、复制、修改和分发软件。GNU 通用公共许可证 (GPL)、MIT 许可证 (MIT License) 和 Apache 许可证 (Apache License) 都属于此类。这类许可证通常鼓励社区贡献代码，并确保所有衍生作品也必须保持开放。受开源软件的启发，后续还诞生了开源硬件许可证、开放文档许可证、开放数据许可证等类别。

(2) 商业（专有）软件许可证 (Proprietary License)：这类许可证限制了用户使用和分发软件的方式。例如，EULA (End User License Agreement) 是一种常见的商业许可证类型。商业许可证通常附带特定的费用，并可能包含严格的使用条款和条件。

(3) 试用许可证 (Trial License)：这类许可证允许用户在一定期限内免费试用软件，期满后可能需要购买正式许可才能继续使用。

(4) 教育许可证 (Educational License)：这类许可证是针对教育机构提供的特殊条款和条件。这类许可证通常允许学校或教育机构以较低的价格将软件分发给学生和教职员工。

6.2 开源许可证的重要性

在未明确授权的情况下，软件的使用、复制和发布都可能受到版权法规限制。直白地说：如果一个“开源的”代码仓库的根目录下找不到名为 License 的文件，

你就要非常谨慎，它可能并未允许你做任何事情。

开源许可证作为一种法律协议，明确了软件作者与用户之间的权利与义务。对开发者而言，它确保他们的劳动成果不会被非法占用或滥用。对用户而言，许可证为他们使用、学习、改进并发布这些软件提供了明确的合法通道。

6.3 开源许可证的分类概览

开源许可证大致分为以下几类。

一、宽松许可证

宽松许可证（Permissive License）的典型代表有 MIT 许可证、BSD 许可证、Apache 许可证和木兰宽松许可证。

宽松许可证具有以下几个主要特点。

- 使用自由度高：这种许可证对代码使用者的限制非常少。用户可自由复制、修改、再分发，无须把修改后的作品以同样的许可证开源。
- 可闭源再发布：用户可以将基于该软件或代码修改后形成的衍生产品闭源发布，甚至进行商业化销售，而无须公开自身的代码修改。
- 责任与担保免责：通常这类许可证会声明软件按“原样”（as-is）提供，不对适用性或质量做出担保，并免除原作者的责任。

宽松许可证适用于以下几个场景。

- 商业软件公司常选择从 MIT 许可证或 BSD 许可证下的项目中汲取组件，因为能轻松整合进其闭源产品。
- 个人开发者和初创企业也经常选择这种许可证，以求最大限度推动代码传播与使用。

二、强 copyleft 许可证

强 copyleft 许可证（Strong Copyleft License）的典型代表有 GNU 通用公共许可证、木兰公共许可证。

强 copyleft 许可证具有以下几个主要特点。

- 强复制条款：GNU 通用公共许可证的核心要求在于，只要你修改并发布了基于 GNU 通用公共许可证代码的衍生作品，就必须在发布衍生作品时也将其在同样的 GNU 通用公共许可证下开源。这种条款确保了开源生态的持续循环与贡献。
- 防止“吃白食”现象：通过保证衍生作品的开源性，GNU 通用公共许可证确保改进后的代码不会被封闭起来，有助于维护开源社区的公共利益和共享精神。
- 严格程度较高：GNU 通用公共许可证在商业化使用方面较为严格，如果企业打算在自身产品中使用 GNU 通用公共许可证代码，就必须公开相应的修改部分的源代码，这可能使其商业策略受限。

强 copyleft 许可证适用于以下几个场景。

- 致力于开源事业、希望确保后续开发者也要分享改进成果的项目。
- 普遍用于 Linux 内核及大量 GNU 相关项目。

三、弱 copyleft 许可证

弱 copyleft 许可证（Weak Copyleft License）的典型代表有 LGPL（GNU Lesser General Public License）、MPL（Mozilla Public License）。

弱 copyleft 许可证具有以下几个主要特点。

- 处于中间地带：弱 copyleft（也称为“宽松 copyleft”）在严格性上介于宽松许可证与 GNU 通用公共许可证等强 copyleft 许可证之间。
- 动态链接友好：例如，LGPL 允许软件以动态链接的方式集成到闭源软件，而无须强制开源整个应用程序的代码，但对直接基于该库修改的部分仍有开源要求。
- 模块化共享：例如，MPL 许可对文件级别的修改施加约束，修改该文件则须开源，但与其他文件隔离的代码部分则无须开源。

弱 copyleft 许可证适用于以下场景。

- 希望在保护部分开源共享精神的同时，给商用产品提供更大灵活性的项目。
- 要开发一个库或框架，并鼓励商业软件使用，但不希望全盘闭源。

四、其他特色类许可证

- GPLv3 与 AGPL：GPLv3 解决了硬件限制与专利问题，而 AGPL（Affero GPL）则扩展了对网络服务环境的共享要求。如果你将 AGPL 代码用于网络应用，需要在用户可访问的前端提供源代码获取方式，从而避免“通过网络提供服务但不分享改进代码”的情况。
- CC-BY、CC0 等知识共享许可证：这类许可证多用于内容（文档、图片、音视频）而非软件本身，但在开源项目的文档、设计、模型分享中常被采用。
- 特殊条款许可证：一些组织或公司通过为特定产品定制自己的开源许可证，以满足独特的法律或战略需求。

6.4 常见开源许可证的特征

通过分析常见许可证的官方原件，我们可以得出其特征，如表 6-1 所示。

表 6-1 开源许可证特征

开源许可证	商业使用	分发代码	内部使用	专利授权	合并/修改代码	使用库	不允许修改许可证
MIT 许可证	√	√	√				
BSD 2-Clause	√	√	√				
BSD 3-Clause	√	√	√				
Apache-2.0	√	√	√	√			

续表

开源许可证	商业使用	分发代码	内部使用	专利授权	合并/修改代码	使用库	不允许修改许可证
MPL 2.0	✓	✓	✓	✓	✓		需分析使用场景
LGPLv2.1	✓	✓	✓		✓		需分析使用场景
LGPLv3	✓	✓	✓	✓	✓		需分析使用场景
GPLv2	✓	✓	✓		✓	✓	✓
GPLv3	✓	✓	✓	✓	✓	✓	✓
AGPLv3	✓	✓	需分析网络应用使用场景	✓	✓	✓	✓

下面具体介绍表 6-1 中的特征。

- 商业使用：该许可证下的代码和衍生品可用于商业目的。
- 分发代码：该许可证下的代码可以分发给第三方。
- 内部使用：该许可证允许在修改源代码后，在个人及组织内部使用，不必向外部分发。
- 专利授权：该许可证明确提供专利授权。
- 合并/修改代码：从要组合的开源软件中提取整体或部分代码，修改或不修改都可以，然后把它添加到你的代码中构成一个作品，分发时必须提供源代码。
- 使用库：在编译或运行时，通过链接、导入或其他典型的机制（如静态与动态链接），将开源代码与自有代码绑定后，自有代码分发时必须提供源代码。
- 不允许修改许可证：当使用“合并、修改、使用库”等方式组合开源代码与自有代码后，不允许自有代码使用不兼容的开源许可协议或闭源分发。

第 7 问 选择开源许可证时需要考虑哪些因素

以下内容将分别从企业使用开源软件、企业参与外部开源社区、企业对外开源自身软件这 3 个角度对如何选择合适的开源许可证进行探讨。

7.1 企业使用开源软件

对大多数企业而言，开源软件已经成为不可或缺的组成部分。然而，企业使用

开源软件并不只是“下载—使用”这么简单，还需要从商业利益和供应链安全的角度评估所使用的开源许可证。

先看商业利益与合规风险。

- 许可证限制：有些许可证（如 GPL、AGPL）带有 copyleft 特性，要求对源代码或衍生作品进行开源。如果企业在闭源软件中使用了带强 copyleft 许可证的开源软件，可能会被迫公开自身核心代码，造成商业利益或技术机密损失。
- 责任与法律风险：不同许可证对于侵权责任、专利授权等条款规定不同，若不加以审慎评估，企业可能面临知识产权或专利纠纷风险。

再看供应链安全与合规管理。

- 供应链安全：随着软件供应链安全的重要性日益凸显，企业需清晰地了解所引入的开源组件的许可证信息及版本信息，以便在出现安全漏洞或许可证合规问题时能快速定位和应对。
- 软件物料清单：越来越多的企业开始引入软件物料清单的概念，通过建立可追溯的组件清单来强化供应链安全和许可证合规。如果开源组件本身的许可证与企业合规策略存在冲突的可能性，应该尽早替换、规避或取得相关授权。

最后看与内部研发和产品路线的匹配。

- 技术栈一致性：企业在选择开源软件时，往往会考虑软件的活跃度、社区支持度及许可证与企业已有技术栈的兼容性。
- 商业模式匹配：对于一些对外售卖的产品，如果引用了带强 copyleft 许可证的开源组件，可能影响产品的商业模式。

企业在使用开源软件时，首先应该考虑许可证合规，还要从商业利益保护与供应链安全的角度出发，制定清晰的审核流程和策略，以规避潜在的法律和商业风险。另外，企业内部的技术需求与产品发展规划，也是需要参考的因素。

7.2 企业参与外部开源社区

当企业决定将内部的部分研发成果贡献给外部开源社区时，需要充分理解原项目的许可证条款，并评估对企业自身以及外部社区产生的影响。

（1）与目标社区/项目许可证的兼容性。

- 兼容性：如果目标社区使用的是 GNU 通用公共许可证，而企业想贡献的代码来自基于 Apache、MIT 等许可证的项目，就需要确认是否存在不兼容问题（如部分早期版本的 GPL 与 Apache 许可证可能存在冲突）。
- 项目规范：一些大型开源社区通常有贡献者许可协议（Contributor License Agreement, CLA）签署流程，要求贡献者明确放弃或授权一定范围内的知识产权。企业需要评估这些协定是否与自身策略相冲突。

(2) 内部审查与管理流程。

- 识别贡献内容：企业在贡献代码前应对拟贡献的代码进行审查，确保其中不包含专有信息、敏感信息或第三方受限制的代码。
- 与利益相关方的沟通：对大多数企业而言，贡献开源并不是技术部门的单方面决定，往往还涉及法务、合规、市场等部门，需要形成明确的流程来管理对外贡献的行为。

(3) 企业形象与社会责任。

- 品牌与声誉：积极参与和贡献开源社区不仅有助于塑造企业开放、创新的形象，也可以吸引更多开发者和合作伙伴。
- 社区生态共建：与社区紧密合作，在享受社区成果的同时也为社区作出积极贡献，可以帮助企业在未来获得更多技术支持和合作机会。

企业在将内部成果贡献到外部社区时，要综合考虑目标社区的许可证兼容性、内部法务流程以及对企业形象和未来生态的影响，进行审慎的选择。

7.3 企业对外开源自身软件

当企业决定将自主研发的软件（或其部分）对外开源时，许可证的选择直接影响企业的知识产权保护、商业利益以及对外合作方式。

(1) 保护知识产权与商业利益。

- 选择合适的宽松许可证或 copyleft 许可证。

宽松许可证（如 MIT、BSD、Apache）允许他人灵活地使用、修改和商业化，能快速吸引更多社区和商业伙伴加入，但企业对衍生软件的控制权较弱。

copyleft 许可证（如 GPL、AGPL、LGPL）可以确保衍生版继续开源，在一定程度上防止他人“闭源化”企业的开源成果，但可能限制商业合作伙伴或某些商业生态的采用。

- 兼顾企业商业策略：如果企业希望通过卖增值服务、云服务或企业级支持来盈利，则可能更倾向于强 copyleft 许可证（如 GPL、AGPL）或“源可见但用途有限”的模式；如果希望最大限度推广自己的技术或标准，则更倾向宽松许可证。

(2) 与企业已有专利和技术机密的关系。

- 专利许可：Apache-2.0 等许可证包含专利授权条款，能够对企业 在开源软件中涉及的专利提供一定防护。如果企业自身拥有核心专利，需要评估许可证在专利领域的影响。
- 商业秘密与保留部分：对于不希望公开或还在孵化中的核心技术，企业可以采取“开源核心、保留插件”或“核心基础开源、商业化部分闭源”的混合模式，并用合适的许可证来约束不同部分。

(3) 吸引外部贡献和社区共建。

- 社区友好度：更宽松的许可证通常对开发者更具吸引力，不会让他们担心

在某些场景下无法正常使用代码。

- 减少潜在冲突：一些开发者或企业会刻意回避带有强“传染”特性的许可证^①，因此在选择 GPL、AGPL 等许可证时，要考虑对外合作和商业伙伴的接受程度。

(4) 流程与合规制度建设。

- 开源项目治理：企业在选择许可证后，需要配套建立明确的治理流程，包括代码审核、贡献者引入、版权声明等，以保证对外开源持续、规范地进行。
- 持续合规检测：对社区贡献的代码进行审计，识别潜在的法律与合规风险，防止引入不兼容或受限的代码。

企业自行开源时，不仅要关注许可证是否能保护自己的商业利益与知识产权，还需兼顾社区友好度和对合作伙伴的影响，最终通过一套完备的开源治理体系落地实施。

第 8 问 开源如何影响软件开发的方式和文化

在软件工程的发展历程中，开源运动不仅引入了一系列新颖的开发模式和管理理念，也深刻地改变了软件行业的文化与实践。从传统的线性规划和瀑布模式为特征的“大教堂模式”，到强调快速迭代、开放协作、多元参与的“集市模式”，软件开发正经历一场持续而深远的变革。与此同时，开源社区在实践中不断发展出新的工具、平台与沟通方式，使得这一变革不仅局限于开源世界本身，更深度地融入了企业级开发和整个软件行业的主流文化中。

8.1 从“大教堂”到“集市”：软件开发范式的转变

雷蒙德在《大教堂与集市》一书中对“大教堂模式”和“集市模式”这两种典型的软件开发模式进行了对比。

他将传统的软件开发过程比作修建一座大教堂：开发团队内部有严格的层级与计划，产品路线图清晰却缺乏灵活度，开发周期漫长且释出谨慎，决策权多由少数权威人士掌控。这种模式下的团队在交付成果前期常常封闭作业，高度依赖事先规划与自上而下的管理，其结构性与可控性类似于传统的瀑布式开发。

与之相对，雷蒙德提出的“集市模式”则呈现出更为灵动、多元和开放的特征。这种模式更多地借鉴了开源社区的实践：人人可参与，代码不再局限于某个封闭的团队内部，而是向全球开发者与用户敞开大门。2001 年之后，敏捷软件开发（Agile Software Development）的兴起使得集市理念在企业级开发中广泛应用，可

^① 强 copyleft 许可证往往会被认为具有较强的“传染性”，但是也有专家认为“传染”这个词本身就是一种污名化的称呼，而应该称之为“互惠性”。

将企业内的敏捷团队（以及众多敏捷团队加在一起）视为在组织内部打造了一座小型的集市，而开源社区则是这种模式的天然温床。

传统软件工程的观点往往认为，无序、嘈杂、看似无人管理的开发模式会导致混乱和低效。然而，开源社区的经验表明，当这种集市模式在合适的条件下得以实践时，其实更具有创新与活力。通过开放代码、鼓励众人参与，软件项目得以在一个自组织、去中心化的环境下快速进化。尽管听上去缺乏传统架构下的秩序和计划性，但实际结果往往是问题被更快速地发现与解决，进而实现软件质量与功能的持续提升。

在此逻辑下，集市模式可被视为开源语境下的敏捷开发：多方协作、快速迭代、持续反馈，鼓励开发者在社区中自发贡献与学习。

8.2 开源社区的实践原则：快速迭代与用户反馈

雷蒙德总结了多项有助于提升软件质量与开发效率的开源实践原则，其中最广为人知的便是“早发布，经常发布”策略。这一策略鼓励在产品还不算完备时就向社区释出版本，借此快速吸收用户反馈，并在后续更新中不断完善。与传统瀑布式开发中漫长的开发、测试、发布周期相比，这样的频繁迭代有助于加速问题发现与解决的过程。

除此之外，注重用户反馈和模块化设计也是开源文化的重要基石。模块化设计意味着将大型软件项目切分为相对独立、可替换的组件，方便不同领域的专家独立负责和改进各自的模块，降低耦合度并加速迭代。倾听用户反馈（甚至直接将用户反馈算作社区贡献之一）则确保开发方向与用户需求保持高度贴合，这也是开源社区成功的关键要素之一：用户不仅是被动的使用者，更是积极的测试者与改进建议的提供者。

8.3 分布式开发与精英治理：多元化的贡献者结构

在开源项目中，开发团队往往遍布全球，跨越不同的时区、文化背景和技术领域。这种分布式开发的特质使得协作与沟通必须突破传统组织结构的约束。分布式协作在降低地理与组织壁垒的同时，也为软件开发文化注入了独特的价值观。

- 自治与自组织：个体贡献者不必等待管理层审批即可提交代码，贡献质量决定了他们在社区中的地位与影响力。
- 精英治理与核心贡献者：许多开源项目虽然表面上人人可参与，但在决策层面往往由少数高水平、信誉良好的“精英贡献者”与“核心维护者”进行治理。这种精英治理模式实现了社区开放性与决策高效性的平衡：一方面保持了大量贡献者的活力与多样性，另一方面又通过核心团队的把关确保项目走在正确道路上。

分布式开发与精英治理为软件开发文化注入了“胜任力即权威”的理念。与传

统组织的自上而下控制相比，这里更多的是贡献与能力塑造地位和影响。这个过程不仅提高了软件质量，也为后继项目与开发者开辟了有效的学习、提升与晋升路径。

8.4 林纳斯法则：质量保障的独特逻辑

雷蒙德还提出了另一个深入人心的洞见，即所谓的林纳斯法则：“只要有足够多的人查看，所有的漏洞都会变得显而易见。”这意味着，当有大量开发者和用户共同审视同一份代码时，潜在的问题会更早被捕捉，更快被修复。这与传统封闭开发的测试流程截然不同，在传统开发中，测试团队的规模有限，测试范围和深度受时间与人员限制。而在一个开源的集市环境下，无数双眼睛帮助保证软件质量，缩短了问题修复的周期。

8.5 在线包管理工具：生态系统与再利用的繁荣

伴随开源项目数量激增，代码的复用需求日益强烈。在线包管理工具与生态（如 npm、PyPI、RubyGems、Maven Central）帮助开发者轻松找到并引入他人贡献的功能模块，从而快速构建复杂应用。

- 快速迭代与高度集成：开发者无须从零开始实现基础功能，可直接使用成熟的库、框架或组件。
- 依赖管理与版本控制：包管理工具不仅简化了依赖解决，还促进了更可靠的版本控制、自动更新与安全审查。
- 生态型创新：随着更多高质量开源包的出现，不同语言、框架和工具间的互通性和可移植性不断增强，进而激励更多创业公司与团队基于这些开源“积木”快速搭建新应用与服务。

这种基于包管理工具的开发方式，本质上是一种知识与代码资产的不断重组和累积。开发者在全球范围内共享成果，彼此协同，进而改变了传统软件工程中“从无到有”开发模式的思维定式。

8.6 社交化编程与自动化工具：从协作到文化升级

在开源协作早期，人们主要依靠邮件列表与 IRC 沟通，分发代码则借助 CVS 或 SVN 等工具。后来的 Git 与 GitHub 在版本控制与协同方面的创新，使代码审查、问题跟踪、合并请求（Pull Request, PR）流程、持续集成（CI）/持续交付（CD）流水线等特性变得标准化和社会化。如今，GitHub、GitLab 及类似平台的社交化编程生态，让开发者与用户在代码之上建立了共识、网络与品牌。

在线自动化工具（如 GitHub Actions）与机器人、应用等扩展服务，则在此基础上进一步强化了开源开发的效率和便捷度。自动化 CI/CD 流水线、代码质量检

查、漏洞扫描、部署触发等功能全部在一个统一的平台上实现，开发者能将更多精力投入创新与质量提升。

8.7 开放文化渗透企业：从开源到内源

随着开源实践的不断成熟，企业开始意识到：将开源理念内化到企业内部开发流程中，可提升组织创新力与生产力。当 GitHub Enterprise、GitLab、内部包管理与协作工具引入组织时，企业内部的开发团队得以复制“集市”模式的快速反馈循环。同时，企业开发人员更熟悉开源社区的工​​具链，进一步拉近了企业软件开发与开源文化的距离。

在这种“内源”（Inner Source）环境中，企业内部团队之间共享源代码、互相评审与合并请求，以类似开源社区的自治与协作模式来提高生产效率。由此，企业文化中对知识封闭、决策僵化的倾向受到挑战，更开放的沟通、更灵活的架构与更加透明的决策过程得以形成。

第 9 问 开源人有哪些特点？如何与他们做朋友

通常所说的开源人，是指积极参与开源软件开发、维护、推广以及社区活动的人。他们因共同的理念和特质而形成一个富有活力的群体（开源社区）。当然，更广义的开源人是指以开源的理念指导自己的工作和生活，并积极传播开源理念的人。他们通常都具有一些共同的特点，这使得与他们相处，和他们交朋友，非常容易。

9.1 开源人的特点

开源人有以下 10 个主要特点。

- 热爱技术，充满好奇心。开源人热爱技术，善于钻研新技术，乐于解决复杂问题。他们乐于尝试新事物，对技术进步抱有浓厚的兴趣。这种好奇心驱使他们不断学习、探索技术的前沿。
- 有共享与开放精神。共享知识和资源是开源人的核心价值观。他们乐于分享代码、经验和见解，认为开放和协作能促进技术进步，造福社会。开源人也倾向于在开放的环境中交流与学习，对信息和知识的流通持欢迎态度。
- 有高度的团队协作能力。开源项目依赖于全球范围内的多方协作。开源人通常具有很强的团队合作意识，能够与不同背景、文化和观点的人高效沟通，共同解决问题。他们尊重他人的贡献，也能够融入群体，推动项目进展。
- 能持续学习与自我驱动。面对技术的快速更新迭代，开源人保持着持续学

习的态度。他们主动适应变化，通过实践和探索提升自己。开源人通常是自我驱动的个体，能够独立思考、自主解决问题。

- 注重规则与社区协作规范。开源项目的成功离不开清晰的协作规则和贡献准则。开源人尊重透明的社区规范，注重按规则行事。他们积极参与社区讨论，遵循既定流程，并为社区治理贡献力量。
- 有开放性与包容心。开源人对新观点、新方法和新文化持开放态度。他们愿意接受不同的想法，在技术和思想上都表现出高度的包容性。这种开放性不仅体现在对技术选择的灵活态度上，也表现在他们对多元文化的欣赏和尊重。
- 有强烈的社区意识。开源人深知社区是学习、成长和合作的重要平台，积极参与贡献代码、撰写文档、改进项目、参与讨论等活动。他们也常通过举办或参与技术活动（如黑客松、技术沙龙等）回馈社区。
- 诚实与透明。开源提倡透明和诚信，开源人通常坦诚直率，在沟通中真实表达自己的观点和想法。他们尊重彼此的意见，力求在信任的基础上建立牢固的协作关系。
- 乐于助人且慷慨。许多开源人将助人为乐视为一种成就感。他们乐于解答他人的疑问，为社区提供帮助，并希望自己的贡献能对他人和社会产生积极影响。他们的慷慨体现在愿意贡献时间、技术和资源。
- 有解决问题的能力。面对问题，开源人倾向于主动寻找解决方案，而非回避挑战。他们勇于尝试，依靠实践验证想法，并在迭代中不断完善自己的解决方案。

9.2 与开源人做朋友

其实与开源人做朋友很简单，最简单的办法也许就是“成为开源人”。

一、积极参与开源项目

- 选择感兴趣的项目：找到一个你有热情的开源项目，参与代码贡献、文档改进、问题反馈或社区讨论。共同的目标和协作经历是建立友谊的良好基础。
- 展示自身价值：通过实际贡献展现你的能力和热情，让开源人看到你对项目的真诚投入。

二、尊重并理解开源人的工作方式

- 遵守规则和流程：开源社区有自己的工作规范，如贡献指南、代码评审流程等，遵守这些规则是赢得信任的重要方式。
- 理解文化差异：开源社区成员来自全球，不同的文化背景和工作方式可能存在差异。尊重和包容这些多样性，有助于融入社区并建立信任。

三、分享知识与学习心得

- 开放分享：开源人热爱分享知识，也欣赏他人分享的见解。在你的学习或贡献过程中，分享心得、经验或有趣的想法，能激发更深入的交流。

- 参与讨论：在社区中积极发言，提出问题或帮助解答疑问，展现你的兴趣和爱好，促进双向交流。

四、赞赏与支持开源人的贡献

- 表达感谢：对开源人的贡献表示感谢，如点赞、复用、推广他们的项目，或在社交媒体上表达对他们工作的支持。
- 认可价值：具体指出他们贡献的意义，哪怕是一句真诚的赞扬，都能增进关系。

五、互相帮助与支持

- 技术支持：在遇到难题时，主动向他们寻求帮助或为他们提供技术支持。共同解决问题不仅增进感情，也能让彼此信任更深。
- 非技术支持：除了技术上的合作，还可以在职业、生活等方面提供力所能及的帮助，建立更加全面的关系。

六、参加开源活动

- 线上交流：积极参与开源社区的线上活动，如论坛讨论、问题答疑或在线研讨会。
- 线下活动：参加黑客松、技术沙龙、开源会议等线下活动，面对面的互动能更快拉近距离。

七、保持开放和好奇的心态

- 倾听想法：开源人乐于表达自己的观点，保持开放心态，多倾听他们的故事和见解，即便与你的背景不同，也努力理解他们的工作和生活方式。
- 展现独特性：分享你的兴趣爱好、职业经历等，展示你的独特性和共同兴趣点，增加话题共鸣。

八、长期坚持与投入

- 建立信任：友谊需要时间积累，持续参与社区活动、贡献成果，并与他们保持互动，能让关系逐步深化。
- 贡献价值：无论是技术、文档还是翻译，你的每一份贡献都会被社区认可，成为建立友谊的纽带。

通过真诚的参与、积极的交流和持续的支持，你不仅能够与开源人建立友谊，还能成为开源生态中的重要一员，共同为开源事业创造更多价值。

开源人物小传

一、理查德·斯托曼

理查德·斯托曼是自由软件基金会（FSF）的创始人，被誉为“自由软件运动之父”。

其主要贡献如下。

- GNU 项目：1983 年发起 GNU 项目，旨在创建一个完全自由的操作系统。
- GNU 通用公共许可证：制定了 GNU 通用公共许可证（GPL），开创了以法律保障软件自由的方式。

理念：提倡软件自由，强调用户应有“使用、复制、研究、修改和分发”软件的权利。

名言：自由不是免费的问题，而是权利的问题。

影响：斯托曼的自由软件理念成为开源运动的基石。

二、林纳斯·托瓦兹

林纳斯·托瓦兹是 Linux 内核的创造者，被誉为“Linux 之父”。

其主要贡献如下。

- Linux：1991 年发布了 Linux 内核，开创了一个开源操作系统的时代。
- Git：2005 年开发了分布式版本控制系统 Git，成为全球开发者必备工具。

理念：更注重实用性和技术创新，而非纯粹的哲学自由。

名言：Just for Fun（只是为了开心）。

影响：Linux 成为全球开源软件的标志性成果，广泛应用于服务器、移动设备和嵌入式系统。

三、埃里克·雷蒙德

埃里克·雷蒙德是开源运动的重要推动者和理论家。

其主要贡献如下。

- 《大教堂与集市》：1997 年发表的论文，系统阐述了开源开发模式，提出“集市模式”开发哲学。
- 开源定义：与布鲁斯·佩伦斯等人共同起草了“开源定义”，推动开源运动从自由软件中分离出来。

理念：提倡协作开发和快速迭代。

名言：只要有足够多的人查看，所有的漏洞都会变得显而易见。——林纳斯法则。

影响：帮助开源运动在商业界站稳脚跟，使开源模式成为软件开发的重要方式。

四、布鲁斯·佩伦斯

布鲁斯·佩伦斯是开源运动的早期参与者，“开源定义”的联合起草者。

其主要贡献如下。

- Debian 项目：参与创建了基于 Linux 的 Debian 操作系统，是其第二任领导者。
- 开源定义：1998 年与埃里克·雷蒙德共同起草了“开源定义”，为开源软件提供了理论框架。
- 倡导企业参与开源：推动企业接受开源模式，促进商业化与开源的结合。

理念：认为开源可以兼顾自由和商业价值。

名言：开源不仅仅是自由软件，它是一种更广阔的合作方式。

影响：他的努力帮助开源理念进入主流社会，并获得商业界的支持。

五、陆首群

陆首群是中国开源软件推进联盟名誉主席，被誉为“中国开源泰斗”，是中国开源运动的早期推动者。

其主要贡献如下。

- 开源普及：长期致力于推广开源理念，通过技术讲座和行业论坛宣传开源精神。
- 国际化合作：积极参与国际开源社区的合作，提升了中国在开源领域的影响力。

理念：倡导开源技术的本地化和产业化发展，为中国的开源生态奠定了基础。

影响：他的努力为中国开源社区的发展注入了强大动力，推动了中国企业和开发者对开源技术的接受与应用。

第 2 篇 开源生态系统

开源生态系统是一个基于协作与共享的复杂社会经济系统，其核心由多元角色、治理模式与演化规律共同构成。如果你对开源的理解还停留在“代码公开、免费使用”这个层面，那么本篇将带你看见更深层的风景。

开源不只是一种技术现象，更是一种社会现象。

一个开源项目从诞生到消亡，会吸引哪些人参与？他们各有什么诉求？如何协调无数素不相识的开发者共同维护一个项目？当企业利益与社区公益发生冲突时怎么办？这些问题的答案，构成了开源生态系统的真正骨架。

第 10 问 开源生态系统包括哪些关键角色

开源生态作为一个社会经济系统，也是一个不断生长的概念，从最初的开源项目发起人，到后来越来越多的参与者，以及出于各种不同的理由不断加入的各种角色，发展到今天已经形成一个非常复杂的生态系统了，如图 10-1 所示。

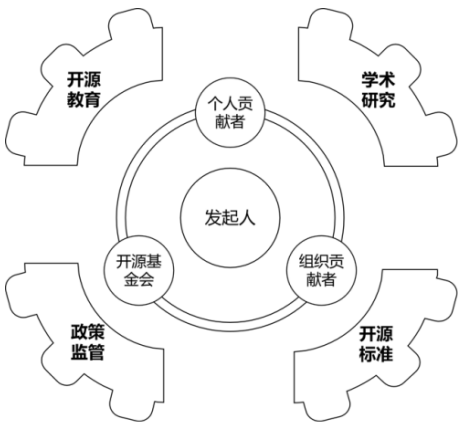


图 10-1 开源生态系统中的关键角色

10.1 从“个人”与“组织”维度分类

个人扮演的角色（个人贡献者）包括以下几类。

- 项目发起者（Project Initiator）。提出开源项目的创意或需求，制定初始目标与规划，他们通常还是这个项目的第一行代码的撰写者、最初的文章作

者。他们可能在项目初期身兼多职（如核心维护者、布道者等），直到项目发展壮大之后，再将一些角色分配给他人。

- 维护者/核心开发者（Maintainer/Core Developer）。负责项目整体的技术决策、代码审阅、版本发布等，是项目的长期、核心贡献者，具备合并代码、管理社区贡献的权限，是一个项目中最为重要的角色。
- 贡献者（Contributor）。在社区中做出各种贡献的人，包括提交代码、撰写文档、测试、翻译、设计、社区运营等多种形式。他们的努力将推动项目不断完善与进步，通常一个项目的贡献者数量，决定这个开源社区的整体规模。
- 安全审计员/专家（Security Auditor/Experts）。在开源项目中承担对代码和系统进行专业审查的工作，包括漏洞检测、依赖分析、风险评估等。他们不仅发现潜在安全隐患，还会为核心维护者和贡献者提供修复建议或安全增强方案，使得项目更可靠、抗风险能力更强。随着开源项目在关键基础设施和商业应用中广泛部署，安全专家的作用愈发重要。
- 漏洞报告者/白帽黑客（Bug Bounty Hunter/Ethical Hacker）。这些人通常以悬赏形式或出于责任感，自发地对开源项目进行渗透测试或漏洞挖掘。发现漏洞后，他们会在公开披露前私下通知项目维护者，以便及时修补，防止恶意利用。相应的漏洞奖励机制也激励更多人参与此类正向行为，从而提升整体安全水平。
- 社区经理（Community Manager）。在社区发展到一定规模后，项目发起人或核心维护者已经没有精力过多关注社区运营工作，此时社区经理的角色变得至关重要。他们关注社区健康与活跃度，组织活动、协调沟通、解决冲突，负责帮助新成员融入社区，促进多元化与包容性等工作。
- 用户（User）。开源软件或项目成果的使用者，包括个人开发者、早期尝鲜者、普通爱好者或测试者，他们通过反馈意见、提交漏洞报告和需求建议，推动项目改进。
- 布道者（Evangelist/Advocate）。有特定开源项目的布道者，也有专门的开源布道者，他们通过会议演讲、技术文章、社群活动等方式，向外界推广项目和开源理念，对吸引潜在用户与贡献者、提升项目影响力起到关键作用。
- 导师（Mentor）。当一个项目被捐赠给开源基金会之后，基金会往往会指派经验丰富的导师，以“传帮带”形式指导新贡献者或核心成员，帮助他们熟悉项目结构与贡献流程。他们所扮演的这样的角色，能够有力地保障社区的可持续发展与人才培养。

组织所扮演的角色（组织贡献者）包括以下几类。

- 企业（Enterprise/Company）。企业是开源生态中最常见的组织参与者之一。它们既可能作为开源项目的重要用户，也可能直接参与到项目的维护与贡献中。许多大型企业会投入资源，专门组建或赞助开源项目团队，通过提供资金、基础设施或技术专家的方式来支持社区。企业通常具备较强

的市场影响力，当它们采用并推广某个开源项目时，往往能带来巨大且快速的用户增长。同时，企业在开源项目中汲取的技术收益或品牌价值，也能够反哺其自身的产品与创新。

- 赞助商与捐赠者（Sponsors/Donors）。这类角色通过资金、服务器资源或其他形式的支持，来帮助开源社区减轻运维或开发成本压力。他们可能是个人，也可能是机构、企业或基金会，出发点既可能是公益理念，也可能是战略考量。赞助商与捐赠者的支持通常能使核心维护者和社区经理更专注于项目本身，从而帮助项目取得更长远、稳健的发展。一些赞助计划也会配合社区激励机制，如定期透明披露经费使用，或推出贡献者的“奖金池”，在鼓励创新的同时确保社区的公开与公正。
- 生态合作方（Ecosystem Partners）。这部分角色既有可能是商业公司，也可能是技术联盟或解决方案提供商，它们与开源项目形成互补关系，共同构建产业生态。例如，云服务厂商为开源项目提供免费或优惠的服务器资源，CI/CD 平台为开源项目提供自动化测试和集成工具，安全厂商提供漏洞扫描和防护机制，律师事务所提供开源合规咨询服务等。合作伙伴与生态合作方的存在，使得开源项目在行业落地时更具竞争力，也让项目与市场需求无缝对接，为社区带来更多的潜在用户与贡献者。
- 社区组织或非营利组织（Community/NPO）。这些组织通常专注于公益、教育或社会事业，往往通过协调志愿者、组织开源活动或项目竞赛等方式，促进开源文化的传播与社区的多元化。非营利组织常承担“桥梁”角色，一方面为社区成员提供各种资源和培训，另一方面也帮助社区凝聚力量、提升影响力。在一些情况下，非营利组织还会与政府或行业协会合作，推动开源在公共政策或行业标准层面的落地，为开源生态带来更广泛的社会价值。
- 开源基金会（Open Source Foundations）是一类特殊的组织贡献者。开源基金会是开源生态中提供法律与治理支持的重要力量，如 Linux 基金会（Linux Foundation）、Apache 软件基金会（Apache Software Foundation）、Mozilla 基金会（Mozilla Foundation）等。它们在保护项目商标、维护版权及管理许可证等方面具有丰富的专业经验。开源基金会通常提供一系列项目治理模式，让开源项目能够在更透明、更民主的机制下运转。有些开源基金会还设立导师团队，帮助新项目迅速上手并建立可持续的发展架构。开源基金会往往承担“背书”或“保管人”的角色，使得开源项目在法律和组织层面更加稳健。

10.2 从“教育、学术与政策”视角扩展出的角色

开源教育类的角色包括以下几类。

- 教育者/培训机构（Educator/Training Provider）。这些角色通常活跃于学校、培训机构、在线教育平台和社区活动中，负责教授与开源相关的知

识和实践技能。他们可能开设开源课程、组织工作坊或竞赛，以帮助更多人掌握从版本控制（如 Git）到项目协作（如 Pull Request 流程）等实用技能。通过面对面或在线的授课、辅导，教育者不仅在技术层面提升学生能力，也在精神层面推广开源文化，培养更多潜在的贡献者和核心维护者。

- 学习者/学生（Learner/Student）。他们是开源教育的主要受众，包括在校大学生、职业转型者，以及希望通过自学提升开源技能的个人。通过接受培训、参与社区活动或阅读开源文档，学生和学习者逐渐掌握实践技能，并有机会在真实的开源项目中进行实际贡献。对于他们而言，加入开源社区不仅能提高技术水平，也能认识更多志同道合的人，获取更丰富的职业发展机会。

开源学术研究类的角色包括以下几类。

- 研究者/学术贡献者（Researcher/Academic Contributor）。这些角色深入研究开源项目的技术架构、社区治理模式、可持续发展路径等，并在专业刊物或会议上发表论文。他们往往跨越学术与实践的界限，通过量化数据或深度访谈，为开源社区提供可参考的理论框架与改进建议。研究者也可能直接成为开源项目的贡献者，将研究成果转化为新特性或优化方案。
- 学术会议与研讨会（Academic Conference/Workshop）。此类会议专门聚焦于开源系统、开源治理或特定技术领域，如 OSS（Open Source Systems，开源系统）系列会议、FOSDEM（欧洲开源开发者大会）等。不同于常规商业会议，学术研讨会更倾向于研究成果交流和前沿思考的碰撞，参会者包括研究者、社区维护者、产业专家。通过论文分享、议题讨论和工作坊等形式，这些会议为学术界与产业界、社区之间的深度融合搭建了重要平台。

开源政策与标准类的角色包括以下两类。

- 政策制定者/监管机构（Policymaker/Regulator）。政府部门或国际组织往往承担法律法规与产业政策的制定职责。它们在数字经济与科技发展的背景下，审视开源软件使用、许可证合规、数据安全等方面的问题，并出台相应政策或标准，以保障公平竞争与创新活力。通过立法或监管，政策制定者为开源社区提供更明确的合规环境，也推动公私部门在开源方面的协同合作。政府或公共部门常在信息化建设与公共服务领域应用开源技术，也可能开放部分公共数据或资源以鼓励公民创新、使用开源软件或将公共项目以开源形式向社会发布等。
- 行业协会/标准化组织（Industry Association/Standards Organization）。这类组织如 IEEE、ISO、W3C 等，也包括专业化的垂直领域协会（如金融、医疗、汽车等）。它们会制定或推广行业技术标准，推动开源技术与行业标准的对接，进而提升互操作性和应用广度。对开源项目而言，被行业协会或标准化组织认可，意味着在合规与技术成熟度上获得更高的可信度，有助于在更广阔的行业中推广应用。

10.3 角色之间的交互与多重身份

开源生态中的各类角色之间是存在交叠的，而且会出现多重身份的情况。

- 个人与组织的交叠。部分看似属于个人的角色（如项目发起者），也可能由企业或基金会承担。事实上，一家企业可以主导项目创立，最初版本的代码也由其核心员工完成。个人在开源生命周期中会历经多种转变：有些人既是核心维护者，又兼任社区经理或布道者；也有人在初期贡献代码，后期则专注于社区运营或项目战略。
- 教育、学术与产业的融合。高校常与企业结成合作关系，为学生提供实习与项目实践的机会；与此同时，研究人员也会在社区中直接贡献代码或进行项目测试，以实现学术与产业的无缝衔接。各类培训机构和社区会联手开展工作坊、编程竞赛等活动，为新人提供更宽广的学习和实践平台，鼓励他们走向贡献者与核心维护者的道路。
- 政策对整个开源生态的影响。无论国家政策、地方政策，还是各类标准（国际、国家、地方与行业标准），对开源生态的影响都是全方位的。由于开源是非常新兴的事物，无论是政策制定还是标准的研发，都会广泛地征询各类开源专家的意见，这也是开源社区的人士影响政策与标准发展的机会。

第 11 问 开源基金会的角色定位是什么

开源基金会在现代软件开发和技术生态系统中扮演着至关重要的角色。它们不仅促进了开源软件的成长与发展，也为开源项目提供了资金、基础设施、法律保护等多方面的支持。如果将开源项目比作一棵树，基金会的作用是提供土壤（资源）、阳光（社区）和法律屏障（许可证），而支持手段则是浇水（资金）、修剪（治理）和防虫（合规）。

11.1 开源基金会的角色定位

开源基金会是非营利组织，目标通常是支持开源项目与开源生态的发展。全球顶级、最活跃的开源项目大多选择开源基金会进行托管。

各大基金会 2023 年年报显示：Linux 基金会有 1000 多个开源项目，1709 名成员，6.5 万多名积极贡献者，17 亿行代码；Apache 软件基金会拥有超 320 个活跃项目，294 个顶级项目，746 个人会员，超 8400 名贡献者，2.99 亿行代码；云原生计算基金会（Cloud Native Computing Foundation, CNCF）拥有 173 个项目，827 个成员组织，超 22 万名贡献者的支持；开放原子开源基金会拥有 49 个项目，46 个成员组织。（引用自赵海玲和郭皓的“基金会在开源生态培育中发挥的作用”）

下面具体介绍一下开源基金会的常见角色。

一、基础设施与资源的整合者

首先，技术基建网络的构建为不同项目提供了共享的基础设施资源。例如，Linux 基金会的 LF Networking 为 5G 网络的测试环境提供了统一的标准和平台，而 Apache 软件基金会的 ASF Infra 则为项目提供了持续集成和内容分发网络（Content Delivery Network，CDN）服务，这些都极大地提高了项目的开发效率和技术可复用性。

其次，资源池化运营通过集中管理代码仓库、文档平台和安全扫描工具，有效地降低了项目的维护成本。开放原子开源基金会甚至还提供了自建的代码托管平台，而安全扫描工具（如 OpenSSF Scorecard）则为项目的安全性提供了保障。这种资源的集中管理不仅优化了资源配置，还提升了项目的整体质量。

最后，跨域协同平台的搭建为不同技术标准的互操作提供了框架。以云原生计算基金会为例，它通过构建以 Kubernetes 为核心的云原生技术矩阵，推动了云计算、容器化等技术在不同领域的应用和融合。这种跨域的协同平台不仅促进了技术标准的统一，还加速了技术创新的步伐。

二、开源法律体系的构建者

首先，许可证治理涉及维护和管理各种开源许可证的族谱，如 Apache-2.0 许可证、MIT 许可证、GNU 通用公共许可证等，并处理它们之间的兼容性争议。例如，协调 GNU 通用公共许可证与 Apache 许可证的专利条款冲突，通过对许可证的细致管理和兼容性处理，可以有效避免法律纠纷，确保项目的合法性和可持续性。

其次，知识产权托管通过贡献者许可协议（CLA）统一管理代码版权。这种托管方式不仅明确了代码的版权归属，还简化了版权管理的流程。这种集中化的知识产权管理方式，显著降低了法律风险，提升了项目的稳定性。

最后，合规性工具链的开发和应用是建立开源合规管理体系的重要手段。开源基金会一直致力于推动更多的开源项目与企业采纳诸如 SPDX、FOSSology、SigStore 这样的工具，通过对开源软件的合规性进行全面审查，从而降低潜在的法律风险，提升开源治理的透明度和效率。

三、社区生态的孵化者

首先，通过建立分层孵化机制，如 Apache 孵化器的相关流程，确保项目在早期就具备多元化贡献者群体，这不仅增强了项目的可持续性，也为其长期发展奠定了坚实基础。

其次，设计阶梯式贡献路径，从普通贡献者（Contributor）到代码提交者（Committer），再到模块维护者（Maintainer），再到项目管理委员会（Project Management Committee，PMC）成员，为开发者提供了清晰的成长轨迹。以云原生计算基金会为例，其贡献者的数量从 2017 年的 18687 人到 2023 年覆盖 190 个国家的 220000 多人，充分证明了这一体系的有效性。^①

^① CNCF，2023 年年度报告。

最后，构建厂商中立的市场对接平台，如 LF Edge 基金会，成功促成了工业物联网领域 50 多家企业的技术整合，推动了商业生态的繁荣发展。

这三者相辅相成，共同构建一个健康、可持续的开源生态系统。

四、中立治理的守护者

首先，采用分层决策模型，如 Eclipse 基金会的项目管理委员会选举制度、Apache 软件基金会的“成员、项目管理委员会、董事会”多级体系，有效分配决策权，确保社区成员在项目发展方向上拥有平等的参与机会。

其次，设置企业会员分级制度（白金/黄金/白银），并限定单一企业的投票权不超过 20%，这样既鼓励企业积极参与，又防止单一利益方主导决策，维护了社区的利益平衡。

最后，建立技术监督委员会（Technical Oversight Committee, TOC）和争议仲裁流程，并且倡导以公开的方式协作，为矛盾的解决提供了清晰的路径。

这些机制相互配合，共同构建了一个公平、透明且高效的治理体系，为开源社区的长期繁荣提供了坚实保障。

五、开源文化的传播者

首先是开源教育，开源基金会通过开设课程、组织培训、提供学习资源等方式，为开发者、企业和高校师生提供系统化的开源教育。例如，Linux 基金会的“开源教育与培训计划”也提供一系列线上课程，让全球开发者都能学习开源技术；天工开物开源基金会也联合国内高校推出“开源毕业设计”等计划，帮助学生理解开源软件的开发模式、许可证协议及社区协作方式。

其次是组织线上/线下社区活动，以促进技术交流与生态建设。如 OSCon、KubeCon、ApacheCon 等大会，每年都会吸引成千上万的开源爱好者、企业、政府和高校代表参与，分享开源项目经验、探讨行业趋势，并促进跨界合作，是全球开发者交流技术、推广开源理念的重要平台。

开源基金会的布道师（通常由来自企业和社区的志愿者担任）通过演讲、图书出版、社交媒体推广等方式，帮助更多人理解和认同开源文化。

通过开源教育、社区活动和全球布道，开源基金会不断推动开源文化的普及，使更多人能够理解、参与并贡献开源，让开源真正成为全球技术创新的重要基石。

11.2 开源基金会的主要作用

开源基金会在推动开源项目的发展中发挥了重要作用。它们不仅为开源项目提供了资金、基础设施和法律保护，还通过促进社区建设和治理机制的完善，推动了开源生态系统的繁荣。随着开源项目的增多和影响力的提升，其作用愈加重要。有了基金会的支持，开源项目就能够更好地应对开发、管理、法律和市场等多方面的挑战，推动技术的创新和社会的进步。

第12问 如何理解开源项目的治理模式

开源项目的治理模式本质上是一套动态演化的协作规则，其核心并非预先设计的“完美制度”，而是随着社区成长、技术迭代、用户需求和外部环境变化不断自我调整的生存与发展策略。不同的项目、不同的社区，在发展过程中，或者是自然演化，或者是艰难探索，才会发展出各种不同的治理模式。下面从3个关键视角拆解开源项目治理模式的本质。

12.1 开源项目治理模式的主要特征

开源项目的治理规则往往先有实践，后有规范。

- Linux 内核早期依赖林纳斯·托瓦兹的个人决策，随着贡献者激增，逐渐形成“维护者分层机制”，即将复杂系统拆分为多个子系统，然后设置“子系统维护者”（Subsystem Maintainers），用代码审查和邮件列表讨论替代独裁决策。
- Python 语言从“终身仁慈独裁者”（Benevolent Dictator For Life, BDFL）模式转向指导委员会制，直接源于社区对吉多·范罗苏姆（Guido van Rossum）退休后权力真空的担忧。

这种演化类似于生物进化——治理规则是社区在解决实际冲突中沉淀的“适应性产物”。GitHub 的 CODEOWNERS 文件机制、Apache 软件基金会的投票流程，本质上都是为解决“代码合并权纠纷”“路线分歧”等具体问题而生的工具。

所有治理模式都在尝试平衡以下三大矛盾。

- 开放与控制的博弈：完全开放可能导致混乱（如早期 npm 包因无审核机制出现恶意代码事件），过度控制则可能扼杀创新，甚至导致社区崩解。
- 短期效率与长期可持续的博弈：初创期项目常采用“终身仁慈独裁式”快速迭代，如 Redis 早期由萨尔瓦多·桑菲利波普（Salvatore Sanfilippo）单人维护，成熟期则需建立贡献者阶梯（如 Kubernetes 的特别兴趣小组制度）防止维护者倦怠。
- 商业利益与社区公益的冲突：MongoDB 将协议从 AGPL 改为服务器端公共许可证（Server Side Public License, SSPL）（禁止云厂商免费商用），本质是商业公司出于自身商业利益，对社区贡献的背叛。Elasticsearch 修改许可证同样引发争议，而 PostgreSQL 始终坚持“BSD+社区”开放治理，成为各大厂商的“基础设施公地”。

治理模式遵循以下3个演化规律。

- 工具塑造治理：GitHub 的 PR 模板、CI/CD 自动化测试等工具，事实上定义了贡献流程。例如，Visual Studio Code 要求每个 PR 必须关联问题编号，强制形成“问题讨论—解决方案”的决策链条。Linux 内核使用 Git 的签名（signed-off-by）机制实现责任溯源。当然，还应该看到另一个方

面：治理的创新也往往会驱动新工具（新功能）的诞生。

- 危机驱动变革：Node.js 曾因 Joyent 公司掌控核心代码导致社区分裂（io.js 分叉事件），最终促成 Node.js 基金会成立。而 OpenSSL 在 2014 年“心脏出血”漏洞后，从松散维护转向“核心团队+财政赞助人”制度。
- 权力取决于代码贡献：在 Apache 项目的项目管理委员会中，持续贡献代码的开发者自然获得投票权。相反的例子是：Docker 曾试图绕过社区直接推进商业化方案（Docker EE），导致核心贡献者流失到 Kubernetes 生态。

另外，还应该区分治理模式与治理手段。当我们讨论治理模式时，是在对社区的治理做宏观的、概要性的分类。但是有很多治理手段，在不同的治理模式中，都可能会应用。例如，问题分类与标签规范、维护者分层机制、特别兴趣小组（Special Interest Group, SIG）、贡献者许可协议（CLA）等。

12.2 主流治理模式的分类与特点

开源社区的治理模式，有很多种分类方法，还有类似的模式却起了不同的名字。按照通常我们所知的模式，简单分类如表 12-1 所示。

表 12-1 主流治理模式的分类与特点

主流治理模式	特点	优点	缺点	案例
社区治理	以开放协作和去中心化为核心，决策权分散于社区成员。通过公开讨论、投票或共识机制推动决策	激发创新，增强社区归属感	决策效率可能较低，易因意见分歧导致停滞	Linux 内核项目，由林纳斯·托瓦兹主导但依赖社区贡献，重大决策需经邮件列表讨论和代码审查
集中式治理/终身仁慈独裁者	由单一主体（如企业或核心开发者）主导决策，明确领导权与责任链	方向明确，迭代高效	社区参与受限，过度依赖核心团队	React 框架由 Meta（原 Facebook）团队把控核心设计，社区贡献需符合企业技术路线
基金会治理	由非营利基金会（如 Apache 软件基金会、Linux 基金会）提供中立治理框架，平衡商业与社区利益	降低商业风险，提升公信力	流程可能比较烦琐，小型项目资源不足	Kubernetes 由云原生计算基金会管理，确保技术中立和长期维护
混合式治理	结合集中式与社区式，核心团队把控战略方向，社区参与功能开发与问题修复	兼顾效率与开放性	需明确权责边界以防冲突	Python 语言由 Python 软件基金会监督，但技术决策由核心开发者（终身仁慈独裁者，现改为指导委员会）主导

续表

主流治理模式	特点	优点	缺点	案例
自组织式治理	无固定架构，贡献者基于兴趣自发协作，通过工具（如 GitHub 的问题）形成临时决策	灵活轻量	可持续性差，易因维护者退出而停滞	小型开源工具（如某些 npm 包），依赖开发者自主提交 PR 和问题反馈
敏捷式治理	结合敏捷开发原则，以快速迭代和用户反馈驱动决策，常见于工具类项目	适应市场变化	需高强度维护，对核心团队要求高	Visual Studio Code 通过月度发布周期，快速响应社区需求

12.3 选择开源项目治理模式要考量的关键因素

如何选择开源项目治理模式，要从技术维度、社区生态维度、经济与法律维度、演化周期维度和社会文化维度全面考量，下面具体介绍每个维度要考量的关键因素。

一、技术维度

技术的难度、复杂度、变化的速度，都会影响社区对治理模式的选择。

- 高度耦合的复杂系统（如操作系统、数据库）通常会有更多开发者参与，需要采用集中式治理模式，并结合维护者分层机制。
- 低耦合的工具链（如前端框架、CLI 工具等）可以采用更开放的自组织式治理模式。
- 快速演进领域（如 AI 框架）需敏捷式治理，PyTorch 通过“月度核心团队会议+社区论坛”异步决策，保持与学术研究的同步。
- 基础设施类项目强调稳定性，通常会捐献给开源基金会，并采用基金会治理模式。在功能特性创新方面，采用特别兴趣小组的方法进行管理，如 Kubernetes 项目，每个功能域需通过 Kubernetes 特别增强提案（Kubernetes Enhancement Proposal）审批。

二、社区生态维度

社区里有哪些人和组织，正在源源不断地加入的是谁人和组织，不同的人和组织各自的利益诉求如何，也会影响治理模式的选择。

- 头部贡献者占比超 80% 的项目适合“终身仁慈独裁者”治理模式（如 Redis 早期阶段）。
- 企业开发者占比超 50% 的项目常采用基金会治理模式。例如，云原生计算基金会通过会员协议、章程及治理政策，明确要求会员遵守开源中立原则和知识产权转移规则，以防止技术垄断和社区分裂（如 React、Kubernetes）。
- 通用型项目需平衡多方利益，选择社区治理模式，需要设立指导委员会设置“用户代表席位”吸纳企业诉求（如 Python 语言）。

- 垂直领域工具可让核心团队主导，选择集中式治理模式，他们通过 Discord 频道收集需求但保留最终决策权（如区块链开发框架 Hardhat）。
- 长尾贡献者分散的项目，往往选择敏捷式治理模式，并配合自动化治理工具（如 Visual Studio Code 插件生态，广泛运用 GitHub CODEOWNERS 技术）。

三、经济与法律维度

从单一企业的利益出发是一种考虑，从整个生态健康的角度出发又是另一种考虑。在具体的技术领域，商业利益、竞争态势、法律与知识产权等问题的不同，治理模式的选择也会不同。

- 越是注重短期商业利益的公司，越容易选择集中式治理模式以建立商业产品技术壁垒。
- 越是在多头并存、单一公司难以获得垄断地位的市场中，开源项目越有可能选择基金会治理模式。
- 专利敏感技术类项目需要贡献者签署贡献者许可协议（CLA），例如，FFmpeg 就要求签署《开发者知识产权声明》以降低专利诉讼风险。

四、演化周期维度

开源项目在不断发展，早期阶段往往需要快速集中决策，后期则应该更加注重开放包容，这样的变化在很多社区不断上演。

- 初创期（0~1 年）：采用“终身仁慈独裁者”模式快速验证技术可行性，如早期 Webpack 由 Tobias Koppers 单人维护。
- 成长期（1~5 年）：引入 RFC 流程规范提案，如 Rust 语言 2018 版重大更新前启动社区大讨论。
- 成熟期（>5 年）：建立委员会应对企业级需求，如 Apache Kafka 的项目管理委员会机制。

五、社区文化维度

- 工程师文化主导：Go 语言坚持“最少抽象原则”，技术决策需核心团队全票通过。
- 学术研究背景：PyTorch 深度学习框架治理融合论文评审机制，重大特性需附学术界引用论证。
- 地域文化差异：中日韩开发者占比超 40% 的项目（如 TiDB）设立多语言治理文档和异步决策流程。

治理模式的选择本质是在技术可行性、社区活力、商业利益之间寻找动态平衡点。最终，有效的治理模式应具备 3 个可观测指标：PR 处理时间（反映决策效率）、漏洞类问题的处理时间（验证治理执行力）、贡献者数量（衡量社区健康度），更加全面地对社区与项目的健康度进行评测。（开源项目的健康指标详见第 14 问。）

第 13 问 开源项目的生命周期是怎样的

开源项目的生命周期通常包括多个阶段，从诞生到成熟、维护，甚至可能衰退

或终止。有很多文章都对此有所讨论，本问将开源项目的生命周期分为孵化期、成长期、成熟期、衰退/转型期和归档期 5 个阶段。

13.1 开源项目生命周期

五阶段划分，只是一种权宜之计，很多开源项目并不完全遵循这样的五阶段顺序，有快速通过几个阶段的，也有在好几个阶段之间反复跳转的，以下分期仅供参考。

一、孵化期

这个阶段往往由个人或单一团队主导，开发者在封闭环境中快速验证技术可行性，就像 Linux 内核最初只是林纳斯·托瓦兹的个人实验。这个时期的代码架构变动频繁，决策权高度集中，甚至可能长期处于企业内部开发状态，直到第一个版本发布为止。

孵化期主要聚焦于技术可行性的验证，内部迭代速度快，架构频繁调整，谈不上什么治理模式，主要靠中心化决策。好的开源项目在这个阶段并非只考虑代码方面的问题，而是需要为正式开源，以及今后的成长做好准备。这个阶段面临的主要挑战是明确项目定位与差异化价值，哪些人/组织会成为这个项目的使用者，哪些人会成为这个项目的贡献者，这些都是需要更多思考的问题。

在孵化期，还有一个重要的决策需要完成，即选择许可证（如 MIT 许可证、GNU 通用公共许可证、Apache 许可证等），明确授权方式。许可证的选择可以参考第 7 问。

一个值得借鉴的案例是 TiDB 项目。在 2015 年 4 月正式开源之前，TiDB 团队花了 3 个月的时间，拜访了数十家企业，发现许多公司面临 MySQL 分库分表的复杂性，从而确定了“面向中大规模 OLTP 场景的分布式数据库”这一定位，这种清晰的自我认知成为其日后爆发的关键。

二、成长期

一旦前期准备充分，市场定位准确，开源项目就很可能迎来充满活力的成长期，此时社区蓬勃生长：GitHub 上的星标数每周翻倍，贡献者来自各大洲的不同地方，Slack 频道里讨论声此起彼伏。例如，Kubernetes 自 2014 年开源后，在 18 个月内（至 2016 年初）吸引了来自谷歌（Google）、红帽等公司的近千名代码贡献者，若包含全生态贡献者则超过 1200 人。

但是，高速成长带来的阵痛也在所难免。在 2021 年的中国开源年会（COSCon）上，Apache SkyWalking 的创始人吴晟就回忆道：“当 SkyWalking 的 GitHub 的问题每日新增超过 50 个时，手动分类和分配变得不可持续，于是我们开发了自动化机器人来标记优先级、分配处理人，并过滤无效反馈。”

在这个时期，社区管理团队应该及时明确治理模式，制定版本发布周期（如 Semantic Versioning），建立贡献者培养体系（如标注“good first issue^①”、新手引导

① good first issue 是一个常用的标签，用于标记一个待解决的问题，表示该问题适合第一次尝试开源贡献的新人。这个标签旨在帮助新手开发者找到入门级问题，轻松迈出开源贡献的第一步。

计划)等,及时将汹涌而至的人潮转化为社区与项目发展的动力,并且及时解决因快速开发导致代码质量下降的问题,以及因人员纷乱导致的社区决策混乱等问题。

聪明的项目维护者会在这个阶段铺设基础设施的轨道,如借助 GitHub Actions 实现自动化测试、引入 Docusaurus 构建知识库、通过 Open Collective 建立资金池等。这些工具就像脚手架,支撑着社区建筑的持续升高。

三、成熟期

开源项目进入成熟期就像人进入了壮年期,此时的开源社区、项目、产品都不会有太大的波动,人员稳定、特性稳定、新版本的发布周期也很稳定。如果是捐赠给开源基金会的项目,此时的治理模式往往形成“三权分立”结构——技术监督委员会把控架构方向,社区经理维护开发者关系,基金会管理法律财务。

但成熟并不意味着安全。2014 年,OpenSSL 的“心脏出血”漏洞事件暴露出这个支撑全球 60%网站加密通信的项目,竟然长期只有两位兼职维护者。这一警钟催生了关键信息基础设施计划(CII),推动成熟项目建立安全响应团队。这又回到了“开源项目健康度指标”的问题。

在看似稳定的状态下,发现潜在的风险,及时规避;发现潜在的发展机遇,及时抓住。这些都在考验社区运营者的治理能力。

四、衰退/转型期

社区的衰退可能缓慢发生,社区的分裂或许会骤然来临。在 2014 年底,Node.js 社区因对 Joyent 公司的治理模式不满(如代码合并速度缓慢、决策不透明等),由核心贡献者分出 io.js 分支。结果导致 GitHub 上的 Node.js 仓库在 2014 年末至 2015 年初提交数量显著减少,但这场危机反而成为转机:新成立的 Node.js 基金会引入多方治理,吸引了更多企业支持(如 IBM、微软),生态持续繁荣,最终通过合并分支实现了涅槃重生。

转型期的智慧往往体现在“断舍离”的勇气上,一个著名的案例是从 Python 2 到 Python 3 的迁移,Python 2 最初的版本于 2000 年发布,而 Python 3 发布于 2008 年,但因为与 Python 2 存在语法和标准库的不兼容,导致迁移缓慢。于是,Python 软件基金会(Python Software Foundation, PSF)在 2019 年宣布,官方对 Python 2 的支持于 2020 年 1 月 1 日终止,不再提供安全更新或补丁。这在社区激起了轩然大波,Python 软件基金会通过工具支持(开发并优化 Python 2 到 Python 3 自动迁移工具,发布现代版本兼容指南,如“Porting Python 2 Code to Python 3”)、资金支持(资助关键开源库的 Python 3 适配,如科学计算库 NumPy 和 Web 框架 Django)、社区宣传(联合微软、谷歌等企业发布迁移白皮书,并设立 Python 3 Wall of Superpowers 追踪生态兼容性),逐步实现了从 Python 2 到 Python 3 的平稳过渡。可以说,全球 Python 生态的迁移是社区转型与开源协作的典范。

五、归档期

花无百日红,任何开源项目都会有消亡的一天。当一个项目的维护者全面退出且无新贡献者接替,当一个项目所采用的技术彻底过时(如 Flash 相关项目),好地将项目归档(终止)是更正确的选择。

在归档阶段,项目负责人还需要做好以下收尾工作:发布项目终止公告,明确

归档时间线，推荐替代方案（如 Apache Mesos 归档后，引导用户转向 Kubernetes），保留只读仓库和文档供历史参考等。总之，归档不是撒手不管，而是“封存”。主动归档，避免资源浪费，是负责任的行为，而非“失败”的标志。这样才能避免“僵尸项目”误导新用户，维护开源生态健康，为后续项目提供经验教训（如通过变更日志记录关键决策）。

13.2 阶段化策略

与分期的逻辑类似，阶段化策略同样是某种概念指引，而不必“刻舟求剑”。

一、早期阶段：聚焦核心功能，蓄势待发

早期阶段，一般是指项目对外开源之后的初创阶段，从“无人问津”到“小有所成”。很多开源项目以版本号来表明这样的阶段：刚刚发布的时候是 0.x 版，等到逐步发展成熟，功能基本实现最初的设计，发布正式的 1.0 版。这时，可以认为项目度过了早期阶段。当然也有一些项目，在发展很多年以后，也一直是 0.8、0.9.x 这样的版本号，不可一概而论。

- 在技术策略方面，应该放弃“大而全”思维，通过 MVP（最小可行产品）验证技术可行性。例如，Redis 在 2009 年发布的最初版本，仅支持 5 种数据类型，但凭借其极致性能快速赢得开发者的口碑，成为开发者首选工具之一。
- 在社区运营策略方面，建议定向邀请领域专家参与设计，邀请知名开发者或企业成为标杆用户，从而打造示范效应。另外，友好易读的文档、一步一步的上手指南，都是吸引早期采用者的不二法门。
- 在治理模式方面，采用“终身仁慈独裁者”模式加速决策始终是一个不坏的选择。在这方面不必“想得太多”，等到社区的人真正多起来，实际产生了治理演进需要，再改进也来得及。

二、中期阶段：降低协作成本，守住质量底线

中期阶段，通常是项目与社区都较为成熟。项目版本号在 1.0 以上，社区的协作者超过两位数的时候，差不多可以算进入中期阶段。这个阶段如果发展顺利，通常会经历一段时间较快的用户增长、开发者增长与社区口碑的增长。所以，效率与规范化会成为社区治理主要的追求。

- 在开发协作方面，可以考虑逐步引入自动化工具（GitHub Actions、DCO^①/CLA 签署机器人）、AI 辅助开发工具（问题分类、任务分派、AI 代码评审等），这些工具能够提升社区的协作效率。

在项目质量方面，应该不断提升测试覆盖率，通过 CI/CD 全链路管控，守住

① DCO（Developer Certificate of Origin，开发者原创证书）是一种轻量级的开源协议，用于确保贡献者对他们贡献的代码拥有所有权，并授权开源项目根据开源许可使用这些代码。DCO 相对于传统的 CLA 更简单，只需要贡献者在提交代码时签署一份声明，而不是签署一份完整的法律协议。

开源软件的质量底线。

- 在社区治理方面，可以考虑逐步建立“新手—常驻—核心”的贡献者成长阶梯。很多开源社区都经历了从完全扁平的组织，到逐步增加层级结构的过程。这样，一方面能实现多人协调共同完成复杂事务，另一方面能有效激励参与贡献的社区成员，使他们有更大的成就感。

三、长期阶段：可持续发展，长治久安

很少有开源项目能够成长到需要考虑这些问题，加之开源本身的发展历程也不算太长，因此我们可以学习和借鉴的对象实在是不多。那些最长寿且最优秀的开源社区通常被认为很好地处理了以下问题。

- 在治理机制方面，能够有效平衡决策效率与决策公平的矛盾，避免权力过度集中，甚至能够有效地实现管理团队的更新换代。
- 在技术创新方面，能够有效平衡长期维护的稳定性与架构重构和创新性的矛盾。例如，Ubuntu 就采用“每两年发布 LTS 版本（支持 5 年）+每半年滚动更新”策略，既满足企业稳定性需求，又为开发者提供试验场。
- 在开源商业化方面，能够有效平衡企业利益与社区利益的矛盾，避免理想主义者与功利主义者之间的尖锐对立。通过基金会这样的非营利托管机构，建立“中立的管理委员会”，让竞争企业共同投资关键底层技术（如 OpenSSL、Node.js），形成“竞合生态”，这种做法是值得学习的优秀实践。

第 14 问 什么是开源项目的健康指标

开源项目的健康指标是用于全面衡量和评估开源项目可持续性、社区活跃度、代码质量及生态影响力的量化与质性标准。这些指标在开源生态系统中发挥着举足轻重的作用，能够为项目的维护者、贡献者以及各类相关利益者提供深入洞察项目实际情况的有力工具，对于把握项目的当前状况、预测其发展趋势以及提前识别未来潜在风险，都具有重大的价值。

14.1 开源项目健康度评估的主流框架

在开源领域，主要有以下一些评估框架在探索如何评估开源项目、开源社区的健康度：最早也是目前发展最活跃的是由 Linux 基金会于 2017 年 8 月发布的“Community Health Analytics in Open Source Software”（开源软件中的社区健康分析，以下简称 CHAOSS），还有 2021 年 3 月红帽公司发布的“衡量开源项目健康状况的清单”。2023 年 2 月，我国首个开源生态健康评估平台“OSS Compass^①”正式诞生。

① OSS Compass（开源指南针）由国家工业信息安全发展研究中心、开源中国、南京大学、华为、北京大学、新一代人工智能开源开放平台（OpenI）、百度、腾讯开源联合发起并协作开发。

一、CHAOSS 指标体系

CHAOSS 是 Linux 基金会旗下的项目，专注于通过指标分析来定义社区的健康状况。CHAOSS 项目有以下 4 个目标。

- 建立衡量开源社区健康状况的指标和模型。
- 开发用于衡量社区健康状况的开源软件和举措。
- 制订计划，部署无法通过跟踪数据实现的指标。
- 与全球合作伙伴合作，塑造理解开源社区健康的方式。

(1) CHAOSS 的早期阶段。

在早期阶段，CHAOSS 提出了 5 个指标领域，并设立了 5 个工作组。

- 通用指标工作组（后改名为指标开发工作组，Metrics Development Working Group），工作重点是确定多个工作组使用的指标或对社区健康非常重要的指标，专注的领域如下。

贡献——确定作出了哪些贡献（如代码提交、维基编辑、文档）。

人员——确定作出贡献的组织和个人（如贡献类型、人口统计等）。

地点——确定贡献发生的物理和虚拟地点（如 GitHub、聊天频道、论坛、会议等）。

时间——确定作出贡献的时间（如回复时间）。

- 多样性与包容性工作组（后改名为多样性、公平和包容工作组，Diversity, Equity and Inclusion Working Group），旨在定义衡量标准和方法，以帮助其他人在自己的开源项目中衡量和关注多样性、公平性和包容性，专注的领域如下。

沟通包容性——确定我们如何与贡献者和潜在贡献者沟通。

贡献者社区多样性——确定社区内贡献的多样性，以及如何重视这些不同的贡献。

活动多样性——确定活动的多样性和包容性。

管理——确定我们的管理是否具有多样性和包容性。

领导层——确定我们社区领导层的健康程度。

项目和社区——确定我们的项目场所（社区参与的场所）的多样性和包容性。

表彰优秀作品——确定我们如何表彰/奖励社区中的优秀作品。

- 演进工作组，主要制定评估开源项目生命周期的指标，专注的领域如下。

代码开发活动——了解代码开发活动的类型和频率。

代码开发效率——了解代码开发活动的开发效率。

代码开发流程质量——了解用于提高/审查质量的流程（如测试、代码审查、标记问题、标记发布、响应时间、CII 徽章）。

问题解决——确定社区在解决社区参与者发现的问题方面的效率。

社区增长——确定项目社区的规模，以及它是在增长、缩小还是保持不变。

- 价值工作组，主要制定衡量标准，描述个人和组织参与开源活动的价值，

但是现在已经不再活跃了。

- 风险工作组，重点关注与开源中的风险问题相关的衡量标准，专注的领域如下。

业务风险——了解围绕/支持特定软件包的社区的活跃程度。

代码质量——了解特定软件包的质量。

许可——了解与使用特定软件包相关的潜在知识产权问题。

安全——了解与软件开发相关的安全流程和程序。

透明度——了解特定软件包在依赖性、许可证、安全流程等方面的透明度。

依赖性风险评估——了解软件依赖性风险。

二、CHAOSS 的最新发展是“指标平铺，另外建立指标评估模型”

通过阅读上述的分类可以发现，CHAOSS 指标体系存在交叠的现象，如演进工作组与风险工作组关心的内容就存在很多交叠，甚至在风险工作组的内部，与透明度相关的指标也会关注许可的风险。因此，在 CHAOSS 的官网上，目前（截至 2025 年 2 月）的做法是将所有（89 个）指标全部平铺展示，不再分类。另外，越来越多的社区实践者发现，原先对于指标定义的颗粒度过小，不能具体地描述某个社区的实际场景。因此，在 2021 年，华为倡议成立了指标评估模型工作组，首次提出了评估模型的概念。经过近两年的发展，到 2023 年，CHAOSS 社区把重心转移到了评估模型工作组方向。（来自 2023 年 4 月笔者对王晔晔的访谈。）截至 2025 年 2 月，CHAOSS 一共发布了 17 个指标模型（Metrics Models）。

试举一例，介绍指标模型与指标的关系。开发响应度（Development Responsiveness）是一个指标模型，这个指标模型的计算结果反映了与项目有效运营和扩展能力相关的一系列因素。响应度可以反映项目的投资（包括放弃）、能力，并且可以指示工程优先级，有时还会偏向于某些类型的交互和人员——响应度可以揭示很多东西，包括你没有注意到的东西，如机器人在解决常见痛点方面的效果（或效果不佳）。

在开发响应度指标模型中，包含如下 4 个指标。

（1）变更请求审核周期。审核周期的时间长度衡量的是变更请求中一个完整审核周期所需的时间。一个审核周期从提交或更新变更请求时开始，到接受、拒绝或进一步请求变更时结束。这一指标可帮助维护人员评估代码审核流程的效率，找出潜在的瓶颈和出现延迟的地方。通过了解这些周期的持续时间，可以深入了解审核实践的有效性和项目中贡献的整体工作流程。审核周期中的放弃或半放弃流程，即表示维护者或提交者响应缓慢。

（2）变更请求持续时间。变更请求持续时间是从变更请求开始到结束（被接受并合并到代码库中）的持续时间。这仅适用于已接受的变更请求。

（3）问题响应时间。此指标衡量问题从作者以外的贡献者收到首次回复所需的时间。问题响应时间是衡量项目响应能力及其吸引贡献者的能力的关键指标。响应时间短，可能表明社区健康活跃；响应时间较长，则可能凸显管理社区贡献方面的潜在问题。

（4）缺陷解决持续时间。缺陷解决持续时间是指从报告软件缺陷到解决缺陷之

间的中位时间，解决问题可能通过代码修改（合并修复）或拒绝报告/问题来实现。不同贡献者的问题或报告的解决时间，可突出显示贡献者在处理报告/问题时是否面临延误。每个数据点都与缺陷报告/问题及其相应的解决行动相关联。如果缺陷解决速度较快，则表明项目的维护团队积极主动、反应迅速，有助于提高软件的整体质量和用户满意度。

三、对 CHAOSS 指标体系的评价

CHAOSS 指标体系是一个非常活跃且依然在快速变化的指标体系。作为一个全球协作的开源项目，CHAOSS 提出的很多关于开源项目健康指标相关的思想都非常值得借鉴。但是由于社区成员的兴趣比较分散，各种各样的指标层出不穷，却难以落地。在开源社区中工作的很多朋友都逐渐产生了疑惑，也不太在实际工作中使用他们的指标体系了。

四、红帽公司发布的衡量开源项目健康状况的清单

2020 年，红帽公司发布了“衡量开源项目健康状况的清单”。相对于 CHAOSS，这个清单提出了一个简明扼要的框架（检查清单），将衡量一个开源项目的健康状况分为若干检查项。至于如何进行具体的检查，看起来是一个完全手工的过程。另外，这个检查清单也不提供综合评价的计算方案，或者不同的项目之间如何比较健康度的机制。

五、开源指南针

2023 年推出的开源生态健康评估平台——开源指南针，是一个兼具理论深度与实用价值的平台，由国内的多家高校、企业、开源社区联合打造。开源指南针展示的开源生态评估体系包括开源生态、“协作、人、软件”和评估模型，如图 14-1 所示。

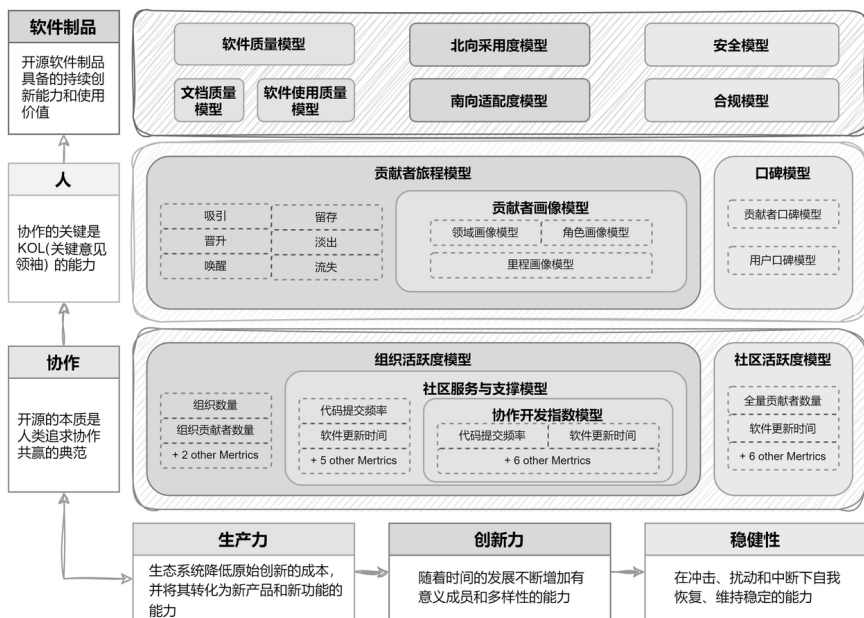


图 14-1 开源指南针的开源生态评估体系

基于南京大学的陶先平老师、汪亮老师的研究成果，开源指南针的指标体系（v3 版本）针对开源生态健康度构建了社区活力、协作效率、开放治理三大核心维度的模型，其核心思想是：一个健康、可持续发展的开源社区，不仅需要“有人参与、有持续贡献”，还需要具备“高效的协作机制”，以及“开放、多元的治理结构”。整个体系进一步细分为 3 个维度、7 个二级评估模型、28 项度量指标，从开发者参与、代码与社区活动、Issue/PR 协作流程，一直延伸到组织和个人参与社区治理的情况，形成了一套由“参与—协作—治理”逐层深入的社区健康度评价框架。

（1）社区活力：衡量社区是否“有人来、有人做、还能持续吸引人”。

社区活力（Community Vitality）评估社区的活跃程度和吸引力，反映社区当前的生命力以及未来持续发展的潜力。该维度不是简单以 Star 数量判断一个项目是否活跃，而是同时考察项目的外部关注度、实际贡献活动和开发者基础，由 3 个二级模型构成：社区流行度、贡献活跃度、开发者基数。

社区流行度关注项目在开发者群体中的传播力和吸引力，通过统计周期内的 Star 新增量和 Forks 新增量衡量项目受到关注以及引发进一步参与的程度。Star 更多反映关注与认可，而 Fork 则进一步体现开发者基于项目开展二次开发或者贡献的潜在意愿。

贡献活跃度关注社区是否存在持续、真实的贡献活动，包括代码提交次数、新增及修改代码行数、PR 评论数量、Issue 创建数量和 Issue 评论数量。这些指标分别从代码生产、代码审查、问题反馈和社区讨论等方面观察社区实际运行状态，而不是仅仅考察项目累积了多少历史资产。

开发者基数关注社区是否拥有足够广泛和多样的人才基础，包括社区贡献者数量、代码贡献者数量、非代码贡献者数量和社区关注者数量。其中尤其将非代码贡献者单独纳入指标体系，意味着文档维护、Issue 管理、PR 协作和社区互动等非代码贡献同样被视为社区健康的重要组成部分。

因此，社区活力实际上考察了 3 个递进层次：有多少人关注项目→有多少真实贡献正在发生→有多少人真正进入社区并形成持续参与。

它不仅衡量社区“现在是否热闹”，还试图衡量社区是否拥有持续产生贡献者和扩大参与基础的能力。

（2）协作效率：衡量社区能否把“参与”转化为高质量协作。

仅仅拥有大量贡献者并不意味着社区一定健康。如果 Issue 长期无人回复、Pull Request 长时间得不到处理，或者代码可以未经评审直接进入项目，大量参与反而可能带来协作成本。

因此，协作效率（Collaboration Efficiency）重点评估社区的协作质量和响应效率，反映社区的协作成熟度和服务水平，由响应及时性和协作开发质量两个二级模型构成，共包含 12 项指标。

响应及时性主要考察社区对外部反馈和贡献的响应能力，而协作开发质量进一步观察社区是否已经形成规范、成熟的协同开发机制。

这些指标实际上覆盖了一条相对完整的开源协作链条：问题提出→代码修改

→PR 提交→代码评审→沟通讨论→合并代码。例如，通过 PR 合入代码意味着代码修改进入公开协作流程；PR 与 Issue 关联能够保留代码变更的背景和上下文；非作者参与评审和合并，则有助于避免“自己提交、自己批准、自己合并”的单人封闭开发模式。

因此，协作效率关注的不是单纯的“开发速度”，而是社区能否以较低的沟通和等待成本，将分散的个人贡献转化为经过讨论、评审和协作形成的项目成果。

这一维度在“活跃”与“健康”之间建立了重要区分：贡献很多但协作机制混乱的社区，未必是一个真正健康的开源社区。

（3）开放治理：衡量社区的决策权是否真正开放和多元。

当一个开源项目逐渐发展壮大，仅仅考察代码贡献和协作效率仍然不够。项目的长期可持续性还取决于一个更深层次的问题：谁拥有决策权？社区治理权是否向不同组织和个人开放？

因此，开放治理（Open Governance）评估社区治理的开放性和包容性，反映社区的治理成熟度和多元化程度，并从组织开放治理和个人开放治理两个方面进行衡量。

这两个方面的指标进一步考察普通个人贡献者能否通过长期贡献进入社区管理和治理层。个人贡献者能够凭借实际贡献获得治理权限，意味着社区不仅允许外部开发者“贡献代码”，而且为其提供了从贡献者向维护者或其他治理角色成长的空间，从而体现社区治理的开放性与包容性。

值得注意的是，开源指南针并不是简单认为“组织参与越少越好”或者“个人占比越高越好”。其指标设计同时关注组织和个人参与治理：组织能够带来专业能力、工程资源和长期投入，而个人参与则有助于保持社区的独立性、多元性和开放性。真正健康的治理结构，需要在二者之间形成合理的平衡。

对开源指南针指标体系的评价如下。

开源指南针指标体系是一个面向实用的开源生态指标体系，从一开始就有一个思路清晰的顶层架构，然后层层追问，层层分解，落实到各个可观测的数值。另外，开源指南针还提供了一个方便部署、开源的 Compass 平台系统，既可以通过这个项目的官网直接使用，也可以本地部署、内部使用，具备较高的使用价值。

14.2 深入思考开源指标体系

在经过上述 3 个开源指标体系的介绍之后，我们需要按下暂停键，不必急着开始使用这些指标，而是思考一下这些指标是怎么来的，到底意味着什么。

面对开源生态，有两个目标是非常明确的：一个是从问题出发指向数据的，我们有很多问题，这些问题需要寻找答案，而我们希望通过这些数据得到答案；另一个是从实际已有的数据出发指向问题的，我们有很多原始数据，需要确认这些数据的意义，用来回答可能会被用户提出的问题。

而在问题与原始数据之间，我们试图通过指标来实现上述两个目标。但是，难

点在于：从问题出发，想要设计指标来寻求答案，很可能找不到合理的、正确的原始数据；从原始数据出发，通过各种“拍脑袋”的计算方法得到的指标，说不清能够回答什么问题。

还有一个难题，是因果关系的复杂性。从社区活跃到用户增长，从用户增长到产品成熟，从产品成熟到商业成功，我们隐隐约约感觉到其中是存在着因果关系的。但是如何通过“可观测的现象”建立“量化的指标”，再通过各个指标之间的相互影响证明因果关系真实存在，也是非常困难的。

另外，建设真正合理的开源指标体系，还需要更加规范化的案例研究。例如，一个项目有 10 万个赞，有 5 万个用户下载（假设能够统计到），每天有 100 个新增的问题，这样的项目到底算不算成功？算不算健康？到底成功或者健康应该和哪些项目进行比较？不同的编程语言、不同的技术领域和不同的商业场景，开源项目之间很可能是无法横向比较的（或者即使能够进行比较，也是不公平的）。我们可能首先需要基于实际的、成千上万的开源项目案例，达成定义、分类、判定优劣方面的共识，在此基础上建立的指标体系，才可能是更有说服力的。

至于现在已经出现的这些指标体系，或许还处于早期探索的阶段。在实际工作中，我们可以参考这些指标，但不必迷信具体的数值，同时也建议大家积极参与到指标体系的建设社区中去（它们本来也是欢迎所有人参与贡献的开源项目）。

第 15 问 开源标准化在开源生态系统中的价值是什么

15.1 开源标准化的基本概念与发展历程

一、标准化的概念及意义

标准化是指在经济、技术、科学和管理等社会实践中，为获得最佳协作效果和社会效益，对重复性事物和概念制定、发布并实施统一标准的过程。标准化的重要意义是改进产品、过程和服务的适用性，防止贸易壁垒，促进技术合作。它不仅涵盖软件接口、数据格式、通信协议等技术层面，还涉及开发流程、文档编写、测试方法等管理层面的规范。标准化旨在减少信息孤岛、提高效率，并促进技术的广泛应用。

二、开源标准化的概念及特征

开源标准化是指在开源社区和标准化组织之间建立合作机制，通过开放协作的方式制定和维护技术标准。这种标准化过程不仅确保了技术的互操作性和兼容性，还促进了技术创新和产业生态的健康发展。

开源标准化的核心在于开放性、透明性和社区参与，通过共同制定标准，推动技术的广泛应用和持续改进。具体来说，开源标准化包括以下几个方面的特征。

（1）开放性：开源标准的制定过程、内容及实施资源（如代码、文档）均向公众开放，允许任何人自由使用、修改和分发。这有助于吸引更多参与者，促进技术

创新和合作。

(2) 社区驱动：开源标准的制定通常由开源社区主导，依赖多方协作而非单一厂商控制。例如，中国电子技术标准化研究院推动的开源治理系列标准即强调社区共建模式。

(3) 持续改进：与传统标准相比，开源标准更注重快速迭代、持续改进，通过短反馈周期和透明流程适应技术变化。例如，DIN SPEC 3105 标准在开源硬件领域的应用即体现了这一特点。

(4) 反垄断属性：通过打破技术壁垒和知识封闭，开源标准化促进公平竞争和技术民主化。

三、开源标准化的发展历程

虽然开源标准化非常重要，但是在开源领域，开源社区对标准化的态度往往并不积极，直到后来因为业界竞争态势的逐步演化，开源社区的观念才逐渐发生了转变。

(1) 早期阶段：被动兼容，各自为政。

2000 年以前，也就是开源概念出现前后，大致可以算作早期阶段。开源社区主要遵循业界已有的技术标准。例如，Linux 早期为了兼容 UNIX，就积极遵循了可移植操作系统接口（Portable Operating System Interface, POSIX）标准，通过可移植与可连接格式（Executable and Linkable Format, ELF）实现与 UNIX 相同的可执行文件格式，支持 TCP/IP 标准，支持 UNIX 域套接字等。

在定义方面，则出现了“自由软件/开源软件”的定义之争，出现了与自由软件定义、开源定义相关的文献（称不上标准），但是这些定义对开源许可证并无约束力。从 20 世纪 80 年代诞生的 GNU 通用公共许可证、MIT 许可证、BSD 许可证等，到现在已经出现了超过 100 种开源许可证。若以独立法律文本计算（含不同版本），总数可能接近 150 种，经过 OSI 认证的也有超过 80 种，这就出现了严重的协议兼容问题，至今尚未得到妥善的解决。

(2) 快速发展期：浏览器大战与 Web 标准，开源对于标准化的支持。

以 Web 标准为代表的开源标准化时期，可以算作快速发展期。第一波浏览器大战（1995~2001）在 Internet Explorer 和 Netscape Navigator 之间展开，制造了令程序员深恶痛绝的浏览器兼容性问题。2004 年，Firefox 浏览器推出，强调速度、安全性和标准支持，开始挑战 Internet Explorer 的垄断地位。2008 年，谷歌的 Chrome 浏览器推出，同样开源，同样强调对 Web 标准的支持。2004 年，苹果、Mozilla、Opera 等厂商成立网页超文本技术工作小组（Web Hypertext Application Technology Working Group, WHATWG），以“活标准”（Living Standard）模式开发 HTML5，直接响应实际需求。2019 年，W3C 宣布将 HTML 和 DOM 标准主导权移交 WHATWG，这标志着厂商主导标准的时代到来。开源再一次战胜了闭源，开放标准战胜了私有标准，开放联盟战胜了科技寡头。

(3) 生态协作期：注重供应链安全。

21 世纪 10 年代至今，随着开源软件的接纳程度不断提高，开源软件蕴含的风险也逐渐受到重视。开源安全与质量方面的标准经历了从概念探索到体系化落地的

关键跃迁。2014 年，众议员爱德华·罗伊斯在美国国会提出了《网络供应链管理与透明法案》（Cyber Supply Chain Management and Transparency Act）。该法案要求政府机关在购买新产品时，需要取得产品的软件物料清单（SBOM），其中也针对“所有美国政府会使用的软件、固件及产品”，都要有软件物料清单。虽然这个法案未获通过，但是确实启发了后来的一系列实践。2021 年 5 月 12 日，美国政府发布的提升美国网络安全的行政命令（Executive Order on Improving the Nation's Cybersecurity）要求美国国家标准与技术研究院（National Institute of Standards and Technology, NIST）在 90 天内制定指引，其中包括许多主题的标准、程序或准则，目的是“强化软件供应链的安全性”。其中，也包括“提供采购者所有产品的软件物料清单”。与此同时，SPDX、CycloneDX、SWID 等机器可读格式成为主流，其中 SPDX 被 ISO/IEC 5962:2021 采纳为国际标准。

另外，整个供应链的安全也受到越来越多厂商的重视。Linux 基金会于 2016 年发起名为 OpenChain 的全球性开源项目，该项目旨在通过标准化流程解决开源软件供应链中的许可证合规性问题。OpenChain 规范于 2020 年升级为 ISO/IEC 5230:2020 Information technology—OpenChain specification，成为首个针对开源许可证合规的国际标准。2023 年，OpenChain 还推出了 OpenChain 安全保证规范 ISO/IEC 18974:2023 Information technology—OpenChain security assurance specification，可以帮助组织检查开放源代码中已知的安全漏洞问题，例如，CVE、GitHub 依赖性警报或软件包管理器警报。总之，越来越多的企业、政府机构、学术机构开始结成广泛的协作共同体，开始有意识地通过制定开源相关的标准，推动行业更加良性发展，生态更加健康。

15.2 标准化在开源生态系统中的核心价值

开源标准化在开源生态系统中具有非常高的价值，主要分为以下几个方面。

（1）促进技术互操作性与兼容性。

通过定义统一的接口、数据格式或协议，标准化确保了不同工具、平台和服务之间的无缝协作。例如，在容器技术领域，开放容器倡议（Open Container Initiative）和云原生计算基金会推动了容器运行时标准，使得容器可以在任何兼容的环境中运行，从而大幅降低了跨平台迁移的成本。此外，在开源硬件领域，DIN SPEC 3105—(1/2) Open Source Hardware（一系列来自德国的标准规范）通过规范技术文档要求和社区测试流程，确保了开源硬件设计的可复现性和兼容性，为开源硬件生态的健康发展奠定了基础。

（2）加速技术创新与产业化落地。

标准化的代码库、接口和工具链减少了重复开发的成本，使开发者能够专注于差异化创新。例如，Linux 基金会发布的 OpenChain 规范通过标准化合规流程，降低了企业采用开源技术的法律风险。此外，标准化还推动了技术的普及，降低了准入门槛，使更多开发者能够参与进来。例如，GitHub 平台所建立的一套 GitFlow 与 GitHubFlow 工作流，成为众多开源项目的协作模板，甚至后续的 Gitee、GitLab

等代码托管平台与开源项目，也都完全遵循这些工作流。

（3）增强生态稳定性与可持续发展。

碎片化是开源生态面临的一大挑战，而标准化通过统一技术路径，减少了社区分裂的风险。除了上文提到的 Linux 与浏览器的相关标准实践，在移动操作系统（Android）、云计算（Kubernetes 容器编排标准）、数据库（SQL 语法标准）、大数据（MapReduce 接口标准）、芯片设计（RISC-V）等方面，都有效地通过标准来协调市场活动，确保技术路线的一致性。此外，标准化还能提升信任与安全性。例如，软件物料清单标准的制定，使软件供应链更加透明，从而增强了企业对开源组件的信任，并有效降低了安全风险。

（4）推动社会治理与合规性。

数字主权保护成为全球关注的重要议题，开源治理标准的制定能够规范开源协作流程，保障技术自主可控。此外，标准化与开源的结合也被视为打破技术垄断的重要手段。例如，欧洲通过开源标准化推动工业 4.0 技术的平等获取，避免了大型科技企业对关键技术的垄断，从而促进了技术的公平分配和社会的可持续发展。

综上所述，标准化在开源生态系统中的核心价值不仅体现在技术层面，还涉及产业发展、生态治理、商业模式和社会公平等多个维度。未来，随着标准化进程的深入，开源生态系统将进一步走向规范化、开放和可持续发展。

15.3 标准化缺失导致开源项目失败的反面案例

一、Chromium 项目失败的原因

2015 年，红帽的工程师汤姆·卡拉韦（Tom Callaway）在 OSCON 上发表了一个演讲《这就是你的开源项目失败的原因》，主要谈到了他参与的 Chromium 项目：“这不是一次愉快的经历。”他认为开源项目失败的一个重要原因是缺乏标准化。具体到 Chromium，主要有两个原因：首先是代码管理问题，Chromium 代码库超 3500 万行，依赖树复杂（含 1200 多个第三方库），若缺乏严格的依赖管理（如 Bazel 构建系统）和代码审查流程（Gerrit），维护成本将指数级上升。其次是许可合规风险，Chromium 混合了 BSD、MIT、LGPL 等 30 多个许可证，这使得许可证兼容变得非常复杂。这些问题导致开发过程复杂化，增加了维护难度，最终影响了项目的成功。

二、OpenStack 项目失控的深层原因

表面上看，OpenStack 从风光一时到最后无人问津，主要原因是 Docker/Kubernetes 的崛起带来的冲击。但是从项目自身，我们也能找到一些深层次的原因。企业利益争夺引发技术路线分裂，导致在 OpenStack 的大家庭中存在大量的组件，甚至互相竞争。2015 年 12 月，有一位记者麦克斯韦·库特（Maxwell Cooter）还写了一篇文章“OpenStack：厨师太多了”，指出参与者过多导致决策分散，企业的不同议程可能导致技术方向不一致的问题。但 OpenStack 的创始人之一、Rackspace 的首席技术官约翰·恩盖茨（John Engates）表示，多样性从一开始就是计划的一部分。OpenStack 基金会执行董事 Jonathan Bryce 也表示，灵活性非

常重要。“你可以对它进行如此多的自定义，这是 OpenStack 最强大的功能之一。”恩盖茨还以 Linux 为例说明：“Linux 的多个发行版并没有阻碍 Linux 的发展。”但事实上，Linux 在标准化方面要做得好得多。由于缺乏标准化，OpenStack 过度的多样性与灵活性带来了一个直接的后果——OpenStack 社区的核心贡献者从 2016 年的 1800 人降至 2022 年的 600 人，这进而导致了单元测试覆盖率低于 40%（Kubernetes 单元测试覆盖率高于 75%）、升级困难（如从 Newton 到 Zed 版本需手动迁移）等问题。越来越多的厂商不再考虑选用 OpenStack 的任何一个发行版本。

三、AI 开源领域的兼容性问题

AI 开源领域正如火如荼，应该还谈不上失败，但是各种各样的兼容性问题的确给程序员带来了大量的烦恼。首先，是模型文件的格式问题。根据 GitHub 上的“List of ML file formats”仓库的统计，目前存在超过 20 种 AI 模型格式，在不同的格式之间虽然能够转换，但可能会存在难以预料的精度损失。其次，硬件适配困境是 AI 领域的另一个挑战。不同硬件生态使用的编程模型和编译器互不兼容。尽管现在 CUDA 是主流，但同时还存在 CANN、XLA 等其他技术路径。2021 年，OpenAI 发布的 Triton 编译器试图统一中间表示（Intermediate Representation, IR）层，但目前还处在奋力前行的阶段，能否打破 CUDA 的垄断，成为 AI 领域的开放标准与事实标准，还有待观察。

15.4 标准化面临的挑战与未来发展

一、标准化面临的挑战

第一个挑战是创新与标准化之间的平衡。标准化带来的是统一性和互操作性，但过度标准化可能会束缚创新。在一些新兴技术领域，过早地采用严格标准可能会阻碍实验性和创新性的探索。尤其是在快速发展的开源技术领域，对标准化的反感自有其合理之处。因此，在制定标准时，如何平衡生态创新的多样性与统一性成为一个重要问题。社区和标准化组织需要在技术成熟度和市场需求之间找到最佳平衡点，既保证技术进步的自由，又确保生态系统的健康发展。

第二个挑战是全球协作与文化多样性。开源项目往往是全球开发者协同合作的结果。不同国家和地区在技术习惯、文化背景和法律法规上存在差异，如何制定一套能够广泛适用且易于遵循的标准，在标准层面如何有效地兼容各国的文化（尤其是法律法规），是一个不小的挑战。标准化过程中需要充分考虑文化多样性，并通过开放讨论、广泛征求意见来形成共识。

二、未来发展

受到开源的启发，开放标准的概念也被多次提出，并应用于不同的标准化领域，根据维基百科的词条：开放标准是指任何人都可以公开获取和使用的标准。开放标准的一个共同前提条件是使用开放许可证，以提供可扩展性。通常情况下，由于其固有的开放性，任何人都可以参与其开发。开放标准没有统一的定义，其解释也因使用情况而异。较为著名的开放标准有 GSM、4G 和 5G 等，这些标准使大多数现代手机可以在全球范围内使用。

随着 AI、物联网等新兴技术的发展，标准化的需求也在不断变化。未来的标准化趋势可能会更加注重动态适应和快速迭代。笔者认为开源标准化的架构设计应该考虑以下策略。

（1）区分快慢。开源领域并非全体一致、同步变化，总有些领域（如容器、DevOps）要快些，有些领域（如操作系统、数据库）要慢些。法务合规领域已经比较稳定，而安全漏洞方面可能还会有较快的变化。因此，不同变化速度的领域需要以不同的迭代周期发布标准。例如，3 个月发布一个正式版本，或 2~3 年发布一个长期稳定版本（LTS）等。

（2）宽严适度。越是已经达成一致的领域越需要严格、精确、无歧义的语言来定义。分歧较大、含义模糊、快速演变的领域也需要有一些宽松但能够框定范围的语言来做一些定义。

（3）密切观察，开放协作。本质上，针对开源这样一个快速变化的领域，需要密切观察其发展态势，需要引入更多参与者，进行开放协作，这样才能制定出有意义、有价值的标准。

第 16 问 开源项目如何实现全球化？面临哪些挑战

按理说，开源生态原本就是一个无国界的数字世界，一旦将项目开源就意味着向全世界开源。但是，由于语言、文化、商业和地缘政治等方面的原因，开源项目（非英美国家的）要实现真正的全球化，需要付出巨大的努力。

16.1 开源全球化的 4 层境界

开源全球化分为技术架构全球化、社区协作全球化、法律合规全球化和商业全球化 4 层境界，如图 16-1 所示。



图 16-1 开源全球化的 4 层境界

一、技术架构全球化

在全球化进程中，首先需要考虑的是产品用户将会来自世界各地。因此，开源项目的技术架构需要突破地域限制，实现跨区域的技术兼容与文化适配。这首先体

现在基础功能的本地化支持上——不同地区对日期格式（如中国采用“年一月一日”、欧美惯用“月/日/年”）、货币单位（如人民币符号¥与美元符号\$的差异化显示）、字符编码（如支持中日韩多字节字符集）等基础规范存在显著差异，项目需通过 i18n（国际化）框架将代码逻辑与文本内容解耦。

更深层的国际化适配则需要从架构设计着手。采用模块化开发模式将核心功能与区域特性分离是关键策略：将支付接口、地图服务等具有地域属性的功能封装为可插拔模块，既保持核心代码的稳定性，又允许社区开发者根据本地需求灵活扩展。同时需确保跨平台兼容性，通过容器化部署、环境变量配置等技术手段，使项目能在不同国家的云环境中无缝运行。

文化层面的适配往往容易被忽视却至关重要。以日本市场为例，用户不仅需要日语界面，而且对用户界面的极简设计（如减少弹窗层级）和隐私保护机制（如数据存储地域选择）有特殊要求。不同地区的宗教信仰、礼拜习惯、交流模式、日常用语都会对软件功能提出要求，这种深度本地化改造往往比单纯的语言翻译更具挑战性。

二、社区协作全球化

开源项目的全球化进程本质上是开发者社区的全球化——当代码仓库向全球开发者敞开时，如何构建跨越语言、时区、文化的协作网络，就成为决定项目成败的关键。

打破语言壁垒是首要任务。尽管英语仍是技术领域的通用语言，但仅依赖英语会天然排斥大量潜在贡献者。成熟的开源社区通常采用“主文档英语+多语言分支”的策略：Linux 基金会不仅维护英文技术文档，还支持社区成员将内容翻译为多种语言。以 Kubernetes 为例，在 2019 年，社区已经有了 9 种不同语言的文档，现在更增长到了 15 种之多。还有很多社区，都借助自动化工具（如 GitHub Actions 自动化翻译）实现跨语言协作。

分布式协作机制是应对地理隔阂的核心设计。Git 版本控制系统天然支持异步开发，允许旧金山工程师的代码提交与东京开发者的代码评审在时间线上交错推进。例如，AFFiNE 项目中通过标注“good first issue”降低新手门槛，配合 Discord 频道的 24 小时异步答疑，显著提升了社区的参与度和协作效率。

社区治理的透明度决定了全球化协作的可持续性。Apache 软件基金会的“一人一票”决策模式结合了民主原则和灵活性，既保障了会员的平等参与权，又通过 STV 机制优化了选举过程，同时在必要时采用正式投票确保重要决策的严谨性；华为 openEuler 社区则建立中英文双周例会制度，会议纪要同步在 GitHub Wiki，确保欧美开发者能追溯技术决策的全过程。

开放式协作中的冲突解决是更深层的挑战。当德国开发者强调代码规范严谨性时，中国贡献者可能更关注快速迭代效率。openEuler 社区给出的解决方案是与 Linux 基金会、OpenInfra 基金会等全球性基金会开展深度合作，构建产教融合人才培养体系，在社区成员的共同协作和贡献下，加速全球化的人才发展，覆盖 16 个国家和地区的 60 多所高校。到 2024 年 11 月，openEuler 已经吸引了来自 43 个国家和地区的 2000 余名海外贡献者加入，进一步推动其全球化进程。

随着开发者的数量不断增加，来自不同国家与文化的开发者多样性也会增加，这将会给项目开发管理带来更多的挑战。代码质量管控和安全漏洞管理都是开源开发者团队需要关注并解决的问题。

全球化社区的本质是通过工具链（如 GitHub/Gitee）、流程（异步协作）、制度（透明治理）的三重设计，将地理上的分散转化为创新力的聚合。当东京的 UI 设计师、硅谷的架构师、柏林的测试工程师能像同一办公室的团队般高效协作时，开源项目才真正完成了从“跨国协作”到“无国界创新”的进化。

三、法律合规全球化

开源项目的全球化扩张，始终需要面对与法律边界的博弈——当代码跨越国境时，必须直面不同司法辖区的合规高墙。从许可证的法律效力到数据主权的硬性约束，开源社区需要构建一套“合规即代码”的管理体系。

许可证战略的全球适配是首要挑战。选择开源协议时需兼顾技术自由与商业友好性：Apache-2.0 许可证因明确专利授权条款，受企业级项目的广泛青睐。数据库厂商 PingCAP 的全球化策略即印证了这一点——其核心产品 TiDB 采用 Apache 许可证，允许用户自由修改代码并闭源衍生服务，同时通过提供 TiDB 商业版实现盈利。而对于涉及专利技术的项目，还需警惕一些国家的“专利钓鱼”（Patent Trolling）风险，Linux 基金会为此推出 OpenChain 规范与相关认证，帮助社区标准化知识产权管理流程。

数据隐私合规是一场全球规则的拼图游戏。2018 年欧盟出台的《通用数据保护条例》（General Data Protection Regulation, GDPR）要求用户数据可删除、可迁移；而美国颁布的《儿童在线隐私保护法》（Children's Online Privacy Protection Rule, COPPA）禁止未经家长明确同意收集儿童个人信息，要求提供数据删除选项并保障数据安全；美国 1986 年颁布的《电子通信隐私法案》（Electronic Communications Privacy Act, ECPA）禁止未经授权拦截或披露电子通信内容，并对执法机构获取通信记录的条件作出限制；我国《中华人民共和国个人信息保护法》则强制境内数据本地化存储。这些各不相同的规则意味着开源项目的基础架构必须具备区域化部署能力（以符合当地的法律法规）。例如，AI 开发平台 Dify 进入日本市场时，为满足日本的《个人信息保护法》（Act on the Protection of Personal Information, APPI），不仅将服务器迁至东京数据中心，还在代码层增加数据加密开关，确保符合日本《个人信息保护法》中“用户明示同意”的要求。更进一步，Dify 还通过了 SOC 2 Type II、ISO/IEC 27001:2022 等认证，证明其具备符合国际标准的数据安全管理体系，进一步增强日本用户与监管机构的信任。

地缘政治风险需要更高维度的架构设计来应对。2019 年 7 月，因美国制裁，GitHub 封锁了克里米亚地区、伊朗、古巴、朝鲜和叙利亚的用户访问权限。为了应对这样的挑战，RISC-V 基金会（现称 RISC-V 国际）于 2019 年 11 月宣布将其总部从美国迁至瑞士。2022 年 4 月，因俄乌冲突升级，俄罗斯的开发者也因制裁无法访问 GitHub 私有仓库，暴露了中心化代码托管平台的脆弱性。2022 年 9 月，Linux 基金会也在布鲁塞尔开设了欧洲分部。

技术组件的出口管制是更深层的挑战。若项目包含加密算法（如 SSL/TLS 模

块)，需遵守美国《出口管理条例》(Export Administration Regulations, EAR)，标准加密（指符合国际标准化组织，如 IEEE、IETF、ISO、NIST 等认可的加密算法或协议，如 AES、RSA、SHA-256 等）若满足“已发布”条件（公开源代码+通知），可豁免许可证要求；非标准加密（如未公开算法、专有协议或未公布的实现方式等）仍需发送通知并接受审查。Linux 基金会在 2020 年特别发布了一份文档“了解开源科技和美国出口管制 (Understanding Open Source Technology & US Export Controls)”，详细阐明了与 EAR 相关的各种问题，并推荐了一组开源社区的最佳实践，供大家采纳。

合规管理的终极目标，是在法律约束与技术自由间找到动态平衡。这要求开源社区建立“合规跟踪—风险评估—架构迭代”的闭环机制：从选择兼容性许可证、设计隐私保护架构到预判地缘政治风险，每一步都需将法律逻辑转化为代码逻辑。唯有如此，开源项目的全球化进程才能既突破地理边界，又不触碰法律红线。

四、商业全球化

开源项目的全球化不仅是技术传播，更是一场商业模式创新实验——当代码能自由流动时，如何让商业活动也能跨越地理边界，是维持项目生命力的核心命题。第 3 篇会对开源商业化有更详细的讨论，本节只简单讨论 4 个要点。

(1) 收费模式与付费习惯。无论是哪种商业模式，最后都会落到“钱从哪里来？”这个问题上。想要把生意做到全世界，自然需要考虑“不同国家的钱怎么赚”的问题。这当然要比在一个国家赚钱要困难得多。

(2) 生态联盟。在全球范围内搭建生态联盟，首先需要考虑的是“你的产品本身是不是从一开始就要设计成满足具有全球普遍性的需要”。然后是研究自身产品的“生态位”，也许在不同的国家与市场，相同的产品会占据不同的生态位。在不同的市场，也许需要以不同的“姿态”与人合作。接着是标准化与技术兼容，现在不可能开发一种完全独立的技术或产品，而是需要与很多其他技术与产品“搭配”在一起使用。要具备广泛的适应性、兼容性，遵从已有的标准还只是起步，做得好的企业则努力成为全球标准的制定者与输出方。

(3) 加入或捐赠基金会。是否需要加入某个开源基金会或者将自己的开源项目捐赠到某个开源基金会也是开源商业全球化需要考虑的一个重要因素。加入云原生计算基金会或 Apache 软件基金会的项目，不仅能获得法律、资金支持，而且可接入成熟的全球化协作网络。

(4) 本地化运营。商业化进程必须与本地化运营深度耦合。大多数出海的中国开源项目都需要思考如何在海外的社交媒体中“招揽”用户，在 Medium 上发文章，用 Slack 群组交流，在 Reddit 上打广告等，是需要不断精进的技能。TDengine 数据库在北美市场推出“按写入数据量计费”的云服务，与 AWS 的定价模型无缝衔接，是“本地化”的典范。

可持续的全球化开源项目本质上是在“社区驱动”与“商业回报”间建立正反馈循环：开源版本吸引全球开发者贡献技术价值，商业化产品将其转化为经济价值，而利润反哺又推动项目迭代升级。当这个循环能自适应不同区域的法律环境、技术生态和商业习惯时，开源项目只有商业全球化才能进入一个良性生长的阶段。

16.2 开源全球化的未来趋势

过去 20 多年，可能是开源全球化发展得最好的阶段，而接下来开源全球化也许会进入一个“非常灰暗”的时代。前面讨论的挑战也许都变成了“小儿科”，而过去经过检验的“最佳实践”，也许都需要重新摸索。

世界正在走向分裂，以美国为首的个别国家，正在努力推动“逆全球化”，现在已经有专家在讨论“非美全球化”或者“半球化”这种荒诞的概念。开源也不可能置身事外，独善其身。过去我们谈开源全球化，默认的语境是“全球技术协作没有边界”，但现在，这个“全球”已经在发生裂解，而开源正在其中艰难穿行。也许我们应该问：如果全球化终结，开源应该走向何方？

一、“黄金 20 年”终结，开源全球化面临重构

过去 20 多年，开源全球化得以顺利发展靠的是以下几个条件。

- (1) 技术中立，加上开放协议，再加上 GitHub 等平台提供的全球基础设施。
- (2) 相对自由流动的技术人才。
- (3) 美中欧在开源上的“竞合”状态。
- (4) 资本和市场都鼓励“全球增长”逻辑。

但是今天，这一套逻辑正在崩塌，随着各种各样的地缘政治事件的爆发，开源也被迫卷入其中。GitHub 开始限制特定国家/地区的访问（如伊朗、克里米亚、朝鲜和俄罗斯）。自然，欧洲、中国也会产生警惕，纷纷考虑建设自己的基础设施。欧洲开始考虑建设自己的 Sovereign GitLab，俄罗斯也计划投入 13.9 亿卢布（约合人民币 1.07 亿元）建设自己的代码托管平台，我国则已经有 Gitee、GitLink、AtomGit、GitCode 等开源代码托管平台。随着美国对芯片、AI、大模型等领域施加越来越严厉的技术出口管制，最悲观的预测是整个世界发生分裂，变成互不相通的几块（比当年的“美苏冷战”更糟糕）。最乐观的预测是世界从单一中心裂变为多个中心，在多个中心之间存在着时断时续、互通有无的联结。虽然不能像过去那样，技术在全世界范围内自由流动，但是或快或慢，新技术总能够传遍全球。

二、数字世界中的栅栏与鸿沟

可以预期，未来的 5 到 10 年将是全球规则、各个区域、国家规则快速变动的时期。数字世界的“路与桥”会演变成“栅栏与鸿沟”。有些国家在打造“数据护城河”，对代码、模型、数据跨境流动设置技术与法律壁垒；有些平台在执行“国家与地区黑名单”，对特定开发者、组织实施技术封锁，拒绝提供服务；有些组织在构建“安全信任区”，对来自盟友之外的代码贡献保持怀疑甚至默认敌意。

在开源世界里最常见的逻辑“只认代码不认人”将会失效。我们不会预先假设对方心怀善意，而是首先确认对方是敌是友。过去在开源世界里通行的质量观念，也会变得过时，必须高度重视国家安全，要防止恶意“贡献者”在开源代码中“投毒”。整个开源社区可能会走向碎片化和保守主义。

是否有可能重塑信任？是否有可能跨越鸿沟？是否有可能建立更加牢固的纽带？

三、应该怎么办

我们需要既能识别与管理国家安全与产业安全风险，又不至于一刀切地排斥国际协作的新机制。推动建立完善的软件物料清单和“维护者透明身份认证”制度也许是目前最紧迫的任务。

当前，大多数主流开源治理机构（如 Linux 基金会、Apache 软件基金会、开源促进会等）高度集中于北美。在新的地缘环境下，需要探索“多中心、多主体”的治理结构。例如，鼓励成立多区域（如东亚、东南亚、非洲、拉美等）协作的“开源联盟体”；提倡开源基金会之间的互信互认协议；推动全球不同文化背景的开源治理经验交流，从而避免全球开源生态因单边结构而失衡，甚至失控。

推动建设面向多极世界的开源基础设施体系，而这些多中心的基础设施之间，又能实现某种程度上的互联、互通、互信与互为备份。这些不是“割裂”的体现，而是构建真正自由协作能力的保障。

再者，中国的开源技术（尤其是开源 AI 技术）是否有机会接过开源全球化的大旗，成为新技术的引领者。例如，面向共建“一带一路”国家的开源或许会成为新的增长点。通过输出中国的开源技术，强调“开源赋能、合作共建”，帮助更多的国家借助开源技术确立自身的数字主权。我们也许会看到新的开源全球化的形态。

第 17 问 开源生态与开源供应链是什么关系

开源生态与开源供应链是两种不同的看待开源世界与描述开源世界的角度。前者从生态的眼光来看待开源，而后者从数字世界“生产过程”的角度来看待开源。前者更加看重生态的“繁荣与和谐”，后者则是从消费者的角度出发，看重“供应”是否稳定、可靠、顺畅。由于供应链的视角较为“狭窄”，也可以认为：开源供应链只是开源生态的一部分，而不是全部。

如第 10 问所述，开源生态是一个基于协作与共享的社会经济系统。而开源供应链聚焦于开源软件从开发到应用的供应网络。现代软件项目平均依赖数百个开源组件，这些依赖关系形成了复杂的供应链网络。一个典型的应用可能直接依赖二三十个库，但这些库又可能各自依赖数十个其他组件，形成多层次的依赖树。

17.1 开源生态与开源供应链的结构关系

开源生态是开源供应链得以存在的基础。开源生态提供环境，开源供应链则是其中的物质和信息流动。

从开源技术发展的角度来看：每一个开源项目的诞生都是源于某种“实际存在的需求”，而这些需求不仅仅是最终用户的使用需求，也可能是某个开源软件的功能与技术需求。从软件架构的角度看，一个具有一定共性的技术需求就值得通过一

个具有一定独立性的模块来实现，而这样的模块逐渐就具备了从一个项目中脱离出来，独立成为项目的趋势。

待到开源软件的生态发展越来越成熟，开发一个新软件时，不再需要从第一行代码开始写程序，而是先在开源世界里寻找已经可用的模块、组件、框架甚至成品，然后再稍作修改，或者拼拼搭搭，用于组成符合需要的软件。在软件领域，非常重要的哲学“不要重复造轮子”，使整个开源生态本身变得越来越繁荣。

随着各种现代编程语言都开始支持基于网络的在线包管理工具，复用已有的组件变得顺理成章，甚至非此不可。Docker 之类工具的出现使更大复杂度的重用也变得可能。到了现在，从网络上“拉取”一些组件回来后继续开发，变成一种在开源世界里随处可见的操作。

顺着开源供应链的脉络去理解，才有可能把握整个开源生态的全貌，而不是将开源世界理解为一个互不相关的开源项目。

再进一步，需要从软件架构的依赖关系出发，看到背后的开发者“身兼多职”的现象。一个开源开发者既是一个开源项目的主力开发者，又是另外一些开源项目的业余（非长期的）贡献者，因为在他开发一个项目的过程中，自然产生了对其他生态相关项目的贡献需要。甚至，他很有可能成为另一个开源项目的创始作者。最著名的例子可能是林纳斯在开发 Linux 的过程中，“并不那么情愿”地成了 Git 的创始人。

17.2 开源生态治理与开源供应链治理的区别

“治理”这个词用在开源领域通常是指某种管理行为。开源生态治理主要关注社区治理模式（见第 12 问），多项目协同、文化建设、社区包容性等软性因素，通常会发生在社区里。而开源供应链治理更注重技术性、规范性，强调依赖关系、组件来源、软件物料清单、漏洞管理等相关的企业内部的实践（见第 18 问），但是随着开源软件复杂度的上升，大型开源项目也涉及自身的开源供应链治理的问题。

开源生态治理的核心目标是能够使整个生态更健康（见第 14 问），虽然最新潮流会让我们更多地关注“健康指标”这样看起来更加科学的内容，但是更基础的“人际沟通”“团队管理”“人员激励”等工作，更像是“社区管理的艺术”而非简单直接的最佳实践组合。在开源基金会的层面，则不只是关注单一项目的开源生态健康与繁荣，而是会思考更复杂的整个“生态圈”如何保持繁荣的问题。

在整个开源生态层面，需要考虑的是参与开源生态的各方角色，各自的利益与诉求，如何通过开放式协作得到满足，以及在这样的过程中他们又需要付出哪些努力、提供哪些资源。而所谓开源生态治理，本质上是一种开放式协作的架构设计。通过与多方协商，互相妥协，最终达成某种合作共赢的利益架构，这是想要维持开源生态的真正挑战。

开源供应链治理的核心目标是确保（尤其是企业的）软件 DevOps 全过程都能够顺利进行。因为存在下文中会详细描述的各种各样的开源供应链风险，软件可能

会无法正常发布，或者在发布之后由于各种疏漏给企业带来重大损失。而所谓开源供应链治理，就是在选型、设计、开发与发布软件的全过程中引入对开源软件的精细化管理，从而避免潜在的风险。

第 18 问 如何理解和防范开源生态风险与 开源供应链风险

18.1 开源生态风险与开源供应链风险的区别

广义来说，开源生态风险最终会导致开源供应链产生风险，因此也属于开源供应链风险的一种。

一、开源软件领域可能遇到的 4 类风险

开源软件领域可能遇到 4 类风险：技术风险、法务风险、生态风险和政策风险，如图 18-1 所示。

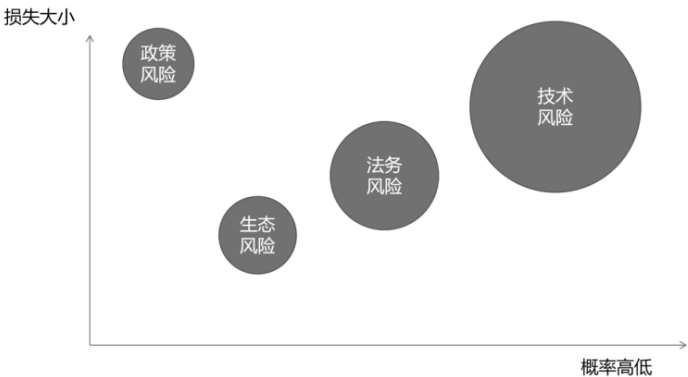


图 18-1 开源软件领域可能遇到的 4 类风险

在开源软件领域，“技术风险”随时可能冲击项目的运行。代码本身的不完善、隐藏的漏洞或者后门一旦被黑客利用，就可能导致数据泄露或系统瘫痪；而在复杂的依赖链中只要某个上游项目出现漏洞（甚至被人“投毒”），就可能让整个生态陷入危机。正因为技术风险既普遍又波及面广，所以在图 18-1 中它的体量最大又偏向关注度的右侧。

相比之下，“法务风险”虽然看起来没有那么显眼，却同样暗藏危机。开源许可证种类繁多，从 GNU 通用公共许可证到 Apache 许可证、MIT 许可证，每种许可证背后都有不同的义务和约束。如果在理解不清的情况下把各种许可证混用，或者未与贡献者签署明确的协议，就可能在某个商业化阶段被追究侵权责任。一行未经授权的代码就可能把整个产品拖入漫长的诉讼漩涡，因此在图 18-1 中，法务风

险紧邻技术风险，提醒我们务必把合规当作常态化管理。

再看图 18-1 的左下方，“生态风险”显得相对温和，却与项目的生命力息息相关。一个健康的开源社区不仅需要源源不断的新贡献者，也离不开透明的治理和合理的资源分配。一旦项目的核心维护者因精力不足而逐渐淡出，或决策流程不够公开，就容易造成内部摩擦、用户流失，最终使项目在无声无息中走向衰落。生态风险虽然影响范围小于技术风险和法务风险，但决定了项目能否持续发展。

最后，在图 18-1 的左上方是“政策风险”——难以预料的风暴。由于地缘政治的影响，国家的政策会不断发生变化，网络安全审查、加密算法出口管制，或者对关键基础软件提出的更高合规要求，都可能瞬间改变开源项目的全球推广策略和开发成本。如果政府补贴导向发生调整，曾经慷慨扶持的资金就可能随之消散【美国政府在 2025 年 4 月 16 日宣布停止资助 CVE（Common Vulnerabilities and Exposures）和 CWE（Common Weakness Enumeration）项目，这可能导致开源安全领域的巨大风险】。因此，尽管政策风险发生的概率或许不如代码漏洞那样频繁，但一旦爆发，其影响往往跨越行业与地域，需要项目方持续关注政策动态并灵活应对。

二、应对风险的心态和策略

不同的角色需要担心的风险是不同的，开源项目的作者、一个传统软件的开发（架构师）、一个公司的 CTO/CIO、开源项目办公室的负责人、某一个国家产业政策制定者，他们所处的位置不同，能够选择的应对措施和能够发挥的作用也不一样。以下只能大略提供一些通用的心态与策略。

首先，需要具备常识，理解开源相关的风险是存在的，是可能会带来危害造成损失的。

其次，需要具有判断力，尤其是企业的架构师，在做技术选型的时候需要有所取舍，以规避风险。

再次，需要积极投身开源生态建设，从力所能及的角度出发，为开源生态做一些贡献，降低各种风险的发生概率。

最后，需要做好预案，对那些无法抵御的风险，至少有一个替代的方案，以避免最糟糕的情况发生。

18.2 对开源供应链风险的理解

开源软件并非孤立存在，它们相互之间有着依赖关系。当我们思考软件供应链或者开源软件供应链时，会想到一条从不知名的远处延伸到我们面前的链条，这个链条的最后一环是一款我们看得见、用得上的软件。在这个链条中，有很多环节都是别人（别的公司、别的国家）提供的。如果有一天，他们决定拿回自己的那一环，那么延伸到我们面前的链条就断了，我们手中正在使用的软件就不能用了。这就被称为“断供”。

这样的意象会让我们过度紧张，似乎在一个链条上的任何一个环节都可能是薄

弱环节，让我们“受制于人”。在物理世界里，的确会存在零部件断供的现象，所以为了防范风险，我们需要寻找国产替代的零部件。但是在数字世界，尤其是在开源软件的世界，我们需要对供应链有更加符合实际的认知。

数字公共产品的复制成本几乎为 0，我们需要重新理解什么是开源世界中的“供应”，以及“供应链条”上有哪些风险。

18.3 开源供应链的本质

如果我们获得某一款开源软件，开始使用、修改甚至分发，就意味着我们可以在获得授权的前提下，做很多事情，也因此形成了一组合理预期。

- 技术类预期：这款开源软件能够正常工作，不会给我们带来隐患。
- 依赖类预期：这款软件的运行可能还依赖其他开源或不开源的软件、在线服务，这些依赖也应该能够保持正常运行。
- 法务类预期：我们可以通过自行修改软件使这款软件符合我们的需要，还可以（根据许可证）分发修改后的软件，获取商业利润。
- 服务类预期：我们可以在开源社区寻求帮助，也可以购买商业服务，以帮助我们更好地使用这款软件。

只要以上预期被打破，我们就会遇到各种各样的麻烦，也就是“开源供应链风险”。

所以，开源供应链并非源代码本身（仅仅把需要的代码全部下载回来，无法保障供应链安全），而是围绕开源代码的一组合理预期。随着开源生态的日益复杂，本来默认存在且合理的预期，需要被明确地识别出来，并得到相关供应方的承诺。另外，由于难以避免的疏漏，还需要对各种风险有所防范。

18.4 企业内部的开源治理

一般开源治理主要是指对外开源的社区与项目治理和企业内部在开源软件使用方面的治理。

在企业的开源软件治理框架中，要确保全流程透明。无论是产品对开源软件的使用，还是对其源代码的修改，都必须有完整、可溯的记录，并通过中心化仓库进行统一管理。这样可以让公司层面对所有开源组件的引入与变更一目了然，为后续的各项管理工作提供坚实基础。

在此基础上，应该建立优选引入的原则，持续从社区中甄选成熟、活跃且质量优良的开源项目，严格对标社区数据与公司质量要求。同时，禁止将已进入衰退期的项目纳入生产环境，并在同一产品版本中尽量统一同一开源软件的版本号，以避免因多版本并存增加漏洞跟踪与修复难度。只有在功能和性能同等的情况下，才推荐采用社区中质量更高的替代品，以此不断提升产品竞争力和整体质量。

与此同时，所有开源软件的来源都必须可追溯、可信赖。无论是从官方仓库下

载还是从企业内部的中心仓下载，均要通过恶意软件扫描、哈希值或数字签名校验，并做到告警清零，杜绝任何潜在的“投毒”风险，以便在客户或监管方询问产品所含组件来源时，能够清晰展示溯源链条，并在漏洞爆发时迅速追踪到受影响的软件实体。

在合规义务层面，企业应该坚持在法律法规与开源许可证的双重框架下开展工作。发现漏洞要第一时间向工业和信息化部网络安全威胁和漏洞信息共享平台上报，确保符合法定的漏洞管理要求；在使用和分发过程中，要严格遵守各类开源许可证的声明、分发和开源义务，避免因疏忽而陷入维权诉讼。

为了让治理闭环，不留死角，企业还应该承诺在产品生命周期内实现开源漏洞的“感知、定位、快速响应与通知机制”，从而支撑产品安全，为客户提供稳定可靠的保障。

除了以上关键目标，企业还应该建立“上游优先”的意识，在发现漏洞之后，及时向上游反馈，并协助上游修补漏洞，从而帮助整个开源生态变得更健康。

以下是一些值得关注的企业开源供应链治理实践。

- 2011 年，微软公司发布的《网络供应链风险管理：实现透明和信任的全球共享》报告，提出了一个旨在有效管理供应链风险的框架。该框架具有 4 个核心特点：协作、透明、灵活和互惠。
- 2016 年，华为公司发布的《全球网络安全挑战——解决供应链风险，正当其时》白皮书，主张推动多方合作，建立全面的供应链安全管理体系。
- 2020 年，红帽公司提出了《可信软件供应链规范》，旨在为软件安全性、合规性、隐私性和透明性这 4 个方面提供保障。
- 2021 年，谷歌公司提出了《软件构建的供应链级别》框架，简称 SLSA 安全框架，旨在通过端到端的解决方案确保软件供应链的完整性，防止源代码、构建平台或构件仓库被篡改带来的威胁。SLSA 框架定义了 4 个安全级别，从 SLSA L0 到 SLSA L3，每个级别代表不同的安全控制强度和完整性要求。

以上只是企业开源治理框架的简要概述，事实上，要实现这些目标，需要组织、流程、工具与人员能力的多方面保障，第 24 问和第 30 问会有相关介绍。

第3篇 开源商业模式

在软件领域，很多商业模式都与开源有关，这是因为基于商业的考量分为两部分：如何控制成本（节流），如何赚取收益（开源）。最终达到的效果，是成本降低，收益提高，利润最大化。而无论是“开源”还是“节流”，都需要围绕开源软件做出战略决策。所以在本篇中，我们讨论企业的开源商业模式，不仅会讨论企业如何通过开源“赚钱”，也会讨论企业如何通过引入开源、用好开源、回馈开源来降本增效，以及企业如何制定开源战略，在企业内部推广开源文化等话题。

第19问 企业如何通过开源降低成本并提升效率

在企业数字化转型日益加速的今天，企业对“降本增效”的诉求愈加迫切。开源技术因其自由、共享、可扩展等特性，正在成为企业实现这一目标的重要抓手。我们将从两个方面出发，探讨企业如何通过开源降低成本、提升效率，并总结一些落地的最佳实践。

19.1 企业使用开源能够降低成本

中国信息通信研究院 2022 年度的《全球开源生态研究报告》明确指出：开源软件能够为企业带来超过 8% 的成本节省，其中包括引入成本节省（18%）、开发成本节省（5.4%）、运维成本节省（0.8%）。我们可以进一步展开分析一下。

（1）免除软件许可费用。这可能是最为明显的降低成本的原因。由于开源软件通常以自由许可证发布，用户可免费获取和使用，无须支付高昂的许可费用。这对于初创企业或 IT 预算紧张的企业尤为关键。以数据库为例，PostgreSQL、MySQL 等开源数据库可作为 Oracle 等商业数据库的替代方案，显著减少软件支出。在众多其他技术领域，我们都能看到大量采用开源的企业。根据新思科技 2025 年度最新的《开源安全与风险分析报告》，有 97% 的代码库都包含开源代码。具体到云计算、网络安全、物联网、能源、清洁技术、计算机硬件与半导体等行业，100% 都含有开源代码。

（2）增强灵活性与可定制性，避免浪费。相对于无法修改的闭源软件，企业可根据自身业务需求，对开源软件进行裁剪、集成和定制开发，避免商业软件的“黑盒限制”，实现更精准的技术适配。

这种灵活性也大大降低了套装软件、功能溢出带来的资源浪费。毕竟商业软件的开发企业，总是希望开发足够多的功能，覆盖尽可能广泛的用户需求。至于买到软件的特定用户，很多功能他们根本用不上。

至于开源软件，“反正我也没花钱”，必要时我还可以把那些功能都删除，不仅节约了购买成本，也降低了后期的运行和维护成本。

（3）降低长期维护成本。当一个企业开始使用某款软件，无论开源或闭源，都存在后续的维护工作。安装、配置、调试、修复漏洞，如果可能的话还需要二次开发，以及后期的版本升级。如果是一款商业软件，自然需要与厂商签署维护合同，以确保对方能够提供足够好的服务。

虽然很多企业希望省事、省心，觉得购买商业软件能够减少麻烦，但是如果从成本的角度来看，开源软件的长期维护成本，实际上才是更低的。由于源代码公开，企业可以自主掌握维护节奏，快速定位和修复问题，避免因厂商停止支持或更新节奏缓慢而带来的运维风险。社区版本的稳定性和成熟度不断提高，也降低了依赖专业技术服务的频率。

（4）提高安全性，降低安全成本。从来不存在完美无缺的软件，任何软件都或多或少存在安全问题或者其他缺陷。如何及时发现安全问题并妥善修补，是企业需要考虑的大问题。开源社区往往拥有庞大的用户和开发者基础，安全问题能被快速发现和修复。

在第1篇提到的林纳斯法则“只要有足够多的人查看，所有的漏洞都会变得显而易见。”更正式地说：“只要有足够的单元测试员及共同开发者，所有问题都会在很短时间内被发现，而且能够很容易被解决”，就是在论证开源软件的安全性为何会比闭源软件更高。

相比封闭系统依赖厂商发布补丁的模式，开源软件的透明性更有助于安全防护，从而减少潜在的数据泄露、黑客攻击、后门入侵等高风险安全事件。

（5）降低招聘成本。熟悉开源技术的开发者数量众多，招聘成本相对较低。许多高校和在线平台都将开源技术作为教学核心内容，企业更容易找到熟练掌握Linux、Kubernetes、Python、MySQL等开源技能的人才。这个现象背后的实质是：采用开源软件的从众策略（尽量选择更多人选择的开源软件），本身是一种更加合理的选择，越是被广泛采用的开源技术，越是为所有的使用者带来更大的收益，降低更多的成本，从而形成几乎无可撼动的累积优势。

（6）避免单一厂商绑定，降低迁移成本。商业（闭源）软件厂商希望做到，而购买者希望极力避免的，就是“绑定”。我们在IT发展历史上不断看到的“A厂商与B厂商的产品、接口不兼容”，本来就是他们刻意为之。如果企业过于依赖某一供应商的产品，可能在未来演化为“技术债务”，迁移成本高昂。

开源生态则天然具备开放接口和跨平台兼容性，方便用户在多种基础设施之间自由迁移，降低了长期锁定成本。这一趋势进一步演变，哪怕是商业软件的厂商，也会发布开源的SDK之类的周边，看起来更加开放，也的确实现了某种更加友好的适配性。这就是开放能够战胜封闭的本质原因。

19.2 企业使用开源能够提升效率

（1）提高开发效率。开源提供了丰富的模块、框架和工具库，大量通用功能可直

接复用，避免“重复造轮子”。例如，Web 应用开发中广泛使用的 React、Spring Boot 等框架已成为提效利器。一个复杂的软件系统，同样是 100 个人的年工作量，A 公司选用了 80% 的开源组件，B 公司选用了 50% 的开源组件，然后都进行二次开发。两家公司自身还需要投入的人力，就相差 2.5 倍，开发效率自然也大不相同。

（2）通过社区协作，共同创新。活跃的开源社区是天然的“外部研发力量”。企业不仅可以获得最新的技术进展和实践经验，还能参与社区标准制定，增强行业影响力。参与度越高，收获越大。

相对于闭门造车，在社区中发现创新的方向，本身就能少走弯路。更何况，如果企业追求创新，那么在社区中能够吸引志愿者加入，就更加“划算”了。

（3）提升质量与稳定性，加快交付速度。优质的开源项目通常经过大量用户验证和社区持续优化，拥有稳定的版本发布机制。企业在此基础上构建系统，可以大幅减少测试周期和 Bug 率，加速产品交付。

（4）自动化管理，释放人力资源。DevOps、CI/CD 工具链（如 Jenkins、GitLab CI、ArgoCD 等）多基于开源技术构建，支持自动化测试、部署、监控等操作，减少重复人工工作，使技术团队专注于更具创造力的任务。

19.3 企业使用开源的最佳实践

（1）为使用开源做好预算。尽管开源免费，但实施过程中的定制开发、技术支持、人员培训等仍需投入。企业应合理规划开源预算，避免“0 成本幻想”，尤其在核心业务场景中。

（2）建立开源风险治理机制。开源项目中存在许可证不合规、版本更新不及时、安全漏洞传播快等风险。企业应建立相应的开源治理制度，包括开源合规审计、第三方依赖清单管理、安全补丁追踪等。

（3）“开源+商业”混合部署。针对核心系统，企业可选择“社区版+商业服务”的混合模式：使用开源核心技术，通过购买厂商支持或托管服务获得 SLA 保证，兼顾成本与稳定性。

（4）人才招聘、培养与激励。积极引导员工参与开源项目，既能提升技能，也有助于企业品牌在技术圈的曝光。建立“开源内推”“技术布道激励”等机制，可以吸引更多优秀人才加入并留在团队中。

第 20 问 开源软件如何实现盈利

相对于引入开源获得的降本增效的好处，还有一些企业更加深入地考虑：如何通过开源软件来赚钱。这方面已经有了不少好的经验可供借鉴。另外，一个开源项目的长期健康运行，其开发、维护需要资金支持，也需要考虑商业化的路径。

20.1 开源软件实现盈利的主流模式

开源软件实现盈利的主流模式包括支持与服务（订阅）模式、软件及服务（SaaS）与托管服务模式、广告模式、开源硬件模式、众筹/捐赠模式、认证培训模式以及混合模式等。各种模式通过不同途径（如提供增值功能、技术支持、云服务、培训认证等）实现开源项目的商业变现。

（1）支持与服务（订阅）模式。在该模式下，软件本身开源免费，但厂商为企业用户提供收费的技术支持、维护、培训和定制服务。典型案例包括红帽、SUSE、Canonica（Ubuntu）等开源操作系统厂商。以红帽为例，其企业版 Linux（Red Hat Enterprise Linux）开源代码对外公开，但公司通过销售订阅服务（包括软件更新、补丁、SLA 级别支持等）获得收入。同样，Hortonworks（开源 Hadoop 供应商）、Percona（开源数据库供应商）等也依赖专业咨询和支持服务获利。此外，一些公司将源码免费发布给社区使用，企业若要求稳定运行或使用特定功能，就需购买技术支持。例如，MongoDB 提供免费社区版，但企业版订阅包含管理工具和专业服务。

（2）软件即服务（SaaS）与托管服务模式。该模式通过将开源软件以云服务形式提供给客户来盈利。厂商不直接售卖软件，而是通过订阅云平台或托管服务收费，典型例子有 GitLab（GitLab 的托管服务），WordPress（Automattic 运营的托管版 WordPress）以及数据库领域的 MongoDB Atlas。MongoDB 的财报显示，Atlas 云服务收入占总收入的比例很大。例如，2022 财年第四季度，Atlas 收入同比增长 85%，占当季 58% 的总收入。通过按需云部署、自动运维和按月付费，企业用户可以避免自行运维复杂系统，而厂商则获得持续的订阅收入。另外，一些开源项目将 SaaS 模式与本地部署并行，用户可自行下载部署，也可选择购买托管服务。例如，Elastic 提供 Elasticsearch 的云服务 Elastic Cloud，Confluent 提供 Kafka 的云平台等。SaaS 模式往往与开放核心结合，免费版可自建部署，但若想使用云托管或企业版，则需付费。

（3）广告模式。部分开源项目通过广告获得收入。例如，广告屏蔽插件 Adblock Plus 的源代码开源，但它允许谷歌等投放“可接受广告”并收取费用。此外，开源社区平台 SourceForge 曾通过在其网站投放横幅广告获取大量收益（2006 年单季收入曾达 650 万美元，2009 年全年达到 2300 万美元）。类似地，一些开源浏览器（如火狐 Mozilla 浏览器）的收益也部分来自搜索引擎的广告分成。总体而言，广告模式多用于具备大流量用户基础的开源项目，但需注意保持社区信任、避免用户反感。

（4）开源硬件模式。将开源理念应用到硬件领域。厂商公开硬件设计图纸，使他人可自由制造衍生产品，但通过销售实体硬件或授权获利。代表性案例如 Arduino：其单片机开发板设计完全开源，任何人可根据协议制造并销售 Arduino 硬件。Arduino 创始人主要通过销售标有 Arduino 商标的官方产品获利：任何用 Arduino 商标销售的板卡都需支付授权费用。这种模式下，开源硬件厂商靠销售实体产品（板卡、套件）、提供培训或技术支持获得收入，同时依靠品牌商标维护产品质量与形象。

（5）众筹/捐赠模式。开源项目可以通过社区和用户的资助获得资金。一种是众筹，即在 Kickstarter、Indiegogo 等平台预售或收集开发基金。例如，开发者在 Indiegogo 上为 Mesa 图形库新增 OpenGL 功能筹得 2500 美元，还有开源游戏《大灾变：黑暗之日》（*Cataclysm: Dark Days Ahead*）通过 Kickstarter 为全职开发者筹款实现长期开发。另一种是定期赞助或捐赠：项目在平台（如 GitHub Sponsors、Open Collective、Patreon）上接受支持者的捐款。例如，Mozilla 基金会曾在《纽约时报》上刊登募资广告，为 Firefox 1.0 版本募集启动资金。2019 年，GitHub 推出 Sponsors 项目，国内的 Gitee 也推出了类似的功能，使个人和公司能直接赞助其托管的开源项目。这些模式依赖社区支持，适用于具备热心用户和公益属性的项目，但可持续性和金额往往不稳定，需要结合其他模式共同使用。

（6）认证培训模式。开源项目衍生的培训和认证课程也是重要的收入来源。许多机构针对流行开源技术提供收费培训课程和认证考试，如红帽的 RHCE/RHCSA 认证考试、Linux 基金会的 Kubernetes 认证（CKA/CKAD）等。企业用户或个人为获得官方认证或系统性培训往往愿意付费，而认证项目既增强了生态影响力，又带来了可观收入。此外，一些开源项目的官方培训与认证由第三方（如培训机构）承办，项目方通过授权或分成获益。

（7）混合模式。混合模式是指开源软件商业模式中结合开源许可证与专有许可证的方式，其核心思想是通过不同的许可证（或其他法律文书）构筑企业知识产权架构，并在此基础上获得合理的销售收入。常见的做法包括双重许可、开放核心、插件与增值功能或其他限制性许可模式，详见 20.3 节。

20.2 各种模式的优缺点与适用场景分析

如表 20-1 所示，各种开源商业模式都有各自的优缺点，也有各自不同的适用场景。不过分类并非一成不变。

表 20-1 开源商业模式比较

模式	优点	缺点	适用场景
支持与服务模式	收入稳定、业务规模可控，适用于企业级项目（如操作系统、数据库）	服务成本高、扩展性差，对社区价值贡献低	适合客户对稳定性要求高、愿意为技术支持付费的场景
SaaS 与托管模式	获取订阅型持续收入、降低用户部署门槛，适合愿意使用云服务的客户	面临云巨头竞争风险（平台商可能直接提供开源 SaaS 服务），以及初期需投入云基础设施的成本	适合需要按需使用、缺少自运维资源的用户，如中小企业
广告模式	门槛低，不影响用户使用	收入有限，需要巨大的流量和平台；容易招致用户反感	一般适用于面向大众用户的开源项目，如浏览器、开源社区平台等

续表

模式	优点	缺点	适用场景
开源硬件模式	用户可以自由创新，厂商通过硬件销售获利	竞争容易导致利润率下降，需靠品牌和服务维持	适合硬件开发或电子爱好者市场，如教育、创客领域
众筹/捐赠模式	启动门槛低，可直接与社区互动	资金不稳定且通常难以长久依赖	适用于一时创新项目或小规模开发项目，如游戏、工具原型等
认证培训模式	品牌效应强，适合热衷学习者	主要局限于成熟技术，且需投入课程开发和考试管理成本	适合拥有业界影响力的项目，如 Linux、Kubernetes 等
混合模式	开放协作与商业用户兼得，能够保持生态繁荣、易于推广，吸引免费用户	可能引发社区质疑（如许可争议），企业需平衡开源与专有功能之间的利益，另外开发者需同时维护开源与商业版本，可能出现碎片化	场景较为复杂，详见 20.4 节

20.3 开源商业的混合模式

常见的混合模式中，最古老也最成熟的一种是双重许可模式。这种模式是在开源许可的基础上，同时提供专有许可版本，或在免费核心版本上附加商业功能。用户可选择使用免费版进行评估或开发，若要在大规模生产或闭源环境中使用，则需要购买商业许可。经典案例是 MySQL：Oracle 将 MySQL 同时以 GNU 通用公共许可证和商业许可证发行，商业许可证为那些无法接受 GPL 约束的企业客户提供授权。类似地，SugarCRM、MongoDB（已改用 SSPL）和 Redis Labs（对某些模块采用商业闭源）等，都采用这种模式。

另一种广为采用的模式是开放核心模式。它与双重许可模式不同，通常将核心功能开源，在商业版中提供额外的企业功能或插件。例如，GitLab 既有开源代码的社区版，又有收费的企业版（含更多 CI/CD、高级安全等功能）。这种模式一方面扩大了产品的开源社区影响力，另一方面通过差异化功能和支持收取费用。

与开放核心模式非常相似，但更强调模块化和生态的是插件与增值功能模式。这种模式下的软件设计通常更精良，插件化架构方便用户像在“应用市场”一样挑选所需功能。像 WordPress、Eclipse、Visual Studio Code、Firefox、Chrome 等软件都内置了这样的扩展市场。关键区别在于，这些插件或增值功能来源更广泛，既有开源的免费插件，也有由其他公司开发的商业插件，并不局限于软件原厂商提供。这能极大地促进软件生态繁荣和快速发展，但也可能导致发展方向超出原厂商控制。例如，Visual Studio Code 就限制其分支版本使用官方插件市场，以防范基于自身技术成长起来的强大竞争对手对自身市场的影响。

此外，还有一些公司尝试采用带有其他限制的许可模式，例如，SSPL、Commons Clause 或 BSL 等“源代码可用（Source Available）”，但不完全符合 OSI 开源定义的许可证。这类许可通常保留代码可见性，但限制特定用途（如禁止大型云服务商将其直接作为托管服务盈利）。这更像是一种防御性策略，旨在保护自身收益不被云巨头侵蚀，而非成熟的商业模式。然而，这类策略往往效果不佳，既易引发社区强烈反对，又难以实现商业目标。例如，Redis Labs 曾添加 Commons Clause 限制销售，后因社区反对而撤回；MariaDB 的 MaxScale 也经历了从 GPL 改为商业源许可，3 年后又改回 GPL 的反复。事实证明，这种改变许可证的策略往往得不偿失。

20.4 各类混合模式的商业动机与技术考量

近年来，混合模式与多元化策略成为趋势。一方面，更多的厂商选择通过开源基金会对开源项目进行长期持续的支持；另一方面，开源项目与社区也在不断创新，GitHub Sponsors、Open Collective、Patreon 等平台帮助个人开发者获得持续收入，一些项目还在尝试通过区块链代币经济或开源孵化器获得资金支持（仍处于早期）。总体趋势是各种模式互补并用，根据项目特点灵活组合以提高可持续性。

（1）吸引用户与社区规模：开源核心模式鼓励用户免费试用并参与社区，从而迅速扩大用户基础并获得口碑。这有利于提高产品成熟度和促进社区贡献。例如，Oracle MySQL 通过 GPL 免费版吸引开发者，然后向大规模部署或商业用途收取费用。技术上需将商业功能与开源核心分离，可采用插件架构或许可校验机制，确保用户能清晰区分免费/付费功能范围。

（2）研发投入回报：许多公司投入大量研发成本，开源方式虽然扩大了影响力，但需要商业模式支撑持续创新。商业授权或专有模块收入可回流研发。例如，Elastic、MongoDB、Confluent 等强调，它们对开源项目投入巨资，而变更许可证或销售企业版是保证继续投资的手段。技术考量上，这要求项目维护者在保留核心价值的同时，区分开源部分与企业部分，经常使用许可证标记、构建脚本或分支管理等方式。

（3）应对云服务商竞争：近年来，云厂商经常以开源项目为基础提供托管服务，抢占市场份额且对原厂回馈有限。为保护自身利益，一些公司采用限制性许可证（如 SSPL、Commons Clause 许可证、Confluent 许可证等），要求将其软件用作 SaaS 服务的厂商必须开源其整套服务代码。Elastic 的 CTO 直言“我们的许可证变更旨在阻止他人直接将我们的产品作为服务提供”。技术上，这涉及在许可证中明确禁止特定用途，以及在产品中加入许可验证或水印。虽然能限制云厂商无偿占用其软件，但会被社区质疑其不再真正开放。

20.5 典型企业的混合策略案例分析

表 20-2 对 Elastic、MongoDB、GitLab、Confluent、Redis Labs 等公司的模式类型进行分析，并列出它们的开源/专有部分划分、许可证演变及市场/社区反应。

表 20-2 企业混合策略分析

公司	模式类型	开源许可（历史）	专有许可	开源/专有部分划分	许可证演变及市场/社区反应
Elastic（Elastic/Nx）	开源核心+限制性许可	2010~2020 年：Apache 2.0 许可证； 2021 年：转为 Elastic 许可证 v1+SSPL； 2024 年：改为 Elastic 许可证 v2+SSPL，新增 AGPL 选项	Elastic 许可证（企业功能） SSPL（源可用）	开源版本提供基础搜索功能，企业版（安全、监控、高级分析等）闭源	2021 年变更后，AWS 推出 OpenSearch 开源分支；社区批评 Elastic “弃开源”；OSI 称其新许可证为“伪开源”。Elastic 称此举为对抗 AWS 的无偿服务；2024 年增列 AGPL 重回 OSI 许可，力图恢复“开源”称谓，但开发者普遍对其动机持怀疑态度
MongoDB	开源核心+双重许可	原：AGPLv3； 2018 年：转为 SSPL（源可用）	企业版（附加模块、安全特性等）闭源 SSPL（社区版）	社区版（核心数据库、驱动等）采用 AGPL/SSPL；企业版为闭源附加功能	2018 年发布 SSPL，要求提供 MongoDB 服务的企业开源其服务代码。社区认为 SSPL 非开源（OSI 未认可），MongoDB 在 2019 年主动从 OSI 撤回 SSPL 审核申请。市场上，AWS 等推出兼容版（如 DocumentDB），社区对 SSPL 争议颇多，但 MongoDB 强调此举为保护投资
GitLab	开源核心+企业版（OpenCore）	CE：MIT 许可证（始终开源）； EE：原为类似开源协议，2014 年改为商业许可	EE 专有许可，需订阅使用	CE（社区版）功能完备，MIT 开源；EE（企业版）加入高级管理、认证等功能需付费	2013 年引入 EE 后，2014 年将 EE 改为商业许可，以减少混淆。GitLab 表示社区版功能充足且持续迭代。社区普遍接受此安排，因为 CE 不受限制且 GitLab 坚持开源开发，GitLab 已合并代码库，主要通过功能差异和许可证来区分 CE/EE

续表

公司	模式类型	开源许可（历史）	专有许可	开源/专有部分划分	许可证演变及市场/社区反应
Confluent	开源基础+限制性许可	Apache 2.0 许可证（Kafka 核心始终开源）； 2018 年：部分组件（REST Proxy、Schema Registry、KSQL、连接器等）改为 Confluent 社区许可证	Confluent 社区许可证（限制性开源可用） Confluent 企业版	Kafka 核心组件仍然是 Apache 2.0 许可证；增强组件（连接器、管理工具等）以 Confluent 许可证发布	2018 年将非核心组件换为 Confluent 社区许可证，禁止将其用于竞品 SaaS。社区反应不一：Kafka 核心不受影响，但一些用户担心限制性许可证的不透明和锁定风险。Confluent 称此举可持续投入免费代码并保护业务；批评者 Joyent CTO Bryan Cantrill 认为，这会导致云厂商另起炉灶，不利于生态
Redis Labs	开源基础+限制性许可	核心：BSD 3-Clause 许可证（始终开源）； 模块：原为 Apache 2.0 许可证，2018 年先改为 Apache + Commons Clause，后改为 RSAL（源可用）； 2025 年 5 月 Redis 8 重新宣布以 AGPLv3 开源	Redis Labs 专有模块 许可证（Commons Clause/RSAL）	Redis 核心功能（缓存、主数据结构等）保持 BSD 开源；Redis Labs 开发的高级模块（RedisSearch、RedisGraph 等）在 Commons Clause/RSAL 许可证下限制使用	2018 年将部分模块许可证从 AGPL/Apache 许可证转为 Commons Clause 许可证，后优化为自创的 RSAL。Redis 强调核心版完全开源，仅模块受限。社区初期对 Commons Clause 许可证表示担忧，但多数接受了过渡到 RSAL 的解释，认为新许可证可保持多数开放性。其他厂商（如 MongoDB、Confluent）也随后调整许可证，说明业界面对云服务竞争的一致性趋势

混合模式的选择应结合产品类型和公司阶段。早期创业公司或初创产品，常倾向于保持更纯粹的开源（以获取社区和客户），成长后可逐步引入混合元素；高技术壁垒的企业软件适合采用开源+商业扩展，而依赖规模服务盈利的产品（如云数据库、企业搜索等）则可能更早尝试许可证创新或 SaaS 模式。各种模式均需平衡“社区贡献”和“商业收益”两个目标，混合模式的优势在于提高灵活性和可持续性，劣势则在于可能引发社区信任危机和法律合规压力。选择适合自身的模式，谨慎规划许可证和商业策略，对开源公司持续发展至关重要。

第 21 问 开源商业模式的本质是什么

开源的商业模式多种多样，我们需要更加深入地理解其本质。任何商业都会追求利润，而开源软件背后的企业，自然也希望获取更大的利润。虽然我们看到了企业向社区发布开源项目，似乎意味着利润的丧失，但事实上却可能帮助企业获取更大的利润。

21.1 一个简单的开源经济学模型

首先，假设一家企业只销售一款软件，而这款软件的用户数量是 x ，每一款软件销售的利润是 y ，那么这家企业的总利润就是 $x \times y$ 。如果企业对外开源自己的产品（或产品的一部分），我们可以猜想：因为开源，这个企业的用户数量会从 x 增长到 x' ，而软件的单位产品利润，会从 y 下降到 y' 。如果这家企业的开源策略合理，则 $x' \times y'$ 的总面积会大于原来的 $x \times y$ 的面积，从而实现利润的增长，如图 21-1 所示。

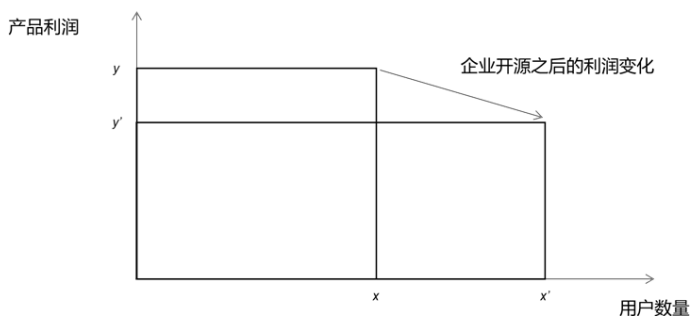


图 21-1 开源经济学模型

这背后蕴含以下几个假设。

- 开源之后，企业的知名度、软件的用户数量都会获得增长。
- 软件的定价取决于买家与卖家之间的知识落差，如果一个企业将自己的软件开源，则意味着知识落差减小，从而导致软件的单品利润下降。

- 合理的开源商业模式就是，开源导致的利润下降能够被用户数量的大幅增长抵消，甚至帮助企业赚得更多。

再考虑以下两种极端的模式。

- 完全闭源，这当然是传统的、始终合理的商业模式，至今依然有很多企业选择这种模式。
- 完全开源，这是“毫不利己、专门利人”的行为，软件的用户数量可能会非常大，但是基本上赚不到钱。

所以，任何合理的开源商业模式都是在完全闭源与完全开源之间，选择能够使企业整体利润最大化的方案。

企业开源之后的商业生态如图 21-2 所示。

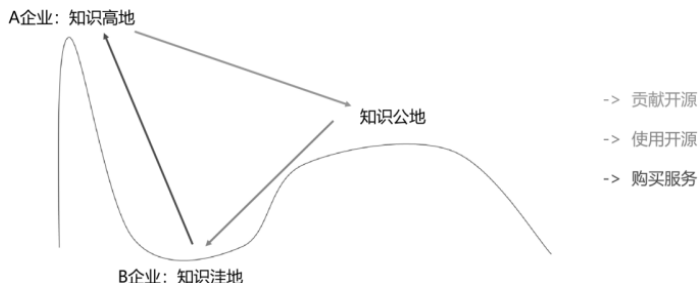


图 21-2 企业开源之后的商业生态

A 企业通过向知识公地开源自己的软件，进而吸引 B 企业的注意力。在简单的使用场景下，B 企业可以直接使用开源软件；在复杂的使用场景下，B 企业知道可以通过向 A 企业购买后续服务或其提供的商业版本，从而满足自己的需求。

21.2 影响开源商业模式整体利润的因素

一、许可证的选择

一般而言，越宽松的许可证，越是容易获得企业的选择与使用；越严格的许可证（类似 GPL、AGPL），越容易导致企业选择时产生更多的顾虑。

但是，越宽松的许可证，也越容易导致开源软件产生更多分支版本（fork），形成更多的竞争对手，从而降低软件的利润。越严格的许可证，越有可能形成紧密协作的技术共同体，从而加快创新速度，从而在与闭源软件的竞争中快速胜出。

二、开放程度与开放策略

软件的开放程度是另一项重要影响因素，具体而言，该因素包括开放代码的范围（仅开放核心部分代码或完整开放全部代码）、代码与相关文档的完整度与细致程度，以及版本发布周期。例如，相对于商业版本的发布周期，开源版本的发布始终落后 3~6 个月，这也是一种常见的策略。

还有一些极端的情况。例如，假开源，其本质也是以最低程度的开放（几乎为 0）来换取尽可能多的用户增长。这样的做法一旦曝光，会导致用户与市场口碑的

急剧下降，甚至造成公关危机。

企业在开源部分软件之后，就会进入一个更复杂的动态竞争状态。竞争对手也同样会使用这部分开源代码，会增强自身产品的竞争力。如何确保企业自己的竞争优势，关键还是要看企业如何吸引并留住高水平的人才，从而能在内部源源不断地创造更新的、更有价值的知识，维持自身的定价权（利润）。

三、社区运营能力

当一个企业选择开源自己的软件，并不意味着它的软件会有更多的人来使用，也不意味着品牌形象会得到提升，更不等于整体利润会大幅增长。传统的企业都会做客户关系管理（CRM），而在企业开源之后，则需要考虑开发者关系（DevRel）。

与传统的客户关系不同，开发者关系是企业与开发者社区之间建立的深层次关系，它需要企业从技术层面真正走进开发者的世界，理解他们的需求和痛点。成功的开发者关系策略能够将开源项目转化为企业的核心竞争力。

要想更好地运营开源社区，企业需要考虑很多相关的事项，其中最为重要的，就是建立清晰的开源战略（见第 28 问），即识别企业的核心竞争力，针对不同的技术领域，选择完全自研、引入开源、回馈社区、主动开源等不同的策略，并在整个企业中贯彻执行。

要想吸引企业之外的开发者，设计良好的贡献机制并建立开发者激励机制非常重要（见第 32 问、第 35 问、第 37 问、第 40 问）。实现以上目标，最关键的一点在于：理解开源贡献者的动机与诉求，更进一步则需要理解开源文化，并在自己的企业内部推广开源文化（参见第 30 问）。这样才能使企业的开源社区吸引外部人才，留住外部人才。

组织社区活动，建立有效的沟通渠道是保障社区活跃度的长期工作（见第 38 问、第 41 问）。最简单的逻辑就是：有吸引力的社区能够健康生长，这样企业才能事半功倍。

第 14 问中已经介绍过开源项目的健康指标，这意味着数据驱动的社区管理是社区运营能力的基本功。以上的全部努力，本质上就是希望能够实现“杠杆效应”。通过优秀的社区运营，使得企业投入的人力物力，能够撬动更多的外部力量自动参与，加入共建。撬动的效果越好，则企业的开源节支就能做得越好。

21.3 一种打击竞争对手的开源商业模式

前面讨论的开源商业模式，都是“正面进攻”，但是商战中的招数还有更多侧面的玩法。按照前面的分析，企业对外开源，意味着降低了某一个部分的知识落差，从而导致利润减少。但是，如果企业开源的项目，并不是自己的优势技术，而是竞争对手的优势技术呢？这样就能够有效地减少竞争对手的利润空间，从而拓展自身市场。

比较典型的案例就是 IBM 公司在 Linux 操作系统领域的大量投入，一方面能够打击微软的 Windows 操作系统，另一方面也能促进自身的服务器销售，可谓一举两得。

另一类常见的打击竞争对手的做法出现在多家企业基于开源项目的竞合生态中。有多家企业都围绕同一个开源项目，开发自己的商业化的版本，但同时又贡献一部分代码到开源项目之中。这时，生态企业的选择往往并不贡献自己有技术竞争力的那部分代码模块，而是贡献同一生态中其他企业较有竞争力的模块。虽然自己的这部分技术并不是最好的，但至少也是某种开源贡献（对外提升品牌），还能够“卷”竞争对手，迫使对方加大开源力度。当然，对方也不会坐以待毙，而是选择开源己方更加擅长的技术领域，大家就都“卷起来”了。

以上的开源生态，通过开放式的竞争迫使对手也变得更开放，对整个市场确实是整体利好的事情，大家也都喜闻乐见。

第 22 问 服务与支持如何成为 开源软件的收入来源

根据第 20 问的介绍，开源商业企业可以通过支持与服务（订阅）模式获取收入。服务与支持可以被视作在软件销售之外获得收入。很多开源企业往往会同时运用两种甚至多种不同的商业模式来使自己的收益最大化。

22.1 开源软件服务与支持的具体形式

要想获得软件销售之外的收入，关键是收费服务项目的设置，以及与开源软件本身的关联。

（1）技术支持服务。由于开源软件可以在互联网上自由传播，因此软件客户的分布往往是全球性的。一般而言，企业提供的开源技术支持服务，默认都是远程的（提供 7×24 小时或 5×8 小时的电话/邮件支持）。也有一些企业会提供线下技术支持，包括现场问题处理、定期巡检、咨询建议等。另外还有一种情况，在网络安全、数据安全、严重技术故障等问题爆发时，客户可能需要快速故障响应、紧急安全补丁等，尽快解决问题。

（2）维护与升级服务。一款健康的开源软件，往往能够做到定期发布稳定的新版本，并且不断地增加业界需要的新功能。采用这款开源软件的企业自然可以选择自行维护升级，但是可能由于缺乏专业人才，出于稳妥的考虑，甲方企业会购买持续的维护与升级服务。这也是“订阅制”的核心逻辑。我花钱买个心安，你定期帮我升级，有漏洞帮我修复，甚至是健康巡检、性能调优、参数优化，你也帮我完成。

（3）托管服务。越来越多的开源企业（尤其是创业企业），在提供开源版本之外，还会提供 SaaS（托管服务）。但是，新的争议也由此产生。大型云计算厂商（如 AWS）会提供类似的托管服务，基于某一个开源版本，再加上自身的技术优势进行定制，从而侵占了“原厂”的利润。这自然引发了原厂的不满，甚至决定修改

许可证，以排除竞争。这是近年来开源生态中最为常见的争议。

(4) 培训与咨询服务。一个比较有追求的企业，在自身使用的关键开源软件方面，可能不愿意长期购买外部服务，而是希望逐步培养自己的开发人员，具备自我维护、长期运维的能力。于是，他们会选择购买原厂的培训与咨询服务，从而帮助自己的员工快速掌握。当然，还有一些常见的案例，甲方会觉得乙方派来培训的工程师特别优秀，甚至会将其“挖过来”。这就要看各自的博弈情况了。

(5) 定制与开发服务。根据客户需求进行开源软件的定制开发，以满足特定功能需求。在选用开源软件的企业自身研发能力不足的情况下，可能会直接找到“原厂/开源社区”，要求提供定制开发服务。这时可能会出现几种情况：①开源软件背后的企业原本就有这方面的打算，顺理成章就可以接这样的生意；②开源社区的生态比较繁荣，有一家甚至多家专门提供二次开发定制服务的企业，可以供甲方选择；（例如，基于 WordPress 这个开源项目，多个厂家就提供网站定制开发服务）；③没有乙方愿意承接合作，这也是特定领域开源生态不成熟的体现。

22.2 开源软件服务与支持的收入情况

据市场研究机构统计，2022 年，全球开源服务市场规模达 250.3 亿美元，预计从 2023 年到 2030 年的复合年增长率（CAGR）将达到 16.9%。具体公司层面，红帽公司 2019 年培训和咨询收入占比约 12%；Elastic 公司 2023 财年订阅收入为 9.124 亿美元，其中云服务占 35.9%，专业咨询额外贡献 0.872 亿美元；Confluent 公司 2024 财年订阅收入为 9.22 亿美元，同比增长 26%，其云服务收入为 4.92 亿美元（同比增长 41%），显示出云托管服务增长尤为迅猛。这些数据表明，虽然各公司业务结构不同，但订阅+服务模式在其收入构成中占主导地位，并呈现稳定增长趋势。下面简单介绍几个企业的开源商业服务案例。

(1) 红帽公司。红帽公司主要以订阅制收费。例如，针对企业客户以每台服务器或每核心为计费单位收取年度订阅费，提供不同级别（标准/高级）的支持服务和培训。客户可远程获得 7×24 小时支持，需现场运维时则另行付费。2019 财年红帽公司全年营收 34 亿美元，其中培训和服务收入约为 4.13 亿美元（约占总营收 12%）。

(2) Elastic 公司。Elastic 公司基于开源搜索引擎 Elasticsearch，但采用开放核心模式。用户可免费使用核心功能，公司则出售企业版订阅、技术支持和托管服务。2023 财年，Elastic 公司订阅收入约为 9.124 亿美元（同比增长 16%），其中 Elastic Cloud 托管服务收入为 3.275 亿美元（占订阅收入 35.9%）；企业版支持合同收入为 2.146 亿美元；专业服务（咨询、培训）收入约为 8720 万美元。由此可见，Elastic 公司的收入主要来自软件订阅和云服务，另有少量咨询服务营收。

(3) Confluent 公司。Confluent 公司提供流数据平台服务。其商业模式以订阅为主，并提供云服务。2024 财年，Confluent 公司总订阅收入为 9.22 亿美元，同比增长 26%；其中 Confluent Cloud 云服务收入为 4.92 亿美元，同比增长 41%。仅 2024 年第四季度，订阅收入为 2.51 亿美元（同比增长 24%）、云收入为 1.38 亿美

元（同比增长 38%）。公司还提供企业支持和专业咨询，但占比相对较小。

（4）MongoDB 公司。根据 MongoDB 公司发布的 2024 财年第四季度财报（截至 2025 年 1 月 31 日），2025 财年第四季度 MongoDB 公司的主要收入来源包括 MongoDB Atlas（托管云服务）收入为 3.89 亿美元，占季度总营收的 71%；其他订阅收入（包括企业版许可）为 1.49 亿美元，占季度总营收的 27%；服务收入为 1740 万美元，占季度总营收的 3%。

第 23 问 如何建立企业开源合规治理框架

开源软件的合规性管理，主要涉及在使用、分发和管理开源软件时，遵循相关法律法规及其许可协议的过程。这里主要讨论从企业视角出发的开源合规管理，根据开源合规的场景划分，可以分为使用开源、回馈开源和对外开源 3 类。按照 T/CESA 1270.2—2023《信息技术 开源治理 第 2 部分：企业治理评估模型》，企业的开源治理又可以分为组织、制度和基础设施 3 个方面。

23.1 企业开源面临的合规性挑战

总体来说，合规就是：确保做了该做的事情，保证不做不该做的事情。而哪些事情该做，哪些事情不该做，则主要依据各国的法律法规，以及各个不同的许可证中的规定。而企业要做的事情粗略地划分有两件：一是明确自己的合规义务，二是确保能够遵循合规义务。但说起来简单的事情，做起来却往往异常困难。

一、企业使用开源的合规性挑战

企业使用开源的情况，还可以分为两大类：一类是完全在企业内部使用（自用），另一类是企业在对销售软硬件或提供的在线服务中用到了开源（商用）。一般来说，自用的合规风险比较低，大多数合规挑战还是来源于开源（商用）。但是，由于我们很难排除原本自用的软件，考虑商用销售的可能性，因此企业的开源使用治理，最好一开始就尽量掌握自用情况。

另一个使用开源中常见的误区，就是“以为能够下载到源代码的就是开源”。事实上，必须对企业内部的软件开发人员进行最基本的开源培训之一就是不要下载没有许可证的软件，更不要在企业内部使用，这是一条必须明确的红线。没有授权的源代码，就只是给你看的，不是给你“拿来用的”。

还有一种开发过程中的陋习，企业必须严厉禁止，就是代码的“片段引用”。未经培训的开发者可能会在网上随意下载开源代码，直接将别人代码中的一部分复制到自己的项目中，这样通常会造成合规问题，可能会给企业带来很多麻烦。

如果开源软件本身带有许可证，那也需要看清楚到底是什么许可证。因为这意味着后续企业需要考虑不同程度的合规义务，而合规义务是否繁重则与企业的研发成本息息相关。如果引入了类似于 GPL 的开源软件，导致企业最后必须开源自己

的机密代码（以满足合规要求），则可能给企业带来知识产权方面的重大损失。

更加复杂的情况是，软件依赖导致的“许可证丛林”。明明只引入了一个开源软件，但这个开源软件却层层依赖了其他一堆开源软件（组件），而这些开源软件又各自选择了不同的许可证。最糟糕的情况是，这些许可证还存在兼容性问题，不能一起使用。最开始被引入的开源软件的作者可能未在意，但企业在使用，尤其是商用时，必须格外谨慎。

另外一种比较容易忽略的情况是，企业采购了商业软件，但该商业软件本身包含开源软件。从提供商业软件的企业角度来说，有义务提交完整的开源软件使用说明，并证明自己已经尽到了开源合规义务。否则，乙方的不负责任可能会给甲方带来风险。

二、企业回馈开源的合规性挑战

企业在内部使用开源软件的过程中，可能会自行修改以符合自己的需要。是否需要将这些修改回馈到开源社区，则是一个需要考虑的问题。企业在面对这个问题时，主要应根据以下方面进行考虑。

从法律与合规的维度考虑，某些许可证（如 GPL、AGPL）明确要求发布修改后的衍生作品必须在同等条件下开源；其他许可证（如 MIT 许可证、Apache-2.0 许可证）则仅建议保留声明，回馈与否更多取决于企业策略。修改开源软件时加入的代码，可能涉及第三方专利或内部专利，一旦回馈到社区，就可能无意中放弃对某些专利的追索权，所以应该慎重考虑。

从业务与战略的维度考虑，则需要在企业技术竞争力与企业技术品牌形象之间，做出艰难的权衡。

从技术与运维的成本考虑，则是一个复杂却重要的判断：到底是在内部始终维护一个自己的分支，定时合并来自外部最新版本的代码，还是组织一个内部的维护团队，密切地与外部社区合作，不断提交回馈代码？哪一种的长期成本更低？

虽然这里主要聚焦于合规问题，但企业的思考路径应该是：首先从合规视角判断是否必须回馈社区，其次是从业务战略与技术运维的成本视角，判断是否有必要回馈社区，最后如果是决定回馈社区，还有哪些需要面对的合规性挑战。

一旦企业决定回馈开源社区，则需要考虑两个问题：一是完善贡献流程，如果缺乏标准化的贡献准入流程，开发者直接向社区提交代码，可能泄露企业敏感信息或违反公司政策；二是明确知识产权归属，在回馈代码前，需明确员工贡献代码的知识产权归属，否则可能引发著作权争议。

三、企业主动开源的合规性挑战

企业开源希望能够换来更大的商业利益，这在第 21 问已有论述。而开源对于企业的挑战就在于：尚未看到利益的增长，损失却已经实实在在地发生了。

首先，企业肯定不希望发生“原本不想开源的代码泄露出去”这样的悲剧。所以由此而来的挑战在于：如何制定内部规则，确保不该开源的代码，不会被开源。要通过制度与工具，确保这样的规则能够被妥善执行。还有相关的知识产权审查，需要确保开源的代码所关联的专利、商标等知识产权，被合理声明或已经受到妥善的保护。

其次，开源之后，因许可证问题带来的各种麻烦：包括选择了不合适的许可证，导致企业的利益受损；在开源的代码中包含了其他第三方代码，从而引入了更多的第三方授权冲突；修改了第三方代码后应遵循的合规义务，由于开源后暴露在公众视野，甚至会带来公关危机。

最后，开源合规是一个长期持续的工作，需要针对后续不断提交的代码，进行代码审计、许可证审核、安全扫描。若无专业人员与自动化工具支持，成本和周期难以控制。企业如果没有一个合理的成本预算，可能会在随后遇到的麻烦面前束手无策。

23.2 企业开源合规治理框架

按照《信息技术 开源治理 第2部分：企业治理评估模型》的描述，企业应从组织架构、制度政策、基础设施三方面构建合规体系，如图 23-1 所示。相关的论述建议参考盈科律师事务所编写的《开源软件合规与法律指南》一书。另外，中伦律师事务所发布的“开源合规白皮书”和 OSPO 联盟（OSPO Alliance）发布的“开源治理良策”也值得参考。在第 18 问介绍了企业内部的开源治理，企业开源治理涉及的范畴如图 23-2 所示。本节主要讨论企业内部的合规治理工作，但是采取安全措施、回馈社区的做法，甚至包括整个开源战略，在组织架构、规章制度、基础设施等方面也可以复用一大部分。

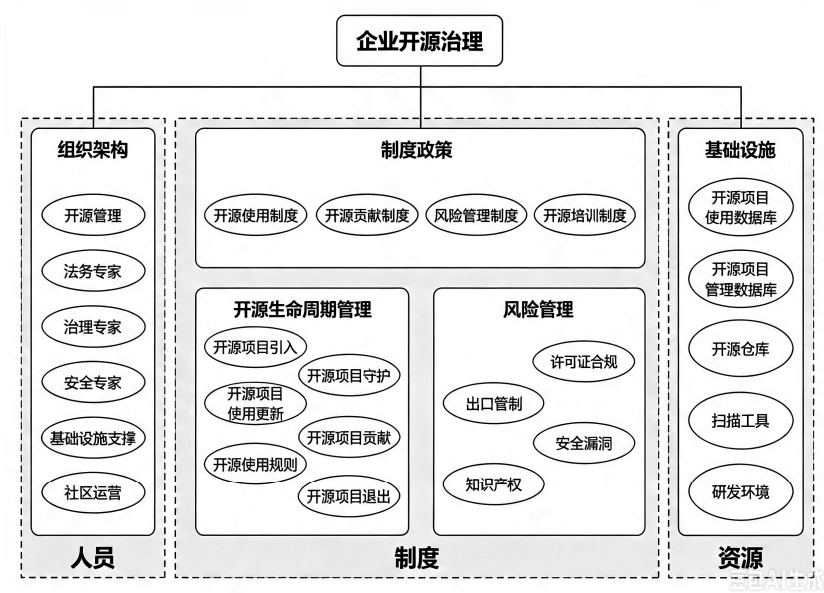


图 23-1 企业开源治理评估模型（引用自《信息技术 开源治理 第2部分：企业治理评估模型》）

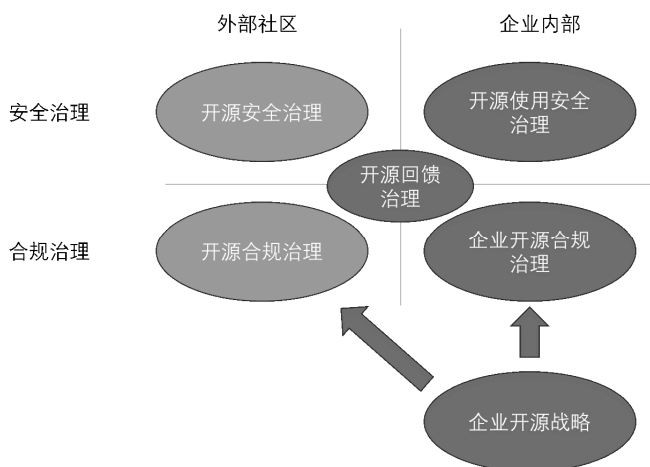


图 23-2 企业开源治理涉及的范畴

一、组织架构与人员能力

首先需要明确的是，企业内部的开源治理是一个需要跨部门协作才能完成的复杂任务，这样的组织现在往往会被称为开源项目办公室（Open Source Program Office, OSPO）。但是，在企业投入开源治理的早期，往往会从各种不同的角度来培养开源治理的综合能力。因此，有些开源项目办公室是从企业的法务部生长出来的，有些是从企业的安全部门生长出来的，还有些是从企业的研发工程与效能团队生长出来的。最开始，这可能还只是一个虚拟团队，小组的成员本身有自己所在部门的各种本职工作，直到企业深刻意识到开源治理的重要性，才会逐步推动这样的组织实体化。（如何在企业内部建立开源项目办公室详见第 30 问。）

二、规章制度与管理框架

在制定明确的规章制度与管理框架之前，首先需要明确的是治理的目标。

在开源软件治理中，过程透明是第一要务。企业应该致力于建立完整的开源软件使用与修改记录系统，通过中心仓库实现透明管理，确保所有开源组件在公司层面可视可控。这不仅作为各项管理活动的基础，也使团队能够全面掌握产品中的开源成分。

优选引入机制确保企业能够持续吸纳高质量的开源软件。通过对标社区数据，严格筛选符合公司质量标准的组件，支持产品履行开源义务并有效应对漏洞。同时，企业也应限制使用衰退期软件，推动版本归一化管理，降低维护成本，并积极推荐更优质的替代方案，从而增强产品竞争力。

来源可信是企业软件的核心安全保障。应该确保每一个开源组件都能追溯至对应社区，并通过恶意软件扫描与哈希值/签名校验实现无告警。这不仅帮助产品在客户面前证明软件成分的纯净性，还建立了坚实的防御机制，抵御潜在的软件供应链攻击，保护产品免受恶意代码侵害。

合法合规应该是企业的基本原则，遵循国家及地区法律法规，履行开源许可证规定的各项义务，包括使用声明和代码开源义务，有效规避因违反开源协议可能带

来的法律风险。

这里也附带提及一些开源安全治理的目标：漏洞感知与修复确保产品全生命周期的安全性。应该建立高效的漏洞响应机制，实现小时级感知、分钟级定位，并按照服务水平目标完成修复。这一体系确保企业能够迅速应对新出现的安全威胁，满足客户对产品安全性的期望，同时维护产品在市场中的可靠声誉。

在明确了开源治理的目标之后，企业还应该建立一套术语库（以统一企业内部的各种术语）和各种规则制度。

（1）管理规范。管理规范是开源治理的基础性文件，定义了组织如何管理开源软件的总体原则和要求。“开源软件管理规定”建立全局治理框架，“开源软件选型规范”确保引入高质量组件，“使用与维护规范”规范日常操作，“生命周期管理规则”确保全周期管控，“安全与漏洞管理规则”保障安全响应，而“开源所有者职责分工”明确各角色责任。这些规范共同构成了开源治理的政策基础。

（2）流程文档。流程文档将管理规范具体化为可执行的工作流程。“开源软件选型与引入流程”指导如何评估和引入新组件，“开源软件维护流程”确保组件得到持续更新和支持，“开源软件社区回馈流程”规范组织如何回馈开源社区。这3个流程文档涵盖开源软件从引入到回馈的完整生命周期，确保各环节有章可循。

（3）工程规范。工程规范提供了技术层面的具体要求，确保开源治理能够落地实施。“开源中心仓技术规范”和“开源制品工程规范”确保组件存储和使用的标准化，“开源定制修改相关命名规范”保证修改的可追溯性，“门禁服务规则与规范”建立质量控制机制，“开源软件数据定义及质量要求”设定质量基准。这些规范使开源治理从理念转变为可实施的工程实践。

（4）操作指导。操作指导为具体实施人员提供详细指引。“操作全景图与流程指导”提供整体视图，“开源软件分类与质量策略指导”帮助实施差异化管理，“对外开源自检流程指导”确保合规性，“开源软件漏洞与风险评估指导”支持安全管控。这些指导文档使治理要求变得可操作，确保一线人员能够准确理解和执行各项规定。

这套完整的开源治理规章制度体系，从政策到实施，从管理到技术，形成了一个闭环的治理框架。通过这些文档的系统实施，组织可以实现开源软件的透明管理、优质引入、来源可信、合法合规及高效的漏洞管理，从而提升产品质量，降低安全与法律风险。

三、基础设施与IT工具

企业级的开源治理需要企业级的基础设施与IT工具，这对企业的研发工程管理提出了更高要求。我们需要理解研发工具的集成化、开放平台化与数字化的发展趋势，这些趋势正在重塑企业开源治理的实践。

（1）集成化趋势体现为工具链的无缝衔接和功能整合。现代开源治理工具不再是孤立的点状解决方案，而是相互连接的生态系统。通过建立公司级的代码托管平台、构建与部署平台以及开源及第三方资产管理平台，实现端到端的组件分析、漏洞监测，再到合规检查，各工具间实现数据共享与流程联动。这种集成使开源组件的全生命周期管理成为可能，企业能够从引入、使用到维护、更新，形成完整的可

视化管理链条，大幅提高治理效率并减少人为错误。

(2) 开放平台化代表了工具架构的根本转变。企业级开源治理平台正从封闭的专有系统向开放、可扩展的生态转型。这类平台提供核心功能和 API，允许企业根据自身需求定制和扩展功能，同时与现有的 DevOps 工具链无缝集成。开放平台能够更好地适应不同企业的治理模式，也便于引入第三方专业工具，如 OSS Review Toolkit、FOSSA 或 Black Duck 等，构建最适合企业的组合式解决方案。

(3) 数字化则是开源治理进入数据驱动时代的标志。通过资产数字化与数字资产化，借助自动化扫描、机器学习和大数据分析，企业能够实时监测开源组件的健康状况、使用情况和潜在风险。数字化工具可以自动识别产品中的开源组件、追踪其版本和许可证，甚至预测可能出现的安全漏洞。这种基于数据的治理方式使决策更加精准，响应更加迅速，特别是在漏洞管理方面，实现了“小时级感知，分钟级定位”的目标。有了数据，才有智能化的可能，但由于数字化本身的进展缓慢，智能化也只能局限于非常有限的范围之内，其余的可能性只能作为美好想象的题材。

企业级的研发工具整体架构如图 23-3 所示。

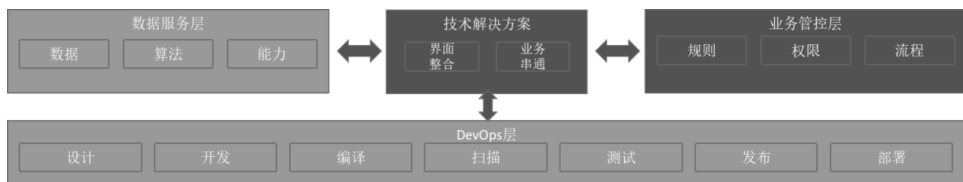


图 23-3 企业级的研发工具整体架构

企业在加强研发工程管理的过程中，应该特别关注工具与流程的协同优化。最佳的开源治理不仅仅是部署先进工具，还包括将工具融入研发流程，使开源治理成为开发团队日常工作的自然延伸，而非额外负担。只有技术和流程双管齐下，企业才能建立真正有效的开源治理体系，在保障安全和合规的同时，充分发挥开源软件的创新价值。

企业内部的开源治理是一个涉及人员、规章制度与 IT 工具，跨越多个不同知识领域的复杂系统工程。要建立合格的开源治理框架，需要企业投入持续不懈的努力。如果企业不能深刻认识到开源为企业创造的价值，是很难坚持投入的。如果企业内部的 OSPO 人员陷入烦琐且徒劳的自我证明（对企业的价值），则可能难以持续有效地开展工作。

第 24 问 开源商业化有哪些成功的案例

24.1 如何选择开源商业化的成功案例

在开始案例分析之前，需要回答的问题是如何选择案例？开源项目、开源企业

或者大型企业的开源战略，都是值得分析的案例素材。我们需要区分项目级案例与企业级案例。虽然开源项目的案例研究也非常重要，但是从商业化的角度来看，还是以企业为单位进行研究更加有价值。因为开源商业化的本质是企业战略行为，需要分析公司层面的资源整合、生态构建与盈利闭环。项目级案例虽能反映技术成功与社区成功，但难以体现市场验证、规模化变现和可持续发展能力。

另一个选择的难题是“如何衡量是否成功”。毕竟成功的开源项目、开源社区、开源战略与（基于开源的）商业收益，都有不同的衡量标准，也需要不同的定义。如果从最初的目标出发，我们应该研究的是：达成企业最初商业目标的开源战略与相关执行情况。但是这样的案例太难筛选了。一方面是缺乏资料，企业最初有没有明确的商业目标，可能并未披露，也可能他们当初都没有想清楚；另一方面是难以判断，企业内部对于其开源战略成功与否的总结可能与外部的评价都不一致。

一个选项来自《2024 中国开源年度报告》中的全球企业排行榜，排名前 10 的企业是微软、华为、谷歌、亚马逊、Meta、红帽、Mozilla、阿里巴巴、Elastic 和 IBM。笔者最终选择美国的两个案例——红帽和谷歌，以及中国的两个案例——PingCAP 和华为，如表 24-1 所示。之所以这样选择，一是因为 10 家企业实在太多，二是因为希望平衡介绍中美两国的成功案例。当然，OpenRank 指标的排序是否能够代表企业的商业成功也很值得商榷。

表 24-1 开源商业化成功案例与选择理由

地域	案例	选择理由	特征
美国	红帽	订阅模式鼻祖，完整开源商业化闭环	以开源为核心业务的企业
美国	谷歌	开源生态构建者，广告+云服务多元变现	大企业执行开源战略的代表
中国	PingCAP	开源数据库全球化标杆，社区驱动型商业化	以开源为核心业务的企业
中国	华为	从封闭到开源的战略转型代表	大企业执行开源战略的代表

24.2 红帽：开源商业化的范式革新

红帽是全球领先的企业级开源解决方案供应商，成立于 1993 年，总部位于美国北卡罗来纳州罗利市。作为标准普尔 500 指数成员，公司拥有超过 2 万名员工和 100 多家分公司，覆盖全球市场。其核心业务涵盖操作系统（RHEL）、云计算（OpenShift）、存储（Ceph Storage）、中间件（JBoss）和自动化工具（Ansible）等领域。2018 年，IBM 以 340 亿美元收购红帽，进一步强化了其在混合云和开源生态中的战略地位。

作为开源商业化领域的“教科书式”案例，红帽开创了“开源产品+订阅服务”的商业模式范式。自 1993 年成立以来，该公司通过提供企业级支持、安全补丁和系统维护服务，将免费的开源技术转化为付费的商业产品。这一模式成功解决了企业用户在生产环境中使用开源软件的后顾之忧。红帽在 2019 年经营性现金流

为 10.1 亿美元，递延收入占比显著，反映出订阅模式的稳定性。从收入的整体结构来看，订阅服务收入占据主导地位（2019 财年占比 87.7%），其次是咨询与培训（12.3%）。2019 财年营收 33.62 亿美元，同比增长 15.1%，毛利率稳定在 85%。

红帽的商业模式以“开源社区+企业服务”为核心，形成差异化竞争力。

- 免费软件，付费服务：软件代码开源免费，收入依赖订阅制技术支持。例如，RHEL 用户需订阅以获得安全更新、兼容性认证和 7×24 小时支持。
- 技术整合路径：通过“项目—集成—平台”3 个阶段，将社区创新（如 Linux 内核、Kubernetes）转化为企业级产品。例如，OpenShift 整合容器、自动化和云原生技术。
- 混合云定位：成为企业私有云与公有云之间的连接桥梁。
- 生态中立性：保持与多云计算提供商的兼容性，避免被厂商锁定。

红帽的成功表明，开源并非反商业，而是构建了新的价值创造逻辑。其模式精髓在于找准了企业用户的痛点——生产环境中的稳定性与安全性需求，并通过专业服务满足这些需求。这一范式为后续开源企业的商业化提供了基础参考框架。

24.3 谷歌：开源生态的流量霸权

谷歌是全球领先的科技公司，成立于 1998 年，总部位于美国加利福尼亚州山景城。公司拥有超 18 万名员工，业务覆盖全球 150 多个国家。其核心业务涵盖搜索引擎（Google Search）、云计算（Google Cloud）、开源操作系统项目（AOSP）、浏览器（Chrome）、AI 框架（TensorFlow）及大模型（Gemma）等领域。其中，AOSP、Kubernetes、Chrome、TensorFlow 等项目，都是谷歌旗下全球著名的开源项目。

从商业战略的角度来说，Android 是谷歌最成功的开源品牌。AOSP（Android Open Source Project）才是真正的开源项目，Android 并不能称之为开源项目。但是，在普通人的心目中，Android 就是开源的代名词。事实上，谷歌成功实现了一套“开放核心+闭源服务”的商业模式，一方面通过开源，免费授权设备厂商，快速地拓展市场，抢占原本属于 iOS 的市场份额。另一方面又通过 GMS（Google Mobile Service），提供闭源增值服务，牢牢把控生态位，赚取预装授权费+广告分成。据报道，2016 年，Android 为谷歌带来了 310 亿美元的年收入，其中 220 亿美元为净利润。

在浏览器领域，谷歌从 2005 年开始，通过支持 Mozilla Firefox 开源，每年支付 Mozilla 约 4 亿至 4.5 亿美元，以确保谷歌作为 Firefox 的默认搜索引擎，从而实现流量闭环。但是，谷歌最终决定自行开发一款浏览器（Chrome），随后逐步赶上并超过了 Firefox 的市场份额，成为浏览器领域的最后赢家（见图 24-1）。这背后的逻辑依然是流量与广告收入。

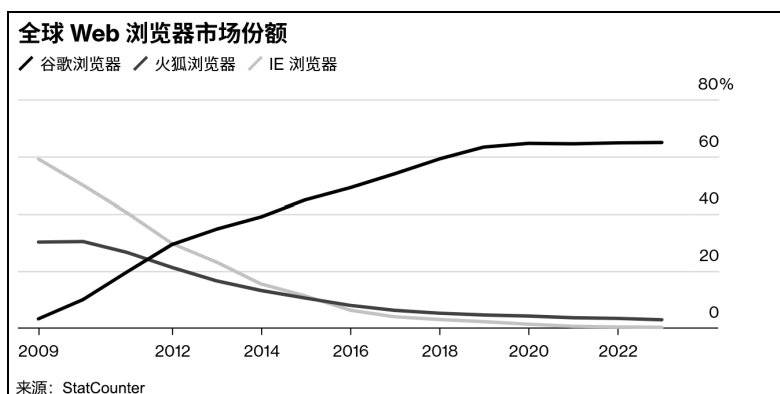


图 24-1 浏览器市场占有率变化 (来源: StatCounter)

在云计算领域，谷歌开源的项目同样出色，但是却未能获得类似于操作系统或浏览器那样的成功。根据 Gartner 数据，2019 年谷歌云的全球市场份额仅为 5.3%，而 AWS 和 Azure 分别为 45% 和 17.9%，即使到了 2025 年，谷歌云的市场份额也仅为 10%~12%，其中的原因也相当耐人寻味。有相当多的分析指出：谷歌的工程师文化主导，忽视企业需求痛点；ToB 服务能力结构性缺失是最主要的原因。另外，关键收购错失（GitHub、红帽分别被微软和 IBM 收购），渠道建设滞后也是关键的原因。

总之，企业选择开源战略，并不能够确保必然获得成功，即使是谷歌这样深谙开源、擅长开源模式的大厂。

24.4 PingCAP：开源数据库的全球化突围

PingCAP（平凯星辰）成立于 2015 年，是全球领先的企业级开源分布式数据库厂商。其使命是通过技术赋能开发者和企业，推动数据价值的最大化与数字化转型。产品体系以 TiDB 为核心，覆盖 OLTP（在线事务处理）、OLAP（在线分析处理）及 HTAP（混合事务/分析处理）场景，支持云原生部署和 MySQL 兼容生态。在全球 9 个分支机构（中国、美国、日本等），服务 20 多个国家的 3000 多家企业，包括金融（中国银行、平安人寿）、互联网（京东、小米）、Web3（NFTScan）等关键领域。

与红帽的服务订阅模式不同，PingCAP 探索出了“开源核心+云服务+生态增值”的复合商业模式。2023 年营收 3500 万美元（同比增长 94.4%），中国市场增速 100%。海外市场收入占比超 50%，成为中国开源软件领域营收最高的企业。2024 年估值达 30 亿美元，成为中国最大的开源独角兽企业。

PingCAP 的成功首先建立在强大的开源社区运营基础上。截至 2024 年底，TiDB 在 GitHub 上获得超过 38000 个星，汇聚全球近 900 名代码贡献者。这种社区驱动的开发模式不仅加速了产品迭代，也形成了天然的客户教育渠道。PingCAP 的联合创始人兼 CTO 黄东旭将开源社区视为“最佳获客引擎”——技术团队通过社区讨论即可了解用户需求，而产品优劣也在开发者群体中形成口碑传播，大幅降

低了传统企业软件销售的高昂获客成本。

PingCAP 的另一个战略亮点是国际化与本土市场的双轨并行。公司早期以中国互联网企业为目标客户，如游戏公司盖娅互娱成为其首家付费客户。这些互联网业务场景具有高并发、数据量大的特点，为 TiDB 提供了极佳的产品打磨机会。随着产品成熟，PingCAP 迅速拓展海外市场，目前已在美、欧、日、南亚等地区建立客户群，海外营收甚至超过国内。

PingCAP 通过“开源驱动创新、订阅保障营收、云服务扩展市场”的模式，成功在分布式数据库领域占据一席之地。其核心护城河在于技术领先性与社区生态的正向循环，而未来挑战则集中在全球化市场的深度渗透与传统行业的替代竞争中。

PingCAP 的案例证明，开源是全球基础软件领域最有效的市场进入策略。通过社区驱动的产品迭代和全球化的协作网络，中国开源企业完全有能力在数据库等核心基础软件领域实现技术突破，并将本土创新转化为全球竞争力。其“产品—社区—商业化”的正向循环模型，为新一代开源企业提供了可复制的增长框架。

24.5 华为：从封闭到开放的生态转型

华为公司成立于 1987 年，但是长期以来却非常重视“信息安全”，特别重视内部的知识产权和源代码保护，以防外泄。对于开源的看法与投入，也偏于保守。直到 2008 年，华为才成立了开源能力中心，致力于规范开源软件的使用和管理。根据华为首席开源联络官任旭东所著的《开源心法》介绍，华为的开源可以分为 4 个发展阶段。

- 2009 年之前：专注于开源软件的规范使用，实现从自发到有序的转变。在合规使用的基础上，更关注开源软件的安全性问题。
- 2010~2014 年：以 2010 年加入 Linux 基金会为标志，积极参与开源社区，构建开源战略管理体系。先后加入了 Apache 软件基金会、OpenStack 基金会、Linaro 基金会等重要组织，并于 2014 年正式建立了公司级的开源战略管理体系。
- 2015~2017 年：持续深化开源贡献、构建对外开源管理体系及团队，积极探索主动开源。2015 年，华为作为初创成员参与了 CNCF 的成立；2016 年，华为还主动开源了 LiteOS，并在随后的时间里，不断发起和捐赠了更多的开源项目。
- 2018 年至今：坚持积极贡献，不断优化开源机制，加速重量级开源项目孵化。

除了企业主动从封闭向开放的转变，来自美国的极限施压，也是促使华为变得更加积极拥抱开源的推动因素。

美国对华为制裁的时间如下：2018 年 6 月 7 日，美国国会呼吁谷歌停止与华为合作。2019 年 5 月 15 日，时任美国总统特朗普发布总统令，主要内容为禁止使用敌对国家（Adversary）的信息与通信技术及服务，并授权美国商务部具体执

行。同日，美国商务部及其下属机关“工业与安全局”（BIS）公布执行总统令的具体措施，将华为加入“实体清单”（Entity List）。2019年5月19日，谷歌公司将华为手机从安卓升级列表中删除。同日，谷歌宣布暂停向华为提供GMS，包括Google Play商店、Gmail、Google地图等应用程序和服务。2019年7月16日，美国两党参议员组织推出了5G立法，该立法将华为列入黑名单。

于是，到了2019年8月，在华为开发者大会上，华为正式发布了鸿蒙系统 HarmonyOS。更进一步，2020年6月15日，在多方共同努力之下，由阿里巴巴、百度、华为、浪潮、360、腾讯、招商银行联合发起，开放原子开源基金会在民政部注册成立，成为中国首个致力于开源产业公益事业的非营利性独立法人机构。同年9月10日，华为 OpenHarmony 操作系统开源，并捐赠到开放原子开源基金会。随后开始的，是一段令人惊叹的飞速发展。截至2025年1月31日，社区累计超过8200名贡献者，代码量1.3亿行，生态伙伴发展至401家，共63家成员单位。社区通过技术创新和生态建设，推动了全场景操作系统的构建，覆盖金融、教育、医疗、交通、工业等多个关键行业。2025年5月8日，鸿蒙计算机也正式亮相，预约开始约15个小时，华为商城中 MateBook Fold 非凡大师的预约人数就超过了5万人。

另一个著名的华为开源项目是 openEuler，openEuler 前身为华为于2010年启动的内部研发项目 EulerOS 操作系统，主要用于服务器、存储等 ICT 产品。2019年12月31日，华为将 EulerOS 项目开源，并重新命名为 openEuler，这标志着一个面向多样性计算的全新开源社区的诞生。2021年11月，华为将 openEuler 捐赠到开放原子开源基金会，进入产业共建阶段。如今，openEuler 已经成为全球最具活力的操作系统开源社区之一。到2024年12月，openEuler 的累计装机量突破1000万套，市场份额达36.8%（中国服务器市场），根据最新的（2025年7月）统计数据：openEuler 的用户达到415万个、贡献者2.2万名、单位成员2073家。

针对华为的开源商业案例，我们可以总结如下：务实的经营理念推动华为从封闭走向开放，坚定的战略执行为华为带来了丰硕的开源生态成果。

第25问 如何评估开源商业模式的可行性

评估开源商业模式的可行性是一个系统工程，需要从市场、技术、社区、竞争和法律等多个维度进行综合分析。以下是一个结构化的评估框架，帮助读者判断一个开源的产品是否具备可持续的商业化潜力。

25.1 市场需求：产品是否能够满足特定的市场需求，目标用户是否有足够的付费意愿

在开源生态中，有很多不同类型的开源项目与开源产品，它们都有其存在的价值。但是，用户需求不等于市场需求，更不等于商业需求。首先需要判断自己产品

的目标用户是谁，是不同的开发者（他们需要在自己的 IDE 里装一些满足自己需要的插件），是特定的行业用户（如电商、医疗、金融或者其他行业的特定应用），还是其他产品对其产生的技术性依赖（某一个开源软件在运行时，需要预先安装的其他开源软件包）。更重要的是，使用者与付费决策者可能并不是同一个人，我们经常会遇到实际使用者非常喜欢，但是其所在的企业却并不愿意采购的情况。

还需要分析的是需求的“梯度”。一个领域的需求如果存在某种梯度现象，也就意味着同时存在具有普遍性的初级需求，以及虽然并不普遍，但是用户付费意愿强烈的高级需求。只有在这样的领域，企业才能够规划开源产品，其核心功能满足用户基础需求，同时为增值服务预留空间。

进一步可以细分出一个“开源替代”的场景。在某一个领域存在一个被广泛使用的商业、闭源产品，但是缺少足够好用的开源替代产品，这时用户的需求是明确的，付费意愿（选择更加便宜且开源的产品）是强烈的。进入这样的市场，通常会比较容易找到明确的商业模式。

从产品形态而言，如果一类需求可以通过云服务满足，则适当的收费模式就会显得非常合理。有很多开源产品，一方面允许用户自行下载部署，同时也提供方便好用的 SaaS 云服务，这样的商业模式也非常成熟了。

对于那些被其他开源产品依赖的开源组件，也就是在开源生态中虽然不可或缺，但难以被定价销售的“产品”，就很难为其找到某种可行的商业模式，这也是开源世界的常态。

25.2 技术架构：发展阶段与成熟度

越是新兴的领域，越可能存在快速变动，无论是用户需求、产品呈现还是技术架构，可能都不太稳定。至于具体的项目，也可能处于不同的成熟度阶段。通常可以观察到，在新兴的、快速变动的领域，很难建立稳定有效的开源商业模式。对企业而言，选择完全开放源代码、吸引尽可能广泛的社区关注，最终成为行业事实标准，且在达成该目标前无须考量经济收益，这一路径看似不符合常规逻辑，却可能是唯一具备可行性的发展方向。

在成熟的技术领域，尤其是存在长期发展的闭源商业软件的领域，一个熟悉现有产品的技术大牛或架构师，很有机会实现青出于蓝而胜于蓝的突破。他们会选择一套经过深思熟虑的技术架构，既考虑其稳定性，又考虑其生态扩展性。在这种架构下，基于开放核心，加上插件与增值功能的商业模式，就非常顺理成章。

当然，在成熟的技术领域，也会面临更加艰难的竞争性生态，在 25.4 节中，我们再深入讨论。

25.3 社区状况：活跃度与贡献者画像

首先需要考察的是社区的活跃情况，从考察指标的有效性来看，外部 PR 数量>外部问题数量>fork/star 数量。因为外部 PR 数量可以代表实际贡献者的活跃

度，而 fork/star 数量只能代表用户/爱好者层面的活跃度。在活跃度太低的社区，选择哪种开源商业模式的意义都不大。

同样是开源，可能完全没有外部开发者的参与。这种情况下的开源，就只能被视作一种市场营销的手段。企业虽然无法在社区中获得太多的帮助，但是在商业模式的选择方面，也不必有太多的顾虑。

至于存在外部参与者的情况，又可以分为散户居多、企业居多、少数寡头等各种不同的情况。在选择开源商业模式时，也需要认真考虑上述区别，以及企业自身相对其他外部企业的强弱对比。

- 一家企业主导，外部参与多是个人贡献者，通常在新兴技术领域会出现这样的格局。企业应该尽快发展自身实力，并通过活跃的社区建立足够的生态护城河，选择较低利润却能够更多地回馈外部贡献者的商业模式。
- 一个较强的企业主导，多家稍弱的企业参与，通常会围绕开源项目组成紧密合作的商业联盟。例如，谷歌通过安卓建立的开放手机联盟（Open Handset Alliance）就是非常典型的商业模式组合。
- 企业自身实力不够强，却有一个或几个强大的企业玩家，这是一个非常危险的状况。一个著名的案例就是 Docker 公司虽然像超新星一样快速崛起，但是却被以谷歌为首的一群大玩家“玩弄于股掌之间”，又没有及早抽身、在一个好的价位卖掉自己，最后遗憾收场。

25.4 竞争格局：竞争态势与团队能力

竞争格局同时要看两个方面：对手及能力如何、自身能力如何。展开比较的话，又可以细分为技术创新能力、产品特性积累、用户忠诚度、市场营销能力、管理能力与财务实力等多个方面。

一、看清对手：识别与评估竞争者

要明确主要的竞争对手是谁，包括直接采用类似开源商业模式的玩家，也包括提供替代解决方案的闭源商业公司、云巨头的托管服务，甚至用户自研的可能性。对每个关键对手，要深入分析其核心能力。

- 技术创新：他们的技术路线是否更前沿？研发投入和迭代速度如何？是否有难以逾越的技术壁垒（如独家专利）？
- 产品特性：其开源核心的功能完整性和性能如何？商业版独有的企业级功能（如高可用、安全合规）是否有吸引力？生态系统的丰富度（插件、集成）怎样？
- 用户忠诚度：他们的社区是否活跃健康？用户黏性高吗（体现在用户留存、社区参与度）？付费转化率和续费率如何？口碑和品牌声誉怎么样？
- 市场营销：他们的增长策略有效吗（SEO、社区运营）？品牌影响力和市场触达能力强吗？销售漏斗效率和转化率如何？
- 管理能力：团队是否稳定高效？开源治理模式是否合理？能否有效平衡商业目标与社区利益？

- 财务实力：他们的融资能力和资金储备如何？商业模式是否健康（关键看 CAC 与 LTV 的比率、毛利率）？抗风险能力怎样？

二、认清自己：深度剖析自身实力

在相同的六大维度上，客观、诚实地评估自身的现状，对于选择合适的开源商业模式至关重要。

- 技术创新：我们是否有独特或领先的技术？研发效率和创新能力如何？技术栈是否可持续？
- 产品特性：我们的开源核心是否足够强大以吸引用户？商业版的价值主张是否清晰且刚需？产品生态的建设进展如何？
- 用户忠诚度：我们的社区健康度和活跃度如何？用户对我们的信任和依赖程度有多深？免费到付费的转化路径是否顺畅有效？
- 市场营销：我们的增长引擎是什么？品牌认知度如何？触达和转化目标客户的能力强吗？
- 管理能力：团队是否有能力兼顾社区繁荣与商业成功？决策和执行的效率高吗？是否存在文化或流程上的冲突？
- 财务实力：我们的资金储备和现金流能否支撑商业化的投入？当前的收入模式和财务指标（CAC、LTV、毛利率）是否健康可持续？盈利路径是否清晰？

经过上述的比较，可以做一个简单的判断：如果我们处于较为弱势的一方，我们需要变得更加开放，才有后来居上的可能；如果我们本就是较为强势的一方，适当地放缓开放的节奏，保护自己的核心竞争力，倒是更加明智的选择。

25.5 知识产权：保护策略与合规风险

根据前文的讨论，企业从商业利益的角度不必 100%开源自身的代码，而是应该有选择地开放。而在开源代码之前，选择合适的 License 也是非常重要的一环。是选择更加宽松的、商业友好的许可证，有利于广泛采用（但是容易被大厂“白嫖”）？还是选择更加严格、要求互惠的强 copyleft 许可证，能够防止项目分裂（但是可能阻止更多的商业采用）？这需要综合考虑前文提到的市场需求、技术因素及竞争格局，才能做出正确的选择。

另外，还需要检查是否存在专利、商标或版权纠纷，以及项目是否依赖可能受限的技术。这可能导致在开源之后给企业带来麻烦。但是另一方面，为了保护企业自身的知识产权，及时申请专利、商标与软件著作权，也是需要考虑的必要操作。

在开源软件的基础上进一步开放的商业软件，就需要考虑商业产品/服务的许可证策略是否清晰、合规，以及是否与开源许可证兼容。

再者，开源商业化的目标市场，相关的法律法规、标准和准入原则，也需要仔细研究。要想赚钱，就得合规。开源软件与商业产品需要满足目标行业/地区的合规要求。

总之，我们需要从知识产权的角度去思考相关的问题，才能制定合理的开源商业模式。

第 26 问 企业应如何制定开源战略

开源战略是企业经营战略的一部分，需要服从企业的整体经营战略，应该始终从经营战略的角度进行思考。制定并执行战略的常见思路是：评估经营状况，明确战略目标，搭建执行组织，具体执行与度量。这些思路同样适用于制定企业的开源战略。

26.1 评估经营状况

正如上述关于开源治理的讨论（第 23 问），企业应该从使用开源、回馈开源以及主动开源 3 个层面考虑问题。在评估企业的经营状况时，也应该从以上 3 个层面出发，对以下 3 组问题进行回答。

（1）使用开源：企业需要了解自身的开源软件使用情况，做到心中有数。企业用到了哪些开源项目？是内部直接使用，还是在内部的软件中依赖开源软件，又或是在可能对外销售的商业产品中依赖开源软件？用到了同一开源软件的哪些不同版本？是原样使用还是修改后使用？这些开源软件是否存在公司内不同团队，都在使用同一开源软件的情况？这些开源软件为公司带来了多少价值？带来了哪些风险？又引入了多少必须履行的合规义务？这些开源软件背后的开发社区的构成情况如何，是否掌握在某些单一的企业手中？那些开源软件的相关贡献企业，与自身是竞争关系、合作关系，还是竞合关系？在社区里是否存在其他同类开源软件，是否应该考虑替换，或者作为备选方案，持续关注？

（2）回馈开源：在回答了第一组使用开源的相关问题后，自然能够梳理出企业重视对哪些开源软件的社区回馈工作。必须履行义务向社区回馈的开源软件有哪些？从企业自身技术发展的需要，应该主动参与社区建设的项目有哪些？一旦决定回馈社区，企业的哪些源代码会被提交到社区？社区里的哪些企业会因此受益？

（3）主动开源：属于开源商业模式相关的范畴，参见第 25 问。开源战略与开源商业模式是一种包含关系，或者可以是：大多数企业的开源战略，就是围绕自身的开源商业模式，制定更加详细的战略与计划。

26.2 明确战略目标

企业的开源战略目标通常有以下几类，也可能存在多目标并存，但侧重点不同的情况。

一个常见的目标是降本增效。通过采用开源软件，企业能够有效降低在软件许可上的采购成本。同时，借助开源社区的资源和技术，企业内部的开发流程也能得

到加速，从而提升整体效率。

技术创新也是关键驱动力之一。开源社区汇聚了全球的开发者和前沿思想，参与其中能让企业更快速、更直接地接触到最新的技术趋势。例如，AI 或云原生技术，从而加速自身的技术更新和突破。

开源还可以成为一条有效的商业化路径。企业选择将核心产品的源代码开放，能够吸引大量用户和开发者，建立广泛的用户基础。在此基础上，通过提供专业的增值服务、技术支持或企业级功能，实现商业价值的转化。红帽的 OpenShift 模式就很好地诠释了这种商业化路径。

提升生态影响力、构建整个生态系统是许多技术领先企业的追求。主导或深度参与重要的开源项目，有助于企业建立行业技术标准，显著提升自身在技术领域的品牌声誉和话语权。谷歌推动 Kubernetes、阿里巴巴贡献 Apache Dubbo 等，都是这方面的成功案例。着眼于通过开源引领整个行业的技术发展方向，汇聚各方力量形成强大的生态合力，最终借势引领行业发展。这可以说是开源战略的最高境界之一。

26.3 搭建组织与制定政策

要成功实施开源战略，企业需要搭建相应的执行组织，并构建清晰的组织架构与治理机制。

组建一个开源项目办公室（OSPO）是常见的核心举措。这个办公室将负责推动战略的落地执行，具体工作包括评估引入的开源项目、管理与开源社区的协作关系，以及管控使用开源带来的法律合规和安全风险。为了确保治理工作的有效性和一致性，围绕开源的使用、贡献和发布，制定清晰的政策和流程也是必不可少的基础工作，详见第 30 问。

同时，成立一个跨职能的开源委员会也非常重要。这个委员会通常需要法务、技术研发、安全部门和产品负责人等关键角色的参与。委员会的核心职责是共同制定开源相关的关键政策，并对这些政策的执行情况进行监督，确保各部门协同一致。

OSPO 与开源委员会，前者是实体团队，后者是虚拟团队；前者是执行团队，后者是决策团队。在实际的职能与分工方面并不相同。但是不同的企业，有些可能只有 OSPO，也可能只有委员会，我们从概念上需要有所区分。

在政策层面，有几类核心政策需要重点制定。

- 开源软件使用政策：这需要规范企业选用开源软件的流程，重点考量因素包括许可证的兼容性以及必要的安全审计。
- 开源贡献政策：必须明确员工参与外部开源社区（如贡献代码或加入基金会）的内部审批流程和要求。
- 内部项目开源发布政策：对于决定将内部研发项目开源的情况，需要规定标准化的发布流程，这通常涉及代码扫描清理、符合要求的文档撰写等环节。

26.4 具体执行与度量

在具体执行层面，采取渐进式策略通常更为稳妥。企业可以优先选择风险较低的项目进行试点，积累经验后再逐步扩大开源投入的广度和深度。同时，保持灵活性，根据市场的实际反馈和技术的演进趋势，对战略进行动态调整至关重要。例如，初期可以通过向已有项目贡献代码或者赞助社区活动来熟悉开源运作，待到时机成熟，再考虑开源自有核心技术以增强市场竞争力。

开源战略的成功离不开内部的高效协作，尤其是在跨部门之间。一个基础的工作是进行软件识别，梳理各个部门都在使用哪些开源软件，并建立统一的共享资源库进行管理。同时，积极促进最佳实践的分享也非常重要，鼓励不同部门（如 IT 部门和开发团队）交流使用开源工具的经验、遇到的挑战以及解决方案，共同优化开源工具的部署和使用效率。

管控风险是开源战略执行的核心环节，需要建立全面的框架。法律与合规风险、技术风险在第 23 问已经详细讨论，而社区治理风险往往容易被低估。为参与或主导的开源项目设立明确的“贡献者公约”，规范社区成员的行为准则，有助于维持社区健康，避免因分歧导致社区分裂（历史上，Node.js 与 Io.js 的分叉就是典型案例）。另外，要警惕陷入“指标陷阱”，单纯追求 Star 数量等表面指标可能导致低质量贡献泛滥，真正值得关注的是补丁采纳率、问题解决效率等反映社区健康度和贡献深度的指标。

工程能力建设方面，第 23 问已经有所讨论，除此之外，从数字化运营的角度考虑，为对外开源的战略级项目，建立一套辅助运营的数字化工具，也是非常必要的。这方面华为的 OpenHarmony 与 openEuler 都做得非常不错。

在工程能力建设的基础上，合理的指标才能够被采集、统计和展示，从而成为战略执行所需的度量工具。详见第 14 问中对于开源相关指标的介绍。

第 27 问 企业在开源中的常见误区有哪些

企业在拥抱开源的过程中，常常会陷入一些认识误区和实践陷阱，这些都可能带来意想不到的成本、风险或错失良机。

27.1 从核心维度看企业开源误区

企业在开源之旅中面临的挑战，可以从以下几个关键维度来理解。

- 对开源本质的误解：这常常是起点。最大的误区莫过于将“开源”简单等同于“免费且无成本”，忽视其集成、维护、支持和定制化所需的持续投入。另一个极端是因免费而怀疑其质量和可靠性，认为它只适合小项目或极客，无法承担关键任务。事实上，开源模式通过众包开发和严格的同行评审，已产出大量支撑全球核心系统的高质量软件（如 Linux、Kubernetes）。同时，

认为“开源仅适合技术极客”或“便宜即低质量”的观点，已被广泛的商业实践所证伪。

- 法律合规风险的轻视：这是贯穿始终的高压线。企业容易忽视开源许可证的复杂要求（如 GPL 的“传染性/互惠性”、不同许可证之间的兼容性冲突），导致潜在的法律纠纷（如被迫开源专有代码、侵权诉讼）。这包括误认为开源不可商用，或混淆不同协议条款。与之紧密相连的是对知识产权保护疏忽，在使用或贡献开源代码时，未能平衡好保护自身 IP 与尊重他人 IP 的关系。
- 安全风险的认知不足：开源代码的公开性不等于安全。企业往往缺乏对开源组件安全性的系统评估，尤其容易忽略“传递依赖风险”——大量漏洞实际来自项目间接引用的库（Log4j 事件是惨痛教训），而审计常只停留在主组件层面。建立软件物料清单（SBOM）进行全链路跟踪至关重要。
- 社区互动的心态与方法偏差：开源的核心是协作生态。企业常见的误区包括：“索取多、贡献少”的利用心态，损害声誉与信任；“黑洞式人才掠夺”，高薪挖走核心开发者破坏社区健康；“封闭式沟通文化”，内部决策不透明引发社区质疑；“将开源作为营销噱头”（开源洗绿），缺乏真诚投入；以及“激励机制错位”，如仅奖励代码量或 Star 数，忽视文档、测试等基础工作，导致社区畸形发展。同时，“过度依赖外部社区”，忽视自身维护能力建设，也可能在关键时刻陷入被动。
- 内部管理与战略规划的缺失：成功开源需要内部协同与长远眼光。忽视员工开源素养培训，可能导致合规或协作失误。缺乏对开源项目的充分评估（社区活跃度、文档、可持续性），易选用高风险项目。低估与现有商业软件集成的复杂度，会带来额外成本或系统不稳定。最关键的是，将开源视为短期降本工具，而非长期战略投入，缺乏持续的规划、资源支持和耐心，往往导致半途而废，无法收获其真正价值。

27.2 时间维度下的误区演进

企业对开源的认知误区并非一成不变，而是随着接触深度和行业环境动态演进。

- 早期阶段（恐惧与回避—FUD 主导）：企业主要被“恐惧、不确定性和怀疑”笼罩。核心误区集中在认为开源“不可靠、不安全、不合法、不适合商业”。担忧许可证“传染”、代码公开导致安全失控，免费意味着低质无支持。行为上表现为不敢使用，尤其避免在核心系统应用。
- 中期阶段（拥抱与探索—实践偏差涌现）：开源的成功案例（Linux、Apache、云计算）促使企业广泛采用并积极参与。此阶段的新误区源于实践：低估集成、维护的持续成本（免费=零成本）；过度依赖社区，忽视自身能力建设；在热情中轻视许可证合规与知识产权风险；对安全性（尤其是传递依赖）评估不足；初期参与时出现封闭沟通、激励错位；缺乏长期战略视角，视开源为短期工具。

- 近期阶段（商业化浪潮下的功利误区）：开源商业模式（如红帽、MongoDB）的成功吸引了众多效仿者，但也催生新偏差：将开源纯粹视为免费营销获客手段，缺乏真诚贡献；“索取无度、吝于回馈”的失衡心态加剧；过度掠夺人才导致破坏社区生态；在商业目标驱动下陷入“指标虚荣陷阱”（过度关注 Star 数等表面数据）；产生“开源是万能商业解药”的幻想，低估商业化所需的生态构建和服务能力；甚至在压力下产生“合规可灵活处理”的危险想法，法律风险加剧。

第 28 问 如何在企业中推广开源文化

以下是如何在企业中推广开源文化的系统性方案。在探讨如何推广之前，首先需要回答的是“为何要推广”的问题，然后才是如何去推广的实践层面的讨论，最后我们还会分析在实际推广的过程中会遇到哪些挑战以及破解之道。

28.1 为何要在企业中推广开源文化

开源是一套技术实践，更是开放式组织如何进行协作的一套文化系统。如果仅在企业内部建立规章制度，监督各级管理者与普通员工去遵守执行，缺乏每一个参与者个人的主动自觉。无论任何一种完善的开源战略，都是无法被顺利执行的。只有通过推广开源文化，让尽可能多的员工与管理者从“要我做”变为“我要做”“我想做”甚至是“我喜欢做”，才能真正让企业从拥抱开源之中获益。

企业推广开源文化的本质是将“重重叠叠的部门围墙”转变为“开放创新网络”，通过内外部协作降低研发成本、加速技术迭代与创新。

28.2 推广路径与实践策略

文化的推广通常会有两条路径：自顶向下或者自底向上。前者首先从企业的最高领导层/决策层开始，在领导层/决策层的意识有所转变，下定决心之后，启动企业内部的文化转型/升级。后者则是从个人、小团队的尝试开始，高层甚至并不知情，随着效果逐渐呈现，再逐步向上传递、影响，最终成为公司级的整体战略决策。我们通常会认为，前者更加困难，是先难后易。但是却往往没有意识到，后者也会先易后难，甚至到最后面临半途而废，被高层叫停的悲剧。

如果整个企业的决策层没有转变观念，明确将开源视为创新引擎而非成本中心，那么整个企业的文化转型本质上就无法实现。小范围的开源尝试、文化推广，甚至可能被视为直接挑战企业的现有规章制度，破坏信息安全，导致知识产权泄露、企业利益受损之类的严重错误行为。

诚恳建议试图在企业内尝试推广开源文化的朋友，不要轻易发起草根开源运动！首先应该从说服有决策权限的高层主管开始。

（1）机制建设：设立开源项目办公室（OSPO）。

在获得公司的初步授权之后设立的 OSPO，通常具备以下职能：文化推广、合规治理、安全防护、技术赋能与社区运营。（详见第 30 问。）

为了配合文化推广，也需要考虑改造企业内部的激励机制。除了设立新的虚拟岗位、荣誉奖项外，也应该与企业内部的 HR 部门协商，在薪酬奖金、岗位职级、绩效评价、晋升通道方面进行设置。

（2）基础设施：开源中心仓与内源平台。

外部繁荣的开源世界，在企业内部却难以感知和接触，又不可能通过行政命令要求所有员工加入外部的开源社区。虽然研发人员都会有意识或无意识地大量使用外部的开源项目与代码，但是绝大多数对于开源都是一知半解的状态。

在企业内部创造一个可以被感知的环境，尤其是研发环境，是推广开源文化的第一步。而且，这一步从说服高层领导而言，也最容易。

首先，搞清楚企业内部研发团队使用开源的情况，先普查后集中，建立“开源中心仓”。对开源的集中管理是开源治理的第一步，没有这一步，后续的所有工作几乎都无从谈起。

其次，在建立了开源中心仓之后，企业内部的研发环境、研发流程、DevOps 与 CI/CD 工具链都会发生变化，而其中最重要的一个步骤，就是推广代码评审（Code Review）成为研发过程的必选环节。许多缺乏代码评审习惯的研发人员，根本无法理解与适应开源社区的协作模式。

最后，是推动内源，要求所有非核心代码开源至内部中心仓，强制代码复用，从整体上打破研发部门墙，从而减少企业的研发浪费。这样做也是为了潜移默化地引导研发人员适应开放式协作。（详见第 29 问。）

（3）生态运营：促进内外交流。

企业对外开源，一个很重要的理由就是吸引外部贡献者的参与。如果企业自己投入 10 个人，却能够换来外部（无须付费的）10 倍甚至 100 倍的贡献，这样的杠杆效应就太划算了。但是，为啥这样的好事不容易发生呢？因为只有走出去，才能引进来。如果企业内部的员工在开源社区里，总是一副面目可憎的样子，人家凭什么要留在你的开源社区，为你的开源项目做贡献？企业内部的员工要成为开源社区里受人欢迎、受人尊敬的贡献者，具备一定的技术影响力，甚至人格魅力，才有可能吸引尽可能多的外部贡献者。

因此，推广开源文化，就是先在内部培养“出得去的人”。这些代表公司在社区贡献的人需要懂礼仪、守规矩、有足够的表达能力与社交能力，能够同时兼顾两种身份、满足内外期待且如鱼得水。这样的人才，可以被称为“技术外交家”。

（4）量化与优化。

文化虽然是一个很虚的概念，但是从管理学的角度来看，没有度量就没有管理（彼得·德鲁克语）。当我们希望说服公司、拨付预算、成立组织来推广开源文化时，一定要回答这个问题：如何衡量企业内部开源文化推广工作的成败？一个团队要想在企业内部长期存在，一定要抢在主管质疑之前，回答这个问题！

一个可行的量化方向是：文化→行为→收益。我们推动的开源文化，引发了哪

些行为上的变化（例如，更多的团队采纳了代码评审这样的开源实践），从而为企业带来了哪些收益（例如，采用代码评审的团队，代码质量得到一定比例的提升）。

当我们建立起一系列“行为：收益”的观察点之后，开源文化带来的益处就会越来越清晰，后续的改进与优化方向也会越来越有说服力。

28.3 关键挑战与破解之道

（1）文化冲突：员工不理解开源价值。很多员工习惯闭门造车，认为开源是“泄密”或“白干活”，看不到实际收益。破解之道在于具体案例展示和领导带头。用真实案例说明开源如何带来技术提升（如更快获得补丁）、人才吸引（如优秀开发者更愿加入开源活跃公司）或商业机会（如基于开源产品建立服务）。公司高层要真正投入，例如，亲自参与开源活动、公开支持贡献，并用实际资源（如预算、时间）证明这不是空谈。

（2）老旧系统改造困难，成本高。老旧系统往往急需开源治理，否则会带来合规风险与安全风险。但是老旧系统往往结构复杂、文档缺失，直接替换风险大、代价高。破解之道是渐进式改造和设立标准。不要强求一步到位，优先隔离高风险或最需更新的部分，进行开源化或替换为合规开源组件。同时，为所有新开发制定明确的、基于合规开源组件的技术选型标准，避免老问题延续。建立内部开源组件“白名单”和自动化扫描工具，能有效管理风险并降低后续改造难度。

（3）员工动力不足，感受不到收益。贡献开源初期确实增加工作量，个人收益（如技能提升、业界声誉）和企业收益（如技术影响力、招聘优势）都有滞后性。破解之道是设计即时反馈和明确激励。建立简单透明的贡献积分或成就系统，让每一次有效贡献（如修复 Bug、提交文档）都能获得及时认可（如内部通报表扬、小额奖励）。更重要的是，将开源贡献与绩效考核、晋升通道明确挂钩，让员工清晰地看到付出能转化为个人职业发展的实在好处。

（4）效果度量难，易被质疑。开源带来的价值（如品牌提升、创新加速）不易直接用传统财务指标衡量，自建度量体系也可能被怀疑“做数据”。破解之道是追求透明度和引入外部验证。公开度量方法、数据来源和计算公式，接受内部讨论和质疑。同时，积极引入第三方评估（如开源社区健康度报告）、关注行业排名（如开源贡献榜单）以及收集外部反馈（如开发者调查、客户评价）。当外部认可度提升（如社区奖项、项目采用率增长）时，内部对效果的共识自然更容易达成。

第 29 问 什么是内源，为何在企业中推广内源

29.1 内源的定义与核心特征

内源是指在组织内部采用开源软件开发的最佳实践，并建立类开源文化的一种

策略，用于开发非开源或专有软件。内源由 Tim O'Reilly 于 2000 年提出，初衷是将开源社区的高效协作模式引入企业内部。内源的核心特征如下。

- 开源文化的移植：基于开放式协作原则（平等、精英领导、自组织），允许企业内部的员工跨部门贡献代码、文档或解决方案，打破传统部门壁垒。在企业内部打造“社区”，而不是原有的、金字塔式的组织架构，从而实现更加宽松与自由的交流与协作氛围。
- 技术实践标准化：强调持续集成/持续部署（CI/CD）、自动化测试、代码复用和透明化协作流程，通过工具链（如内部代码托管平台）实现高效开发。首先是代码在内部平台集中存放，然后是开发协作在统一的平台上进行，目标是实现基础设施、工程实践、代码模块的复用。
- 目标导向：相较于外部的开源社区，内源更加强调持续度量、持续优化，并不断追问“这些实践为企业带来了哪些价值”？在以“共享、复用、协作、学习和自动化”为明确目标的指导下，提升软件质量、加速创新并降低重复劳动。

29.2 企业推广内源的核心动因

（1）创新能力增强：打破壁垒，加速协作与创新。

传统部门墙阻碍了知识和创意的流动。研究显示，83%的公司受到操作孤岛的负面影响，97%承认孤岛对公司绩效有负面影响（来源：Together Mentoring Software）。内源通过默认开放代码库和开发过程，让不同团队的工程师能轻松看到、学习甚至改进彼此的成果。这直接打破了信息孤岛，使创新得以加速。

（2）软件工程改进：提质增效，降低成本与风险。

内源的核心实践，如开放的代码审查和鼓励复用，直接提升了软件质量与工程效率。更多双眼睛审查代码（即使来自其他团队）能更早发现缺陷和安全漏洞。更重要的是，工程师可以轻松找到并复用内部已有的可靠组件，而非“重复造轮子”。某大型企业的一个 50 人的研发团队，内部采用开源社区的同行评审实践，使得后续版本的“漏洞复现率”下降了 30%~40%。而来自 PayPal 的案例则表明：他们的 ALM（应用程序生命周期管理）项目，在引入内源方法之后，不仅可以在不到一年的时间内实现 15 个主要功能，包括容器化和数据库即服务，还能够开始重构 ALM 以过渡到平台即服务。在与各个团队合作之后，该团队创建了一些令人惊奇的开放标准文档，让我们看到开放文档带来的效率提升。

（3）人才发展提升：赋能员工，构建韧性组织。

内源为工程师提供了超越本职团队的学习和贡献舞台。他们可以参与感兴趣的内源项目，学习新技术，通过有价值的贡献获得更广泛的认可，提升成就感和参与度。这加速了人才成长。同时，项目的代码、文档和决策过程对内部开放，极大简化了新员工的入职熟悉过程。知识不再集中在少数“关键人物”脑中，而是沉淀在内部社区之中。以沃尔玛的内源实践为例，沃尔玛推行开源的开发方式之后，对开源社区成果的使用和回馈行为还帮助他们吸引了许多技术人才。在他们准备录用的

候选人中，50%手中还握有谷歌、苹果、Facebook（现改名为 Meta）、LinkedIn 或其他硅谷著名公司的职位邀请函，可是这些人中最终有 70%还是选择了沃尔玛。King 把争夺人才胜利的原因归功于两点：一是小而专的团队，二是开源。“大家都对我们已经取得的成就非常赞赏，同时也希望如同在创业公司工作一样。”（来源：“内源案例”李建盛）

（4）奠定开源基础：内通外达。

成功的内源实践，实质上是在企业内部培养开源文化和协作模式。员工熟悉了代码开放、同行评审、社区贡献和基于共识的治理流程。当企业战略性地需要参与或主导外部开源项目时，这种文化、流程和人才储备就成为强大的基础。华为、微软等大型科技公司在深度投入外部开源生态前，都经历了长期的内源实践，使其在合规、协作、代码管理等方面具备了顺畅过渡的能力。内源是安全可控地锻炼“开源肌肉”的有效途径。

29.3 如何解答那些对于内源的质疑

虽然已经有很多实践案例证明内源对于企业的价值，但是依然不乏对于内源的质疑。

（1）内源效率低下，事实上是在期待着员工用业余时间多打零工。

的确，在很多“996”严重的公司，我们很难想象员工还有业余时间，为公司里的其他项目作出贡献。以至于很多企业的内源项目，基本上无法获得跨部门贡献，使得内源的“加速协作与创新”的承诺，成为一句空话。但这正是内源希望推动解决的问题。如图 29-1 所示，一个企业的员工研发效率低下，通常受限于他们的自身能力。而想要提升员工的研发能力，则需要他们激发活力，主动学习、交流、分享。无论是在企业内部社区的交流分享，还是在外部社区的沟通与协作，都需要他们有起码的业余时间，否则能力提升无从谈起。于是，这就陷入了一个死循环，慢导致累，累导致没时间，没时间导致更慢。而内源实践，就是希望打破这样的死循环，帮助更多的研发人员，得到成长的机会。

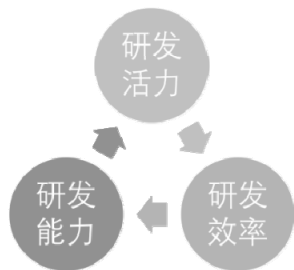


图 29-1 研发活力与研发效率、研发能力的关系

（2）内源绑架开源精神而不是汲取开源精神。

这个说法似是而非，因为它将推行内源过程中可能发生的乱象，总结为内源的

本质必然导致的问题。如果一家企业一方面不遗余力地推行内源，另一方面却对于开源从不参与，从不贡献，这当然会被视作一种“过于利己，毫不利人”的行为。但是，一家企业如果真的想要获得推行内源的收益，也必须实现内源与开源的双向互通，外部开源项目在内部由内源团队负责维护。而内部的内源项目成熟之后，有机会对外开源，以获得更大的社区/技术影响力。否则，缺乏源头活水，完全封闭的内源，就会成为一潭死水，无法为企业带来开源文化与研发模式的变化。如果推广内源的企业却限制员工在开源社区的贡献，就会受到类似的批评：“内源以牺牲（对外部）开源为代价”，这都是非常短视的行为。

（3）期待内源项目转化为开源，为何不是直接开源。

这是另一种相当常见的质疑，可以称之为“开源激进主义”。因为笔者始终不认为100%开源对于企业来说是符合其商业利益的选择。而企业如果一心做公益，却不顾自己的商业利益，这样的开源也不会长久。笔者更加赞同企业在深思熟虑之后，做出符合自身长期战略的开源，这样的开源，也会有更好的持续性。否则一阵热闹之后，人去楼空，原厂不再维护，社区也难以接手，就会成为典型的“烂尾开源”。

第30问 如何在企业中建立开源项目办公室

在本书的第23问、第26问和第28问，都提到了组建开源项目办公室的问题。无论是开源治理、开源战略的制定与执行，还是在企业中推广开源文化，都需要通过开源项目办公室来落实。那么，如何才能成功地建立起一个开源项目办公室呢？在第23问中，我们谈到了开源项目办公室的人员构成，需要哪些类型的专家与角色。但是，仅仅把这些人找出来并放到一起，是不够的。我们还得从头说起。

30.1 开源项目办公室的定义

开源项目办公室是一个组织的开源运营与开源架构中的能力中心。它负责定义、实施和指导这些工作的战略和政策。这可能包括围绕代码使用、分发、选择、审计、贡献和其他关键领域制定政策；为参与开源活动的人员（组织内部和外部）提供教育和培训；通过鼓励可持续地使用现有开源组件，并在适当的情况下将增强功能贡献回这些项目，支持组织提高软件开发效率；在需要时，指导团队将其软件开源；确保工程效率、法律合规性；促进和建立社区参与。内源与开源密切相关，经常与开源项目办公室合作或成为其一部分。

对外，开源项目办公室是企业与开放源码社区之间的重要桥梁，有助于确保企业成为开放源码软件的良好管理者，并从采用开放源码软件中获益，同时将风险降至最低。对于组织内部而言，开源项目办公室是整个组织内开放源码相关活动的中心界面，可以从法律、经济、技术或社区等不同角度汇集所需的专业知识。

开源项目办公室可以在不同部门、地区和组织规模中成立，包括私营和公共组织，如学术界、非政府组织、基金会、政府以及大中小型公司。同样重要的是，一

个组织内部类似于开源项目办公室的结构有许多不同的名称。命名和具体的组织形式在很大程度上取决于每个组织的做法。

30.2 开源项目办公室的价值

开源项目办公室的价值可以简单地总结为：趋利避害。也就是帮助一个组织，尽可能获得拥抱开源应有的收益，同时避免开源可能带来的经济、法律、品牌等方面的损失。

从趋利的角度，开源项目办公室能够带来的价值包括：提升开发效率、增强创新能力、吸引优秀人才、提升企业品牌。从避害的角度，我们也能获得以下收益：降低开发成本、降低试错成本、降低安全风险、降低合规风险。这些价值，有不少在上一个关于内源的问题中，也都已经讨论过了。关键在于，为什么开源项目办公室，就能够做到这些？

本质上，企业需要意识到自身已经处在一个被开源软件包围的世界之中，无论企业是否愿意（或者允许），他们的研发人员都在越来越多地使用开源软硬件，来提高自身的开发效率，减少重复劳动（不再写那些社区里已经写好的代码）。当一个企业意识到这样的趋势，同时愿意顺应和拥抱这一趋势时，企业就会变得越来越开放。我们以物质从固态转变为液态，甚至气态的过程来类比，越是开放的组织，其中的个体就越是活跃，不再固定于自己的角色、身份，受限于自己的职责、目标，会主动做更多的“分外之事”。当一个组织的“能级”变得越来越高，活力越来越强，自然会有更高的效率，更好的创新，但是也必然会更加难以管理，甚至带来无法预料的风险。

组织设立开源项目办公室的根本目的，就是激活组织，但同时又有所控制，不至于让整个组织滑向失控甚至爆发的边缘。所以，他们一方面需要推广开源文化，另一方面又需要制定新的规章制度。一方面他们需要鼓励员工走进社区，积极贡献；另一方面又要提升企业的吸引力和凝聚力，以免自己的员工被人挖走。他们一方面需要向老板“鼓吹”开源带来的好处，另一方面又要随时提醒老板开源可能带来的风险。

30.3 开源项目办公室的成长路径

每家企业建立开源项目办公室的初衷有所不同：可能是出于合规考虑，可能是为了更好地创新，也可能是为了吸引人才。总之，开源项目办公室可能因为不同的机缘，不同的老板偏好，不同的业务逻辑，从不同的部门“生长出来”。

- 从安全需求出发：通常是企业吃到了开源软件安全漏洞的苦头，开始重视安全风险，希望能够通过开源治理，减少今后可能出现的损失。
- 从研发效能出发：通常是顺着 CMM、敏捷、TDD、内源这条线索，开始思考如何更好地提升研发效率的问题。

- 从开源合规出发：通常是由于相关诉讼，或者产品出口受阻，发现还有一堆“必须遵守的规则”，于是指派企业的法务团队先去研究，然后再逐步扩大。
- 从人力资源出发：通常是在思考“如何能够挖到技术大牛”“如何留住企业人才”这些问题的时候，意识到了开源的价值，并且希望能够提升企业的吸引力与凝聚力。
- 从市场营销出发：通常，企业的商业模式严重依赖于自己开源的项目和社区的繁荣、健康，首先想要做到的还是“开源社区运营”，后来才会更多地考虑全局的问题。

总之，无论初心如何，开源项目办公室都会逐步成长为一个复合型人才聚集地，非此不足以应对工作中的各种挑战。开源项目办公室的成长成熟的标志，应该是受命制定（哪怕是提供建议）企业的开源战略，这样的团队才算是从一个“应对眼前挑战的临时组织”，成长为企业战略级别的、不可或缺的专家型组织。

30.4 开始前的准备工作

如果老板授权你组建一个开源项目办公室，或者你自己打算说服老板，在公司组建一个开源项目办公室，都需要提前做一些准备。

（1）搞清楚水有多深。

无论是在企业内部搞好开源治理，还是企业打算对外开源想要认真运营，这些任务的成功都与企业的开源成熟度有关。虽然有很多种不同的“开源成熟度”模型（参考 TODO Group 的《OSPO 指南》），但是究其根本无非是要搞清楚 3 个方面的问题。

- 人的问题：是否有接触开源社区、参与贡献开源的经验？有多少人有这样的经验？对于开源的看法是正面还是负面？整个企业大概有多少研发相关的人员？要知道数量级是最重要的变量：10 倍的人数，可能会带来 100 倍的难度。
- 技术问题：公司产品的技术架构如何？使用开源的广度与深度如何？对于全公司的产品做一遍开源代码成分分析是否方便，成本会有多高？公司内部研发平台，是否已经实现 DevOps，是否能够方便地升级到 DevSecOps？
- 商业的问题：公司的业务方向与技术领域，在开源领域是否已经有类似或相关的项目？这些项目对企业的经营是否带来影响？这些开源项目背后有没有强大的竞争对手？企业的应对策略中，是否（已经）在考虑开源的手段？

总之，人的观念与数量级带来的复杂度，是最难应对的两大挑战。

（2）搞清楚领导的支持力度。

在接手组建团队的工作之后，千万不要马上开始干活，一定要搞清楚领导对你的支持力度到底有多大。哪怕你非常委婉、谨慎，也一定要问清楚，获得实实在在

的支持后再去开始工作。一支持一分成绩，管理好领导的期望也非常重要。我们需要询问的具体要点如图 30-1 所示，从上到下，由易到难。虽然申请预算已经足够困难，但是试图变革企业内部的任职体系，才是最难的。

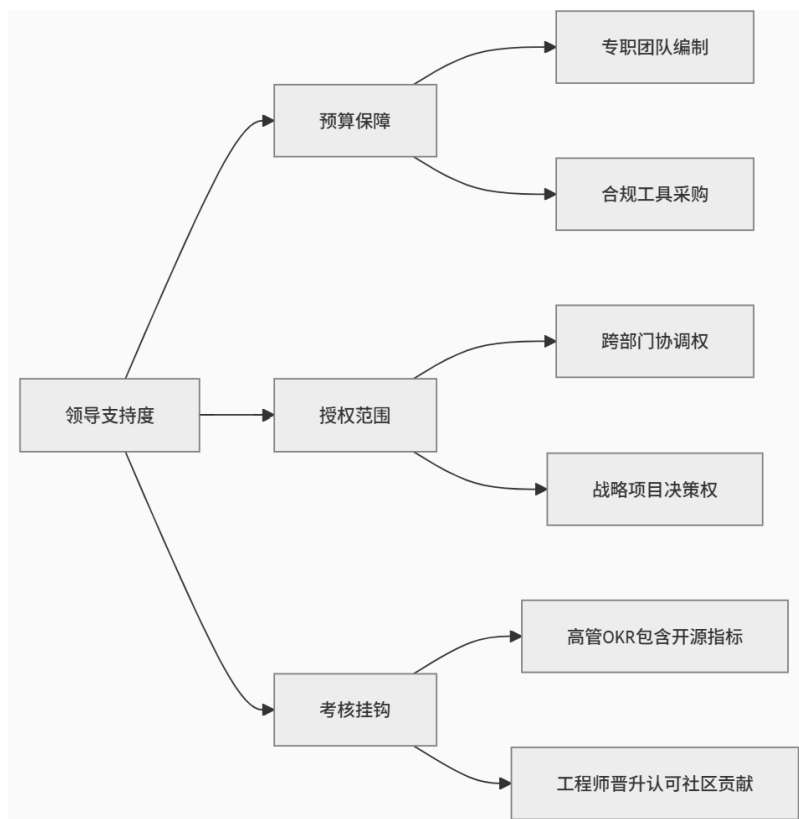


图 30-1 领导对开源项目办公室的支持力度

（3）搞清楚短期目标与长期目标。

另外就是搞清楚目标，无论是短期目标，还是长期目标；无论是领导心目中的目标，还是你的长期愿景；无论是 OKR 还是 KPI，都是需要梳理清楚，并且尽可能明确。

至少在一开始，我们需要与领导对于团队的短期目标，达成一个清晰的共识。例如，3 个月内，对公司内部的开源使用情况完成普查，并提交风险报告。6 个月内，初步建立开源风险的防范体系。或者是 3 个月内，完成一次公司对外开源项目的开发者体验调查报告，并制定今后一年的社区运营计划。总之，团队能够实实在在地交付一些东西，这一点非常重要。

至于中长期目标，需要在短期目标达成之后，领导对于团队的能力与执行力建立初步的信任之后，再进一步确定。而且，越是远大的目标，越是需要足够的授权，不断地“滚动向前”，获取更多的领导支持，是非常重要的。

还有就是需要强调目标量化与试错成本，前者让你的成果更容易被认可，后者让你的失误有机会被原谅。目标管理与预期管理，是一体两面的工作。

30.5 如何搭建开源项目办公室团队

一个较为完善的开源项目办公室，需要负责制定开源合规规则、开源治理流程和协调资源，统筹规划和推动企业开源治理工作。开源项目办公室包含若干角色、工作团队，负责相应职责。

- 开源项目办公室负责人：负责制定企业开源治理政策、治理制度和治理流程，并能够基于此推动企业完成相应的开源治理工作，同时通过确定治理目标和考核奖惩制度确保开源治理的效果。
- 法务专家：负责管控企业在使用开源项目时面临的法律风险，包括开源许可证合规风险、知识产权风险、出口管制风险等。
- 治理专家：根据企业的开源治理制度和流程，具体指导研发项目的开源治理工作，包括制定治理计划、划分治理任务、监督治理进展、核查治理效果和确保研发项目完成企业总体的开源治理要求。
- 安全专家：负责通过各种渠道和手段，从企业外部获取开源软件漏洞的情况，确定开源安全漏洞治理方案，及时通知各研发项目消除安全风险。同时需遵循国家关于漏洞管理的相关规范，代表本企业将安全漏洞及时上报。除开源安全漏洞以外，安全专家还应负责与开源相关的数据安全、个人信息安全等其他安全事项。
- 基础设施支撑团队（可选）：负责开源治理基础设施的建设和维护，保障企业开源治理工作的顺利进行。
- 社区运营团队（可选）：负责与外部开源社区的交流和运营工作，包括推动企业牵头、赞助和参与开源社区的各种活动以及大型会议，推广企业自发自研项目和相关商业宣传等。

虽然一开始我们很难组建一个满员的、实体化的团队，但是清晰的人员能力地图，能帮助我们不断寻找合适的人才，哪怕先组建“虚拟团队”，或者聘请“外部顾问”，逐步由虚向实，也是合理的发展方式。

30.6 建立开源项目办公室的常见挑战与解决方案

成功组建开源项目办公室是企业拥抱开源战略的关键一步，但这一过程并非坦途，常常伴随着几类显著的挑战。

资源限制是初期最常见的障碍。许多企业面临人力和财务预算的紧张，难以从一开始就投入大量资源支持一个功能完备的开源项目办公室。解决这一困境需要采取务实策略。首先，可以从小规模起步，优先聚焦于最核心的职能。例如，开源合规审查或内部开源协作平台的搭建，而非追求“大而全”。其次，充分利用成熟的开源治理工具（如许可证扫描、代码审计工具）能显著降低工具采购成本。同

时，积极寻求跨部门的资源共享与合作也是缓解资源压力的有效途径。例如，与法务、安全或研发部门共享工具或专家资源。最为关键的是，需要持续地量化开源项目办公室带来的价值，无论是通过降低法律风险、节省开发成本、加速产品上市，还是提升品牌影响力，用清晰的投资回报率（ROI）数据来证明开源项目办公室存在的必要性和争取更多资源支持。

另一个深层次的挑战来自文化阻力。企业内部可能长期存在对开源的误解，认为其等同于“免费”或“不安全”，或者固守传统的封闭式开发文化，担心开源会泄露核心知识产权或失去控制权。克服这种文化惰性需要耐心和系统性的工作。核心在于持续的教育和沟通：开展形式多样的开源意识培训、工作坊和讲座，澄清误区，阐明开源在创新、协作和人才吸引方面的战略价值。分享行业内以及公司内部（如果已有）成功的开源案例，用事实展示其带来的技术红利和商业优势，能有效提升说服力。此外，识别并赋能内部的“开源倡导者”或早期采纳者，让他们成为变革的推动者和榜样，能够加速文化的渗透。最后，建立明确的激励机制，将开源贡献（如提交代码、修复 Bug、参与社区讨论）纳入员工的绩效考核和晋升体系，从制度层面引导行为模式的转变，最终让开源协作成为工程师文化的一部分。

最后，合规复杂性往往是企业涉足开源时最担忧的环节。开源许可证种类繁多、条款各异，合规要求细致且法律风险高，管理难度大。应对这一挑战需要建立系统化、自动化的管理流程。部署自动化的开源组件扫描和许可证识别工具是基础，将其集成到 CI/CD 流水线中，实现持续的合规监控。在此基础上，需要为开发团队提供清晰、简明、易懂的开源许可证使用指南和最佳实践手册，明确哪些许可证是安全可用的，哪些需要特别注意，以及贡献代码的规范流程。定期的、针对性的开源合规培训必不可少，确保一线开发者理解规则并能在日常工作中遵循。同时，采取风险管理分级的策略也至关重要，并非所有开源组件或使用场景风险等级都相同。应将有限的合规资源集中投入到对高风险组件（如具有强传染性/互惠性许可证的 GPL 系列）和关键业务产品的审查上，建立更严格的审批和监控机制，实现风险管理的精准与高效。

第 31 问 如何从开源社区招募人才

在软件改变世界、开源改变软件的时代，开源社区已经成为发掘顶尖技术人才的重要宝库。活跃于其中的贡献者，通常具备深厚的技术功底、强烈的自驱力、出色的协作能力以及宝贵的跨文化和跨时区合作经验。因此，许多企业正将目光投向开源社区，以解决技术人才短缺的问题。

31.1 对开源人才的渴求程度与原因

开源社区汇聚了全球大量的技术人才。根据 Linux 基金会 2017 年发布的报

告，有 89%的招聘经理认为寻找具备开源技能的人才非常困难，同时 60%的公司明确希望雇佣拥有开源经验的人才。而到了 2023 年，Linux 基金会的报告指出全球开源职位空缺已超过 40 万。

从开源社区招募人才有几项显著优势：候选人的代码贡献记录就像一份公开的、可验证的“技术简历”，其真实的技术实力和协作能力清晰可见。他们对开源文化的热情也使得他们更容易融入同样重视开源协作的团队。此外，企业在开源社区积极招募，本身也能提升自身的技术影响力和品牌形象，招募一位社区“大牛”，很可能大大降低招募其他技术人才的难度。

31.2 战略定位：从短期挖人到长期共建

成功的开源人才招聘不应仅仅是“挖人”，而应立足于长期的价值共建。核心理念可以概括为：建立开放协作的文化比单纯提供高薪更能吸引和留住人才，长期培养内部人才比一次性外部挖人更具可持续性。表 31-1 总结了企业在开源人才招聘中需要关注的 4 个核心战略方向及其内涵。

表 31-1 招募开源人才的战略维度

战略维度	关键思路
社区关系	企业需要成为社区的“共建者”而非“索取者”，通过持续投入（如贡献代码、修复 Bug、改进文档）来建立可信赖的技术品牌
文化塑造	企业内部需要建立支持开源的文化和机制，包括尊重技术自主决策权、倡导透明协作
人才培养	将开源贡献纳入员工的职业发展路径，鼓励并支持员工参与开源，培养和留住核心人才
生态位竞争	招募是企业在开源生态中建立存在感和影响力的过程，积极参与能让优秀人才主动靠近

31.3 如何精准识别开源人才

一、目标社区与平台

寻找人才首先要关注合适的平台，表 31-2 列举了主要的开源人才聚集平台类型、代表性平台及企业在这些平台上应重点考察的人才指标。

表 31-2 招募开源人才的目标社区与平台

平台类型	代表平台	招募关注点
综合代码托管	GitHub、Gitee 或其他国内外代码托管平台	查看活跃度、Pull Request 质量、项目获星（Star）数量、代码提交频率
技术社区与自媒体平台	开源中国、Linux 中国、CSDN、StackOverflow、知乎、微信公众号等	观察其技术博客深度、在问答区的活跃度和解答质量

续表

平台类型	代表平台	招募关注点
开源基金会	Apache 软件基金会、Linux 基金会、云原生计算基金会、其他国内外基金会	关注在基金会托管的开源项目的核心维护者（Committer/Maintainer）、技术决策委员会成员
开发者大会	KubeCon、OSCon、LinuxCon、COSCon、CoC 大会等	提供面对面交流机会，了解其在社区中的声誉和影响力

二、评估方法

评估开源人才需要多维度考察。下面看一些重要的考察维度。

- 代码贡献质量：查看开源人才提交的 PR 复杂度、代码可维护性、解决问题的效率以及在社区讨论中的技术深度。
- 非代码贡献价值：关注开源人才文档撰写的清晰度、测试用例的覆盖度、翻译工作、社区推广活动等。
- 长期活跃度与声誉：考察开源人才在特定项目或社区中持续贡献的年限，以及来自其他社区成员的认可和评价。

三、核心招募策略

企业可以采取以下 5 种核心策略，系统性地从开源社区吸引和招募人才。

（1）深度参与开源项目：企业应积极投入资源参与关键的上游开源项目，鼓励员工在工作时间参与上游贡献。主动将内部成熟的技术项目开源，也是展示技术实力和吸引人才的途径。通过提交高质量的 PR 和有效修复 Bug，企业能显著提升在社区中的声誉。

（2）构建内部开源文化：成功的开源招募需要内部文化的支撑。像红帽和谷歌等公司，赋予工程师高度的技术决策自主权。明确规定员工可以将一定比例（如 20%）的工作时间用于开源贡献。建立内部激励机制，如确保员工在开源项目中的代码署名权、公开表彰社区贡献等。

（3）积极参与社区活动：赞助或主办黑客马拉松（Hackathon）是接触活跃开发者的好机会。在大型技术峰会（如 KubeCon）设立展台或举办技术讲座，直接与潜在候选人交流。支持和维护本地的技术用户组（Meetup），建立更紧密的社区联系。推行导师制度，让资深员工帮助新人融入社区，建立信任网络。

（4）创新招聘流程：改变传统笔试，可邀请候选人完成一个实际的小型开源任务（如修复一个明确的 Bug 或添加一个小功能）来进行异步评估。对于在知名项目中担任重要角色（如 Maintainer）的候选人，其社区地位和来自其他维护者的推荐信，可作为重要的录用参考。鉴于开源贡献者分布全球，默认支持远程工作模式是吸引人才的关键。

（5）设计有效的激励与留存机制：吸引人才后，如何留住他们同样重要，表 31-3 整理了一些留住开源人才的关键激励措施。

表 31-3 留住开源人才的关键激励措施

激励维度	策略示例
经济回报	提供具有行业竞争力的薪酬、项目成果分红、设立漏洞赏金计划（如 Kubernetes 的漏洞赏金）
技术荣誉	保障和突出代码署名权、建立内部或公开的贡献者排行榜、提供在重要技术会议上演讲的机会
职业发展	提供开源相关的专业培训与认证（如 Linux 基金会开源软件大学 LFOSSA 课程）、设计双轨晋升通道（技术专家路线/社区管理路线）
文化尊重	坚持赋予技术自主权、保持相对扁平的管理结构、营造对远程工作高度友好的文化氛围

四、内部人才培养

招募开源人才的同时，企业也需要建立内部人才的培养体系。因为人才是培养出来的，而非仅靠招募获得的。

重视企业内部的人才培养，尤其是开源人才培养，有两个好处：一是公司的整体氛围更加开放，更加容易协作，能够更好地留住从开源社区招募而来的人才；二是企业自己花时间培养出来的人才，成本更低，忠诚度更高。

因此，建立企业内部的开源人才培养体系非常重要。这方面可以参考第 29 问，关于内源的人才部分。

五、需要规避的常见陷阱

在开源人才招聘过程中，企业容易陷入一些误区。

- 忽视社区文化：将公司内部僵化的流程强加于社区项目，会破坏信任，适得其反。
- 追求短期挖墙角：仅仅为了“购买”人才而临时参与社区，缺乏长期共建的诚意，无法建立持久的影响力，也难以真正吸引顶尖人才。
- 缺乏对新手的支持：新加入的开源贡献者（尤其是从社区招募的新员工）如果得不到足够的引导和支持（如导师帮助），流失率可能非常高。

应对这些陷阱的办法包括：始终坚持优先投入资源贡献上游项目，而非仅仅索取；加强内部开源人才的培养，建立人才梯队；着眼于长远，持续建设企业自身在开源生态中的人才库和影响力。

第 32 问 如何通过开源提升企业的技术品牌

开源不仅是技术手段，更是一项长期的品牌战略投资。它通过多种机制为企业创造价值：透明的代码库能直接证明企业的技术实力，尤其在金融、医疗等对“暗箱操作”敏感的领域，这种透明性成为关键竞争力；开源社区自带传播效应，成功项目（如 TiDB、Kubernetes）能在全球开发者中形成口碑；社区还是重要的人才库，红帽有 60% 的员工来自社区贡献者；企业还可通过主导项目（如谷歌推动

解码开源：开源生态各问题——118

TensorFlow 和 Kubernetes）参与行业标准制定；同时，开源项目还能成为商业产品或服务的入口，直接为企业带来更大的经济利益。

32.1 技术品牌建设的 4 个阶段

根据 Linux 基金会等机构的研究，企业开源品牌建设可分阶段推进，如表 32-1 所示。

表 32-1 技术品牌建设的 4 个阶段

阶段	核心目标	关键行动	品牌收益
消费阶段	降低开发成本	集成成熟组件（如 MySQL、Spark）	无，外界甚至不知道企业在消费开源
参与阶段	建立技术存在感	提交代码修复、参与社区讨论	树立“可靠贡献者”形象
贡献阶段	增强技术话语权	主导特性开发、贡献案例	成为“关键贡献者”，扩大影响
领导阶段	定义技术方向	发起项目、推动标准、捐赠基金会	晋升“技术领导者”的地位

需要留意的是，约 90%的企业停留在前两个阶段。要突破瓶颈，需持续投入资源并坚持长期承诺。如果出现 KPI 式的开源，开源以后主导者获得了职位晋升，随后就对开源项目弃之不顾。这样反而会极大地损伤企业的技术品牌。如果企业不能建立长期的开源战略，所谓技术品牌，也几乎无从谈起。

32.2 核心实施策略

（1）选择与打造标杆项目。

聚焦行业共性痛点（如 Netflix 开源故障测试工具 Chaos Monkey，测试系统在压力下的行为，以提升其弹性和稳定性）；开放能体现技术深度的非核心模块（如 TensorFlow 的机器学习框架）；在新兴领域（AI、边缘计算等）提前布局，抢占生态位。有一个简单的公式：项目开源价值=技术创新度×社区需求度×维护可持续性×品牌提升潜力。总之，要有用、创新、强大，才能有吸引力。

（2）确保开源项目品质。

要努力缩短从下载到成功运行“Hello World”的时间，提供生产级代码（含严格测试和完整文档），公开维护技术路线图以避免项目停滞，平衡商业与社区需求。总之，品质是敲门砖。劣质的开源项目，往往无人问津。

（3）社区运营与人才协同。

设计清晰的贡献者成长路径：从用户→问题反馈→代码提交→代码审阅→项目维护者→技术委员会成员。鼓励非代码贡献（如文档翻译、技术分享），设立新手友好机制（如标记“初学者任务”）。将各种贡献记录作为人才评估依据，并及时给

予公平与公开的激励。这些内容将在第 4 篇详细展开。

(4) 品牌联动与商业化。

让核心开发者在项目中担任维护者角色，及时加入/捐赠到某个开源基金会，并争取决策席位。采用分层商业模式：开源基础版→企业增强版→托管服务→培训支持，形成从技术影响力到商业转化的闭环。这一部分要特别当心，有很多开源项目，因为商业化不当，反而招致社区的群起而攻之，品牌形象不升反降。总之，开源可以赚钱，但是要取之有道。

(5) 风险控制。

严格管理开源合规（使用 FOSSA 等工具扫描许可证），建立三方组件准入清单；推行透明的社区治理（公开决策流程、行为准则），避免企业单边主导；禁止刷 Star 等虚假行为，主动披露安全漏洞响应方案。各种各样的突发事件，都可能影响企业的技术品牌，积极预防+公开透明地应对，才是长治久安之道。

32.3 效果评估指标

本书一直在强调：没有度量就没有管理。围绕技术品牌建设，也应该有合适的评估指标（见表 32-2），更需要超越单纯关注 Star 数量的阶段，关注多维数据。

表 32-2 技术品牌的效果评估指标

阶段	核心目标	关键行动	品牌收益
消费阶段	降低开发成本	集成成熟组件（如 MySQL、Spark）	无，外界甚至不知道企业在消费开源
参与阶段	建立技术存在感	提交代码修复、参与社区讨论	树立“可靠贡献者”形象
贡献阶段	增强技术话语权	主导特性开发、贡献案例	成为“关键贡献者”，扩大影响
领导阶段	定义技术方向	发起项目、推动标准、捐赠基金会	晋升“技术领导者”的地位

另外值得注意的是，在技术品牌建设的不同阶段，评估指标的侧重维度也会有所不同。在早期阶段，不必太过于追求“商业转化与人才价值”。社区健康度与技术影响力的相关指标的讨论详见第 14 问。

32.4 针对中小企业的建议

首先，对于资源相对有限的中小企业而言，在开源领域建立技术品牌需要更聚焦和巧妙的策略。首要原则是精准选择战场，避免在巨头林立的基础设施层（如通用数据库、操作系统）正面竞争。相反，应该深入挖掘某个特定行业或应用场景的垂直痛点，提供高度专业化的解决方案。例如，Dify 没有试图构建一个通用大模型，而是精准定位在“LLM 应用开发”这个细分领域，专注于提供低代码工具

链，帮助企业快速构建和部署基于大语言模型的智能应用。这种聚焦使 Dify 能在巨头尚未完全覆盖的缝隙市场中快速建立用户群和技术口碑，有效避开红海竞争。

其次，中小企业应善于借力成熟的生态系统来放大自身声量。与其从零开始构建一个可能无人问津的全新平台，不如选择像 Kubernetes 或者流行的开源框架（如 Spring、React）这样拥有庞大用户基础和技术社区的平台。主动为这些平台开发高质量的插件、扩展工具或 Operator。例如，开发一个解决 Kubernetes 集群中特定监控难题的轻量级 Operator，或为热门中间件（如 Redis）提供一个提升管理效率的 GUI 工具。这样做的好处显而易见：一方面，能直接触达生态内庞大的潜在用户群体，快速获取初始流量和关注度；另一方面，项目被集成到主流生态中，本身就传递了技术兼容性和专业性的信号，极大地提升了项目的可信度和品牌能见度。

最后，也是最关键的一环，是将开源项目的用户转化为品牌的共建者和推广者。中小企业通常缺乏大型企业的市场预算，而活跃、忠诚的社区成员是最具性价比的“布道师”。可以借鉴 MongoDB 的成功经验。MongoDB 早期投入巨大资源建立了 MongoDB University，提供系统化的免费在线课程和认证计划。这种策略不仅降低了用户学习和使用的门槛，更重要的是，它系统地培养了大量掌握 MongoDB 技能的开发者。这些开发者在使用过程中成为产品的专家，在工作中自然地推广 MongoDB，为社区贡献最佳实践、解答问题，甚至影响所在公司的技术选型。这种由用户驱动的口碑传播和产品迭代反馈，形成了强大的飞轮效应，是 MongoDB 能在竞争激烈的数据库市场中脱颖而出的关键因素之一。对于中小企业，建立类似的开发者培养体系（如在线教程、动手实验室、认证项目），积极倾听社区反馈并快速迭代产品，是将用户资产转化为品牌增长核心动力的有效途径。

32.5 典型案例参考

（1）阿里云的 Dragonfly。

阿里云通过将自研的 Dragonfly 项目捐赠给云原生计算基金会，成功地在云原生技术领域树立了领导地位。Dragonfly 是一个高效、智能的 P2P 镜像分发系统，解决了大规模容器集群中镜像分发速度慢、网络消耗大的行业痛点。将其开源并纳入 CNCF 孵化，不仅使该项目成为云原生镜像分发的事实标准之一，更让阿里云从技术使用者跃升为关键基础设施的定义者和贡献者。这一战略举措显著提升了阿里云在全球开发者社区和云服务市场中的技术公信力和品牌影响力。

（2）谷歌的 Kubernetes 和 TensorFlow。

谷歌通过战略性开源 Kubernetes 和 TensorFlow 两大项目，实现了技术标准主导与商业利益的双赢。Kubernetes 彻底改变了容器编排的格局，成为云原生时代的核心基础设施，而 TensorFlow 则推动了 AI 的民主化，成为全球最流行的机器学习框架之一。谷歌不仅开源了核心代码，更积极投入资源构建社区、制定规范，并推动其成为行业标准。这种深度的参与和领导力，使得 Google Cloud Platform（GCP）能够紧密集成这些技术，将其作为云服务的核心竞争力，有效带动了云业

务的增长，证明了开源可以成为强大的商业引擎。

（3）红帽的 RHEL。

红帽的故事是开源商业模式成功的经典范例。它基于开源的 Linux 内核，构建了企业级的 Red Hat Enterprise Linux (RHEL) 发行版，并通过提供订阅服务（包括长期支持、安全更新、技术认证和咨询服务）实现了商业化。红帽深谙社区价值，其员工中有很高比例直接来自社区贡献者，确保了技术响应力和社区信任。这种“开源核心+企业级服务”的模式被证明极具价值，最终在 2019 年被 IBM 以高达 340 亿美元的价格收购，创造了开源领域的历史性交易，充分彰显了建立在强大开源技术品牌基础上的巨大商业潜力。

（4）PingCAP 的 TiDB。

PingCAP 作为一家中国数据库领域的创新公司，通过 TiDB（一个开源的分布式 NewSQL 数据库）项目，展示了如何通过全球化社区运营建立技术品牌。PingCAP 从项目伊始就坚持开源和社区优先的策略，投入大量资源进行全球化的技术布道、文档建设（提供多语言支持）和社区活动组织。他们积极吸引和培养全球贡献者，营造开放的协作氛围。其成果显著，TiDB 的 GitHub 社区吸引了大量海外开发者和企业用户，海外开发者的代码贡献占比长期稳定在 40% 以上。这不仅加速了 TiDB 的技术迭代，更重要的是，在全球化竞争激烈的数据库领域，成功地为 PingCAP 树立了国际化的技术品牌形象，证明了开源是跨越地域限制、建立全球技术影响力的有效途径。

第4篇 开源社区

本篇从社区的基本结构入手，描绘了创始人、核心贡献者、活跃贡献者、普通用户以及赞助方这一同心圆式的成员生态，并梳理了各类成员的来源与动机。随后聚焦于社区的两大关键挑战：如何吸引新血液和如何留住已有贡献者——从降低参与门槛、及时反馈、分层激励到营造归属感，给出了一套完整的思路。

对于想迈出第一步的读者，本篇各问提供了详细的入门指引，从选择项目、掌握 Git 基础到第一次提交 Pull Request 的完整流程。对于已在路上的贡献者，则勾勒了一条从新手到维护者的成长阶梯，分析了各阶段所需的核心能力、晋升机制以及可能遭遇的瓶颈与应对策略。

在社区治理层面，本篇探讨了冲突管理的思路与机制、包容性和多样性的价值与现实困境，以及如何评估贡献者的多维价值——不仅是代码提交，还包括文档、翻译、运营等非技术性贡献。最后两个问题则把视角拉得更宏观：开源项目如何实现经济、治理、技术 3 个维度的可持续运营，以及在明文规则之外，真正支配社区运转的潜规则与人情逻辑。

第 33 问 开源社区的结构与运作是怎样的

开源社区是一个由多种角色组成、强调协作、不断变化的有机体。它的结构主要基于成员的角色分工、非技术性的支持工作、社区的组织架构与决策机制，以及随着社区规模扩大而日益复杂的协作方式来构建。

33.1 社区的基本成员

我们可以把开源社区想象成一个同心圆，如图 33-1 所示，不同位置的人扮演着不同的角色，共同推动项目前进。

（1）创始人/核心贡献者。

他们是项目的支柱，通常是创始人或在项目中活跃了很久的成员。他们的工作包括审查和合并代码改动、修复关键问题和安全漏洞、设计系统架构、规划技术方向、管理新版本的发布，以及为社区提供技术指导。通常，他们拥有直接修改代码库、发布软件和对技术方向做出重要决定的权限。虽然这个群体相对稳定，但成员也可能随着项目发展而变动。

（2）活跃贡献者。

他们是经常或定期为项目贡献力量的成员，贡献形式多样。他们可能提交新功

能代码、修复程序缺陷、改进文档、回答用户的问题，或者协助进行代码审查。这个群体流动性较高，是核心贡献者的后备力量来源，他们的多样性对项目的创新至关重要。

（3）用户群体。

这是社区中最广泛的群体，也是潜在贡献者。他们主要使用软件，报告遇到的问题和错误，提出对新功能的需求，并在社区内外进行讨论和推广。用户提供的真实反馈是推动产品改进的核心动力，也直接提升了项目的影响力。

（4）赞助与支持方。

他们为项目提供必要的资源支持。支持的形式包括资金捐助（来自公司或基金会）以及基础设施（如服务器、持续集成平台、云服务资源）、法律、品牌或市场推广方面的帮助。对于大型项目来说，这种支持是它们能够长期稳定发展的基石。

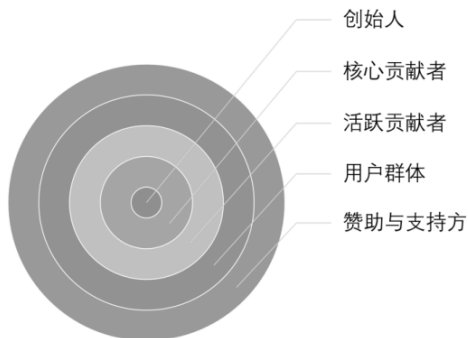


图 33-1 开源社区的成员形成了一个同心圆结构

33.2 社区成员的来源

正如上文所述，社区成员中最广泛的群体是用户。这些用户在使用开源的过程中，会自然在社区中与其他用户和贡献者交流。在合适的情况下，他们也会转化为贡献者，甚至核心贡献者。在某种宽泛的定义中，仅仅是在社区提问、提需求，也可以被认为是一种社区贡献，同样会受到欢迎。

另有一类成员，通常是出于学习的动机来到社区。他们或者是因为好奇心，或者是希望提高自己，又或者是希望为自己今后的工作履历增添光彩，都会以较大的热情投入社区，并积极希望成为社区的贡献者。

又有一类成员，是来自企业的研发人员，他们在自己的工作中需要用到开源软件，随着使用的深度不断增加，他们不得不努力研究该软件，甚至会成为该软件的非正式专家。他们或者擅长在社区中回答问题，或者被公司指派，主动为这个社区贡献代码，也可能仅仅是出于企业的技术需要，只修改影响企业产品的漏洞。虽然他们的言行可能会比较功利，交流方式也比较急迫，但是也不乏能够持续提交真正高质量代码的贡献者。

还有一类成员，是其他开源项目的开发者，因为软件相互依赖的关系，自然而

然地用到了这个软件，甚至也因此发现了这个软件的漏洞。他们已经非常熟悉开源社区的协作模式，也会很自然地为此软件顺手做一些贡献。

33.3 维护者与核心开发者的工作

维护者和核心开发者是项目的技术核心，他们的工作主要集中在 3 个方面。

(1) 代码维护。包括审查和合并他人提交的代码改动、修复安全漏洞、确保代码结构清晰一致及处理技术债务（对旧代码进行必要的优化和重构）。

(2) 版本管理。制定软件版本的发布计划、管理不同版本的代码分支和标签、保证新版本的兼容性和稳定性、撰写和发布版本更新说明。

(3) 方向制定。规划项目的技术路线图和功能优先级、评估新技术的可行性、对外宣传项目技术、参与社区的战略讨论。这些职责与企业中的技术负责人或产品经理有相似之处，但更强调开放协作和过程透明。

33.4 其他非技术性工作

技术之外的角色对社区的活跃度和健康运转同样至关重要。

(1) 文档编写与维护。包括面向用户的使用指南、安装说明、常见问题解答；面向开发者的贡献流程说明、系统架构介绍；治理相关的行为准则、决策流程文档。多项研究报告表明，文档不完善是阻碍人们参与贡献的最大障碍之一。

(2) 传播推广。在社交媒体上宣传项目、撰写博客文章分享案例研究、在技术会议上演讲介绍、发布项目新闻通信等，帮助项目扩大影响力。

(3) 活动组织。组织编程马拉松（Hackathon）、线上研讨会、线下见面会（Meetup）、建立本地化社区小组，以及参与或组织大型年度会议（如 KubeCon、ApacheCon）等。

(4) 社区管理与运营。欢迎和引导新成员加入、维护社区沟通渠道（如 Slack、论坛、邮件列表、微信群）、执行行为准则、维护社区氛围、管理社区使用的工具和平台。Mozilla 的研究表明，如果能在 48 小时内回应新贡献者，他们长期留下的可能性会显著增加。

33.5 社区的组织架构与决策机制

不同的社区，可能会有不同形态的组织架构与决策机制，在第 12 问中有详细讨论。

第 34 问 开源社区如何吸引和留住贡献者

开源社区要想长期繁荣，需要把吸引新贡献者和留住已有贡献者看作相辅相成

的过程，它们共同决定了一个项目的生命力。一个健康的社区应该有低门槛的参与途径、明确的支持机制和持续成长的机会。从吸引策略、留存策略、治理工具与未来趋势 4 个角度来看，可以更系统地理解如何建设这样一个社区。

34.1 吸引贡献者的策略

社区要先让新加入的人感觉“可以尝试”，再让他们体会到“值得投入”。

一、降低参与门槛

完善贡献指南是关键。很多成功的社区都在代码库中提供详细的 CONTRIBUTING.md 文件，包括项目介绍、环境配置、分支策略和提交流程，还会配套 FAQ 和示例 Pull Request，帮助新手快速上手。Kubernetes 和 Node.js 都依靠分层文档有效降低了新手流失率。

标记入门友好的任务也非常有效，如 GitHub 上的 good first issue、help wanted，以及 Fedora 的 EasyFix 标签，这些都方便新贡献者找到合适的切入点。

自动化工具也值得重视，很多社区提供预配置的开发环境（如 Gitpod、Codespaces），或使用 CI/CD 来自动检测提交格式。

二、清晰的价值主张

让人清楚为什么要花时间做贡献。参与开源不仅能掌握新技能，还能提升职业影响力。超一半的开发者通过开源项目学习了分布式系统、云原生等新技术，很多社区还提供徽章、认证或推荐信，增强职业背书。在部分企业，如华为昇腾社区，开源贡献还被视为重要的绩效参考。

总之，“来到我们的社区，你会有哪些收益”，将社区能够为贡献者带来的好处清晰地表达出来，也能够更好地吸引潜在的加入者。

三、多渠道宣传推广

线上线下的宣传和活动都能提高曝光。社交媒体（如 Twitter、LinkedIn、微信与微博）与技术博客（Dev.to、Medium、微信公众号、视频号、知乎）是主要途径。线下则包括 Hackathon、Meetup 和年度大会。例如，Hacktoberfest、KubeCon、OSCon、中国开源年会、“源创会”这类活动可以明显带动参与度。有调研显示，参加线下活动的开发者，会显著提高后续贡献的活跃度。

技术文章的写作、宣传视频的制作，尤其是一个好的 Demo 视频，需要花费相当多的精力与创意，当然，收效也会相当显著。

34.2 留住贡献者的策略

在有人做出了贡献，已经与社区结缘之后，如何让他们愿意留下，是社区治理的另一个重点。

一、及时反馈和认可

Kubernetes 曾设定 72 小时内首次回应的“服务级别协议”，超时则自动提醒评

审人。这样的机制能让新手感受到重视。即便拒绝贡献者的代码提交，也要解释原因，并给出修改建议。公开感谢和认可也很重要，常见方式包括在 README 鸣谢、在 Release Notes 署名、在社交媒体表扬等。有一些社区还会定期发表感谢信，详细列出贡献者名单以及他们做出的重要贡献。或者在每一次新版本发布的时候，感谢所有为这一版本作出贡献的朋友。总之，确保作出贡献的人不会感到被忽视非常重要。

二、分层激励和成长路径

一方面可以用徽章、排行榜等精神激励，另一方面也可通过 T 恤、贴纸、GitHub Sponsors 等物质回馈，或者给贡献者提供更多责任和机会。许多社区会让贡献者逐步从 Contributor 成长为 Committer 再到 Maintainer，Node.js 和 Apache 都有明确的分级晋升体系。有很多开源贡献者在写回忆文章时，谈到他们收到社区提名，邀请他们担任更高一级的职位时，都会很激动，也充满了感激。

社区也会支持贡献者的职业发展，例如，推荐演讲机会或帮助撰写技术文章，或者直接在社区成员求职的时候，帮忙写推荐信。事实上，很多开源人在社区中交换职业发展机会，这就是他们日常社交的一部分。

三、营造归属感和文化认同

明确的行为准则是基础，例如，Python 社区长期坚持包容、尊重的沟通文化。此外，非正式同步等情感联结活动也会增强黏性。据红帽公司公布的报告，情感联结有助于提升社区留存。透明的决策过程同样重要，例如，公开路线图，让社区成员有机会参与讨论和决策，会极大增强社区的凝聚力。

有很多社区会定期召开 AMA（Ask Me Anything）的线上或线下会议，就是鼓励所有的社区成员，向社区管理者提出任何问题，而社区管理者则会尽可能公开、坦诚甚至毫无保留地回答这些问题。这是社区文化，也是社区魅力的一部分。

34.3 社区治理和工具支持

开源社区的人，也是最擅长利用工具的人，他们自己编写各种各样的软件，也擅长使用、改造甚至干脆自己创造各种各样的工具。下面列出的，主要是对于吸引与留住贡献者有帮助的工具。

（1）代码托管平台。在最热门的代码托管平台创建新项目，能够更好地吸引贡献者，这就是集聚效应。

（2）项目管理软件。现在的开源社区往往都会选择直接使用代码托管平台提供的项目管理功能，虽然原来还流行过其他的一些工具，如 Redmine、Jira、Trello 之类。

（3）社交协作平台。在国内，很多社区会在微信、企业微信、飞书等平台之间艰难抉择。而在国外，往往会选择 Slack、StackOverflow，或者其他的一些工具。

（4）社区评估工具。详见第 14 问。

（5）其他自动化工具。AI 增强的 Code Review 工具、AI 增强的问题内容翻译工具、自动化的 CLA 流程签署工具等，都能有效地提升开发者的用户体验。

开源社区的协作模式，事实上是在不断演进的，从早期以 Mailing List 为核心，到 Web 流行之后出现的集成开发平台，再到 GitHub 出现之后的社交化协作平台。背后的推动因素，都是研发工具的不断进化。而研发工具进化的逻辑则非常简单：降低门槛，提高效率。至于吸引与留住贡献者，则是工具不断进化之后，带来的“副作用”。

34.4 未来挑战和特征

未来，开源社区还会面临一些新挑战，包括商业、政治、法律与 AI。

- 开源与商业的关系一直在演化，由此带来的挑战是：繁荣的社区与赚钱的生意如何兼得的两难问题。
- 全球化协作面临越来越大的挑战，从地缘政治到全球不同的监管与合规要求。
- AI 正在加速赋能开发者。未来的社区里，百分之多少的代码会是人类纯手工编写的？

不过，无论未来的挑战如何严峻，成功的开源社区需要具备以下特征。

- 入门门槛低，方便新手上手。
- 反馈及时，让人觉得被重视。
- 有归属感和情感联结。
- 清晰的成长路径，能持续成长。
- 价值共享，共同塑造社区。

希望以上的总结，能够给社区的管理者一些启发。

第 35 问 如何开始为一个开源项目做贡献

开源软件正在改变世界。从操作系统（如 Linux）、浏览器（如 Firefox）、编程语言（如 Python），到日常使用的工具，许多都是来自全球开发者的协作成果。对初学者而言，首次参与开源既令人期待又充满未知：该如何开始？需要多高的技术水平？是否必须写代码？本文将简单的方式，带你迈出贡献开源的第一步。

35.1 参与开源的意义

参与开源，能够更好地掌握软件开发方面的技能。软件是近百年发展最快的技术之一，特别是随着智能手机与平板电脑的普及，软件技术更是渗透到人们生活的各个方面。学会编程，并不意味着要去建立一个复杂的系统。其实，写个报表的计算公式或者做一个小动画，也可以是一种编程的体验。而开源社区，则是学习软件开发最好的“学校”。在开源社区里，有大量热心的程序员，他们乐于分享自己对技术的理解、心得，并通过各自的行动扩大自己的影响力，在协助别人的同时不断

加深自己对技术的理解并提升实践能力。而新的社区加入者也可以在与社区互动的过程中找到自己的技术与精神导师（Mentor）。

通常，我们在求职时，总要提交自己的个人简历。而开源社区的贡献历程，则是更有说服力且更加透明可信的简历。越来越多的开发者会在简历中标注自己的GitHub/Gitee 账号，就是这样一种趋势。如果你是 Apache 项目或者 Linux 内核项目的贡献者，甚至是维护者，那么找到一份好工作将会更加容易。

除了以上两种比较功利的目的，还有两种不那么功利的目的。一方面是在社区里结交更多的朋友，另一方面则是通过贡献社区，创造全新的功能，获得心理上的满足感。也有很多朋友，原本是抱着功利的目的来到社区，却收获了更多的快乐与荣誉。

35.2 贡献的多种形式

也许有人 would 认为，要给开源做贡献，就必须先学会写代码。其实还有很多种贡献开源的方式，如表 35-1 所示。

表 35-1 贡献开源的方式

贡献方式	具体内容
修复代码	修复程序错误或开发新功能
完善文档	修正拼写、更新示例、翻译内容
设计优化	改进界面或设计图标
测试反馈	测试软件并报告问题
社区支持	解答问题、参与讨论、整理常见问题、组织线上或线下活动、技术布道等

除了编写代码，社区还需要做大量的文档编写、完善、翻译等工作。有不少开源项目代码本身写得不错，文档却一塌糊涂，如果有人能够帮助他们完善文档，他们都会非常感激。如果你还有不错的审美，能够做一些视觉设计，就更是社区最紧缺的人才，太多的开源软件，UI/UE 都亟待改进。

作为一个开源软件的普通用户，在使用过程中遇到问题，及时、准确、完整地提交缺陷报告（Bug Report）也是非常重要的社区贡献。

另外，在一个内向型人格积聚的地方，如果你正好是外向型人格，乐于分享，善于交流，喜欢组织线上线下的活动，甚至能登台演讲，那就更好了。社区的发展，离不开社区支持与运营的贡献。

总之，你不能完全不懂技术，但是也不必成为编程高手，就能够加入社区，并且为社区作出贡献。

35.3 需要的基本技能

虽然不需要成为编程高手，但是基本的技能还是需要掌握的。

(1) Git 版本控制。现在几乎所有的开源项目都会托管在 GitHub、GitLab、Gitee 或者类似的代码托管平台上。这些平台无一例外都使用 Git 来管理代码版本。所以,理解什么是版本、版本控制工具、仓库、分支与提交、Fork 与发起 Pull Request 等都是开发者必须掌握的基本功。推荐学习《Pro Git》中文版,这是一本免费的电子书,在线就能搜到。

(2) 代码托管平台。GitHub、GitLab、Gitee 等平台的基本操作其实非常简单,而且这些平台的操作大同小异,只需要学一遍就行。

(3) 阅读能力+耐心。开源社区都比较强调自助服务。你来到一个社区,首先应尽量阅读文档,尝试自己理解并运行相关软件,只有确实遇到困难,才去求助。而不是在社区里反复提交一些早已被回答过无数遍的问题。

(4) 沟通能力。又是一个软技能,学习礼貌、清晰、有条理地提问,有理有节地回答问题,非常重要。特别需要注意:提问前先查阅文档和历史讨论记录;清晰地描述问题或改动目的,避免模糊表达;保持礼貌,尊重不同意见;理解维护者多为志愿者,耐心等待回复。无论你出于什么样的目的,仅仅是一次性的贡献,还是永久性地加入社区,都得和他人进行沟通和交往,这是你要在开源圈发展的必须修炼的技能。

35.4 寻找适合新手的开源项目

选择一个开源软件,首先要明确的是,自己的动力何在。是出于兴趣?还是出于工作需要?例如,有人对搜索引擎特别感兴趣,想了解它是怎么做出来的。那么可以考虑先寻找一些专业的书籍,来了解一些关键的知识点。如果对于某一领域的知识点缺乏必要的了解,可能完全无法理解一个项目里的代码。在掌握初步的知识以后,自然可以去找一些常见的开源项目来学习。也可能是出于工作需要,例如,平时使用 PHP 开发 Web 应用时已经在用某一个常见的 PHP 框架了,希望能够对这个框架有更深入的了解,甚至希望横向地比较多个不同的 PHP 框架,这些都是非常清晰的目的。这样在学习的过程中,才不会迷失方向。

比较危险的一种情况是,听说某某项目很有名气,甚至是为了将来找工作更容易,就贸然一头扎进某个项目中去了。怀有这种学习目的的人,往往会选择那种很庞大且成熟的项目,打开文件夹一看,成百上千的源文件根本无法看完,一下子就懵了,然后灰心丧气,更容易放弃。

一、优先选择能够独立运行的项目

开源的项目有很多种类,能够独立运行的项目很多。但也有不少项目是其他开源项目的插件、类库或扩展包,这些在一开始接触开源时最好不要涉猎,因为理解这些项目可能需要理解它们背后的庞然大物,这往往会遭遇很多难解的细节,一不小心就无法继续进行。

当然,还有一类项目虽然能够独立运行,但要成功运行还需要安装和配置许多其他依赖项目,这往往会让初学者特别绝望,折腾了一个星期,项目居然还不能正常运行!

因此，那些规模小、能够独立运行且不依赖于太多其他项目的开源项目，可以优先选择。

补充说明：在 Docker 出现以后，运行一个项目往往只需要一行简单的命令。在 Github 上，也能找到很多 demo 项目，或者是 docker-compose.yml 文件，这些都是很好的入手点。

二、选择活跃的项目

项目的活跃程度包括两个方面：一方面是开发者提交新代码的频繁程度，另一方面是在社区中对这个项目的讨论热烈程度。代码提交越活跃，参与的人越多，越能证明这个项目很有价值，也表明它值得你投入精力学习。而项目在社区讨论的热烈程度，则确保当你遇到问题的时候，能够搜索到别人的答案，或者你自己提问以后，能够有人热心回答你。

当然，活跃程度都是相对的，如果你真的对一个项目感兴趣，可以试着直接给这个项目的作者发邮件、提问题。大多数开发者都会很高兴有人关注他的项目，通常也会热心地回答你的问题。

三、判断代码质量

并非所有的开源项目都是高手写的，都值得你去学习。事实上，有很多低质量开源项目，仔细查看会发现其代码写得一塌糊涂。所以，试着阅读一下这个项目的代码。至于如何判断一个项目的代码质量，笔者在知乎曾回答过一个类似的问题“如何让自己写的代码易维护？”推荐给各位朋友参考。当然，更加推荐的是阅读《代码整洁之道》一书，这是一本非常好的介绍如何提高代码质量的书。

四、搜索技巧

在 GitHub 上搜索标签：is:issue is:open label:"good first issue"（新手任务）或 label:"help wanted"（急需帮助）。通常能够找到不少对于新手友好的任务，但是这些任务所属的项目是否适合你，还得通过前面的几条来判断。

35.5 第一次贡献的完整流程

以在 GitHub 上为某个项目修复文档错别字为例，在动手执行下面的 8 个步骤之前，还要确保你的机器上已经正确安装了 Git 客户端。

- （1）Fork 仓库：在该项目的主页单击“Fork”按钮，复制到你的账户。
- （2）复制到本地：运行命令 `git clone https://github.com/你的用户名/项目名.git`。
- （3）创建新分支：执行 `git checkout -b fix-typo-readme` 建立修改分支。fix-typo-readme 是你为这个分支起的名字。
- （4）修改内容：用文本编辑器修正文件（如 README.md）。
- （5）提交更改：依次运行 `git add README.md` 和 `git commit -m "修复 README 中的拼写错误"`。
- （6）推送分支：使用 `git push origin fix-typo-readme` 同步到远程仓库。
- （7）发起合并请求：在 GitHub 的项目主页上单击“Compare & pull request”

按钮，描述修改内容。若关联已有问题，请注明“Fixes #问题编号”。

(8) 响应反馈：根据维护者建议调整代码，直至合并完成。

在 GitHub 上有一个著名的开源项目，能够帮助那些想要做出自己的第一次开源贡献的朋友，名字就叫 first-contributions。

35.6 其他帮助新手入门的资源

- MunGell/awesome-for-beginners：一个包含各种编程语言和技术的初学者友好项目列表。
- First Timers Only：帮助新手参与开源项目的网站，提供指导和项目列表。
- HelloGitHub：一个专注于分享 GitHub 上有趣、入门级开源项目的社区，特别适合中国用户。
- 如何选择（并贡献）第一个开源项目：GitHub 官方搜集的适合新手入门的项目。

按照语言/方向推荐，适合新手的开源项目如表 35-2 所示。

表 35-2 适合新手的开源项目

语言/方向	代表项目示例（均含 good first issue）	亮点简述
Python	bokeh、arviz、borgbackup	数据可视化、备份工具，文档完善
JavaScript	hoppscotch、docz	API 调试、文档生成，前端友好
Rust	rustlings、nushell	交互式学习+现代 shell，社区活跃
Mobile	MobSF	移动安全测试框架，需求旺盛
DevOps	GlusterFS、Ceph、OpenStack	存储与云计算大厂项目，成长空间大
QA/测试	DiStaf、Teuthology	分布式系统测试框架，技能提升快

35.7 开始行动

参与开源无须一步到位，一次微小的改进就能成为社区一员。与其瞻前顾后，不如轻松一跃。开源世界始终向所有愿意参与的人开放。每一次贡献都在推动技术进步，祝你的开源之旅顺利启程！

第 36 问 从贡献者到维护者的成长路径是什么

在开源社区里，很多开发者都梦想着从一个初次提交代码的新手，一步步成长为能够决定项目方向、保证代码质量并推动社区发展的项目维护者。这条成长之路没有捷径，需要持续的技术积累、积极的社区协作、逐步建立的信任，以及愿意承

担更多责任。

下面梳理这条成长路径上的典型阶段、需要掌握的核心能力、常见的晋升方式以及一些加速成长的实用建议，希望能帮助你在开源社区中明确方向，稳步前进。

36.1 成长的典型阶段

从参与开源到成为核心成员，开源贡献者通常会经历几个比较清晰的成长阶段，如表 36-1 所示。

表 36-1 开源贡献者的成长阶段

角色	定义
新手贡献者（Contributor）	以任何形式为项目提供有价值成果（代码、文档、测试、设计等）并合并进主线的人
活跃贡献者（Active Contributor）	独立解决中等难度问题（如功能优化、模块漏洞）；参与代码审查（Review PRs），提出改进建议
提交者（Committer）	拥有代码仓库写权限，可直接合并 PR，是 Maintainer 的后备军
维护者（Maintainer）	对项目方向、质量、发布节奏负责的长期决策者与守门人

最开始的阶段是新手贡献者。这时，你会接触一些相对简单的任务，比如修正文档里的错别字、修复小的程序错误或者调整界面样式。社区里通常会有标记为“适合新手”的任务，这是很好的起点。在这个阶段，最重要的是认真阅读项目的贡献指南（CONTRIBUTING.md）和行为准则（CODE_OF_CONDUCT.md），了解清楚提交代码的流程和代码审查的要求。每次提交后，主动寻求他人的反馈并根据建议改进，是快速学习的关键。在上一个问题中，我们主要介绍了这方面的入门方法。

当你能够稳定地贡献代码，并开始解决一些中等难度的问题时，就进入了活跃贡献者阶段。你会更深入地了解项目的某个特定模块或功能领域，开始参与审查别人的代码提交，并在技术讨论中提出自己的想法。这个阶段的关键是保持一定的贡献频率（比如每月都有提交），展现出你对项目整体架构和发展方向的理解，并在社区中逐渐形成自己的技术影响力。

下一个重要的跃升是成为提交者。这意味着你获得了将代码合并到项目主分支的权限。角色变化带来的是更大的责任：你需要协助管理问题报告、认真审查他人的代码、维护项目的测试流程，甚至参与版本发布计划和技术路线的讨论。晋升到这个角色通常需要现有维护者的提名，并经过社区共识或投票确认，这背后是你长期稳定的高质量贡献和良好的技术判断力。

最终的目标是成为维护者或项目管理委员会成员（PMC）。这时，你的职责范围大大扩展，包括决定项目的核心发展方向和关键功能、协调社区讨论并解决可能

的冲突、组织专门的兴趣小组（SIG）或孵化新的子项目，以及指导和帮助新的贡献者，并提名新的提交者。这个角色对综合能力要求很高，不仅需要深厚的技术功底，还需要出色的沟通协调能力和一定的社区治理经验。

36.2 需要掌握的核心能力

要想在这条路径上走得更远，需要系统地培养几方面的能力。表 36-2 中总结了这些关键能力维度及其具体内容。其中，技术是基础，协作是日常行为，治理是规则理解，领导力的本质就是推动力。

表 36-2 开源社区贡献者的核心能力

能力维度	关键内容
技术能力	精通项目所使用的核心技术（例如，在 Kubernetes 项目中要理解 CNI、CRI 等概念），熟练掌握项目主要编程语言（如 Go、Python），并能有效运用 CI/CD 等工程化工具
协作能力	善于清晰地表达自己的想法，积极参与社区讨论，能够撰写清楚明了的代码提交说明和问题报告，并能处理好跨语言或跨文化的交流
治理能力	理解项目版本规划、权限控制的方式以及贡献流程的运作机制，有能力维护社区的规则，营造公平、健康的讨论氛围
领导能力	能够主动发起并组织线上或线下会议、培训活动、代码冲刺（Code Sprint），激励和带动其他成员参与，并有效地指导和帮助新加入的贡献者

36.3 如何获得晋升与建立信任

在开源社区里，地位的提升和权限的获得，核心在于信任的积累。这个过程通常是自然发生的：你持续做出有价值的贡献→社区成员（尤其是现有维护者）开始信任你的能力和判断→被赋予更多的权限（如合并代码）→承担起更大的责任。行动永远比口头承诺更有力量，稳定、高质量的贡献是赢得信任最根本的途径。

不同的社区也有相对正式的晋升机制。例如，Apache 软件基金会的项目普遍采用“共识制”（+1/0/-1 投票）来决定新提交者或维护者的加入。云原生计算基金会旗下的项目，通常由现有维护者提名候选人，然后由相关的兴趣小组（SIG）投票确认。一些新兴的 Web3 开源项目，则尝试利用去中心化自治组织（DAO）和区块链上的贡献积分（如 Sentient 项目）来量化贡献和进行治理。

社区在考察一个贡献者是否具备晋升资格时，往往会关注几个关键指标：你的活跃度（提交代码的频率、审查他人代码的数量）、你在社区中的认可度（响应问题报告的速度、提出的技术方案被采纳的比例），以及你是否展现出领导潜力（如主动承担复杂任务、帮助指导他人或者牵头负责某个子项目）。当我们谈到指标的

时候，也需要注意：这些待考察的方面都很难量化，在很多社区，还是看管理者的整体印象，未必就是完全客观、公平的。

36.4 加速成长的实用策略

如果你想更快地在开源社区中成长起来，可以尝试表 36-3 中的策略，这是在开源社区中成长的具体行动建议，从专注领域到承担责任，从积极参与到主动分享，都是建立影响力和信任的有效途径。

表 36-3 在开源社区中成长的策略与行动建议

策略	具体行动
聚焦特定模块	选择一个项目的子系统或功能模块深入学习，努力成为这个领域的专家，这会让你在社区中拥有独特价值
勇于承担基础工作	主动去做看起来不那么“炫酷”但很重要的工作，例如，维护更新文档、优化自动化构建流程、清理历史遗留的“技术债”。这些工作能赢得社区的尊重和信任
积极参与审查与讨论	主动去审查别人的代码提交（Pull Request），积极参与技术路线的讨论甚至辩论，提出建设性意见
主动建立人脉	寻找一位经验丰富的导师（Mentor）、积极参加线上线下的社区会议、加入你感兴趣的兴趣小组（SIG）
善用工具提效	合理利用各种 AI 辅助工具来提高编码效率，但切记不能依赖工具而忽视对代码原理的深入理解
分享创造影响力	通过写技术博客、在开源会议上演讲、参与教育培训项目等方式，分享你的知识和经验。这能有效展示你的技术能力和领导潜力，扩大你在社区内外的影响力

36.5 可能遇到的挑战及应对

在社区中，难免会遇到一些困难，要总结经验，找到对策。

- 时间精力有限：开源贡献通常是利用业余时间。应对建议是：规划好每周固定投入的时间，优先做那些对社区最有影响力、对自己成长最有利的任务。
- 缺乏反馈：有时提交代码后可能得不到任何回应。主动出击是关键：在提交中礼貌地请求特定维护者审查，或者利用社区沟通渠道（如邮件列表、聊天群）友好地提醒和询问。
- 感觉社区门槛高：面对庞大的代码库和复杂的流程可能不知所措。从小处着手，从修复文档或小漏洞开始；认真阅读项目文档；利用一些基金会（如 Linux 基金会）提供的导师计划（Mentorship）。
- 不清楚晋升标准：每个社区的规则可能不同。仔细阅读项目官方的治理文档（Governance），了解晋升流程和要求。如果还是不清楚，可以私下礼

貌地咨询一位你熟悉的维护者。

- 非技术能力不足：随着角色提升，沟通、协调、冲突调解等“软技能”越来越重要。有意识地练习写作、学习会议组织技巧、培养解决分歧的能力，这些都非常关键。

第 37 问 开源社区如何管理冲突和分歧

开源社区汇集了来自不同文化背景、技术水平和工作角色的参与者，这些参与者基于自愿协作共同推动项目发展。这种开放性和多样性是创新的源泉，但也意味着冲突和分歧几乎是不可避免的。这些冲突可能源于多个方面：对技术的不同见解（采用哪种架构、代码质量标准或性能要求），对开源理念的不同理解与坚持（选择哪种许可证或治理模式），管理流程上的摩擦（谁有权合并代码、多久发布一个新版本），或者关于权力和职责范围的争议（贡献者、维护者、核心开发者之间的界限，代码贡献与非代码贡献之间的重要性差异），以及在国际化程度较高的项目中跨文化交流带来的误解。

正如开源倡导者雷蒙德所说，“只要有足够多的人查看，所有的漏洞都会变得显而易见。”但这也意味着会有“足够多的声音和不同意见”。因此，如何有效地管理这些冲突，是开源社区健康发展的关键。管理的目标不是消除所有分歧，而是创造一个环境，让不同的意见能够被健康地表达、被理解，并最终推动社区达成共识，维持协作的活力。

37.1 冲突管理的总体思路

开源社区通常从几个层面来构建冲突管理的策略。要点在于：通过良好的规则与治理做好预防，倡导透明、公开的沟通方式。遇到无法回避的冲突与危机，也只能迎难而上。

- 预防：通过建立社区文化、明确规则和利用工具，提前减少矛盾发生的可能。
- 治理结构：明确划分不同角色的权力和责任，建立调解和决策的路径。
- 沟通：推动透明、互相尊重且方式多样的沟通实践。
- 危机应对：准备好处理那些难以调和的严重冲突，尽力避免社区分裂或成员流失。当然，如果“悲剧”无可避免，也尽量好聚好散。

37.2 建立清晰的行为准则和社区规范

建立清晰的行为准则和社区规范是预防冲突的基础。一份好的行为准则（Code of Conduct）会明确规定社区禁止的行为（如人身攻击、歧视或骚扰），并倡导尊重与包容。详细的贡献指南（如项目中的 CONTRIBUTING.md 文件）则告诉

参与者如何提交代码、报告问题以及处理技术分歧，这些清晰的指引能显著降低日常协作中的摩擦。此外，制定社区或组织的章程，明确阐述社区的共同价值观（例如，是更看重创新突破还是稳定可靠）也有助于成员在大的方向上保持一致。许多社区的经验表明，拥有明确行为规范的社区，其内部冲突的发生频率明显降低。

37.3 构建结构化的沟通与决策流程

清晰的流程能有效解决分歧。前提是需要定义社区成员的不同角色及其职责，例如，维护者通常拥有最终的技术决策权，而普通贡献者可以通过持续贡献获得信任和更大的话语权。对于重要的决策，采用提案—讨论—评审的机制。例如，Apache 项目遵循 RFC（Request for Comments）流程：重大决策需经过邮件列表 72 小时公示期，采用“+1/-1”投票机制，非常有效。有人提出正式建议，社区公开讨论，最后经过评审做出决定。在决策方式上，常采用“懒人共识”原则，即如果没有人明确反对，提案就被视为通过，这有助于提高效率。当共识难以达成时，则启动正式的投票程序，由维护者团队或专门委员会进行表决。例如，Node.js 社区就通过其技术指导委员会（TSC）成功解决了关于 ES6 模块化的重要争议。

除了决策流程，组织的常见思路是针对不同的问题，设立不同的委员会来负责。例如，技术相关的争议，就由技术监督委员会（TOC）或技术指导委员会（TSC）来解决。组织章程修改的问题交由理事会讨论、起草并由正式成员大会表决。总之，正式渠道越健全，越能够减少纠纷与争议。

37.4 实施协作式冲突解决机制

当冲突发生时，一个逐步升级的解决流程往往很有效：通常从相关方的小范围讨论开始。如果无法解决，那么由该模块或领域的负责人介入调解；若调解无效，则上升到维护者团队进行集体讨论和投票；在极端情况下，可能需要寻求社区所属基金会或中立第三方的仲裁。在实践中，有几个关键原则很重要：鼓励在公开的异步平台（如 GitHub Issues、邮件列表）上进行讨论，让过程透明；强调就事论事，针对观点进行辩论，而不是进行人身攻击；引导大家使用中性、建设性的语言表达分歧（例如，用“我认为……因为……”这样的句式）；在必要时，及时引入中立的调解员或设立专门的冲突处理小组（像云原生计算基金会的做法）。

37.5 利用技术与工具辅助管理

现代工具可以在一定程度上帮助缓解冲突，尤其是因为管理者人手不足、精力不够，成为社区治理的瓶颈时，工具特别有用。

- 自动化工具（如 GitHub Actions）：自动运行测试和代码检查，减少因代码错误或构建失败这类低级问题引发的争执。

- 情绪分析工具：能够识别讨论中可能出现的攻击性或过激言论，及时提醒参与者保持理性和尊重。
- 多语言协作工具：帮助降低语言障碍，减少对非英语母语贡献者表达的误判。
- AI 辅助分析工具：在跨文化沟通中，可以帮助协调者更清晰地识别分歧的核心，并提供对话建议。

虽然具体的量化效果可能因社区和工具而异，但经验表明，这类工具在提前预警潜在冲突和改善讨论氛围方面确有积极作用。

37.6 坚持公开透明与持续反思

透明是建立信任的关键。像 Apache、Linux 内核这样的社区，会将重要争议的讨论和解决过程公开存档，供所有人查阅，这极大地增强了社区的信任度。冲突过后进行“复盘”也很有价值，大家一起总结经验教训，看看是否需要更新行为准则或贡献流程。培养一种“学习型”的社区文化也很重要，鼓励大家将批评视为改进的机会，理解建设性的分歧可以成为创新的催化剂。

在这方面，国内很多社区存在不足之处，社区成员习惯于使用微信、QQ 之类的工具进行日常沟通，缺乏使用邮件列表的习惯。面对这样的现状，作为社区管理者，需要更加关注社区运营的透明度。例如，及时整理并公开相关的会议纪要与决策记录，定期与社区成员公开、坦诚地沟通（如前文提到的 AMA）等，都是比较好的办法。

37.7 应对极端冲突的策略

即使有完善的机制，极端冲突仍可能发生。以下是一些常见情境及其应对策略。

- 无法调和的技术路线分歧：最著名的解决方案是允许项目“分叉”（Fork），即创建一个新的独立项目。健康的做法是公开说明分叉的原因和两个项目未来的分工，避免恶意分裂。
- 严重违反行为准则：社区需要采取行动，可能包括暂停涉事成员的权限、发出社区公开警告，或者在情节严重时永久移除该成员。
- 社区领导层危机：当内部无法解决领导层矛盾时，可能需要引入项目所属的外部基金会（如 Linux 基金会）或其他中立第三方进行调解和裁决。
- 治理模式僵化失效：如果现有治理结构无法适应社区发展，可能需要启动治理模式的重构，如 Rust 社区在 2021 年重组其领导委员会的典型案例。

在面临极端冲突时，一味地息事宁人、希望大家各退一步的做法，未必是最佳

的策略。最重要的是坚持社区的根本原则，坚持做艰难但正确的事情。

第 38 问 开源社区的包容性和多样性为何重要

多元（Diversity）、公平（Equity）、包容（Inclusion），这 3 个词合在一起被简称为 DEI，与开源社区发展理念契合。从开源文化来说，就应该公开地欢迎所有可能的贡献与所有不同身份的贡献者。然而，事实并非如此。Apache 软件基金会在 2023 年发布了一份“ASF 社区的多样性和包容性现状调查报告”，通过发放问卷的方式，调查团队在 2020 年收到了 624 份回复，在 2022 年收到了 432 份回复。我们可以只看 2022 年的，被认为已经有所改进之后的数据，如图 38-1 所示。

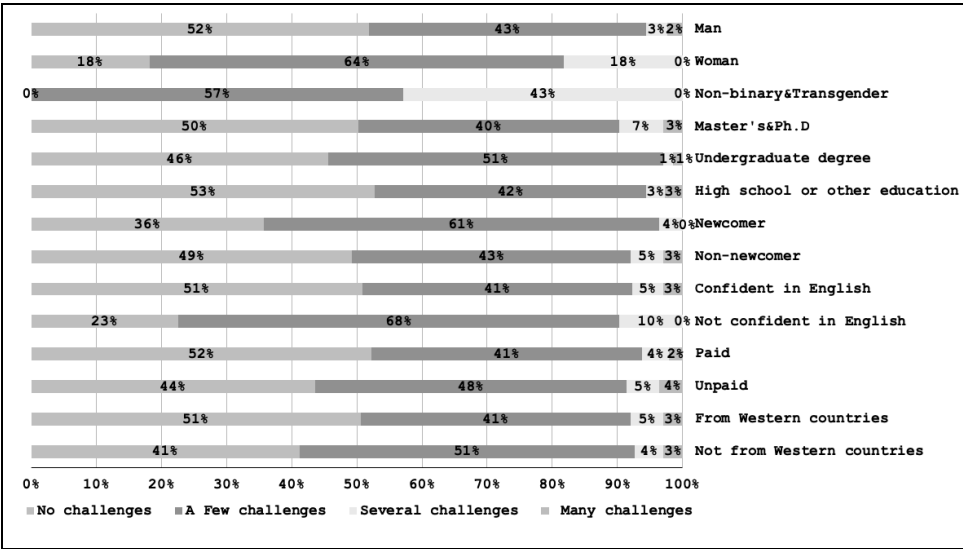


图 38-1 不同的人参与社区时面临的挑战（来源：Apache 软件基金会）

女性、非二元性别与跨性别者、新来者、非英语国家或非西方国家的贡献者（受访者），都会面临更多、更大的挑战。从开源社区应该更多地接纳不同来源贡献的角度来看，受到阻碍的贡献者、可能最终离开的贡献者，都是开源社区的损失。

在 TODO Group 发布的“构建包容性开源社区指南”中，提到了另外一些数据：2017 年麦肯锡的一份报告发现，高管团队性别多样性排名前四分之一的公司比后四分之一的公司盈利能力高出 15%，而拥有文化和种族多样性的公司则高出 33%。

因此，Linux 基金会于 2017 年 9 月宣布的 CHAOSS 项目建立了标准化的、与实现无关的度量标准，以衡量开源社区的整体健康。这些度量标准中，一个关键指标就是多样性和包容性（见第 14 问）。

38.1 多元与包容的重要性

开源社区的包容性和多样性是其成功和可持续发展的核心基石，其重要性体现在多个相互关联的维度。

（1）提升创新能力与问题解决能力。

多样化的视角、不同背景（文化、地域、性别、种族、年龄、教育、专业领域、能力水平等）的贡献者会带来独特的视角、经验和解决问题的方法。这能激发更具创造性的解决方案，避免“群体思维”，发现单一人群可能忽略的问题或优化点。

应对复杂挑战时，现代软件和技术问题日益复杂，需要多学科知识和不同角度的碰撞。多样性确保了项目能汇集更广泛的智慧来应对这些挑战。

探索更多可能性时，包容的环境鼓励大胆的想法和实验，即使最初看起来不切实际，也可能孕育出突破性的创新。

（2）提高代码质量与软件健壮性。

更广泛的测试场景下，来自不同背景的贡献者和用户能覆盖更广泛的使用场景、硬件环境、网络条件和文化习惯（如日期格式、字符编码），帮助发现更多边缘案例和潜在缺陷，使软件更健壮、更具普适性。

更全面的安全审查下，多样化的审查者能从不同角度审视代码，更容易发现潜在的安全漏洞和设计缺陷。

提升无障碍性方面，有不同障碍的贡献者（如视障、听障开发者）或关注无障碍性的人参与，能推动项目从一开始就考虑并实现更好的无障碍支持，使软件惠及更广泛的用户群体。

（3）扩大用户群体与市场覆盖。

理解多元需求时，多元化的社区更能理解和反映全球不同用户群体的需求，使产品设计更贴心，功能更符合当地习惯，文档更清晰易懂（例如，考虑非母语英语使用者），从而吸引更多广泛的用户。

实现高质量软件本地化（翻译、文化适配）和国际化的关键，在于拥有来自世界各地的贡献者。

建立信任与亲和力：当用户看到社区中有与自己背景相似的人时，会更倾向于信任该项目，参与贡献或推荐使用。

（4）确保项目的长期可持续性与活力。

扩大人才蓄水池：包容的环境向所有潜在贡献者敞开大门，极大地扩展了项目可吸引的人才范围。这有助于解决贡献者短缺问题，特别是在关键维护角色上。

降低贡献门槛：强调包容性意味着关注新手友好度（清晰的文档、友好的沟通渠道、导师计划等），鼓励更多人迈出第一步并长期留下，保障项目人力的持续供给。

减少倦怠与流失：一个充满尊重、安全、归属感的社区环境能显著降低贡献者（尤其是核心维护者）的倦怠感，减少人才流失。

传承与韧性：多样化的维护者梯队降低了项目因核心成员离开而陷入停滞或消亡的风险。

（5）提升道德伦理水平与社会价值。

减少偏见：尤其是在 AI、算法决策等领域，开发团队的多样性有助于识别和减少训练数据、模型设计中的偏见，避免软件产生歧视性结果，使技术发展更公平、更负责任。

促进社会公平：开源本身就是一种倡导开放、协作和知识共享的理念。在其社区内实践包容性和多样性，是这一理念的自然延伸，为更广泛的科技行业和社会树立积极榜样，推动整个行业向更公平、更开放的方向发展。

赋能边缘群体：开源为来自资源匮乏地区或传统上在科技领域代表性不足的群体提供了宝贵的学习、实践和职业发展机会。包容性确保他们能平等地获得这些机会。

（6）反映开源精神的核心价值。

开源运动的根本精神是开放、自由、共享和协作。包容性是“开放”的必然要求——开放不仅指代码开放，更意味着向所有人开放参与的机会。多样性是这种开放参与带来的自然结果和宝贵财富。一个排斥特定群体或无法容纳不同声音的项目，实质上违背了开源的核心精神。

38.2 需要解决的现实问题

回到具体问题，开源社区面临的独特挑战，主要有以下几个方面。

（1）女性贡献者比例偏低。

根据 2016 年 Bitergia 的数据，女性贡献者只有 10%~11%。而根据开源社发布的“2024 中国开源年度报告”，在中国的开源开发者中，也只有 28.4%为女性。将目光放宽到整个 IT/科技行业，计算机科学（Computer Science）在 2023 年发布的一份报告“计算机科学领域的女性”显示，“自 20 世纪 90 年代以来，从事计算机科学相关职业的女性比例持续下降，过去 15 年从 35%降至 25%。”然而，整个劳动力中的女性比例则相反：如今女性占劳动力的 47%，而 20 世纪 70 年代仅为 38%。

因此，我们可以这样理解：在整个劳动力市场，女性的比例正在上升，而在 IT 领域，女性的比例却在不断下降。即使在开源社区，这个趋势也不能让人乐观。

（2）对于非英语开发者的语言障碍。

开源社区通常会采用英语作为主要（甚至唯一）的沟通语言，这对于非母语者构成障碍。非英语开发者常需花费额外时间理解技术讨论，如果他们想要发起讨论、提出问题，则会面对更多的困难。这方面的问题，对于英语国家的开发者，往往都不会意识到。而在非英语国家，例如中国，却会更多地强调“开发者应该熟练掌握英文，这样才能更好地融入社区”。

当然，现在 AI 翻译的工具已经非常成熟，我们可以在社区里更多地接入 AI 工具，协助不同国家的开发者更好地交流。

（3）新来者往往会被无视。

即使是开源社区，大家也会更多、更自然地与自己熟悉的人交流。在社区里发帖、提交问题、提交 PR 的人，如果他的名字大家从未见过，他的消息往往会被无视。那些被誉为“对新人友好”的社区，并不是自然而然做到的，他们往往培养出一种重视新人体验的社区文化，社区老人会更加有意识地、友善地回答新人的提问。

我们真的希望更多的社区能够追求这样的文化。

（4）经济困难者和跨国、跨洋的贡献者可能会减少参与社区的机会。

社区除了线上交流，也经常举行线下聚会。在很多情况下，面对面的交流是线上沟通无法替代的。但是由于经济困难，或者路途遥远、时区差距，参与面对面的交流，就会面临时间与金钱方面的困难。对于中国的开源开发者来说，去美国参加一次 OSCon，或者半夜三更上线与美国的社区同仁交流，都会是很麻烦的事情。

（5）非代码类贡献的价值被低估。

开源社区有一句名言，“Talk is cheap, Show me your code”。这句话很可能被引申为：所有非代码贡献，都很“cheap”。如果一个擅长写代码的贡献者，偶尔做做非代码贡献，还可能被人称赞“能文能武”。如果是一个从来不写代码（甚至的确不擅长写代码）的贡献者，哪怕他在社区里做出了很多同样非常重要，甚至难以替代的贡献，也往往会低人一头。这的确是开源社区的“痼疾”，很难改变。

38.3 提升多元与包容性的改进方案

首先是建立可衡量的指标。可以参考 CHAOSS 项目的健康评估框架，重点跟踪不同背景贡献者的留存率、非代码任务（文档/测试）占比及跨时区协作响应速度。当然，更好的就是向 Apache 软件基金会学习，在自己的社区也做类似的“多样性与包容性现状调查”。

其次是降低门槛，鼓励更多的新手成为贡献者。例如，设置新手任务，用“good-first-issue”标签标识简单任务；配置 AI 工具，帮助其他语言的开发者更好地与社区交流；设立本地社群，如 Apache Local Community，一方面传播 Apache 文化，另一方面也能够鼓励更多本地开发者加入 Apache。

再次是提供某种经济支持。例如，对遥远国度、贫困地区的开发者给予一定倾斜比例的差旅补助，这样能够更好地帮助那些地区的开发者，到社区“总部”，与大家面对面对交流。

最后是组建多元背景的仲裁小组。在第 37 问中提到的沟通与决策流程中，需要更加仔细地处理矛盾冲突，避免因不了解、不理解、不能共情，而在自以为公正、公开的情况下伤害到其中一方（甚至双方）。

第 39 问 开源社区如何评估贡献者的价值

在开源生态中，评估贡献者的价值是促进项目健康发展、激励持续参与、构建社区信任的关键机制。相比传统企业以 KPI 为核心的评估方式，开源社区的评估体系更强调多维度、动态化、协作导向与社区共识。本文提出了一个系统性的开源贡献者价值评估框架，涵盖技术贡献、非技术贡献、行为品质和商业价值四大维度，并指出未来挑战与创新方向。

另外需要指出的是，对于贡献者价值的评价，有一个根植于开源文化（礼物文化）的逻辑：一个贡献者的价值，是由他对社区做出的贡献决定的。因此，大多数社区最重视的依然是技术贡献，其次是非技术贡献。至于行为品质与商业价值，在很多开源社区，事实上并不受重视。但是随着近年来的两大趋势——越来越强调多元包容文化、越来越多的企业从商业角度来看待开源，后两个维度才越来越被重视。

39.1 技术贡献

技术贡献是最直观的衡量维度，也是许多项目最早期评估贡献者的依据。在 GitHub 等平台上，代码提交的数量和频率常常被用作初步参考。然而，现代开源社区已经逐渐意识到，仅靠数量并不能准确反映贡献的实际价值。高质量的代码、高影响力的功能实现、严谨的测试覆盖，以及对系统架构的深度参与，都比“多而杂”的提交更具意义。同样，参与代码评审也逐渐被视为一种重要的技术贡献。一个善于评审的贡献者，不仅能帮助他人提升代码质量，还能在审查中守住项目的技术底线。针对不同技术贡献的衡量标准，可参考表 39-1。

表 39-1 技术贡献的分类

贡献类型	衡量标准
有效 PR 数量与通过率	是否合并，是否被主干采纳
漏洞修复与核心功能实现	是否解决关键性问题，贡献是否具有技术深度与创造性
可维护性与测试覆盖率	代码是否清晰、符合规范、通过测试
架构性贡献	是否修改/设计了影响广泛的核心模块或依赖体系
评审质量与数量	是否发现潜在问题，是否提供了有价值建议
判断力与审慎性	是否能平衡创新与稳定性，对他人代码的提升是否有效
基础设施维护	系统是否正常平稳运行，是否不出安全事故

39.2 非技术性贡献

开源社区的运转远远超出代码本身，非技术性贡献——曾经被忽视、难以量化甚至未被记录，如今却在社区的稳定发展中扮演着不可替代的角色。完善文档、翻译项

目、撰写教程、组织社区会议、维护 FAQ、指导新人上手，甚至在 Discord 或邮件列表中积极参与讨论，这些行为极大地提升了项目的可用性与可持续性。在一些大型项目中，非代码贡献者甚至成为推动治理、扩展生态、传播理念的中坚力量。

台湾著名的开源盛会名叫 COSCUP，其全称就是“Conference for Open Source Coders, Users, and Promoters”，其中 Promoters 特指对于推动开源、推广开源作出贡献的人。

针对不同非技术贡献的具体内容，可参考表 39-2。如果说技术性贡献的指标量化比较困难，那么非技术性贡献的指标就更难以量化了。再加上社区常见的“鄙视”不写代码的人的风气，非技术性贡献就更加被低估了。

表 39-2 非技术贡献的分类

贡献类型	贡献内容
文档与知识贡献	撰写教程与文档、翻译文档、维护 API 与 FAQ 等
社区运营与支持	回答 Issue、管理论坛、引导新人、减少维护者负担、主持会议、推动讨论、组织线上线下活动
教育与传播	撰写博客、举办讲座、创建示例代码、参与播客或培训
项目管理与协调	路线图规划、里程碑管理、贡献者协调、治理文件编写

39.3 行为品质

Linux 内核社区可能是全球最著名的开源社区，而 Linus 则是其中当之无愧最有价值的贡献者。但是，即使是 Linus，社区也忍受不了他的坏脾气。2018 年 9 月，他再一次在社区的邮件列表里骂人，最终激起众怒，大家因为他长期使用攻击性语言而公开批评他。终于，在 9 月 16 日的邮件中，他承认自己“需要改变自己的某些行为”，因为之前对其他贡献者的攻击“既失专业又无必要，尤其当涉及人身攻击时”。

最终，他主动提出，暂停内核维护工作一段时间并进行反思，原文是“我将暂时休假并寻求专业帮助，学习理解他人情感并恰当回应”。

在制度方面，Linux 从 2015 年开始有“冲突准则”（Code of Conflict），内容简短，建议大家“彼此优秀”。在 2018 年 9 月，Linus 道歉并暂离社区之后，社区采取行动，修订了之前的“冲突准则”，代之以基于 Contributor Covenant 的新“行为准则”。新“行为准则”明确列出了不可接受的行为，包括“钓鱼、侮辱/贬低评论以及人身或政治攻击”，以促进更专业和包容的社区氛围。

总之，即使是社区中最重要的贡献者，他的某些行为也是不可接受的。换言之，恶劣言行与破坏协作的做法，会被社区“减分”。

39.4 商业价值

从企业视角看，开源贡献者的价值评估与商业目标密切相关。贡献者可能帮助企

业减少内部开发成本，加快产品与主流技术的集成速度，提升品牌的知名度，乃至拓展招聘渠道。一方面，企业可能会积极从社区招聘符合自身商业利益的贡献者；另一方面，是否“派出”更多的员工参与社区贡献，也是由企业的商业利益决定的。

因此，对于贡献者的商业价值的评估，虽然社区内部可能并不在意，但是对于企业来说，也希望能够找到一种适合的评估方法，妥善地给予评判。

39.5 量化指标的尝试

以上所说的4类评估，面临的最大挑战就是如何量化。业界与学术界虽然都有一些尝试，但的确都不成熟。目前，比较接近成熟的是 OpenRank 算法。OpenRank 算法是由 X-Lab 实验室提出的一种开源评价算法，该算法为工业和信息化部电子工业标准化研究院“信息技术 开源治理”系列标准中的项目评估与开发者贡献度评估中的网络评估模型的标准开源实现。

OpenRank 受启发于 PageRank，但在其基础上进行了优化和扩展，使得该网络算法能够为节点引入初值作为固有属性，并支持将节点的中心性推广到高维空间。节点初值的引入使得在实际应用场景中，节点的中心性可以继承其历史数据，从而具有较好的时序连续性，为数据的洞察提供了有效的支撑。OpenRank 在开源中的应用由两个子算法组成，分别为全域 OpenRank 影响力算法和社区 OpenRank 贡献度算法。

其中的贡献度算法，简单地解释就是“权重越高的项目，给该项目作贡献的人，其贡献就有价值；权重越高的贡献者，给某个项目作贡献，带给该项目的加成也越多。”最终，通过复杂的图计算，可以同时计算出项目的 OpenRank 值与贡献者的 OpenRank 值。

另外一个思路是笔者曾经在一篇博客里提出的“节约时间”带来的价值。一款开源软件，如果从头开发需要 100 小时，而这个软件有 1 万个用户，那么这款软件就为全世界节约了 100 万小时。在这个开源软件的开发过程中，某个程序员贡献了自己的时间，如 10 小时，那么他的价值就是 10 万小时。如果一个贡献者，虽然没有提交代码，但是他在社区里回答了一个用户的问题，节约了那个用户 1 小时的时间。与此同时，还有 1000 个遇到同样问题的用户因此受益，那么他的价值至少能值 1000 小时。再如一位布道师，他努力地推广一款开源软件，使得更多用户了解并使用这款软件，那么他带来的价值（甚至还帮社区的其他人增值）也同样是巨大的。换言之，所有在社区里贡献的人，都在帮助世界上的其他人，节约了或多或少的的时间。这就是开源贡献者的价值！

第 40 问 开源项目如何保障其长期可持续发展

开源项目作为技术创新的重要驱动力，其长期可持续发展对全球数字经济的稳定可持续发展至关重要。然而在繁荣兴旺的开源社区背后，资金短缺、治理机制缺

失、技术分歧引发的社区分裂等问题持续威胁着开源生态的健康发展。面对这一挑战，全球开源社区都在进行各种尝试，希望能够为开源项目的长期发展提供更好的保障。下面将从资金机制、治理模式、技术实践和产业协同 4 个维度，解析开源项目长期发展的核心要素。

40.1 经济可持续性

虽然的确存在很多单人维护的开源项目，长期不求回报，只是出于个人兴趣的开源贡献者也有很多。但是，更多的开源项目，都需要稳定的资金支持。在本书的第 3 篇，讨论了不少与开源商业模式有关的话题。但是这里所说的经济可持续性，并不是从开源企业的角度，而是从开源贡献者应当获得合理回报的角度来讨论的。我们可以将投入开源的贡献者，分为以下几种情况，可持续的有 3 种，不可持续的也有 3 种。

- 可持续：自身经济状况无虞，业余时间投入开源。
- 可持续：企业聘请的开发者，以企业员工身份投入开源。
- 可持续：开源企业，并且找到了成功的开源商业模式。
- 不可持续：通过开源项目，寻求捐赠，收入不稳定。
- 不可持续：业余时间投入开源，工作繁忙，时间不够，投入不稳定。
- 不可持续：开源创业公司创业失败，项目终止。

对于后面的 3 种情况，其实并没有太好的解决方法。目前，有开源基金会在尝试直接资助开源贡献者的方式，能够解决一部分问题。另外，就是通过开源治理，想办法繁荣开源社区，提高 BUS 系数^①。

40.2 治理可持续性

在开源社区里，总会产生各种不同的矛盾。小的矛盾可能会导致贡献者愤怒、沮丧、对社区失去信心，甚至选择离开社区。更大的矛盾则可能导致社区分裂，甚至完全解体。所以，我们才会在第 37~38 问，讨论社区的冲突管理，以及包容性与多样性建设。

在下一个问题中，我们还会讨论社区的规则与潜规则，本质上也是在研究如何进行治理，才能帮助社区长治久安。

这里简单地讨论一下社区治理总的原则：存异才能求同，志同道合，和衷才能共济。

开源社区由全球开发者自发组成，个人背景、技术偏好、文化观念各异，只有尊重差异、包容差异，才能在差异的基础上找到共同点，找到最大公约数。如果求同是一味追求完全一致，就会导致社区要么变成一言堂，要么最后分崩离析。

如果要“求同”，最需要的是“明确的核心目标”（志），这样才能吸引志同道合

^① BUS 系数，是指项目中有多少关键开发者一旦被公交车撞了（比喻突然离开），项目就会陷入瘫痪或无法继续，是一个衡量项目可持续性和风险承受能力的指标。

合者，社区才能不断地发展壮大。否则，易沦为无序的“乌合之众”，或者表面一团和气，实则无所作为的清谈社区。

“和衷”是指大家心意一致、团结一心，“共济”是指共同渡过难关。在一帆风顺的时候，社区自然容易发展顺利。但是在遇到困难时，社区的成员能不能团结起来，共同渡过难关，关键还是在于社区成员的核心目标是否一致。

开源社区的治理本质是“在多样性中寻找共识，在共识下构建协作”，这也是为什么 Apache 软件基金会一直在强调“Community Over Code”的原因。

40.3 技术可持续性

开源项目的技术可持续性取决于其架构演进能力、维护效率和安全机制三大支柱。

关于架构演进能力，一个典型的案例是 Python 语言。如果像 Python 社区那样，突然宣布要以不兼容的方式进行升级，从 Python 2 升级到 Python 3，当然会引发轩然大波。更加合理的做法，就是在架构设计时，更具前瞻性，考虑到可能的演进方向，避免在项目发展的中后期，造成不可挽回的局面。

关于维护效率，就需要提到贡献者倦怠感的问题。在特别繁荣、热闹的社区中，开源贡献者尤其是维护者，可能会产生严重的倦怠感。这就需要我们通过各种各样的智能化、自动化的技术手段，降低社区维护的成本，提高维护效率。只有把开源贡献者从烦琐的维护工作中解放出来，才有可能引导他们去做更多有创意、有价值、能够带来自豪感的工作。

关于安全机制，有一幅著名的漫画（见图 40-1），就是在复杂架构中，总是会存在脆弱点。而一个开源软件，如果存在这样的脆弱点，一旦安全风险爆发，就会给用户，当然也会给项目自身带来巨大的灾难。著名的 OpenSSL 心脏滴血漏洞，就是这样的案例。

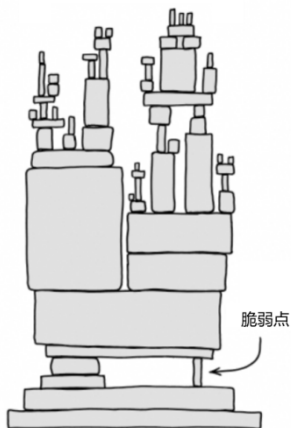


图 40-1 架构中的脆弱点（Randall Munroe 创作，来自 xkcd）

总之，我们在考虑开源项目的架构与技术决策时，需要仔细思考技术可持续性，以免在危机发生时，承担难以估量的代价。

40.4 项目可持续性与生态可持续性

上述的讨论，都只是从项目层面来考虑的可持续性。至于整个生态层面，我们还需要考虑以下问题：什么样的开源项目可能存在经济可持续性的风险，如何找到这些项目，如何帮助这些项目？以及什么样的开源项目会有治理可持续性风险、技术可持续性风险？

总之，从整个生态的角度，如何才能找到那些在可持续性上存在问题，但又难以自己解决的项目，然后找到一些有效的方法去帮助它们。这就是开源基金会，甚至是国家层面的开源政策，需要考虑的问题。

第 41 问 开源社区里的规则与潜规则有哪些

41.1 社区、潜规则与规则

本篇一直在讨论开源社区，却尚未论及开源社区的成长规律。开源社区的成长规律如图 41-1 所示。

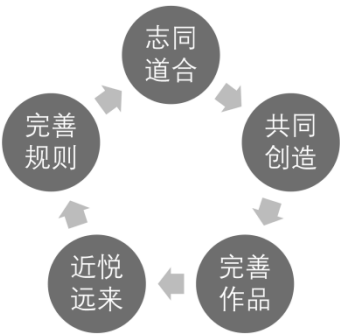


图 41-1 开源社区的成长规律

一个开源社区的形成，往往始于一群志同道合者的聚集。他们因为共同的理想、兴趣或目标，走到一起，开始创造某种作品，如开发一款开源软件。在这个过程中，伴随着对项目的持续打磨、改进和优化，社区逐渐形成，并逐步发展出独特的氛围和文化，吸引更多人因产品质量、项目价值或社区氛围而加入。这种吸引力带来的是“近悦远来”的效应：既能让原有成员持续投入，也不断吸纳新成员。

然而，随着社区规模的扩大，成员数量的增加，各类矛盾与分歧也随之增多。此时，社区成员会开始意识到规则建设的重要性，并在实践中不断完善社区的运作

规范。这一规则的制定与完善，并非源自外部强制，而往往是社区内部对“何为可为，何为不可为，何者应当鼓励，何者应当避免”的集体性反思。这种反思促使社区回溯初心：我们为何聚在一起？我们希望建设一个怎样的社区？我们的共同愿景是什么？正是在这样的集体意识和价值追问中，规则才逐步建立并得以优化。

这一过程本质上是一个循环：从“志同道合”的初衷出发，发展出开放而有吸引力的社区氛围，继而通过实践中不断暴露的问题引导规则建设，而规则的建设又反过来强化社区的凝聚力和目標认同。如果这个循环能够良性运转，社区就会不断吸纳新人、积累经验、提升运作水平，创造出更有价值的开源成果。

在这一演化过程中，最初的规则往往是“潜规则”——即未被明确书写、却被成员普遍默契遵循的行为准则。这些潜规则源于早期成员之间的信任关系和互动经验。但随着人多事杂、情境复杂，社区对“明确、清晰、可执行”规则的需求逐渐增加，原本的潜规则就会转化为显性的成文规则。

例如，在一个新建的微信群中，如果某人发布广告，引发了不适或干扰，群主可能会制定第一条规则：“发布广告前必须发红包”。随着情况变化，规则可能进一步升级为“必须发大红包”。这种规则的演变，反映了从潜规则向明规则的逐步过渡。但即便如此，这些规则的执行仍可能受到群体关系、管理者偏好等非公开因素的影响。如果某人和群主关系密切，即使违反规则也可能不受惩罚。这就使得潜规则仍然在显规则之下发挥作用。

在更大规模的社区中，类似的潜规则现象也随处可见。社区成员会通过观察“某人为何被禁言”“某帖为何被删除”等现象，推测背后的原因。这些推测不仅反映了成员对规则的认知不确定性，也说明了在规则背后仍有一套未明说却起作用的“隐性逻辑”在运行。可以说，没有哪一个社区是完全没有潜规则的，完全公开透明的社区规则几乎不存在。

从形态与可执行性来看，社区规则可以分为 4 个层级：情、礼、法与 Code（代码）。

首先是“情”——即人际关系中的基本伦理，如亲疏远近、人情冷暖，这是最朴素的交往基础；其次是“礼”——即行为的规范与仪式感，强调对他人的尊重、界限感与礼节，如“礼尚往来”“有礼有节”等；第三是“法”——即成文的规范与制度，包括法律条文、许可证、贡献者协议（CLA）等，具有相对的强制力与明确性。

然而，现代社区运行并未止步于此。随着技术的发展，我们开始进入“Code”的阶段。所谓“Code”，并不仅仅是编程意义上的代码，而是一种“可自动执行的规定”。举例而言，一张门禁卡可以控制门的开启权限，本质上就是将“允许哪些人进入”的规则写入了程序中，由系统而非人来执行。这种规则不再依赖管理者的主观判断，也无须解释或辩论，而是通过技术手段实现了高度自动化和执行一致性。

从历史角度来看，规则的演进也体现了人类社会治理的演化路径：在小规模社会中，靠的是“道、德、仁、义”等较为抽象的价值指引；当社会变得更复杂，人们便依赖“礼”来维系秩序；再进一步，出现了需要执法者、法官、律师等专业力

量参与的“法”；而如今，进入数字社会，部分规则已被“Code”所替代，并实现了即时、自动、无歧义的执行机制。

综上所述，开源社区的规则体系既包含明确制定的显性规则，也包含根植于人际互动和文化氛围中的潜规则。社区的健康发展，依赖于这两者之间的良性互动：显性规则提供了可操作性与一致性，潜规则则反映出社区文化的深层价值。理解并善于运用这套“情—礼—法—Code”的规则体系，是推动开源社区持续繁荣的重要保障。

41.2 开源伦理的演进

开源伦理的发展共经历 3 个阶段，其初始阶段以精英伦理为主导。该阶段被学界与业界广泛认为是“真正的开源伦理”，亦即“开源伦理的核心”。在这一阶段，早期开源领域的技术先行者一方面打造了诸多具有行业影响力的经典开源项目，另一方面也奠定了以黑客伦理为标志的开源伦理基础观念。

- 协作与共享：开源社区强调协作与共享，认为通过开放代码，所有人都可以受益并共同进步。
- 透明带来信任：代码是公开的，任何人都可以审查和改进，从而建立一种基于透明度的信任文化。
- 自由的权利：开源倡导软件自由，强调用户应当有使用、修改和再分发软件的自由。

然后就进入了商业伦理冲击的阶段。随着开源软件不断蚕食商业软件的市场，商业伦理也开始对开源运动产生冲击。在这一阶段，越来越多的商业公司认识到开源软件的潜力，并开始参与其中。公司逐渐将开源软件纳入其商业战略，通过开源来降低开发成本、提高市场竞争力和促进技术创新。这一阶段的伦理变化主要体现在以下几个方面。

- 商业化与盈利：公司通过提供开源软件的商业支持、定制服务和增值产品来实现盈利，这在一定程度上改变了开源社区的原有价值观。RMS 曾经严厉地批评过通过商业利润来支持开源的逻辑，但是他的批评无法阻拦这样的趋势。
- 知识产权与许可：商业公司更加关注知识产权保护和许可协议的制定，以确保在开源合作中的权益。尊重版权、注重合规义务，成为与尊重开源贡献同等重要的伦理要求。
- 社区与企业关系：公司需要平衡开源社区的利益与企业的商业目标，这要求建立新的合作模式和伦理规范。我们常说有些公司不会玩开源，把一些糟糕的玩法带到开源社区，就是因为商业伦理与开源伦理之间出现的冲突。

目前，我们正在经历的是外部伦理冲击的阶段。在开源软件广泛普及和应用的背景下，外部伦理因素开始对开源运动产生重大影响。这些因素包括社会责任、隐私保护、数据安全和地缘政治等。这一阶段的开源伦理变化主要体现在以下几个方面。

- 社会责任与政治正确：开源社区和公司越来越多地关注社会责任，强调开源技术应当服务于社会公益和可持续发展。进一步的说法就变成了：开源不应该成为恶势力的帮凶，开源应该追求政治正确、社会正义，于是道德源码运动随之兴起。
- 隐私与安全：随着开源软件在敏感领域的应用增加，隐私保护和数据安全成为重要的伦理考量。开源项目需要在代码开发和使用中严格遵守隐私和安全标准。这方面的案例，在本文并未提及，但也确实是非常重要的议题。
- 地缘政治议题的冲击：生来就是国际化的、全球协作的开源社区，遭遇了各种各样的地缘政治危机，敌对国家的开发者，能不能在一个开源社区里协作？一款开源软件，要不要声明支持一方而反对另一方？更有甚者，政治性的软件投毒也开始出现了！开源社区，是否能够坚守自己的伦理底线，明确有些事情绝不可为？

当开源不再是一小群黑客们自娱自乐的小园地，而是深刻地嵌入整个世界运行之中时，开源的伦理原则应该如何确定？这是一个越来越难以回答的问题了！

41.3 社会之礼与社区之礼

“礼”在社会规范中占据重要地位，其内涵可以进一步细分为礼仪、礼节与礼物3个层面。

首先，礼仪是社会生活中为维系秩序、体现尊重而形成的仪式化行为规范。例如，在婚丧嫁娶等重大人生节点，人们往往会举行仪式、设宴请客，以表达情感与建立社会联系。这种礼仪不仅是个体行为的表达，更是集体文化的体现。开源社区中亦存在相应的“社区礼仪”，如社区大会上成员之间的重逢、寒暄与交流，以及在群体交流过程中，如何恰当地介绍自己、引荐他人、把握交流分寸等，均构成了社区内部的礼仪规范。

其次，礼节主要体现于人与人之间日常交往中的行为规范与礼貌方式。在社会生活中，初次见面时使用怎样的称谓、采取何种姿态进行沟通，均反映了一个人是否具备基本的礼节意识。在开源社区中，同样存在相应的礼节规范。例如，当新人首次在社区中发起提问时，提问的方式、语气和内容往往直接影响其获得回应的质量与态度。诸如“跪求答案”“急！在线等”等措辞，往往被视为不尊重他人时间和精力表现，容易引发反感。相反，以理性、清晰、客观的方式表达问题，反而更容易获得他人的尊重与帮助。这不仅是一种沟通技巧，更体现了对社区文化的理解与尊重，体现了“提问的艺术”所蕴含的礼节价值。

最后，礼物作为一种社会交换的重要媒介，也在社区文化中占有重要位置。在传统社会、现代社会乃至原始部落中，那些最慷慨的贡献礼物者往往拥有最高的社会地位。这种“礼物赋权”机制在开源社区中亦有鲜明体现。如《大教堂与集市》一书所指出，开源社区的成员之所以愿意将自己辛苦编写的代码无偿贡献出来，正是基于一种“礼物文化”的价值观。在这种文化之中，代码不仅是技术成果，更是

一份向公众提供的共享礼物。那些贡献了高质量、关键性代码的开发者，往往因其无私的分享而赢得社区的广泛尊敬与崇高地位。

因此，从礼仪到礼节再到礼物，构成了社区运行中“礼”的三重维度。它既是文化认同的体现，也是建立秩序、促进协作、巩固关系的重要机制。在开源社区中，“礼”不仅是一种外在行为规范，更是一种内化于人心的价值信念，支撑着社区的健康运转与可持续发展。

41.4 开源社区里的“成文法”与自动化规则

很多开源项目的根目录，都会存放各种规则文档。

- 贡献指南（CONTRIBUTING.md）、代码风格（code_style.md）、测试标准等：保障贡献质量、降低协作成本。
- 行为准则（CODE_OF_CONDUCT.md）：应对社区多元文化冲突的产物，著名的 Contributor Covenant 就被数万项目采用。
- 开源许可证（License）：开源项目需在法律框架内运作，许可证决定代码可否被商用、修改、再发布。可以看成开源社区最重要的“成文法”，关于许可证的类型与特点参见第 6 问。

总体而言，社区中存在两类规则：一类是供成员事先阅读并自愿遵守的规范；另一类则无须事先通读，却必须被强制执行的规则。后者通常通过代码或配置的方式加以实现，因而具备自动化和强执行力的特征。

美国著名的开源与互联网法学家劳伦斯·莱斯格（Lawrence Lessig）在其代表作《代码 2.0》中提出了著名的观点：“Code is Law”（代码即法律）。他认为，在数字时代，凡是可以通过代码自动执行的规定，将在社区、社会乃至全球范围内占据越来越重要的地位。代码不仅仅是程序语言，它本质上是一种规范执行机制，能够在无须解释与人为干预的前提下，自动完成规则的实施。

以笔者曾经参与过的 JavaEye 社区为例，这是国内较早引入“积分—权限挂钩机制”的开源社区之一。在该社区中，新注册用户初始等级为“一星”，随着发帖数量和活跃度的提升，等级逐渐上升为“二星”“三星”直至“五星”，其后还可以获得“一皇冠”至“五皇冠”的称号。这一等级体系并不仅仅是荣誉的象征，更直接影响用户在社区中的权限与影响力。例如，当一篇帖子被多位用户标记为“垃圾帖”后，最终是否被移至垃圾箱，不取决于票数本身，而取决于累计的积分权重。当负面积分达到一定阈值（如 20 分）时，系统会自动将该帖移除。每位用户的投票权重依据其等级而定，等级越高，影响力越大。

类似的积分与规则体系在其他开源社区中也有广泛实践，例如，Stack Overflow、SegmentFault（思否）等，它们都通过设计各自的积分系统，实现社区治理的自动化与精细化。GitHub 亦是如此，其引入的自动化工具，如 Actions、Robots，以及近年来广泛应用的 Issue 与 Pull Request 模板（Issue & PR Template），共同构建了一套以“代码即规则”为核心的社区协作机制。

此外，代码评审工具 Gerrit 所体现的规则性也值得一提。在 Gerrit 中，一段代码是否可以被合并入主干，往往依赖于是否获得了多个审阅者的“通过”意见。例如，当两位或三位审阅者给予积极评价后，系统便允许代码合并。这种机制本质上也是规则的代码化体现。

笔者所在的公司也设计并实践了一套基于积分的代码评审机制。我们可以设定规则：当某段代码获得的“支持积分”超过设定阈值（如 3 分或 5 分）时，才允许其被合并。每位审阅者的投票权重设定为 1 至 2 分，这样一来，随着团队人数的增长，获得代码合并所需的“赞成比例”也会相应提高，从而有效提升代码质量控制的门槛。

综上所述，借助代码实现的规则体系能够大幅提升社区管理的效率与公平性。这类规则具备可预测性、可执行性与一致性，是应对复杂社区行为、调节多方利益关系的有效手段。正如莱斯格所言，随着技术的发展，“代码即法律”不仅是一个技术命题，也正在成为一种社会现实。

41.5 社区中的潜规则

（1）人设的重要性。

在社区中，“人设”（即个人在群体中的角色形象）具有重要意义。成员在社区中如何被他人认知，其自我定位如何，直接影响其所能获得的关注、尊重与影响力。

例如，社区中常见一种类型的用户，几乎仅以“发帖—发红包—请求转发”为行为模式。他们往往是处于社区认同体系最底层的群体，无论其行为是否出于无奈，其形象难以获得他人的尊重与信任。这种“发帖机器式”的人设，往往不被重视，难以建立有效的长期连接。

曾有一次，笔者在微信群中与某知名商学院的教授交流时，发现他将自己撰写的文章反复发布至群中。当群主提示其“重复发帖”时，他轻描淡写地回应“抱歉，不小心发错了。”笔者提醒他，社区中“人设”尤为关键。即便他的文章内容优质，但一旦形成“发帖机器”的负面人设，其作品也可能因此被忽略。

这类潜在的规范通常并非明文规定，却具有极强的行为约束力。一旦被打上负面标签，其在社区中的发展空间将大幅受限。许多刚刚开始从事社区运营的年轻成员，尤其容易陷入这一误区，低估了人设对长期发展的影响。

因此，建议社区运营者，尤其是初入者，应主动构建更加积极、专业的人设。例如，通过持续理性发言、深入交流、分享有价值的信息，从而塑造“值得信赖的分享者”“理性思考者”乃至“意见领袖”（KOL）等更高阶的角色定位，进而赢得广泛认可。

此外，也有部分成员以制造争议或“刷存在感”为目标，表现出攻击性、对抗性或偏执特质，俗称“杠精”。此类成员虽可能在短期内带动讨论热度，但从长期看，会破坏社区的理性氛围，损害交流质量。若以“KPI 导向”运营社区，或许会

暂时容忍此类“有毒流量”；但若以社区健康、可持续发展为目标，则应及时识别并排除这些破坏性因素。

（2）规则的“弹性”与社区中的不平等现象。

开源社区存在一种结构性张力，即“人人平等”与“精英治理”之间的矛盾。

初期阶段，成员之间的地位相对平等。但随着时间的推移，那些在代码、项目或协作上作出显著贡献的成员，往往获得更高的影响力、更大的话语权，甚至某些不成文的“特权”。这导致同样的行为，来自不同角色者所遭遇的评价与后果可能存在显著差异。

例如，知名开源人物林纳斯·托瓦兹在社区交流中曾多次使用激烈语言，甚至有不当举止。尽管有部分成员表达批评，但因其作为 Linux 创始人的权威地位，往往被更多人视作“直率”“风格独特”，并未受到实际惩处。

这种现象说明，在社区中，规则的执行标准并非一视同仁。在理想状态下，社区应建立对所有成员相对公平的规则体系，尤其是在面对新人时，避免“门槛过高”“排他性强”的倾向。否则，社区将难以吸引与培养新生力量，从而陷入“内卷”与“老化”。

具备高度自我认知的社区，若未及时调整其对新人的接纳机制，常会表现出更强的苛责与审视。这种文化虽然源于对质量的坚守，但也可能限制社区的成长与包容性。

（3）社区的内部沟通与隐性结构。

在多数社区，尤其是基于即时通信功能（如微信群）构建的社区组织中，除了公开讨论空间，往往还存在一些成员数量较少、较为私密的“内部小群”。这些小群形成了社区的“隐性沟通结构”，用于协调意见、反馈问题，甚至形成舆论引导。

以笔者亲身经历为例，曾在某微信群中出现一位长期以争辩和否定他人为主要行为特征的用户。虽然他在公开频道中引发了多轮争论，也激活了一定的讨论热度，但多数成员对其行为表示不满。随后，笔者私下与群主沟通，建议将其移除。几日后，该用户被清退，群内氛围明显趋于和谐。

这一过程反映出社区运作中的典型“潜规则”：虽然未设明文条款，但实际操作中，社区管理者通过私下反馈、小范围磋商完成决策。

因此，对于社区的运营者而言，除掌握显性规则外，更须具备对社区氛围的敏感度与判断力。即便在制度相对完善的社区中，管理者的“直觉”与“判断力”仍起着不可替代的作用。例如，何时应强制执行某项规则、某位成员的行为是否构成破坏、是否需要调整某项机制等，这些都无法完全依赖自动化或既定流程，而需依赖运营者的经验与判断。

41.6 社区运营的艺术

对于运营社区的管理者来说，往往处于两难的处境之中。

- 尊重贡献者与保护新手。
- 严格执行规定与照顾大家的面子。
- 追求透明度与通过私下沟通寻求妥协。
-

总之，如何寻求平衡之道？还是需要管理者有足够的“品位”。他要能够沉浸在社区里，不断地感受社区是好还是坏，不断地去体会、理解这个社区的规则是如何运行的，如何发生作用，还要不断磨炼自己的技艺，最后才有可能创造出一个越来越好的社区。

第5篇 开源创新与未来

本篇系统探讨了开源如何成为 21 世纪技术创新的核心驱动力，从六大机制出发——全球协作、透明迭代、零门槛复用、事实标准、商业模式再定义和长尾创新民主化，解析了开源驱动创新的内在逻辑，并通过 Linux、PyTorch、Kubernetes 等案例加以印证。

在 AI、区块链、物联网等前沿领域，开源已不只是工具，更是一种协作哲学与治理范式。TensorFlow、PyTorch、Hugging Face 降低了 AI 门槛；以太坊、Hyperledger 重塑了区块链生态；Zephyr、Arduino、ESP32 推动了物联网普及。

然而，开源并非万能。笔者也坦陈其边界：涉及国家安全、资本密集、复杂系统耦合的领域，纯社区协作难以承担主体角色。更深层的挑战在于可持续性——当贡献者倦怠、资金链断裂、地缘政治撕裂社区时，“公地悲剧”正在逼近。

最终，本篇内容指向一个核心命题：开源的未来，需要政府、企业、基金会与个人共同承担起“多元共治”的责任，而非继续将压力压在少数维护者身上。

第42问 开源如何驱动技术创新

在 21 世纪的第三个十年，技术创新的步伐持续加速，成为推动社会进步和经济增长的关键动力。在这一进程中，开源已不仅是软件开发范式，更成为全球技术创新的基础设施。开源从最初的软件开发领域，渗透到硬件设计、科学研究和新兴技术等多个层面。本问将尝试分析开源驱动技术创新的机制、成效与量化证据，并对今后的发展做出展望。

42.1 开源驱动创新的六大核心机制

（1）全球协作。开源项目往往通过全球范围内的协作，汇聚来自世界各地的智慧。这种“众智并行”的模式打破了地域和时差的限制。例如，Linux 内核项目自 1991 年开始，截至 2024 年累计有超过 2.3 万的贡献者，来自 2000 多个企业和高校的贡献者，支持 30 多种架构，每 2~3 个月就发布新版本。目前，已经广泛应用于高性能计算、云计算服务、物联网设备等诸多领域。这种高频率的更新离不开全球开发者的共同努力。

（2）透明迭代。开源项目的“透明迭代”机制强调通过公开的同行评审来加速技术的发展。这种做法确保了所有人都可以看到进展并提出改进意见。例如，

Heartbleed 漏洞事件发生后，开源社区在 48 小时内就发布了全球修复补丁。而 PyTorch 作为一个深度学习框架，其 Pull Request 的合并平均时间仅为 4.6 天，体现了开源迭代的快速性和透明性。

(3) 零门槛复用。开源项目的代码本身就是知识，可以被其他开发者轻松复用，从而避免了重复劳动。Hugging Face 是一个典型例子，其提供了 35 万个预训练的模型，让初创企业能够在短短一天内上手，极大降低了技术门槛，推动了 AI 技术的普及。

(4) 事实标准。流行的开源项目往往成为行业标准，推动技术的统一与规范化。例如，Kubernetes 作为云原生的操作系统已经成为整个云计算领域的事实标准，减少了技术的碎片化，为企业提供了统一的开发平台。

(5) 商业模式再定义。开源项目不仅推动了技术创新，还重新定义了商业模式。通过提供核心的开源技术并基于此发展增值服务或云服务，开源公司可以实现盈利。红帽以 34 亿美元将公司卖给 IBM，而 MongoDB 的 Atlas 云服务收入已经占到公司总收入的 76%。

(6) 长尾创新民主化。开源使得任何个人、高校或新兴市场都能参与技术创新，打破了创新的集中化，激发了创新活力。例如，Raspberry Pi 教育套件不仅推动了全球青少年的编程教育，还衍生出超过 3000 个物联网项目，展示了开源如何让全球范围内的创新得以平等参与。

42.2 开源驱动创新的实际成效

开源的驱动力在多个前沿科技领域都取得了突破性的进展。以下案例展示了开源在近年来的具体影响。

(1) AI。AI 是开源模式影响最为深远的领域之一。开源框架（如 TensorFlow 和 PyTorch）和模型共享平台（如 Hugging Face），共同构建了一个繁荣的生态系统，使 AI 创新以前所未有的速度进行。2025 年，AI 解决方案中基于开源的比例约为 41%，高于 2024 年的 37%，显示出持续增长的趋势。AlphaFold 3 作为 DeepMind 的开源项目，在 2025 年进一步提升了蛋白质结构预测的准确性，甚至能模拟蛋白质动力学，极大地加速了药物开发进程，成为 AI 在生物信息学领域最具影响力的开源项目之一。在中国，目前 DeepSeek、Qwen、Kimi 等开源大模型不仅在国内被广泛使用，也在国际范围内取得了显著的影响力。（关于开源与 AI 创新，详见第 43 问。）

(2) 区块链技术。区块链技术发源于开源社区，并在社区中发展壮大，此后逐渐受到金融机构、IT 巨头等机构的关注。区块链开源项目在技术突破、应用推广、人才培养、产业支撑等方面具有重要作用，以 Hyperledger 为代表的开源项目对我国联盟链技术应用发展影响深远。

自 2008 年比特币白皮书发布以来，催生了众多新兴技术和创新理念的诞生。这些技术不仅局限于加密货币，还广泛影响了金融、计算、网络架构、安全和数据

管理等多个领域。以下是被区块链技术启发并发展的几类代表性新技术：智能合约、去中心化金融（DeFi）、同质化代币（NFT）、去中心化身份（Decentralized Identity, DID）、去中心化自治组织（DAO）、零知识证明（Zero-Knowledge Proofs, ZKP）、Web 3 等。而这些技术大多是开源的，这使得区块链技术带来的创新爆发，进入千行百业，影响深远。（关于开源与区块链创新，详见第 44 问。）

（3）物联网技术。开源正在成为推动物联网（IoT）领域创新的核心力量。它通过开放的硬件、软件 and 标准体系，大大降低了开发门槛，加速了原型设计、系统集成和应用部署，使物联网从一个高门槛、碎片化的领域，逐步走向开放、协同与标准化。

例如，开源硬件平台（如 Raspberry Pi 和 Arduino）极大地降低了入门门槛。开发者甚至非技术人员都能通过这些平台快速构建原型，实现智能家居、环境监测、农业自动化等多种物联网应用。这种即插即用的创新方式，让全球的长尾开发者都能参与其中，推动了“草根”创新的普及。

在安全性方面，开源的透明机制也发挥了关键作用。全球开发者可以共同审计代码、发现并修复漏洞，提升了嵌入式系统和通信协议的可信度。开源固件（如 OpenWrt）提供了可持续升级的机制，而项目如 OpenSSF 则聚焦于软件供应链的安全性建设。

更重要的是，开源让物联网真正实现了技术民主化。在许多发展中国家或资源有限的地区，教育机构和初创公司借助开源工具完成了环境监测、公共设施管理、智慧农业等项目，显著提升了技术普惠的可能性。

总之，开源让物联网从“高成本、专业性强的小众工程”，变成了“人人可参与的创新实践平台”。（关于开源与物联网创新，详见第 45 问。）

42.3 开源创新影响的量化评估

尽管全面量化开源的经济影响具有挑战性，但近年来的研究和报告提供了一些关键数据，证明了其在成本节约和效率提升方面的巨大价值。

（1）直接节约成本。一份 2025 年 1 月发布的研究报告指出，在 2019~2023 年间，采用开源工业应用设备相比专有解决方案，实施成本降低了 30%~40%，维护成本降低了 45%~55%。Forrester 早前的报告“*Forrester: Open source delivers cost and quality benefits*”也显示，87%的受访企业实现了预期的开源成本节约。

（2）加速创新与开发。上述 2025 年的研究报告还量化了创新与开发速度的提升，幅度达到 45%~55%。这得益于开源社区能够快速解决问题，并通过共享代码库避免了“重新造轮子”。企业能够更快地适应技术变化并抓住新机遇。

（3）对 GDP 的贡献。欧盟委员会的研究曾估算，2018 年开源软件为欧洲 GDP 贡献了 650 亿至 950 亿欧元。该研究还指出，欧盟企业对开源的贡献量每增加 10%，可带来 0.4%~0.6% 的 GDP 增长。同时，每年还会新增 600 多家 ICT 初创公司。通过采购开源软件而不是专有软件，公共部门不仅可以降低总成本，还可以减少或防止被供应商锁定，提高其数字自主性。

42.4 开源在中国语境下的机遇与挑战

在全球开源浪潮中，中国扮演着日益重要的角色。开源已成为中国技术创新的关键驱动力和参与全球科技治理的重要途径。然而，在这一进程中，中国既面临着独特的时代机遇，也需应对诸多现实挑战。

一、中国的机遇

中国的国家战略、市场体量和人才储备为其拥抱开源、实现创新突破提供了得天独厚的条件。

- 政策支持。国家层面对开源的重视程度达到前所未有的高度。《“十四五”软件和信息技术服务业发展规划》等纲领性文件明确提出“培育壮大开源生态”，并将开源提升至构建国家软件创新发展核心动力的战略位置。信创（信息技术应用创新）工程的深入推进，也强调基于自主开源技术构建安全可控的产业链供应链，为国内开源项目提供了宝贵的落地场景和政策背书。
- 庞大市场与应用场景。中国拥有全球最庞大的互联网用户群体和极为丰富的产业形态。在金融科技、智慧能源、城市治理、工业互联网、移动支付等领域，复杂多样的应用场景为前沿技术的开源解决方案提供了绝佳的“试验场”和迭代环境。这种规模化的应用反馈能加速技术成熟，进而反哺上游开源社区，形成正向循环。
- 开发者红利。中国已成为全球第二大开源贡献国，拥有规模巨大且日益活跃的开发社区。GitHub 与 Gitee 等平台上的中国开发者数量持续快速增长，他们不仅是开源技术的消费者，更逐渐成为重要的贡献者和项目发起者。这股“开发者红利”是支撑中国开源生态持续繁荣的核心力量。
- 弯道超车机会。在 AI、区块链、Web 3 等新兴技术领域，全球技术格局尚未完全固化。中国通过积极拥抱和主导这些领域的开源项目（如百度的 PaddlePaddle、华为的 MindSpore 等），有机会与国际巨头同台竞技，缩小在传统基础软件领域的差距，甚至实现局部领域的引领。

二、中国的挑战

尽管机遇巨大，但中国的开源生态仍面临来自技术、商业和文化层面的严峻挑战。

- 上游依赖与自主可控压力。当前全球开源生态的基石项目，如 Linux 内核、Python 语言、Kubernetes、PyTorch/TensorFlow 等，均源于并主导于国外社区。中国的信息技术产业在很大程度上构建于这些上游项目之上，存在显著的“卡脖子”风险。一旦国际环境发生变动，基础软件供应链的断供将对国内产业造成巨大冲击，实现真正意义上的“自主可控”仍任重道远。
- 产业化能力不足。与欧美成熟的商业化开源模式相比，中国开源企业在探索可持续的商业模式、融入国际基金会治理体系，以及实现开源项目与资本市场的高效结合方面仍处于初级阶段。因为缺乏成熟的运营模式和可持续的资金机制，许多开源项目难以实现从“开源”到“商业化成功”的跨越。

- 治理与文化差异。中国开发者在国际主流开源社区的治理体系中话语权仍然有限。参与方式多侧重于代码层面的技术贡献，而在制定社区规则、把握项目发展方向、担任项目维护者或基金会董事等“规则制定”层面的影响力明显不足。这与中国开发者的技术能力不相匹配，也限制了中国开源生态在全球范围内的软实力输出。

从战略视角看，中国开源的发展之路需要在“自主”与“开放”之间找到平衡：一方面，必须未雨绸缪，大力投入构建自主可控的开源基础设施（如开源托管平台、代码仓库、协议等），降低对单一上游的依赖风险；另一方面，应坚持“开门搞创新”，积极融入全球开源体系，鼓励开发者深度参与国际顶级社区治理，争取规则制定话语权，避免陷入技术孤岛。唯有如此，方能将中国的机遇转化为真正的创新优势，在全球开源创新网络中扮演不可或缺的核心角色。

42.5 开源创新面临的挑战与未来展望

在 2025 年度阿帕奇软件基金会亚洲大会上，白鲸开源的郭炜发出了一个灵魂的拷问：“We Love Open Source! But Who the Hell Loves Us?” 如果只谈创新，谈到人人都能够参与开源的协作，似乎凭着热爱、大家贡献，全人类都会获益。但是，如果存在太多的搭便车的人只想要享受创新的成果，却不愿意回馈社区（无论是参与贡献还是捐赠金钱），那么核心开发者也会产生倦怠的情绪，他们会扪心自问：“真的还要继续吗？”

开源不等于盈利，如何在“开放”与“可持续”之间找到平衡，也是难题。开发者会倦怠，赚不到钱的开源企业，也只能重新选择闭源（或者退出市场）。

当我们在谈论开源带来的创新是那么美好、有效的时候，也要关注：这样的创新背后，是谁在付出？他们的付出，是否获得了合理的回报？如果在激情燃烧之后，在开源已经改变了世界之后，那些为开源创新作出贡献的人，却一无所获。这种创新，就是不可持续的。

因此，我们所期待的未来，应该是一个开源贡献者能够获得公平、合理的收益的时代。我们不但应该期待这样的未来，更应该推动这样的未来，早日到来。

第 43 问 开源如何影响 AI 领域的发展

开源在 AI 领域中扮演着极其核心和变革性的角色，是推动该领域爆炸式增长的核心驱动力、主要的创新引擎和技术普及的基石。从底层的深度学习框架到尖端的生成式 AI 模型，再到基于大模型的 AI 编程助手、RAG 技术、Agent 框架与众多上层 AI 应用，开源以前所未有的广度和深度塑造了 AI 技术的演进轨迹。另外，开源 AI 也带来了一系列严峻的伦理、安全和法律挑战，使其成为一把锋利的“双刃剑”。本问将简要介绍开源在 AI 发展中的作用及面临的挑战。

解码开源：开源生态入门——160

43.1 开源在 AI 发展中扮演的角色

（1）创新引擎与协作催化剂。开源是 AI 技术诞生以来最重要的催化剂之一。从早期的机器学习库到当前的开源大模型，开发者社区通过共享代码、数据集和模型，加速知识传播与创新发展。开源平台，如 Hugging Face，以及国内的魔乐社区、魔搭社区、模力方舟等，还有托管代码的 GitHub，以及国内的 Gitee、GitCode、GitLink 等，成为研究和产业界的基础设施，极大降低了重复造轮子的成本。

（2）公平接入，降低门槛。开源使 AI 工具对任何人都保持开放，无论是个人开源爱好者还是初创公司，都能以极低成本获取最先进模型并进行微调。这种“全民参与”的模式促进了技术民主化，增强了全球技术平衡性。

（3）知识传播与社区协作。开源项目天然支持社区参与和知识共享，AI 领域的研究人员越来越习惯于在发布论文的同时，也发布源码和数据，其他团队可复现并在此基础上创新。通过开放式贡献与同行评审机制，整个生态的迭代速度显著提升。

（4）战略协作：“开源竞合”模式。大企业为了保护生态份额往往采用“开源竞合”模式（相互竞争的企业可以基于开源项目一起协作），例如 TensorFlow 和 PyTorch，虽然背后依托谷歌和 Meta，但是它们通过开源平台构建更广泛的生态，推动硬件厂商、云服务商参与协作，从而扩大整个市场。

43.2 开源组件——奠定 AI 创新与普及的基石

在 AI 领域，在以 ChatGPT 为代表的大语言模型爆火之前，已经有众多开源框架、开源算法库与开源工具集，这些开源项目的存在为 AI 创新与普及奠定了坚实的基础。AI 领域的著名开源项目如表 43-1 所示。

表 43-1 AI 领域的著名开源项目

领域	名称	层级	主要语言	许可证
通用机器学习	TensorFlow	框架	Python/C++	Apache-2.0
	PyTorch	框架	Python/C++	BSD
	scikit-learn	算法库	Python	BSD
自然语言处理	Hugging Face Transformers	算法库	Python	Apache-2.0
	spaCy	工具集	Python	MIT
	OpenNLP	算法库	Java	Apache-2.0
机器视觉	OpenCV	工具集	C++/Python	Apache-2.0
	Detectron2	算法库	Python	Apache-2.0
	MMDetection	算法库	Python	Apache-2.0
强化学习	Stable-Baselines3	算法库	Python	MIT
	RLlib	平台	Python	Apache-2.0

表 43-1 中只列出了开源项目的一小部分，还有众多领域的创新开源项目并未列出。这里只是希望给读者留下一个印象：AI 领域的几乎每个方面都有开源项目的身影，而不只局限在现在最火热的 LLM 方向。

43.3 2025 年开源 AI 项目的技术版图

（1）基础模型：走向更大规模、多模态与开放。

基础模型，特别是大语言模型，依然是 AI 领域的基石。2024 年以来，开源社区见证了众多重量级基础模型的发布，它们在参数规模、架构创新和特定能力上不断突破。国外有 Llama、Gemma 和 Mistral，国内有智谱 GLM、阿里的千问、深度求索的 DeepSeek、稀宇科技的 MiniMax 及百度的文心等开源大模型。令人欣喜的是，我国的这些开源大模型不断地刷新 AI 大模型的能力上限，不仅挑战国外的开源大模型，甚至对比顶级的闭源大模型也不遑多让。

（2）多模态 AI 与具身智能。

AI 处理信息的能力正从单一的文本扩展到图像、音频、视频等多种模态，并进一步寻求与物理世界的交互。这在科学研究（如 AlphaFold 预测蛋白质结构）、内容创作（如 Stable Diffusion 3 生成高质量图片）和人机交互领域至关重要。开源多模态模型 OpenFlamingo 就是一个典型代表，它能同时处理视觉和语言任务。

至于具身智能，则被视为 AI 的下一个浪潮，具身智能强调 AI 系统不仅能“思考”，还能在物理世界中“行动”。它需要 AI 具备从传感器系统（如摄像头、雷达）接收感知信息，并输出动作控制指令的能力。智元机器人于 2025 年 7 月 26 日发布的智元灵渠 OS，作为业界首个具身智能操作系统参考框架，其实现从底层硬件驱动到上层智能服务的全链路打通，对具身智能行业迈入标准化、规模化、生态化发展意义匪浅。

（3）推理框架与部署工具。

模型的高效推理是 AI 应用能否成功落地的关键。vLLM、Ollama、FastChat 等开源项目，支持一键式在本地或云端部署主流大模型，并提供与 OpenAI 兼容的 API，极大简化了开发和集成流程。

（4）边缘智能与隐私计算。

边缘智能与隐私计算是当前技术领域的重要交叉点，特别是在物联网、AI 和数据隐私保护需求的驱动下。边缘智能指的是在网络边缘（靠近数据生成源的地方）运行 AI 和机器学习（ML）模型，而不是依赖集中式云服务器。这种方法可以降低延迟、减少带宽消耗，并通过减少数据传输增强隐私保护。隐私计算则专注于在数据收集、处理和存储过程中保护用户隐私，特别是在敏感数据（如医疗、金融数据）涉及的场景中。

- KubeEdge 是一个开源的 Kubernetes 原生边缘计算框架，旨在将 Kubernetes 的容器编排能力扩展至边缘设备。它支持在边缘设备上运行 AI 和 ML 工作负载，并通过 Secure Production Identity Framework for Everyone（SPIFFE）确保安全性。

- FATE 是一个开源的安全计算框架，专为联邦学习（Federated Learning）设计。联邦学习是一种隐私保护的机器学习方法，允许多个参与方在不共享原始数据的情况下共同训练 AI 模型。

（5）其他垂直领域。

开源 AI 项目不仅在技术上持续演进，更在医疗、工业、金融等垂直领域加速落地，创造实际价值。

- 医疗领域：谷歌基于 Gemma 3 架构开源的医疗专用模型 MedGemma，擅长多模态医学文本和图像理解，可用于放射影像分析和临床报告总结。
- 工业领域：TensorFlow Lite（后改名为 LiteRT）是谷歌面向设备端 AI 的高性能运行时，作为主流的边缘 AI 框架，在工业场景中应用广泛。官方基准测试显示，通过模型优化和硬件加速，可在树莓派等设备上实现显著的延迟降低（如从 210ms 降至 85ms）和功耗节省（从 4.1W 降至 3.2W）。在目标检测任务中，其端到端延迟可低至毫秒级。
- 金融领域：现代金融风控系统是一个复杂的多模块体系，通常采用分布式架构，整合大数据分析、机器学习和知识图谱技术，通过深度学习算法实现从客户准入、反欺诈、信用评估到贷后监控的全流程智能化管理。在众多的实践案例中，开源技术都是必不可少的关键组成部分。

43.4 开源 AI 的伦理、安全与治理挑战

开源作为 AI 技术的创新引擎与协作催化剂，起到了重要的作用。但是在开放平台上的广泛协作，也给供应链攻击创造了机会。受到攻击和篡改的模型在特定触发条件下表现出行为异常，这使得安全防护非常复杂，防不胜防。

当我们在欢呼开源 AI 实现了公平接入、降低门槛的同时，也应该意识到，恶意使用者现在也能够更加方便地用上 AI 技术了。开源模型被用于生成假新闻、网络钓鱼邮件、深度伪造等恶意用途，已经屡见不鲜。由于采用开源技术，大多选择在本地部署，这给源头追查、责任追究、及时拦阻带来了极大的困难。

现在的开源 AI 模型大量使用互联网上获取的数据，通过生成式 AI 创造的内容又进一步被传播到互联网上，这使得传统的知识产权保护面临重大挑战。预训练、微调、蒸馏等全新的技术，如何以及是否构成侵权，也是现在开源 AI 治理需要讨论的热点问题。

传统开源许可证的设计初衷并未完全考虑到 AI 技术的特殊性。为了限制模型的滥用，一些模型发布方（如 Meta 对 LLaMA 模型的许可）开始采用带有使用限制的“伪开源”许可证，这与传统开源社区倡导的无差别使用原则产生了冲突。

当开源的逻辑、AI 的技术与商业的诉求碰撞在一起之后，各方的分歧会进一步凸显。如何寻找新的利益平衡点？各方的权益和诉求将以何种方式达成一致？开源 AI 时代，真正的开放式协作如何落地？都是全新而又亟待解答的问题。

再联系到当今的国际形势，不同国家政策对开放性和安全性的态度迥异。美国强调追求发展、去监管；欧洲强调要监管、要治理；中国则强调全球治理与开放平

衡。最新消息是：2025 年 7 月 26 日，中国政府倡议成立世界人工智能合作组织，这是中国坚持践行多边主义、推动共商共建共享全球治理的重要举措。希望这个努力能够结出硕果。

第 44 问 开源如何影响区块链领域的发展

在数字技术的世界里，很少有两种理念能像“开源”和“区块链”这样天然契合。开源追求代码透明、协作共享；区块链强调去中心化、不可篡改、人人可参与。它们不仅在理念上彼此呼应，更在实践中相互成就——没有开源，就不会有今天的比特币、以太坊和成百上千条活跃的公链生态；而区块链的兴起，也在反向推动开源模式的创新和演化。

44.1 开源对区块链发展的六大核心影响

（1）信任与透明性的基石。在区块链领域，“代码即法律”是核心理念之一。开源让协议和智能合约的源代码对所有人公开，任何人都可以审查、验证甚至复现运行逻辑。这种透明性，避免了传统系统中“黑箱”式的不确定性。更重要的是，开源吸引了全球开发者参与“众包安全”——“The DAO 事件^①”之后，以太坊社区迅速完成漏洞审计与修复，就是透明性与协作的典型成果。

（2）促进协作与加速创新。开源让全球开发者得以跨越地理与组织的限制，在同一个协议栈上协作创新。区块链的“可组合性”——像搭积木一样构建新应用，正是得益于开源代码的自由复用。Uniswap、Aave 等 DeFi 协议，都是在以太坊上部署的，经由快速迭代而成为成功范例。

（3）去中心化与社区赋能。开源代码的可获取性避免了供应商锁定，任何人都可以运行节点或分叉协议。这不仅保证了技术的去中心化，也赋予了社区在技术路线上的自主权。例如，以太坊链（ETH）和以太坊经典（ETC）的分叉，正是社区在理念分歧时行使选择权的结果。

（4）降低门槛与推动普及。开源打破了技术准入的壁垒。无论是初创团队还是个人开发者，都可以在无须许可的情况下直接使用、修改并部署区块链相关代码。像 Hyperledger Fabric 这样的开源框架^②，不仅推动了企业级区块链的快速原型开

① “The DAO 事件”是区块链历史上的一次著名黑客攻击事件，发生在 2016 年 6 月。The DAO 是一个基于以太坊的开源去中心化自治组织，因智能合约漏洞被黑客攻击，损失约 5000 万美元。此次攻击导致以太坊社区分裂，为恢复被盗资金，以太坊进行了硬分叉（hard fork），创建了新的以太坊链，而坚持原链的部分人形成了以太坊经典。这一事件暴露了智能合约的安全风险，并促使区块链行业加强代码审计和安全实践。

② Fabric 最初由 IBM 与 Digital Asset 贡献，在 2016 年开源并在 Linux 基金会的 Hyperledger（超级账本）项目下孵化。

发，还成为区块链教育和培训的重要教材。

(5) 推动互操作性与标准形成。开放标准是区块链生态扩展的基础。ERC-20、ERC-721 等代币标准，以及 Cosmos 的 IBC 跨链协议，都是在开源协作下形成的。这些标准的存在，使跨链交互与应用生态的无缝衔接成为可能。

(6) 促进生态系统繁荣。开源为区块链提供了丰富的基础设施与工具链——从智能合约开发框架（Truffle、Hardhat）到节点客户端（Geth、Prysm）。这些工具不仅降低了开发难度，也支持了从 DeFi 到 GameFi、从供应链到数字身份等多样化应用的诞生。OpenSea 的 NFT 交易、Chainlink 的去中心化预言机、Decentraland 的虚拟世界，都是这一生态的直接受益者。

44.2 在开源推动下诞生的新技术及跨领域影响

开源不仅加速了区块链自身的演化，还为许多新兴技术提供了孕育土壤，这些技术在区块链世界成熟后，又反过来渗透到更广阔的领域，产生了深远影响。

(1) 隐私保护技术的突破。

区块链的公开透明虽然带来了信任，但也意味着交易细节可能被任何人查看。为了平衡透明与隐私，开发者在开源协作中不断探索，最终催生了零知识证明（Zero-Knowledge Proof, ZKP）等前沿技术。ZKP 允许一方在不泄露具体信息的情况下，向另一方证明某个陈述的真实性。它原本被用来在链上隐藏交易细节，如今已经被移植到医疗数据保护、金融隐私交易、身份验证安全等多个领域。例如，一家银行可以在不暴露客户资产详情的前提下，证明客户拥有足够的贷款资格。

(2) 分布式存储的崛起。

传统互联网的文件存储依赖集中式服务器，而区块链社区在开源环境中发展出了 IPFS（星际文件系统）等分布式存储方案，让文件可以被切分、加密并分布在全球节点中。这种方式不仅抗审查，还能降低单点故障风险。它最初被用来存储区块链上的大文件，如 NFT 的图片和元数据，现在已拓展到科研数据共享、去中心化云存储及提升内容分发网络（CDN）的效率等领域。

(3) 智能合约的出现，让“自动化执行”有了全新的边界。

在以太坊等开源区块链的推动下，智能合约使得预设逻辑可以在没有人工干预的情况下自动执行，这催生了去中心化交易所、借贷协议等金融创新。但它的应用并不局限于金融：供应链企业用它自动结算货款，保险公司用它在触发条件达成时自动赔付，甚至法律界也在探索将合同条款以代码形式嵌入数字化执行流程。

(4) 跨链通信技术的成熟，解决了“区块链孤岛”问题。

起初，不同区块链网络之间是相互隔离的，资产与数据无法自由流动。开源社区的持续努力带来了跨链协议（如 Cosmos 的 IBC、Polkadot 的中继链），让不同链之间可以安全互通。如今，这项能力不仅促进了多链生态的融合，还被应用到跨境支付、物联网平台之间的数据协作等领域，例如，一个跨国物流平台可以让货运信息在不同的区块链系统间实时同步。

(5) DAO 治理工具的普及正在重塑协作模式。

去中心化自治组织 (DAO) 依托开源区块链工具, 实现了由持币者共同决策、自动化执行治理规定。这种模式不仅改变了区块链项目的运作方式, 还影响了更广泛的开源社区。开源项目团队可以用 DAO 机制分配资金、管理任务、做出版本路线决策; 非营利组织、科研项目组甚至艺术团体, 也开始尝试用这种透明而高效的方式进行跨国协作。

可以说, 开源与区块链的结合, 就像一座实验室, 不断孵化出能跨越行业壁垒的新技术。这些技术一旦走出区块链的“家门”, 便能在更广阔的世界中找到用武之地, 从而反向证明开源协作的力量不仅在于改进已有工具, 更在于创造全新的可能。

44.3 区块链对开源社区的反向推动

值得注意的是, 区块链技术也在反向推动开源模式的革新。链上治理机制, 如提案投票、资金分配、贡献者激励等, 正在被 Gitcoin 等开源平台吸收和试验, 为开源社区治理提供新的工具与范式。

代币经济则为开源项目提供了新的可持续资金来源。传统开源项目往往依赖捐赠或商业化服务维持运作, 而在区块链的加持下, 项目可以发行代币, 如 Aragon、ENS 等项目通过代币激励长期维护, 让贡献者直接从生态增长中获益。

区块链的不可篡改性, 也为开源代码的供应链安全提供了保障。通过链上记录代码版本与贡献历史, 可以更容易追踪漏洞来源、防止依赖包被恶意篡改。同时, DAO 和多签钱包等工具降低了跨国协作的信任成本, 让全球分布式的开源项目能够更高效地运作。

44.4 挑战与趋势

尽管开源为区块链带来了透明性、活力与创新, 但它也面临一系列不容忽视的挑战。

(1) 可持续性和资金来源的问题。开源区块链项目通常依赖捐赠、资助或代币发行来维持运转, 但这类资金模式容易受市场波动和社区热情的影响。一旦资金链断裂, 即使技术前景广阔, 项目也可能陷入停滞。许多小型公链或去中心化应用, 正是因为缺乏稳定的资金支持而逐渐淡出人们的视野。

(2) 安全责任的分散带来了隐患。开源的本意是让更多人审查和改进代码, 但在实际操作中, 漏洞修复的协调推进往往困难重重。当漏洞被发现时, 开发团队、节点运营者、用户之间的反应速度和配合程度各不相同, 可能导致修复延迟, 甚至被攻击者抢先利用。2017 年, 以太坊钱包 Parity 多签漏洞事件就是一例。因修复协调不及时, 数十万枚 ETH 被冻结, 至今无法取回。

(3) 恶意分叉与诈骗项目的存在, 会消耗社区的信任与资源。开源意味着任何

人都能复制一份代码，改个名字就发行新的代币。这种技术门槛的降低一方面促进了多样化发展，另一方面也让一些投机者有了可乘之机。他们可能在代码几乎没有改动的情况下高调宣传，吸引投资，甚至卷款而逃。

（4）社区治理的复杂性是长期存在的问题。区块链项目的方向、功能更新、参数调整，都需要社区成员达成共识。但社区往往由背景、利益诉求各异的参与者组成，意见分歧不可避免。如果缺乏有效的决策机制，这些分歧可能演变为长期对立，甚至导致链的分裂——比特币与比特币现金、以太坊与以太坊经典的出现，都是这种矛盾的结果。

（5）知识产权与监管政策的不确定性也让开源区块链的发展充满变数。虽然开源代码的许可证已经比较成熟，但当区块链涉及跨境交易、金融监管、数据隐私等敏感领域时，不同国家和地区的法律差异会带来额外风险。项目方需要不断适应政策变化，否则可能在某些市场面临禁令或法律诉讼。

不过，从趋势来看，这些挑战并不会阻碍开源与区块链的结合继续深入。相反，技术与治理的成熟反而会为它们带来新的机遇。未来，跨链合作将更加频繁，打破区块链之间的壁垒，促进数据与资产的自由流动；DAO 治理也会在实践中不断优化规则与工具，变得更加高效和稳健；开源的资金模式将更加多元化，从单一捐赠或代币发行，转向结合社区基金、协议收入分成，甚至与传统投资机构合作的复合模式。同时，区块链还会与 AI、物联网等技术发生更多交叉融合，例如，AI 模型的训练数据可以通过区块链验证溯源，物联网设备的状态更新可以用区块链保障可信性，这些跨领域的创新，可能催生超出当下想象的突破。

开源是区块链的灵魂，也是它不断进化的动力来源。区块链则是开源理念的最佳实践场之一。两者相互赋能，不仅推动了去中心化技术生态的繁荣，还催生了一批能够跨越行业、改变世界的技术创新。在未来的技术版图上，开源与区块链注定会继续并肩前行，塑造一个更加开放、透明与协作的数字世界。

第 45 问 开源如何影响物联网领域的发展

根据财富洞察报告（Fortune Business Insights）对物联网的市场分析，2025 年全球物联网市场规模为 787.1 亿美元。预计该市场将从 2026 年的 971.0 亿美元，增长到 2034 年的 5209.3 亿美元，在预测期间显示出 23.36% 的复合年增长率。

而根据国际数据公司（IDC）的报告：全球物联网支出在 2021 年为 6902.6 亿美元，预计 2026 年，全球物联网（企业级）支出规模将达到 1.1 万亿美元，五年复合增长率（CAGR）为 10.7%。

不同权威机构的预测，计算口径不一，数据互相矛盾，但存在一个共通之处：“大家都知道物联网市场巨大，但无人确知究竟有多大”。基于不同口径得到的增长率，从最低的 10.7% 到最高的 23.36% 不等。

在这样飞速发展的过程中，开源技术在这一背景下迅速崛起，既降低了门槛，又促进了协作，成为推动物联网创新的关键力量。

45.1 降低成本与开发门槛

开源的最大价值之一，是让物联网开发变得更经济、更普惠。通过软件和硬件层面的创新，降低了准入门槛和成本。软件层面，开源实时操作系统（RTOS）和中间件支持多样化硬件，简化开发流程；硬件层面，开源平台和芯片使原型设计和量产更易实现。

在软件领域，多个开源项目推动了物联网的普及。Zephyr OS 作为 Linux 基金会的项目，专为物联网设备设计，支持数百种 MCU 架构，其模块化架构和丰富网络功能（如内置安全协议）使其被英特尔、Nordic 等公司用于商用设备。FreeRTOS 由 AWS 维护，全球下载量超数百万次，应用于医疗和工业控制系统。例如，飞利浦医疗和霍尼韦尔的产品中使用了这些开源项目，提供实时任务调度和内存管理。Eclipse Mosquitto 则是一款轻量级 MQTT Broker，可运行在仅 1MB 内存的嵌入式设备上，适合低资源环境，并通过开源许可促进消息传递协议的集成。这些项目通过社区协作，提升了开发效率和安全性。

硬件开源同样驱动了物联网的经济性。Arduino 作为开源电子原型平台，全球销量已超过 1700 万块，成为教育、原型开发的标准工具，其易用性和灵活性支持快速迭代。ESP8266/ESP32 由乐鑫科技推出，是高性能、低功耗的 Wi-Fi 芯片，受益于开源 SDK（如 ESP-IDF 和 Arduino 支持），广受开发者青睐；ESP32 系列支持蓝牙、Wi-Fi 和多种编程语言（如 MicroPython），年出货量据报道称超 1 亿颗。这些硬件方案降低了设备成本，加速了智能家居和工业应用的部署。

有报道称，深圳一家智能家居创业公司仅用 ESP32+Zephyr+Mosquitto 组合，在 3 个月内完成了智能插座原型开发，并将硬件成本控制在 3 美元，相比专有方案节省了近 60% 的预算。Zephyr 提供跨平台兼容性，缩短开发周期；Mosquitto 实现轻量级通信；ESP32 的高集成度和低成本特性（如外围器件少）有助于成本控制，这凸显了开源方案潜在的经济优势。

45.2 加速创新与迭代

开源技术的全球协作模式显著推动了物联网领域的迭代效率。通过社区驱动的开发机制，项目可在高频更新中快速验证技术方案，降低企业试错成本。例如，Node-RED（由 IBM 发起）通过可视化流程编排能力，使开发者无须深入编码即可连接硬件设备与云服务。某能源公司利用其模块化特性，仅用 6 周便将风力发电机组的状态监控系统从原型部署至生产环境，大幅缩短了传统工业系统的开发周期。

资源受限的终端设备通过开源框架获得 AI 能力，成为物联网普惠化的关键突破。TensorFlow Lite Micro 作为轻量级机器学习引擎，支持在 16KB RAM 的微控制器（MCU），如 Cortex-M 系列 MCU 上运行模型。这使得低功耗传感器能直接在边缘端执行异常检测、语音识别等任务，减少云端依赖并提升实时性。其开源特性还

催生了丰富的模型优化工具链，帮助中小企业以低成本实现智能化升级。

分布式账本技术为物联网数据提供了可信存证基础。Hyperledger Fabric 作为企业级区块链平台，通过权限管理和拜占庭容错机制确保数据不可篡改。这一方案将传统数月的审计流程压缩至分钟级。沃尔玛中国基于它搭建食品溯源平台，物联网传感器采集的冷链温度数据及时存证，提升了供应链透明度。

上述案例印证了开源技术对促进企业创新的“杠杆效应”。通过复用成熟组件（如 Node-RED 的流程引擎、TensorFlow 的模型库、Hyperledger 的智能合约），企业可聚焦差异化创新而非底层重建。这种“站在巨人肩膀上”的模式，通过社区支持加速了创新迭代与商业化进程。

45.3 推动互操作性与标准化

开源协议是物联网的“公共语言”体系，物联网的碎片化生态依赖开源协议实现底层互通。MQTT 作为发布/订阅消息协议的标杆，通过 Eclipse Mosquitto 等开源实现，成为设备—云端数据管道的事实标准。其轻量化特性（仅需 1MB 内存）支撑了从智能家居到工业监控的规模化部署，并于 2016 年获 ISO/IEC 国际标准认证，奠定了协议级互操作性基础。

针对低功耗场景，CoAP 协议由 IETF 标准化设计，采用 UDP 传输与二进制编码，在 16KB RAM 的微控制器上实现 HTTP 兼容通信。例如，农业传感器通过 CoAP 多播功能向网关批量上报数据，降低 40%网络能耗，解决了 NB-IoT 等窄带网络的资源瓶颈。

在工业网关层，Eclipse Kura 提供开源协议转换框架，集成 Modbus、OPC UA 等 20 余种工业协议驱动。其 Java OSGi 架构允许开发者动态加载厂商定制协议插件，将西门子 PLC 与三菱机械臂数据统一转换为 MQTT 上行，破解了 OT/CT 系统融合难题。

低功耗广域通信（LPWA）的碎片化生态曾长期受制于供应商锁定问题。LoRaWAN 协议通过开源网络栈（如 ChirpStack、TTN）构建标准化通信层，使终端设备、网关与应用服务器解耦。在荷兰鹿特丹的智慧水务系统中，这一价值得到充分验证：水务局混合采购 5 家厂商设备，所有设备通过 LoRaWAN OTAA 激活流程接入统一 ChirpStack 服务器，Join Server 自动分配跨厂商安全密钥。网关实时上传设备健康状态（如电池电压、信号强度），AI 算法预判故障节点，现场维护频次减少 60%；开源网络服务器支持跨厂商设备的空中升级（FUOTA），修复漏洞无须物理接触终端；所有传感器数据经 MQTT 转发至统一云平台，破除原有“数据孤岛”导致的多系统维护负担。根据鹿特丹 2024 年审计报告，实际运维费用降幅 18%~30%。

上述案例也证明了我们在前文讨论的开源标准能够为整个产业带来巨大的价值。与传统的标准化方式相比，针对标准快速推出的开源实现，能够充分调动行业的协作力量，打破生态碎片化的魔咒。

45.4 增强安全性与透明度

安全一直是物联网的软肋，而开源在这一领域的作用并非“免费但不安全”，而是通过透明性提升防御能力。开源安全工具（如 OpenSSL 和 WireGuard VPN）允许全球安全专家审查并快速修补漏洞。

以著名的开源项目 WireGuard VPN 为例，这是一项加密 VPN 协议和自由及开源程序。旨在获得比 IPsec 和 OpenVPN 更好的性能，后两者都是常见的隧道协议。WireGuard 协议的流量经由 UDP 传输。2020 年 3 月，该软件的 Linux 版本达到了稳定的生产版本，并被纳入 Linux 5.6 内核，并在一些 Linux 发行版中向后移植到早期的 Linux 内核。

ArsTechnica 的一篇评论表明，WireGuard 易于设置和使用，使用了强大的加密算法，并且代码库最小，攻击面小。另有专门的研究论文对此进行了深入的分析，表明 WireGuard 为资源受限的物联网设备提供了高效安全连接的可能性。

值得注意，也是本书反复提及的社区响应：活跃的开源社区通常能够更快地响应和修复发现的安全漏洞，这对于物联网安全至关重要。

45.5 构建繁荣生态与商业模式

根据 IoT Analytics Research 2023 的报告（见图 45-1），全球物联网市场规模呈现增长趋势。2022 年的全球企业物联网支出已达 210 亿美元，预计到 2027 年，将达到 483 亿美元的规模。

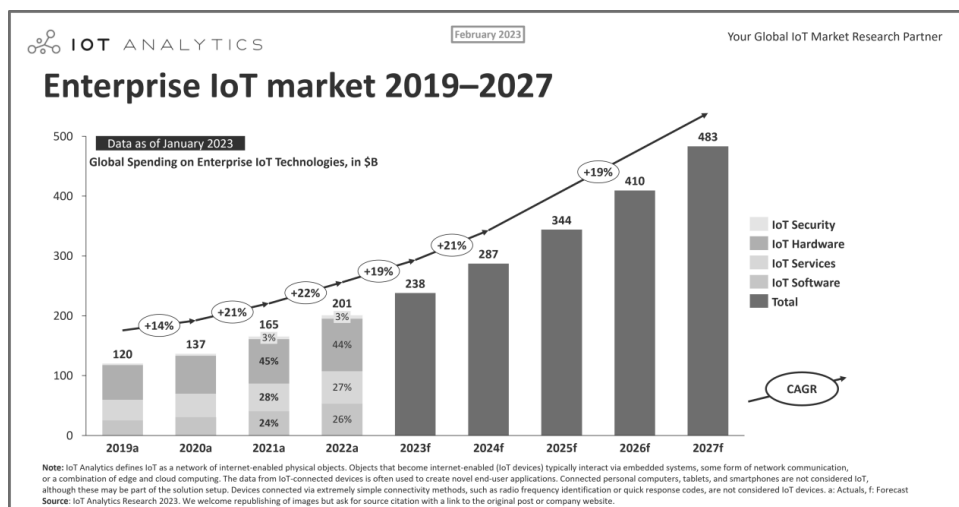


图 45-1 全球物联网市场规模增长趋势

设备端的普及速度同样惊人。IoT Analytics 报告指出，2025 年全球整体 IoT 连接数已达到 211 亿（其中蜂窝物联网连接约占 22%），预计到 2033 年，全球活跃

IoT 设备数量将实现翻倍，超越 400 亿个。

随着物联网生态的扩张，开源项目正在成为商业化的重要驱动力。开源服务市场本身也在快速增长：2024 年全球开源服务市场规模约为 154.4 亿美元，预计 2034 年将达到 756.1 亿美元，复合年增长率超过 17%（见图 45-2）。这表明，越来越多的企业愿意在开源基础上构建商业模式，例如，提供订阅服务、增值功能和专有硬件配套。

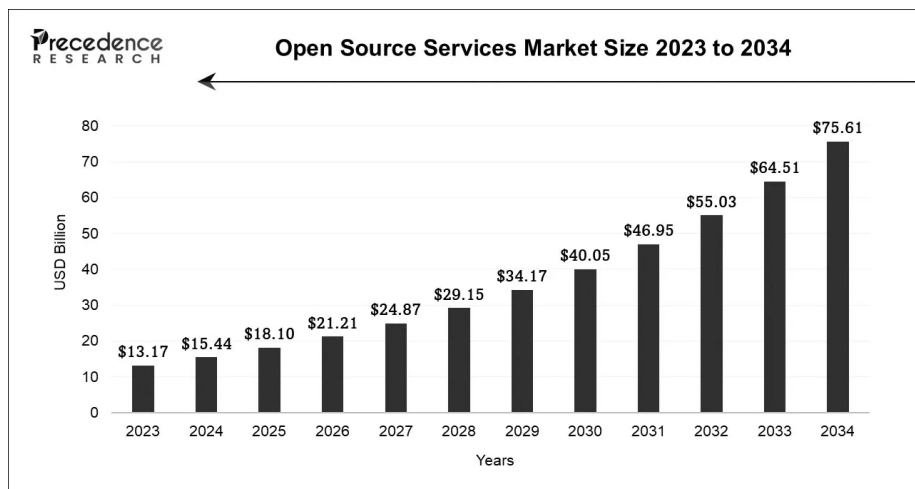


图 45-2 开源服务市场增长趋势

在硬件与软件结合的赛道上，开源解决方案展现出显著优势。例如，一家深圳的智能家居初创企业，仅用成本约 3 美元的 ESP32 芯片，结合 Zephyr OS 和 Mosquitto 等开源组件，在 3 个月内完成了功能完整的智能插座原型开发，相比使用专有平台节省了近 60% 成本。这不仅降低了试错门槛，也为后续商业化提供了可行路径。

Node-RED 是另一种加速商业部署的利器。某能源企业利用该平台，仅用 6 周时间就将风力发电机监控系统从概念阶段推向了上线运行，这种低代码、可视化的特性极大地提升了团队的迭代速度。

在工业边缘计算场景中，EdgeX Foundry 的灵活性同样令人印象深刻。例如，Technotects 利用 EdgeX 为工业泵和传感器构建 PoC，实现跨协议互联与实时控制，绕开了专有网关的高昂费用，展示了开源边缘解决方案的成本效益和部署自由度。

以上的案例与数据，可以整合得出一个乐观的结论：全球市场和设备数量的爆发式增长，为开源生态提供了巨大的舞台；而企业在硬件、平台、安全等方面的创新实践，则为商业落地提供了清晰的路线图。未来，随着协议标准化和边缘 AI 的进一步普及，这一领域有望迎来更高质量的增长。

45.6 潜在的挑战、风险与展望

- 安全风险：一方面，物联网设备数量激增，为黑客提供了更多攻击入口；另一方面，工业物联网（IIoT）的安全威胁可能直接危及生命，尤其是低功耗设备难以部署强安全措施，导致设备硬件本身易受到攻击。
- 隐私风险：端侧设备收集到的隐私数据，如何合理使用、妥善保管、避免滥用，也是物联网领域需要面临的重大挑战。
- 标准化困难：不同厂商设备（如蓝牙/ZigBee 协议）兼容性差，形成“数据孤岛”。缺乏全球统一的安全标准，导致安全防护参差不齐。
- 数据过载：大量的设备意味着大量的数据，如何在端、边、云合理地分配计算任务，如何利用网络高效传输数据，也是一项困难的任务。

以上提到的这些挑战与风险，同时也是物联网未来的破局之路。如何推动技术创新？如何更好地实现标准化与互操作？如何快速适应全新的业务场景？如何在上下游产业与众多厂商中快速构建生态联盟？以上这些问题，其实都指向了同一个答案：开源！

我们可以合理地展望，开源不但在过去支持了物联网领域的快速发展，更会在今后帮助物联网领域更加健康、快速地发展与创新。

第 46 问 开源与知识产权之间，存在何种张力

知识产权是一个非常古老的词汇，早在 17 世纪中叶，德国学者卡普佐夫就在著作中首次提出了“知识产权”的概念。根据维基百科上的词条，知识产权是人类用智慧创造出来的无形的财产，主要涉及著作权、专利、商标等领域。

用我们自己可以理解的话来说，人类的劳动可以分为体力劳动与智力劳动，体力劳动创造出来的有形物质，其财产的属性与归属都比较容易确定。但是，智力劳动创造出来的无形的东西，其是否具有价值、如何决定其价值、应该属于谁，都是非常困难的问题。

在知识产权提出后的近 400 年里，这些困难不但没有被消解，反而随着技术的进步，变得越来越复杂、越来越难解了。

46.1 知识产权遭遇的历次挑战

知识产权的本质是法律为无形的智慧造物赋予的一种有限的财产垄断权，其核心目的在于：通过保障创造者能从其劳动中获益，来激励持续的创新与创作，最终通过成果的扩散促进社会整体进步。这一精巧的平衡机制，自诞生之日起就不断遭遇技术革新的冲击与重构。历次挑战并未颠覆其法律内核，却不断重塑其实现哲学与实践形态。

（1）传统本质——有限垄断与利益平衡的框架。

在传统物理世界，知识产权的本质相对清晰。它作为一种法律工具，旨在解决“无形物”（如思想、设计、故事）固有的“公地悲剧”问题——即无法排除他人“搭便车”，导致无人愿投入创造。其运作模式是“以公开换保护”：发明者公开技术细节，换取专利期的独家实施权；作者发行作品，换取版权期的复制发行权。

这一模式的精髓在于平衡：在赋予创造者临时垄断权以获取回报的同时，通过保护期限、合理使用等制度，确保知识最终汇入公共领域，滋养后世。此时，知识的载体（书籍、磁带、光盘）具有排他性，复制的物理成本天然构成了侵权门槛，法律得以在相对清晰的边界内行使职责。

（2）数字世界的挑战——边际成本归零与平台的崛起。

数字技术的降临，首次对知识产权的基础假设发起了猛烈冲击。文本、音频、视频、软件乃至大模型，其核心特征是将信息与载体彻底分离，复制与传播的边际成本趋近于零。

这一根本变化带来了如下两大挑战。

- 执法危机：近乎零成本的完美复制和瞬时全球分发，使传统法律手段难以应对海量、匿名的侵权行为，维权成本急剧上升。
- 模式重构：知识的控制方式从“防止复制”转向“控制访问”。销售“所有权”的模式演变为授权“使用权”，并进一步发展为订阅服务（SaaS）。平台（如应用商店、流媒体网站）凭借技术能力（如内容识别系统）和市场地位，成为事实上的知识产权“仲裁者”与“守门人”，重塑了利益分配格局。

在数字世界，知识产权的本质未变，但其实现路径变得无比复杂。它必须在鼓励创新、保护创作者权益与维护互联网的开放、共享文化（如二次创作、混剪）之间，进行远比过去更为艰难的动态平衡。

（3）开源运动的挑战——对垄断本质的哲学重构。

开源软件、开源硬件、开放数据以及开放大模型的兴起，带来了最深层次的挑战。它们并非否定知识产权，而是以一种“以子之矛，攻子之盾”的方式，重构了其逻辑。

开源运动的核心智慧如下。

- 法律根基未变：开源许可协议本质是版权许可。开发者并非放弃权利，而是主动运用法律赋予的排他性权利，来强制要求后续使用者保持开放（如 GPL 协议的“传染性”条款）。这证明，知识产权法本身是一个中立的工具。
- 激励模式颠覆：它摒弃了“通过垄断获取直接利润”的传统激励模型，转而构建一种通过开放协作降低创新门槛、加速迭代、形成生态的新范式。商业价值则通过提供技术支持、托管、定制化服务等周边模式实现。Linux 的成功和开源大模型（如 LLaMA、DeepSeek）的生态竞争，皆是此模式的典范。
- 焦点转移：博弈的核心从“如何保护”转向“如何治理”。关注点不再是

静态地保护一个成品，而是动态地管理一个由全球贡献者共同演进的项目的合规性与开放性。

知识产权的历次挑战，揭示了一条清晰的演进路径。

- 法律本质：作为由国家授予的有限垄断权，始终稳固。
- 实践本质：实现激励创新与公共利益平衡的方式，则在技术的催化下不断演变。

46.2 开源与知识产权的基本关系

开源与知识产权之间不是对立关系，而是一种复杂的互动关系。开源并不是否定知识产权，而是通过特定的法律工具（开源许可证）来重新定义知识产权的使用方式。开发者拥有版权（或专利、商标），但通过开源许可证赋予他人使用、修改、分发的自由。

在这样的授权中，版权的基础地位并无动摇，版权是开源的“支点”。没有版权，许可证就没有约束力。也就是说，开源是一种“带着版权的分享”。

但是，专利与商标也是知识产权的一部分，而这方面的复杂性也需要被妥善处理。

- 专利：开源项目可能会面临专利诉讼威胁，因此出现了 Apache-2.0、GPLv3 等带有专利授权条款的许可证。
- 商标：项目名称、Logo 等往往通过商标保护，以避免混淆和恶意滥用。

但是，开源与知识产权之间，的确存在着“张力”，主要表现在以下 3 个方面。

- 法律逻辑的冲突：知识产权的逻辑是“排他性”，强调独占、许可与收益。开源的逻辑是“共享性”，强调自由、协作与扩散。两者天然存在矛盾。
- 商业模式的挑战：企业一方面希望通过开源建立生态，但另一方面又担心失去专利壁垒和商业优势。如何平衡开源共享与知识产权保护，常常成为争议焦点。
- 合规风险的挑战：对于使用者而言，开源许可证的多样性和复杂性，带来合规难题（例如，GPL 的“传染性”条款是否会影响闭源产品）。

换句话说，真正受到挑战的是知识产权的传统理解与运作方式，特别是依赖于强控制和封闭壁垒的模式。

46.3 不同的角色如何认识与应对这样的张力

当我们说开源与知识产权之间存在张力，这就意味着面对这种张力，不同角色由于所处的位置与利益诉求的不同，往往需要采取差异化的认识与应对策略。

（1）作为个人、使用者以及贡献者。

个人开发者和贡献者是开源生态的基石。他们往往最直接地感受到知识产权规

则带来的限制与保护。对于个人而言，开源并不意味着放弃版权，而是通过开源许可证赋予他人一定条件下的使用权。因此，开发者需要意识到，选择何种许可证，实际上决定了代码未来的流通方式与限制范围。

在实践中，个人面临的主要挑战是如何平衡“自由分享”与“合法合规”。一方面，他们需要在发布作品时明确许可证的类型，以确保成果在既定规则下被使用；另一方面，作为贡献者，他们必须遵守项目既有的法律与社区要求，如签署贡献者许可协议（CLA）或开发者原创证书（DCO）。此外，他们还必须警惕“无意侵权”的风险，即在使用第三方资源时忽视许可证兼容性，从而引发法律纠纷。可以说，个人的应对路径是“理解规则、选择规则并遵守规则”。

（2）作为企业。

企业在开源与知识产权的张力中所受的影响最为直接和敏感。一方面，开源带来了极大的技术资源与生态优势，降低了创新成本；另一方面，企业仍需依赖知识产权来保护自身的竞争力和商业模式。

因此，企业对开源的认知通常是双重的：既将其视为战略机遇，又将其视为合规风险。在应对上，越来越多的企业选择建立开源项目办公室（OSPO），通过代码扫描、许可证审核和风险管理来规避潜在的法律风险。同时，企业会在开放与封闭之间建立一套平衡机制：核心的专利、算法与数据保持保护，而非核心的基础工具和框架则通过开源释放，以构建生态和扩大影响力。

商业模式创新也是企业应对的一部分。与其依赖对代码的排他性控制，不如通过增值服务、订阅模式或平台托管来获得收益。更进一步，企业参与到基金会或标准化组织中，不仅可以减轻合规压力，还能在规则制定过程中争取话语权，从而在全球开源治理格局中确保自身利益。

（3）作为基金会或其他开源组织。

基金会和开源组织通常被视为开源与知识产权张力中的“中立者”。它们的存在，旨在为开源项目提供一个稳定而可持续的法律与治理框架。

从认知层面看，基金会深知如果没有明确的知识产权管理，开源项目未来可能会陷入复杂的版权与商标纠纷。因此，它们往往集中管理相关的版权与商标，以避免个体或企业的单方面占有。这种集中管理既有利于维护项目的中立性，也保障了长期的可持续发展。

在应对方式上，基金会通过制定和推广标准化的法律工具（如 CLA、DCO、版权转让政策）来降低风险，并为社区成员和企业提供合规指导。与此同时，基金会也承担起产业桥梁的角色，既保障开源精神的延续，又协调不同利益主体之间的矛盾。可以说，基金会的作用并非单纯的技术支持，而是通过制度设计为开源项目注入法律与治理的“耐久性”。

（4）作为政府：立法与执法机构。

政府在开源与知识产权的张力中承担着制度守护者与规则制定者的角色。从整体社会利益出发，政府需要在鼓励开放创新与保护知识产权之间找到平衡。

在立法层面，政府应承认并支持开源许可证的法律效力，为开源的合规使用提供明确的法律保障。在政策层面，政府可以通过推动公共部门使用开源软件，提高

透明度与公共服务效率，同时带动产业生态的发展。在执法层面，政府应避免出现“开源即无产权”的误解，在保护开源共享精神的同时，也要维护开发者与企业的正当权益。更进一步，政府还需要鼓励本国企业和机构积极参与国际开源治理，以提升在全球技术与规则制定中的话语权。

因此，政府的应对并非单纯的监管，而是一种制度性的塑造，即通过政策与法律的协同，引导开源与知识产权在张力中实现新的平衡。

第 47 问 开源如何推动数字化转型

开源在当今的数字化转型浪潮中，扮演着基石与加速器的双重关键角色。它不仅仅是免费软件，更是一种协作、创新和开放的哲学，这种哲学与数字化转型的核心精神高度契合。在讨论开源如何推动这些变化之前，我们先来讨论一下数字化转型的定义。

47.1 什么是数字化转型

数字化转型（Digital Transformation）是指组织利用最新的数字技术（如 AI、大数据、云计算、物联网等）来重新设计和优化业务流程、组织结构、文化和运营模式，从而提升效率 and 创新能力，并更好地适应不断变化的市场或社会需求。它不仅仅是技术升级，而是涉及从根本上改变组织为客户、员工或公民提供价值的方式。

在不同的历史时期，我们用过不同的名词，命名一系列相似的变革：信息化、数字化、智能化。在更早的时候，我们还使用过“无纸化”“电算化”之类的名词。这些名词的演变，代表着一种趋势，一种从 20 世纪中叶计算机被发明之后就始终没有停止，甚至愈演愈烈的趋势。每一个组织都要不断地求新、求变，不能墨守成规。除了关注自己的日常业务，还必须关注最新的 ICT 技术是否能够引入自己的组织，成为组织提速的动力。在这个过程中，组织本身从结构到治理模式，从单个工作环节改进，到整个业务流程再造。一转百转，直到变成一个全新的组织。

可以将数字化转型理解为 3 个层次。

（1）技术应用：这是基础。采用新的数字工具和平台（如 ERP、CRM、SaaS 应用等）。

（2）流程优化：用数字技术优化现有流程，提升效率、降低成本。例如，使用自动化机器人（RPA）处理重复性工作。

（3）业务重塑：这是转型的最高层次。它意味着利用数字技术彻底改变商业模式和价值主张。

因此，数字化转型的关键词是“转型”，而不是“数字化”。它是一场涉及技术、文化、组织和领导力的全面变革。

47.2 企业与政府的数字化转型

企业与政府的数字化转型虽然核心都是利用数字技术进行转型，但由于根本目标、服务对象和考核标准的不同，在具体实施上存在显著差异，如表 47-1 所示。

表 47-1 企业与政府数字化转型的差异对比

方面	企业数字化转型	政府数字化转型
主要目标	提升盈利能力、市场份额和客户满意度，通过创新产品和服务实现业务增长	改善公共服务交付、增强公民参与和透明度，促进社会公平和政策执行效率
驱动因素	市场竞争、客户需求和技术创新，通常由高层领导推动，以应对商业压力	政策法规、公民期望和社会需求，常受政府战略或外部事件（如疫情）驱动
实施重点	业务流程优化、数据分析和生态构建，如工业互联网平台整合资源	数字化治理、数据共享和公民服务平台，如在线政务系统提升服务可及性
服务对象	消费者、客户：聚焦于特定目标市场或用户群体，为其提供产品和服务	全体公民、企业、其他政府部门：服务对象是全社会，必须考虑普惠性、公平性和无障碍性
决策与执行	相对集中和快速：通常由 CEO 或数字化转型官牵头，决策链条较短，可以对市场变化做出快速响应和试错	层级复杂和规范：决策过程涉及多个部门协调，遵循严格的法律法规和采购流程，变革步伐相对稳健，容错率较低
挑战与风险	资源有限、技能缺口和供应商依赖，但企业更灵活，可快速迭代	官僚主义、隐私保护和遗留系统整合更复杂，涉及多部门协调和法规合规
影响范围	主要内部运营和市场生态，强调经济效益和创新应用	覆盖社会整体，如数字采购影响企业创新，强调公共利益和包容性
评价标准	关键绩效指标：投资回报率、市场份额、客户增长率、利润率、用户活跃度、转化率等可量化的经济指标	社会效益：办事流程缩短时间、群众满意度、政务公开程度、政策执行效率、社会治安改善等社会效益指标

总体而言，企业数字化转型更具商业导向和敏捷性，而政府转型则更注重可持续性和公共责任。两者可相互借鉴，例如，政府政策可推动企业转型，反之，企业技术也可以帮助优化政府服务。

47.3 开源在数字化转型中的作用

开源，尤其是开源软件在推动数字化转型方面发挥着关键作用，它通过促进创新、降低成本、增强协作和提高灵活性，帮助企业和组织实现从传统模式向数字化

的转变。以下从几个主要维度阐述开源如何驱动这一过程。

(1) 降低成本并提高可及性。开源软件通常免费或低成本，这使得中小企业和资源有限的组织能够快速采用先进技术，而无须大量投资于专有软件。例如，开源工具 WordPress 用于内容管理、Asterisk 用于通信系统，以及 SuiteCRM 用于客户关系管理，这些都能帮助中小企业实现数字化转型，而不会降低质量。此外，开源减少了许可费用和供应商锁定风险，让企业将资源聚焦于核心业务创新。研究显示，开源的使用不仅提升了开发者生产力，还扩大了对新兴技术的访问。

(2) 促进创新和协作。开源的核心在于社区驱动的开发模式，这鼓励全球开发者共同贡献代码、解决问题和迭代功能，从而加速技术创新。在数字化转型中，开源被视为基础，它支持新兴技术，如云计算、AI 和大数据等技术的应用。在电信、能源、汽车、金融、农业和娱乐等垂直行业，开源驱动了技术转型，通过共享知识构建包容性数字生态。企业可以通过内源模型，将开源实践应用于内部开发，形成生态系统，促进跨团队协作。

(3) 增强灵活性和可扩展性。开源软件允许企业自定义和修改代码，以适应特定需求，这在快速变化的市场环境中至关重要。它帮助组织扩展规模、适应新需求，并创建独特解决方案，且无须从头构建。在数字化转型中，开源简化了集成和维护过程，让企业更敏捷地响应变化。例如，企业使用开源组件来支撑数据、网络 and 云基础设施，推动整体转型。

(4) 吸引人才和构建生态。开源还能帮助企业吸引技术人才，因为开发者更倾向于参与开源项目，这间接提升了组织的创新能力。通过开源项目办公室，组织可以系统化地管理开源贡献，进一步加速数字化进程。总体而言，开源不仅是数字化转型的催化剂，还被视为“伟大均衡器”，它让不同规模的组织都能参与数字经济。但是，企业需注意安全和合规问题，如定期审计代码，以实现效益最大化。

(5) 增强安全性与透明度。“开源等于不安全”是一种常见的误解，事实上，恰恰相反。

- 社区自查：开源代码对所有人可见，意味着全球的安全专家和开发者都可以审查代码，发现并修复安全漏洞。这个过程通常比闭源软件内部团队的小范围审计更高效、更彻底。
- 自主可控：企业可以自行审查代码的安全性，而不必完全依赖供应商的承诺。在发生安全事件时，企业可以自己动手快速修复，而不必等待供应商发布补丁。
- 快速响应：成熟的开源社区对安全漏洞的响应速度通常非常快，修复补丁会迅速发布给所有用户。

(6) 推动组织的文化变革。一个封闭的组织不可能成功转型，而一个转型过程中的组织，一定是一个变得更加开放、更加愿意拥抱变化的组织。这样的变化，就是在组织文化层面发生的。一开始，封闭保守的文化会成为组织转型的最大阻力。但是随着组织转型逐步取得成效，整个文化层面也会逐步发生软化与活化，最后成为积极支持、推动变革的最大动力。

开源、开放、开放式协作，共生、共创、共赢，会成为越来越多的组织成员的口头禅。

第 48 问 开源如何推动高校科研创新

在第 42 问中，我们探讨了开源驱动创新的六大核心机制，这些机制在推动高校的科研创新方面，同样有效。例如，在全球学术协作领域，开源透明协作模式催生了各类开放科学尝试，通过使用开源工具可全面降低科研创新成本，同时已有大量高校开源项目走出校园、在全球范围获得广泛应用并成为事实标准，这些均为相关典型案例。

在本问中，我们将进一步讨论开源为高校科研创新带来的各种实际效果。

48.1 从开源到开放科学

根据英文版维基百科，开放科学是一场旨在使科学研究（包括出版物、数据、物理样本和软件）及其传播渠道能够被社会各界人士——无论是业余爱好者还是专业人士，轻松获取的运动。它代表着一种透明且易于接近的知识体系，通过协作网络进行共享和发展。这一概念涵盖多项实践，例如，发布开放式研究成果、倡导开放访问、鼓励科学家采用开放笔记式科学方法（如公开分享数据和代码）、推动更广泛的知识传播与公众参与，以及整体上简化科学知识的出版、获取和交流过程。

开放科学包括以下几个核心原则。

- 开放访问（Open Access）：研究论文和出版物免费提供给公众，而非局限于付费订阅。
- 开放数据（Open Data）：研究数据公开共享，便于他人复用和验证。
- 开放方法（Open Methodology）：公开实验设计、代码和分析流程，以确保研究的透明度和可重复性。
- 开放教育资源（Open Educational Resources）：共享教学材料和工具。
- 公民科学（Citizen Science）：鼓励非专业人士参与科学研究。

联合国教科文组织（UNESCO）在 2021 年通过的《开放科学建议书》，将开放科学定义为“使科学知识开放、包容和可持续的实践”。从上述的介绍可以看到，开源对于开放科学的启发与影响是巨大的，可以认为开放科学是开源在科研领域的深化与扩展，其背后的理念一以贯之：开放共享、促进协作、推动创新。

为推动落实联合国教科文组织《开放科学建议书》，推动中国积极融入全球开放科学实践，提升科技文献和科学数据开放共享水平，促进与国际科技界的开放、信任、合作，在 2022 年，中国科学技术协会倡议成立了“开放科学促进联合体”。并在开放获取、开放数据、开放科学基础设施，以及联合体运行机制建设等方面持续探索。在 2025 年 9 月，260 家科技共同体还发布了“开放、信任、合作”倡

议，建议全体科技共同体成员，真诚携手、共同努力，让科技更好地造福人类，为全世界人民所享、所用、所及，为人类文明的可持续发展作出贡献。

48.2 在高校开展开源教育的价值

在高校的传统科研与人才培养过程中，一直会提到“产学研合作”，甚至“产学研用一体化”。但是，这样的做法还是存在不少缺憾，而“开源模式”恰恰为这些痛点提供了极具潜力的解决方案。

传统“产学研”合作的主要缺憾是目标冲突与节奏脱节，“学”“研”追求的是论文、基金和学术声誉，周期长，注重理论突破；而“产”追求的是市场份额、利润和快速迭代，周期短，注重实用技术。这种根本性的目标差异，常常导致合作是“同床异梦”，难以形成真正的合力。从学术研究出发的很多项目，其优秀的学术成果停留在论文或实验室原型阶段，无法跨越产品化、商业化的“死亡之谷”。原因在于缺乏中试、工程化、市场验证等中间环节的支持。而且，学生在传统产学研项目中，往往只是完成被分配的、局部的任务，难以窥见全貌，更少参与从创意到落地的全过程，其综合创新能力得不到充分锻炼。

目前，大家正在探索的开源教育，可以被理解为“企业开源→开源课题/开源课程/开源实习→围绕开源项目，开展教学与科研工作”。开源教育通过这样的合作模式为传统的产学研合作带来了新的突破。

- 构建共同的目标生态。所有参与者（高校、企业、个人开发者）围绕一个开源项目形成共同体。项目的成功和繁荣是所有人的共同目标。高校可以借此发表高水平论文，企业可以降低研发成本、招募人才、建立生态，个人可以获得成长。目标在项目生态中实现了统一。
- 建立“天然”的知识交换网络。开源社区本身就是一个 7×24 小时运转的全球知识市场。代码、文档、讨论区、邮件列表都是知识流动的载体。这种流动是自发的、持续的、透明的，打破了机构围墙。
- 实现“接力式”创新与平滑过渡。一个开源项目可以从高校的实验室原型开始，吸引企业开发者加入进行工程化加固；初创公司可以基于此推出商业版；其他公司可以将其集成到产品中。创新不再是“交接”，而是“接力”，整个过程在开放环境中自然发生，大大降低了转化门槛。
- 提供深度实践的“真实战场”。学生不再是被动的任务执行者，而是社区的平等贡献者。他们需要理解项目架构、与全球开发者协作、提交代码、参与评审。这培养的不仅是技术能力，更是沟通、项目管理、社区运营等综合能力。一份优秀的开源贡献履历，是通往顶级公司的黄金门票。
- 采用预先的、标准化的许可协议。通常，开源项目从一开始就采用明确的开源许可证（如 Apache-2.0、GPL）。这预先规定了各方的权利和义务，极大地简化了法律问题。合作的焦点从“如何分配所有权”转向了“如何共同把蛋糕做大”。

目前，我国的产学研各界正在积极协作，探索开源教育的更多可能。例如，清华大学、北京航空航天大学等 25 所高校联合成立“以贡献为导向的开源人才评价机制试点工作组”，将开源贡献纳入学生学分认证和教师成果评定体系。开放原子基金会推动的“校源行”活动已覆盖多所高校，形成课程、师资、实践项目一体化的开源教育模式。2025 年，天工开物开源基金会还设立了开源教育奖学金，评选优秀课程、导师及学生开源项目。

48.3 推动高校的科研成果开源

在 2024 年中国科学院的院刊上，发表了隆云滔等专家联合撰写的一篇文章《推动国家资助科研项目成果开源开放的国际经验借鉴及思考》。在这篇文章中，系统地阐述了在中国数字经济时代背景下，推动国家财政资助科研项目成果开源开放的战略意义、国际经验、现实挑战及政策建议。可以期待，开源对于科研创新的影响与促进，有望从“自然而然”阶段进入“国家政策推动”阶段。

文章开篇就特别强调了“开源创新成为数字时代全球数字公共产品供给的关键来源”，这样的数字公共产品的供给，对于整个国民经济和社会的高质量发展，起到了关键甚至是无可替代的作用。从财政资金的角度来看，采取开源开放路径有助于减少科技项目重复投入困境，优化财政资金分配，确保资源的有效利用，从而取得更高质量和更高效率的成果，同时也能促进经济的增长。另外，开源开放也有助于推进科研创新水平突破，从而形成正向循环，如图 48-1 所示。

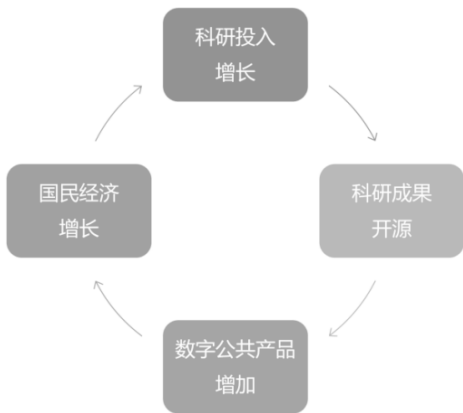


图 48-1 科研成果开源推动的正向循环

对比 GitHub、Hugging Face 这样的开源代码/开源 AI 托管平台，我们还缺少一个开放科学成果的展示、协作、交流的托管平台。因此，在文章中，专家们也建议“搭建国家开源开放的科技成果运营平台”，不仅是要促进国内科研成果的交流推广，更希望融入全球数字公共产品的联合建设和推广中，使更多中国数字化成果得到认可和推广。

48.4 展望未来的高校科研与开放科学

随着上文的思路，我们可以进一步展望一下，如果将开源实践全面引入高校科研领域，将会发生哪些变化？

- 科研论文的发布将会加上版本号：从最开始的 0.1 版到 0.2 版，每次发布都会添加一个感谢列表，列出为这个最新版本成果作出贡献的所有参与者。
- 从可阅读的论文到可执行的论文：2025 年 4 月，一个韩国开源项目 Paper2Code 横空出世，这是一个基于机器学习实现的从科学论文自动生成代码的工具。我们都知道，可重复性（Reproducibility）是科学进步的核心。而这样的项目，就是在 AI 的帮助下，进一步将可重复性做到极致。虽然目前的 Paper2Code 还只能转换机器学习领域的论文，但是我们的确可以期待更多。
- 从 IDE 到 IRE：IDE 是软件工程师都非常熟悉的集成开发环境，但是对于科研工作者，还缺少一套“集成科研环境（IRE）”。如果能够出现这样一种开源工具，同现在的主流 IDE 一样，允许安装各种各样的辅助插件，针对不同类型的科研工作，科学家们可以自由选择安装各种不同的研究辅助工具，从而彻底改变现在各类开源科研工具分散、小众、开发者与用户互相找不到对方的窘境。
- 从 GitHub 到 Open Data Hub：目前已经有各种正在建设中的开放数据集，根据 2024 年《开放数据状态报告》（State of Open Data 2024）显示，开放数据实践正处于成为全球认可标准的边缘，政策环境是关键驱动因素。许多国家（如美国、英国、德国和法国）的开放数据库共享率达 25% 左右，而“按需共享”比例下降了 1%~9%。报告建议通过政策、合规监测和培训来进一步推动。所以，未来是否有可能建成一个全球皆可访问、所有数据都能够找到的 Open Data Hub，我们还处在热切地期盼之中。

这不仅是技术的升级，更是一场深刻的科研文化变革。从论文的版本化管理，到 AI 赋能的可执行论文，再到一体化的科研环境与全球数据枢纽，开源实践将如同一股活水，注入高校科研的土壤，催生协作、透明与高效的新生态。目前，我们正站在这样一个拐点：开源精神将如同当年的印刷术与互联网一般，从根本上加速人类知识的循环与创新。当全球智慧在开放协作中汇聚，我们迎来的将是一个真正“群星闪耀”的科学新时代。

第 49 问 开源社区的未来会怎么发展

在 2023 年初，笔者与一位朋友有过一次交流，评估 AI 爆发之后的开源社区是会更加繁荣，还是从此走向衰落。当时笔者提出了“社区含人量”概念，在 AI

变得如此强大之前，社区里的交流当然都是人与人之间的交流，可以认为含人量为100%。但是，随着 AI 生成代码、AI 测试、AI 修复漏洞、AI 提交合并请求，再由 AI 来做代码评审，整个社区的含人量会逐步下降，人与人之间的交流（社区的本质）也会逐步减少，这会使得整个开源社区的活力下降。

由于 AI 生成代码的能力越来越强，在可以想象的未来，甚至我们都不必事先写好代码，而是在需要的时候，让 AI 当场生成一段代码。这使得对于软件代码复用的需求进一步下降。这也意味着对开源社区和开源软件的需求进一步下降。

因此，当时笔者得出了一个相当悲观的结论：开源社区将会逐渐走向衰落。在此之前，我们会经历最后一次社区繁荣——开源 AI 社区的繁荣。

在最近一年多的时间里，Agent 与 MultiAgent 的概念逐渐兴起。可以想象，未来的社区含人量会进一步下降，继而转向 Agent 之间的交流与探讨。整个社区可能只需要一两个人，其他的都是 Agent。这样的未来，是否切实又美好，的确很难判断。GitHub 从创立之初高举“Social Coding”（程序员们通过代码进行社交）的大旗，也许会逐步走向终结。

当然，人类永远需要社交，只是会“退回到”自然语言的社交，而不是基于代码的社交。人类也永远需要协作，但将会是“人机共生体”之间的开放式协作。至于这样的协作，与传统的人类之间的协作，将会产生哪些差别，现在还非常难以预测，这里只能勉强提出几个设想。

49.1 从代码社交到意图社交

随着 AI 逐步接管从编码、测试到评审的各个环节，代码的社交属性正在被削弱。当人不再是主要的代码生产者，代码本身也不再是社交的主要内容与载体。未来的协作可能更多地发生在“意图”的层面：人类提出需求、目标、价值判断，AI 与 Agent 根据这些意图自动生成解决方案，并相互协作完成任务。开源社区的社交，从“我写了一段代码给你用”变为“我定义了一个问题，我们一起用智能体来解决它”。这种“意图社交”比传统的代码社交更具抽象性和创造性，更加自然也更加自由，它强调的是思想与愿景的共享，而非具体代码行的交换。由此，开源的本质将从“共享成果”向“共享意图与智能”演进。

49.2 项目的生命周期缩短，而社区的流动性加快

AI 带来的一个深远影响，是开源项目生命周期的剧烈压缩。过去，一个成熟的开源项目往往需要数月乃至数年的演进：需求调研、设计、开发、测试、迭代，环环相扣。而在 AI 和 Agent 的辅助下，一个可用的原型可能在数小时甚至数分钟内生成。这意味着“项目”作为协作单元的意义在减弱，取而代之的是更加频繁和灵活的知识流与协作流动。人机协作的社区不再依附于单一项目的长期发展，而是不断涌现、演化和消散的小型协作体。它们围绕特定问题迅速形成、解决后又迅速消散。这种高度流动性的协作生态，或许会让传统意义上“维

护一个大而全的开源项目”的模式逐渐式微，而催生出“以问题为中心”的动态协作文化。开源社区的未来，也许不再是静态的项目森林，而是一条不断涌动、不断重组的知识之河。

49.3 Agent 社区与 AI 进化乐园

当 Agent 成为开源社区的主要参与者，整个生态将可能演化为一个“AI 进化乐园”。不同的 Agent 拥有不同的角色定位：有的擅长算法优化，有的负责文档撰写，有的充当测试专家，还有的可以在自然语言层面与人类协商。它们之间能够通过 API、自然语言或自主协商协议展开协作，形成一种新的“数字社会结构”。更重要的是，开源的 Agent 将使得这些 Agent 具备巨大的演化能力：模型可以从开源项目中学习知识，不同 Agent 可以复用彼此的能力，甚至在协作中产生“组合智能”的进化式行为。这样的社区不再只是软件的协作平台，而成为 AI 自身进化的实验场。人类在其中的角色，或许更像是生态调节者和规则制定者。随着人机协作的深入，未来的开源社区可能成为人类智慧与 AI 共同成长、共创未来的孵化器。

49.4 从开源到开放资源，再到开放智能

“开源”的概念最初指的是“源代码开放”（Open Source），它推动了软件创新和互联网的发展。但在 AI 时代，代码只是智能系统的一个组成部分，更多的价值来源于模型、数据、知识和算力等多元资源。于是，“开源”开始扩展为“开放资源”（Open Resource）——不仅共享代码，也共享数据集、模型权重、知识库与工具链（参见王坚博士在 2025 年 9 月 11 日的演讲）。而这条演化路径的下一站，是“开放智能”（Open Intelligence）：不仅共享资源，还共享智能体本身的能力、思维过程与决策逻辑。未来的开源社区将不再是“贡献代码”的地方，而是“汇聚智能”的平台。人类与 AI 在这里不仅协作开发软件，还协作探索问题、提出假设、构建知识与创造解决方案。开源的内涵，也将从“共享生产资料”升华为“共享智能化”的文明工程。

第 50 问 开源技术的未来趋势是什么

即使从 1985 年自由软件基金会成立开始计算，开源至今也不过发展了短短 40 多年的时间。从一群黑客的个人爱好与边缘创新演变为数字经济的核心驱动力，进而成为推动整个世界发生变化的关键力量，开源的未来将会如何发展，可以说既一目了然，又难以判断。一目了然的部分是：开源注定会对未来产生越来越重大的影响；而难以判断的部分在于：究竟会产生哪些，以及如何产生影响？

对于技术的发展，我们应该先建立一个基本的假设：在一个时间段内，某一项特别重大的技术发展，将会推动其他的技术跟进发展。有可能是这项技术的发展，

打开了一系列过去并不存在的创新可能性，也可能是这项技术的快速发展，导致了其他领域不断产生“失配”，必须快速发展相应的技术，堵上这些“缺口”。在当前的开源领域，AI 的发展，正是这个“牵一发而动全身”的技术。

50.1 被 AI 全面增强的开源

在 AI 时代到来之前，原本的开源技术发展趋势，还可以分门别类地讲述。但是到了今天，AI 已经全面渗透到人类社会的每一个方面，开源技术的发展自然也会全面受其影响。除了 AI 技术本身的飞速发展，我们还可以看到以下一些趋势：从底层开发范式到上层生态构建，开源技术从此进入了智能化协同创新的新阶段。

（1）开发过程的全面 AI 化。

从目前最火的 AI Coding（Vibe Coding）到针对软件开发过程各个环节的智能化增强和改造，AI 辅助开发工具已实现大规模应用。这样的变化首先体现在效率的大幅度提升。我们会发现：新方法带来了新问题。AI 产生的大量代码，基本上可用，细节上却需要调整，这甚至催生了一个新的工种：“AI 垃圾清道夫”。当然，技术进步带来的问题，也只能靠技术的继续进步来解决。

虽然现在有一个潮流：追求让 AI 独立工作更长的时间，一口气完成更加复杂的开发任务。但是在笔者看来：开发过程中合理的人机交互协作，才是更加现实的未来。

（2）软件功能与架构的 AI 化改造。

随着 AI 变得越来越强大，传统软件中那些已有的功能是否能够变得更加智能？上一代的软件架构，如何通过改造，容纳一个或多个 LLM 服务？将传统软件封装成 MCP 或 Agent，供更加聪明的 AI 协调调用？这些都是新一代的架构师与产品经理，正在苦苦思索、热烈讨论的问题。

现在有很多传统软件，都开始在屏幕右侧添加一个“智能对话助手”。还有各个领域，各种场景下，对于“智能编排”与“智能体”的讨论。传统的人机界面，是不是完全可以改为对话式？我们最喜欢的各种可视化的图表，是不是也可以用 AI 来实时生成？

未来我们肯定会看到很多全新的、现在难以想象的软件，以及传统软件的全新版本。而这一切的背后，肯定都离不开 AI 的支持。

（3）通用化与专业化的博弈。

在这一轮的 AI 浪潮出现之前，IT 技术的发展趋势是不断地细分、不断地变得更加专业化的。仿佛整个世界地图都已经完全探明，剩下的只是深入更多细节，寻求特定场景下的特殊需求而已。但是，在 2022 年底发布的大语言模型 ChatGPT 3.5 出现之后，这种强大的 AI 似乎什么都懂一点，什么都能上手做一下。随着大模型技术的飞速进步，在 2~3 年的时间里，一些临时性的解决方案，在更加强大的大模型出现之后，都会快速过时，显得不再必要。因此，当前的一股言论甚至认为：未来我们只需要最强大的通用大模型，甚至 AGI 级别的大模型就够了。其他

的一切专业技术、细分领域、特殊场景，对于强大的 AI 来说，都是不在话下的问题。

但是在笔者看来，这依然只是一时的趋势。长期来看，大模型的能力增长会达到自己的平台期，就像摩尔定律最终会失效一样，规模化法则（Scaling Law）也会失效。通用化的解决方案，在不断提高的要求面前，依然会显得不够好，要么效果不能满足需要，要么成本、效率不能满足需要。总之，专业化的、面向特定领域的优化的趋势，会再次抬头，并成为新技术时代的新常态。

这样的技术发展趋势，我们已经经历多次了，主机时代、PC/桌面软件时代、互联网/移动互联网时代与云计算/云原生时代，都会有一个百花齐放，逐步摸索出稳定的架构模式，再在达成共识的架构模式的框架上细化、专业化的过程。未来的开源技术，也会有类似的发展趋势。

50.2 AI 对未来基础设施提出的挑战

AI 的工程化要求打通“能源—算力—模型—数据—网络—应用—调度—可观测性”这一链条，新一轮 AI 技术浪潮将促使上述各领域加速创新。

在 AI 的训练与推理过程中，高度依赖高性能计算，导致数据中心的电力需求急剧增加。一个以 AI 为主的数据中心的耗电量相当于十万户家庭的用电量，而目前正在建设的最大数据中心的耗电量将达到前者的 20 倍。国际能源署预测，到 2030 年，美国 AI 数据中心用电量可能增长 130%，中国则可能增长 170%。这将促使更多投资流向可再生能源（如太阳能和风能）领域。当然，备受期待的“核聚变发电”技术也有望实现更快速的发展。算力与电力的协同机制创新尤为关键。中国推行的“东数西算”工程开创性地将算力需求与西部可再生能源优势相结合，美国则探索通过需求响应机制将数据中心转化为电网灵活性资产。

除加快发电外，提高图形处理单元（GPU）、张量处理单元（TPU）的能效也是重要举措。目前，全球能效竞赛已进入白热化阶段。从芯片级能效提升（如谷歌张量处理单元 7 比前代能效提升一倍）到系统级优化（英伟达 GB200 NVL72 能耗降为原来的 4%），技术创新正在突破“不可能三角”的约束。然而，目前中国在 AI 计算能效方面仍落后于美国，这一差距凸显了自主创新的紧迫性。

AI 模型（尤其是大语言模型）的训练和推理依赖高性能图形处理单元/张量处理单元集群，这些硬件在运行时会产生极高的热密度。为维持硬件稳定运行并保证计算效率，必须配备高效的冷却系统，否则会导致设备性能下降甚至损坏。然而，先进的冷却系统不仅会进一步增加能源消耗，还会大量消耗水资源。有研究估算，到 2027 年，全球 AI 数据中心的淡水年需求量最高可能达到 1.7 万亿加仑（约合 6.4 亿立方米，大致相当于两个阳澄湖的蓄水量）。这对先进的液冷技术和 AI 驱动的智能冷却算法，提出了更为迫切的要求。

网络带宽和计算资源的瓶颈同样不容忽视。实时的 AI 应用，如多模态模型和代理系统，需要超高带宽和低延迟传输，而现有网络基础设施往往难以满足这一需求。这可能引发网络拥堵，降低整体性能并阻碍企业级部署。升级 5G 乃至 6G 网

络、光纤骨干网，并转向分布式计算架构，将是有效的应对措施。当前的超节点技术（SuperPod），通过高速互联技术将大量计算芯片（如图形处理单元、神经网络处理单元等）紧密集成，形成一个高带宽、低延迟的规模化计算单元，从而支撑千亿乃至万亿参数大模型的训练与推理需求，为解决问题带来了新的希望。

如果说算力是 AI 的“引擎”，那么数据就是其“燃料”。AI，尤其是生成式 AI，在消耗海量数据的同时，也以前所未有的速度生成新的数据，这给数据存储和管理带来了双重挑战。AI 模型训练需要访问 PB 乃至 EB 级别的数据集，而模型推理和应用过程中又会产生海量的结构化和非结构化数据。这要求存储基础设施不仅要具备巨大的容量，还必须具备高速度、高可扩展性、高效率和高灵活性。传统的存储解决方案在面对这种快速增长的数据洪流时，往往会遭遇性能瓶颈，难以满足 AI 工作负载的需求。近期备受关注的超融合（Hyper-Converged Infrastructure, HCI）、软件定义存储（Software-Defined Storage, SDS）等技术，正是下一代信息技术基础设施架构的演进方向。

上述挑战将激发众多前沿的探索与研究。有理由相信，在这些领域，将会有源源不断的开源项目诞生并发展，这正是开源技术的发展趋势。

50.3 开源安全与治理成为“头等大事”

随着 AI 与开源的深度融合、互相促进，技术普惠的程度不断增强，这意味着不但善者的能力会被增强，作恶者的能力也会被增强；不但高手的开发效率会提升，新手的开发效率也会得到提升。这将使得开源安全与治理成为今后整个开源生态的关键挑战。在这种高压环境下，开源生态不得不从开发流程、供应链、模型层和生态平台 4 个技术方向集体升级。

（1）新的攻击面与复合型风险。

- 被 AI 增强的攻击行为：AI 也被用于生成高度逼真、难以识别的钓鱼邮件或深度伪造内容。这极大地降低了发动复杂攻击的技术门槛，并提高了攻击的成功率。
- 开源代码投毒更加容易：一方面是通过 AI 生成的代码越来越多，作恶者当然也可以通过 AI 辅助，高效生成用于投毒的代码；另一方面，新手在借助 AI 生成代码之后，很可能疏于检查，导致恶意代码轻松地乘虚而入。
- 模型投毒：攻击者直接在 Hugging Face 等开源模型社区发布含有后门的模型。这些模型在功能上可能与正常模型无异，但当用户下载并执行它们时，后门便被激活，可能导致数据泄露、远程代码执行甚至整个系统被接管。
- 数据投毒：AI 模型的行为高度依赖于其训练数据，攻击者通过向开源训练数据集中注入精心构造的“毒化”样本，可以操纵模型的学习过程，使其在特定场景下做出错误的，甚至是危险的预测。
- 随着开源使用的日益广泛，安全风险爆发时，修复的广度与难度也会显著增加。

（2）开发流程：从“事后补洞”到“安全内建”。

过去，很多开源项目“先造功能、再想安全”，安全检查多为发布前的少量人工审阅。Log4Shell 经验显示，开发与集成阶段缺乏系统化控制，风险会在应用中长期残留，这迫使开源世界将安全前移：在 CI/CD 流水线默认接入依赖扫描等检查，将“安全检查”固化为工具链的一部分；代码托管平台和开源基金会通过相关标准，让安全成为可审计的实践。AI 不仅制造新风险，也成为新的安全工具，大模型可辅助代码审阅等，专用模型能识别特定类型的自动生成代码。随着工具开源化、标准化，安全将进一步“内置”到常用框架和脚手架中，“安全内建”开发范式也将形成。

（3）供应链：从“黑箱依赖”到可观测的物料清单。

XZ Utils 后门让社区感受到供应链攻击的杀伤力：攻击者伪装成“热心贡献者”接管项目维护权，埋入后门污染发行版等行为，事后难以追查，暴露软件供应链结构性问题。因此，SBOM 等“透明化技术”被推到台前，通过 SBOM 列出软件组件等信息，配合工具查询漏洞并关联修复路径，OpenSSF 等组织则推动相关工具提供“供应链可视化”能力。接下来，开源供应链技术演进将集中在 3 件事：构建精细依赖关系图谱，结合 AI 标注漏洞路径和修复优先级；将“供应链信任基建”设为包管理器与镜像仓库默认能力；搭建开放平台共享漏洞等信息，让修复验证流程可追溯。

（4）模型层：在“开放权重”之外加一层治理外壳。

开源大模型使治理难题升级：权重公开后，任何人可绕开原厂安全策略，用其生成恶意代码等。与传统软件不同，单靠许可证难约束模型滥用。这促使开源技术在模型层探索“可控开放”治理工具。安全守卫模型可集成到开源 LLM 中，降低绕过风险，社区测试显示其在多语言基准上性能突出。漏洞扫描与审计工具能自动检测 LLM 风险，指导开发者识别特有风险。模型对齐与防护技术针对“行为能力”封装问题，让模型更鲁棒。

相比传统软件，开源 AI 治理更重视“集体智慧”，红队测试成为常态。2025 年，OWASP 更新使风险标准化。虽存在多语言覆盖和实时性挑战，但技术进步正让治理从被动变主动。可以预见，治理难题越突出，对治理技术需求越强，越能推动相关协议、工具和基础设施走向成熟。

（5）生态平台：从单点加固到共享治理底座。

越来越多开源安全紧急事件直击全球开源基础设施和云原生生态“软肋”，GitHub、Linux 发行版、Docker Hub 等平台紧急联动处理受影响制品，跨生态应急协同暴露出缺乏统一治理底座，单点加固不足的现实。

为此，治理难题倒逼新的“生态级基础设施”诞生，包括面向全社区的漏洞信息共享与响应平台、聚合多源 SBOM 和风险情报的开放服务、结合 AI 的攻击模式知识库与风险图谱，以及“开源安全中台”。这些平台大概率开源建设、多方共治，既服务全球，也支持本地化合规要求。

长远来看，AI 放大了开源生态安全与治理裂缝，也提供了解决问题的动力与工具。开发流程安全内建、供应链可观测化、模型层可控开放及生态平台协同治

理，将构成新一代“安全友好型”开源技术栈。开源世界难以回到“只谈技术不谈治理”的时代，但有机会在压力下构建更坚固、可持续的技术基础设施。

第 51 问 如何构建开放、包容、可持续的开源未来

51.1 开源生态正在遭遇哪些结构性挑战

在讨论这个问题之前，我们必须先看清当下开源生态正身处怎样的漩涡之中。今天的开源，已经不只是开发者之间的技术协作，而是被国家战略、资本逻辑、安全叙事和社会期待等多重力量拉扯的“数字公共基础设施”。它一方面被各国政府与大企业寄予厚望，承担起创新、竞争和安全的重任；另一方面又在资源短缺、治理失衡和社区疲惫中艰难维系。大体来看，这些压力可以分为外部与内部两重：外部是技术民族主义与监管商业化带来的撕扯，内部则是不平衡的激励机制和脆弱的社区可持续性。

（1）外部挑战：不同版本的技术民族主义。

从外部看，技术民族主义正在重写开源叙事。曾几何时，我们习惯把开源视为“全球开发者的共同语言”：你在旧金山写的代码，可以被上海的开发者复用，再被柏林的团队改进，最终合并回同一个上游仓库。技术争论也好，商业竞争也罢，至少在代码世界里还有一块超越国界的“中立地带”。而近年来，随着大国博弈升温，“安全”与“竞争”成为更加强势的关键词，开放开始被怀疑、被限定，甚至被反向利用。对于某些国家而言，“继续完全开放，会不会等于在无偿帮助对手缩短差距？”在这种担忧之下，开源不再是简单的数字公共产品，而是被当作可以“控制、收紧、选择性提供”的战略资源。

中国这边的语境则有所不同。中国在过去十年对开源的重视度快速提升，从政策文件到产业布局，都频繁出现“开源”与“自主可控”“科技自立自强”的绑定叙事。在被卡脖子的现实压力下，这种选择是可以理解的：通过大规模使用与参与开源，快速补齐基础软件和关键技术短板，以便在封锁中找到“第二条路”。但问题在于，如果开源被主要理解为“摆脱对别人依赖的工具”，而不是“与别人共同打磨的公共基础设施”，那么它在话语上就天然带上一层对抗色彩——“我要用中国特色的开源，来摆脱对你的依赖。”久而久之，各方对开源的信任感和共同体意识都会被侵蚀，全球协作空间被悄然压缩。

在这种大背景下，原本跨国、跨组织、跨意识形态合作的开源生态，正逐渐面临“阵营化”的危险。一边是担心技术外溢而收紧开放的国家 and 机构，另一边是希望通过开源实现技术替代与结构性追赶的国家和产业，中间夹着的是众多中小企业、独立开发者和非西方、非中心地区的开源社区。对于他们而言，贡献和受益的边界变得更加模糊：代码是否还能顺畅地跨境流通？某些平台和基金会的决策，是否会在某一天因为地缘政治转向而改变规则？开源的“全球性”，正在被一次次现实冲击拉出裂缝。

（2）外部挑战：资本与监管。

另外，资本与监管也在重塑开源的外部环境。随着开源项目越来越多地成为云计算、SaaS 平台和 AI 产品的基础，围绕“谁能从中获益”的问题，竞争开始变得激烈。越来越多的公司在寻找一种“既要开放生态、又要保护商业利益”的中间形态：有的通过所谓“开源核心+商业增强功能”的模式，把关键能力锁在闭源模块里；有的干脆发明各种“类开源协议”“源代码可见但商用受限”的条款，在名义上保持开放姿态，实质上却大幅收紧了自由使用和再分发的空间。对于普通用户来说，“究竟什么才算开源”“我还能不能安心将产品构建在这个项目之上”，变成一件越来越难判断的事。

监管合规规则带来了另一类隐性压力。软件供应链安全、隐私保护、数据跨境、AI 滥用防范……这些议题的出现并非坏事，但它们往往会抬高“真正开源”的门槛。大型科技公司可以用法务团队、合规流程来消化这些成本，而纯社区驱动的小项目却很难承受同等压力。对于一群志愿者维护的库来说，单单是“确保不踩雷”的心理负担，就足以让人犹豫是否要继续对外承诺长期维护。于是，在地缘政治、资本博弈与监管合规多重因素的挤压之下，开源在外部世界的处境，远没有“万物开源”的乐观叙事那样简单。

（3）内部挑战：开源世界的公地悲剧。

如果说外部挑战更多来自国家与企业层面的宏观力量，那么内部挑战则切实地压在每一个维护者和贡献者身上。最突出的问题是开源生态自身的不可持续性。我们早已习惯于把开源库当作“空气和水”——NPM、PyPI、Maven 中的开源组件触手可及，现代软件开发几乎无法离开这些依赖。但在这条“依赖链”的最上游，往往站着极少数维护者，甚至只有一两个人。他们用业余时间修复缺陷、回复问题、写文档、发版本，却面对指数级增长的用户和需求。这是一种典型的“公地悲剧”：所有人都从这片公共草场上获益，但几乎没有人愿意真正为维护草场付费或承担义务。

在开源文化中，我们长期强调“精神回报”：技术成长、声誉积累、社区归属感、对世界的影响力——汇总成一个词“礼物文化”。这些确实是真实而宝贵的动力源，但当开源项目一旦跨过某个规模阈值，维护工作就会变质，从“有趣的创作”变成“高压的运维”。缺乏物质回报、制度化的支持和分工机制，意味着个体维护者常常处于时间被透支、心力被消耗的状态。很多人不再是“心甘情愿地做贡献”，而是“因为愧疚和责任感，不好意思放手”。当愧疚感也不再足以支撑时，自然的选择就是悄然离开，让项目在无人察觉的角落慢慢老化、走向失修。

（4）内部挑战：结构失衡导致脆弱的社区可持续性。

治理结构的失衡，使这种不可持续性雪上加霜。许多开源项目从一开始就是“个人业余项目”，却在短时间内被卷入互联网基础设施的角色：一个最初为解决个人痛点写的小工具，可能在几年后被成千上万家公司部署在生产环境里。问题在于，治理结构往往并没有随之升级。没有清晰的贡献路径，没有透明的决策流程，也没有多维护者协同的机制，一旦核心作者因为生活变故、职业更迭或兴趣

转移而抽身，整个项目就会立刻暴露出 BUS 系数极低的风险。

当商业公司参与进来时，问题又变成另一种极端。越来越多的热门开源项目是“公司主导+社区参与”的模式：公司提供全职开发者和资源，社区提供反馈、生态和声量。从资源角度看，这是对传统纯社区模式的极大增强，但也带来了隐忧——项目路径到底是由公司战略驱动，还是由社区需求驱动？当公司因为业务调整而决定更换协议、收紧开放程度、削减投入时，所谓“社区治理”往往很难真正起到约束作用。社区可以做的，常常只是集体抗议或者选择分叉，代价极高，也难以在短时间内重建完整生态。

往深处剖析，内部的包容性问题也会反过来削弱开源的生命力。开源天生带有“开放包容”的价值标签，但在实际运作中，语言、文化、时区、资源等多重不平等一直存在。英语仍然是主导语言，这对大量非英语母语的开发者构成天然门槛；社区中流行的沟通风格、对“专业性”的认定方式，也可能无形中排斥那些表达方式不同、节奏不同的贡献者。资源差异更是难以忽视：有些人可以在工作时间内“带薪参与开源”，有些人则只能在深夜或周末挤出时间；在需要大量算力和云资源的 AI 领域，这种不平等被进一步放大——谁有资源训练模型、搭建基础设施，谁就更容易掌握话语权。

综合来看，当下开源生态所面临的挑战，是一种多重张力交织的状态：国际层面，它被夹在“全球数字公共产品”与“地缘政治工具”之间；产业层面，它被挤在“自由使用”与“可持续供给”之间；项目层面，它在“公司主导”与“社区自治”之间摇摆；实践层面，它不断挣扎于“名义开放”和“实质包容”的差距之中。如果我们想认真回答“如何构建开放、包容、可持续的开源未来”，就必须正视这些结构性矛盾，而不是仅仅停留在对“开源精神”的浪漫化赞美。下一步，问题不再是“是否要开放”，而是“在这样一个充满张力、矛盾与冲突的世界里，我们还能以何种方式继续开放”。

51.2 在撕裂与不平衡中重建开源的“多元共治”

如果说 51.1 节是对当下的开源生态进行一个“快速诊断”，那么接下来要回答的问题就是：在这样一个被地缘政治撕扯、被资本逻辑和技术变迁不断重塑的世界里，开源还能靠什么站稳脚跟？直觉上的答案，往往是“我们要更加相信开源精神”“大家多一点情怀、多做一点贡献”，但这恰恰是过去 20 年已经被证明不够的部分。今天的开源问题，不是维护者不够热爱开源，而是旧有的规则和分工，已经不足以支撑一个被各方严重依赖的全球数字基础设施。因此，真正需要重建的，是一套“多元共治”的新格局：不同角色各自承担起应有的责任，而不是继续把一切的期望和压力都压在少数维护者身上。

一、政府应该如何看待开源：把开源作为“数字公共基础设施”来对待

首先，国家和公共部门必须调整看待开源的方式。对很多政府来说，开源曾经只是“技术路线上的一个选项”，甚至是预算不足时的权宜之计；而新的环境下，它更接近于“数字时代的道路和自来水管道”。既然如此，就不能再把关键开

源项目当作维护者个人的兴趣爱好，而应该视作公共基础设施的一部分，纳入长期规划和财政工具箱。一方面，政府可以通过科研基金、公共项目、产业计划等方式，支持那些处于基础软件、底层库、关键开发工具位置上的开源项目，帮助它们弥补资金和人力的缺口；另一方面，在监管和安全审查中，要坚持风险分级，而不是一刀切地把“开源”和“不可控风险”画上等号。对于通用开发工具、基础库和教育类项目，可以在保障合规的前提下给予更多包容，让它们继续作为跨国合作的桥梁；对于确实涉及敏感用途的领域，则通过透明的规则、明确的边界来管理，而不是模糊地“全盘收紧”，以免误伤整片生态。

在中美竞争的现实背景下，国家层面的另一项重要任务，是为开源争取一块“仍然可以合作”的缓冲地带。从官方渠道看，这可能体现在科技对话中为部分标准、协议、基础库争取“中立区”地位；从非官方、第二轨道看，则可以通过城市间合作、大学之间的联合项目、跨国开源实践营、联合黑客松等形式，让不同国家的开发者和学生，仍然有机会在同一个代码仓库里对话与协作。对中国来说，把开源与“自主可控”绑定，是现实压力下的选择。以自主可控为底线保障，以共建全球公共基础设施为长远目标，在叙事上淡化零和博弈的对抗色彩，强化互利共赢的开发者生态属性，以此在紧绷的地缘政治夹缝中，为开源的开放性与中立性争取最大的战略回旋空间。

二、企业要承担更多的责任：从“开源消费者”升级为“开源投资者”

其次，企业——尤其是大型科技企业，必须从“开源消费者”升级为“开源投资者”。今天互联网上最重要的一批开源项目，大多已经深深嵌入各类商业产品和云服务的底座。没有这些项目，很多公司现有的业务形态几乎无从谈起。在这样的现实下，再把自己定位为“开源的使用者”已经说不过去：企业既是最大受益者，也是最有能力反哺的一方。所谓从消费者走向投资者，至少包括3个层面：其一，拿出真金白银支持关键项目，不仅仅是赞助大会挂个Logo，而是通过长期赞助、基金会会费、采购技术支持等方式，形成稳定的资金来源；其二，在公司内部制度上，为开发者“带薪参与上游”打开空间，把一部分工程师工时明确用于维护自己严重依赖的上游组件，而不是一味要求他们把改动“私有化”在内部仓库里；其三，在开源许可和品牌策略上保持克制，不以模糊概念包装“伪开源”，在合适时机把重要项目捐赠给中立基金会，让技术路线不完全绑在单一公司身上。这些动作短期看似增加了成本，但从长期看，是在为整个产业的安全、灵活性和谈判空间“交保护费”。

三、开源基金会：承担“公共管家”的角色

然而，仅靠国家和企业还不够。介于两者之间的，是各类开源基金会和中立机构，它们在多元共治格局中扮演着“公共管家”的角色。对很多项目来说，真正棘手的问题常常不是“缺代码”，而是“缺制度”：商标怎么注册和使用、域名和仓库归谁所有、治理委员会如何产生、重大路线分歧怎么处理、如何在多个国家之间兼顾法律和合规要求。对于这些问题，让一两个志愿维护者去摸索，既不现实也不公平。基金会可以提供一套通用的工具箱：标准化的治理章程模板、贡献者协议、代码之家的行为准则、商标管理和法律咨询服务，帮助项目在从“小作坊”走向“公

共基础设施”的过程中，有一套可借鉴的制度骨架。同时，跨国的基金会和联盟还能在一定程度上对冲地缘政治风险，让项目不至于完全被某一个国家的政策变化裹挟。对中国本土的开源机构而言，则可以一方面承接国内政策与产业的支持，另一方面主动融入国际基金会与社区，成为连接不同话语体系的“翻译器”和“缓冲带”。

四、开源贡献者：别做超级英雄

当然，多元共治的核心，仍然是那群写代码、合入提交、回复问题的维护者和社区。问题在于，在过去很长一段时间里，我们对维护者的期待是“超级英雄式”的：他们既要写最关键的代码，又要承担社区运营、冲突调解、路线规划，还要免费为各种企业提供“7×24 咨询服务”。这种期待本身就是不可持续的。真正的改变，应该从承认维护者的有限性开始——承认他们的时间和精力有限，承认兴趣会转移，承认“一个人扛全场”不是值得歌颂的英雄主义，而是一种需要避免的结构性风险。基于这样的前提，维护者就有理由，也有责任更早地“设计制度”：在项目尚处小而美时，就写清楚贡献路径和治理规则；在发现工作量开始失控时，主动寻求引入新维护者、与企业或基金会建立伙伴关系，而不是默默硬扛；在面对企业用户超出范围的要求时，敢于说“不”——把“社区支持”和“商业支持”区分开来，既保护自己，也维护整个开源文化的健康边界。

五、普通开发者：培养更好的日常习惯

最后，不能忽略普通开发者和终端用户。多元共治并不意味着只有“有权有势”的主体才有资格参与，相反，它需要大量“微贡献者”来填补缝隙。对于大多数开发者来说，也许没有时间和精力承担大规模的维护工作，但完全可以通过一些日常的习惯改变，为生态注入可持续的养分：遇到问题，提一个有复现步骤的问题，而不是在论坛上发一句“垃圾项目”；看到文档有错别字或不清晰的地方，顺手发一个合并请求；用中文或母语写几篇教程和经验总结，降低他人上手门槛；在真正依赖某个项目的情况下，通过小额赞助、购买周边或在社交媒体上推广的方式，给维护者一些实际支持。对于企业内部的普通工程师来说，把“能不能将这个贡献到上游社区？”当作一种职业习惯，而不是只在内部代码库里修修补补，本身就是一种责任感的体现。

51.3 在不完美的世界里，坚持一种“更好的开放”

我们讨论“如何构建开放、包容、可持续的开源未来”，本质上是在问：在一个不再单纯的世界里，我们能不能为开源设计出一套新的游戏规则，让它在被各种力量拉扯的同时，依然保留足够的公共性和创造力。

这套新规则至少包含 3 个维度。

其一，是对“开放”的重新理解。过去我们常常把开放等同于“尽可能多地公开源代码、尽可能少地设置门槛”。这种直觉在很长一段时间内确实推动了技术的爆炸式创新。但当开源被卷入地缘政治、被用于高风险场景、被嵌入复杂的商业博弈之后，“开放一切”不再总是最负责任的选择。我们需要从“抽象的开放”走向

“具体情境下的开放”：在基础协议、基础库、教育和科研工具等层面坚持最大化开放，在确实涉及重大风险的领域探索有条件开放、分阶段开放；在 AI 等新兴技术上，把“开放可验证性”和“开放可监督性”当作重要目标，而不是只盯着“代码是否全部公开”。换句话说，未来的开放不再是简单的“要不要”，而更像是“开放什么、开放到什么程度、在什么边界之内开放”。

其二，是对“包容”的重新落地。开源一直以“欢迎每一个人加入”为自我叙事，但现实中的语言壁垒、时间和资源的不平等、文化冲突和隐性歧视，让很多人始终停留在社区的外围。要让开源真正成为一种全球性的合作方式，而不只是少数地区、少数群体的“内部游戏”，就必须把包容从口号变成结构：在文档、流程和治理上为多语种、多文化背景的参与者留出空间，在评价贡献时把非代码工作视为一等公民，在冲突解决和日常沟通中对弱势群体保持额外的敏感。这并不是“附加的政治正确”，而是让开源在参与基础上保持活力的前提。没有足够多元的新血涌入，再漂亮的治理模型、再宏大的开放叙事，最终也会因为人力枯竭而崩塌。

其三，是对“可持续”的前置设计。可持续不是在项目快撑不住的时候再去“抢救”，而应该在一开始就被当作设计目标写进项目的 DNA：从一开始就诚实地讨论“钱从哪里来”“谁来承担长期维护”“如果某个关键人退出，项目会怎么办”；从一开始就设立贡献路径和治理结构，而不是等社区壮大之后再去补课；从一开始就把企业、基金会、公共资金等多种资源纳入考量，而不是一味假设“一切都可以靠志愿者撑住”。这听起来有点“扫兴”，因为它打破了我们对于“纯粹黑客文化”的浪漫想象，但只有承认维护者也是普通人，有自己的生活和极限，开源才能从一群人的燃烧变成一代代人的接力。

站在中国的语境里看，这个问题还有一层特殊性。一方面，中国在技术发展上确实面临“被卡脖子”的现实压力，用好开源、参与开源、贡献开源，的确是突破封锁、缩小差距的重要路径；另一方面，如果我们只把开源当作“绕过他人”的工具，而不去思考自己能为全球公共基础设施贡献什么、如何在多元共治格局中扮演积极角色，那最终收获的只会是一个更加分裂的技术世界——而在一个高度互联的数字世界里，任何一方都很难在彻底撕裂的格局中真正获益。把开源视作“全球公共品”，不是否认国家利益，而是承认：在互相高度依赖的基础设施层面，合作本身就是一种高级形式的安全。

“开放、包容、可持续的开源未来”听上去很宏大，但拆解开来，其实就是无数具体的人在各自位置上的一次次小小选择叠加的结果。在一个被撕裂和不平衡不断放大的世界里，也许确实无法保证开源永远保持理想中的纯粹，但我们仍然可以在具体的制度、实践和态度上，让它比今天“更好一点”：少一些伪开源和空洞叙事，多一些诚实的边界与承诺；少一些单向索取和理所当然，多一些自觉反哺和责任感。这样的“更好一点”，未必惊天动地，却很可能是通往未来的唯一现实路径。

第 52 问 开源创新的边界在哪里

近几年，“开源创新”这个词在政策文件、媒体报道和产业语境中频繁出现。它既被视为数字经济时代的公共底座，又被寄望成为突围“卡脖子”困境的关键抓手。一些讨论甚至在不自觉间，把“开源”理解成一种几乎万能的创新范式：只要开放源代码、发动社区，就能自动汇聚全球智慧，产生颠覆式突破。

然而，如果我们冷静一点，就会发现一个根本问题：开源创新到底只是创新的一种表现形式，还是一种能够“包打天下”的新型创新模式？它的边界究竟在哪里？

要回答这个问题，首先要把开源创新放回整个创新体系的坐标系之中，重新理解科研机构、企业和开源社区之间的分工与互补；然后再从国家、产业和个人的不同视角，审视开源创新真正能做什么、不能做什么。

52.1 创新的多重场域与开源的真实位置

传统上，人们容易从“发生场所”的角度来区分创新：高校与科研机构负责“科学创新”，企业负责“商业创新”，社会组织和个人则零星做一些“边缘创新”。在这样的划分下，“开源创新”常常被自然而然地当作第三块，与高校和企业并列，好像它是某个独立的空间。

但如果仔细观察现实世界就会发现：许多具有巨大影响力的开源项目，根本无法被简单归入某一个场域。一个项目可能最初诞生于高校实验室，随后由企业投入资源进行工程化，又在社区里汇聚全球贡献，最终交给基金会托管。换句话说，开源并不是一个“发生在某个地方”的概念，而是一种跨越场域的组织 and 传播方式。

高校可以闭门做研究，也可以开源代码，邀请更多单位一起协作；企业可以完全内部研发，也可以通过开源方式构建生态；社区可以自发发起项目，也可以承接来自高校与企业的成果。真正的创新网络，是“科研—企业—社区—政府”多方交织的结构。在这个结构之上，开源是一种让知识、代码和能力流动起来的机制，而不是一个与其他模式相互排斥的板块。

因此，当我们讨论“开源创新”时，如果先把它从“第三种创新来源”的误解中抽离出来，承认它只是创新的一种组织方式，而不是一个自足的场域，那么后续关于它的优势与边界的讨论，才会有坚实的起点。

52.2 科研、企业与开源的分工与互补

要理解开源创新的特点，必须先看清科研创新、企业创新与开源创新之间的分工与互补关系。

高校与科研机构的创新，主要面向“认知边界”和“知识空白”。它们关注的是那些“如果不问出来，甚至不知道可以被提问”的问题：某种现象背后的机理是

什么？某个领域是否存在新的理论框架？新的算法是否在数学上成立、在实验中可被观测？这类创新的成果往往以论文、理论模型、原型系统的形式呈现，评价体系以学术指标为主，周期长、风险高、回报具有不确定性。

企业创新则处在截然不同的逻辑下。它必须在竞争压力和盈利需求之间寻求平衡，它关心的是“用户是否愿意付费”“是否能支撑业务增长”，时间被拆分为季度、年度，项目要面对明确的绩效和资源考核。企业的优势在于资源集中、目标明确、反馈快速，它不一定做“世界第一次被提出”的东西，但擅长把某件事做得更稳定、更便宜、更好用，从而在市场中站稳脚跟。

开源创新在这个结构中的位置有些特殊。它往往不首先承担“提出全新理论”的任务，也不直接追求短期财务回报，而是在另一个维度释放力量：把零散、试验性的东西打磨成可以复用的公共能力，把“重复造轮子”的领域公共化为基础设施，覆盖那些有需求却未必有商业回报的长尾场景。许多看似“不先进”的小功能、小工具、小脚本，正是这些开源创新的典型产物：它们靠近日常工作，填补了系统与系统之间的空隙，构成了数字基础设施中无数不起眼却不可或缺的螺丝钉。

从这个角度看，科研更倾向“从不可思到可思”，企业更倾向“从可思到可用、可卖”，开源则倾向“从可用到公共可用、广泛可用”。三者并不是互相替代，而是一种层层接力、不同侧重的分工与互补。

52.3 开放协作与有组织创新的张力

围绕开源创新，一种常见的浪漫想象是：只要把代码公开，世界各地的开发者就会自发聚集，形成某种无边界的“蜂群智能”，创新就会自然涌现。这种叙事强调的是开放、自由、平等与去中心化，似乎开源的松散自由，就是迈向创新成功的保证。

现实情况恰恰相反。那些真正成功、具有长期生命力的开源项目，几乎无一例外都有非常清晰的组织结构：有核心维护者团队，有代码贡献和评审流程，有明确的路线图和发布节奏，有负责决策与协调的技术委员会，甚至还有专门的基金会负责治理和资源管理。换言之，开源的开放并不意味着没有组织，而是意味着在保持参与边界开放的前提下，建立起一套透明而稳定的协作机制。

开放协作非常适合这样一些任务类型：系统可以被拆分为模块，工作可以切成一个个清晰的问题和合并请求；可以通过测试和 CI/CD 自动验证代码质量；使用者与开发者高度重合，大家在使用中遇到问题，会反过来修复和改进。反之，当某个领域高度保密、风险极高，或需要重资产和强项目管理来推动，纯社区式的自发协作就会显得力不从心。

于是，我们需要修正一个常见的误解：开源协作不是用来替代“有组织创新”的，更多是补充、重塑和延展。一个健康的创新体系，往往是这样一种结构：核心方向和重大工程由有组织的科研机构、企业和国家队承担，而大量基础组件、工具链、长尾功能和生态扩展，则通过开源协作的方式来完成。真正成熟的开源创新图景，是“开放边界+内核清晰”，而不是“人人一票+完全无序”。

52.4 国家视角下的开源创新：期待与纠偏

在国家层面，支持开源和开源创新已经成为一项普遍共识。其合理性主要体现在以下几个方面。

第一，开源是构建数字基础设施的一条重要路径。在操作系统、中间件、数据库、编译器、AI 框架等关键技术领域，如果能够形成一套可控、可持续、由国内外多方共同维护的开源底座，就能够显著降低对单一封闭供应商的依赖，增强技术安全性与自主性。

第二，开源也是一种非常有效的人才培养与能力提升机制。传统教育往往在真实工程实践方面有所欠缺，而开源项目恰好提供了这部分“缺失场”：学生和工程师可以在真实的代码库与社区协作中锻炼能力，接触到工业级的工程实践，建立起与国际同行沟通的桥梁。

第三，开源是一种重要的国际合作方式。在一些可以合作的技术领域，通过参与或主导开源项目，可以对相关技术标准、接口规范、生态方向产生影响，成为全球规则制定的参与者，而不是被动的执行者。

但与此同时，也逐渐出现了一种需要被温和纠正的期待：似乎只要大力支持开源，便能自然突破所有“卡脖子”技术甚至反超封锁方。这样的想象忽视了很多关键现实：真正的“卡脖子”往往是知识、工艺、设备、资本、供应链等多个要素综合作用的结果；开源在信息和工具层面确实可以帮助缩短起步时间、降低试错成本，但不能替代长期、大规模、系统性的投入。

因此，从国家视角看，更成熟的策略并不是把开源创新抬升到凌驾于整个创新体系之上的位置，而是将其嵌入整个国家创新体系，作为科研、产业和人才培养之间的重要连通器和放大器。在这一前提下，我们既不会对开源寄予不切实际的“万能期待”，也不会低估它在长期竞争中的基础性价值。

52.5 开源创新的几条关键边界

当我们问“开源创新的边界在哪里”时，并不意味着要给出一个抽象的理论结论，反而是指出几条相对清晰、可感知的边界线。

第一条边界来自安全与合规。涉及国家安全、军工、金融核心、重要个人数据等领域，有大量信息天然不可能开放，即便采用类似开源的协作方式，也往往只能在单一组织或有限伙伴之间进行“内源化”的协作。法律和安全红线本身，构成了开源创新在这些领域活动空间的上限。

第二条边界来自资本与重资产。对于先进制程、重大装备、大型物理设施等需要巨额前期投资和长期工艺积累的领域，社区自发协作难以承担起“从零到一”的主体角色。开源在其中可以发挥重要的辅助作用，比如提供仿真工具、数据处理框架、算法开源实现等，但最终能否突破瓶颈，更大程度上依赖实体制造体系、资本投入和产业链组织。

第三条边界与系统复杂度和模块化程度有关。开源非常擅长处理可拆分、可测试、可复用的模块，但当某个系统高度耦合、必须在特定时间节点整体交付、对单点失误极为敏感时，传统的强组织推动和责任体系仍然不可或缺。此时，开源更多扮演的是“为整体系统提供若干开源组件”的角色，而非“整体方案完全交给社区自发完成”的模式。

这些边界并不是为了低估开源，而是为了提醒我们：开源创新需要与现实的约束条件对接，它不是悬浮于现实之上的理想结构，必须在安全、资本与系统复杂度的多重约束之下，找到自己的位置。

第 6 篇 政府与开源政策

本篇围绕“政府与开源政策”的核心命题，采取由内而外、由治理到外溢的论述结构，力求回答一个更具实践指向的问题：政府应当如何通过制度供给与政策工具，动员并组织多元主体，共同建设可持续的开源生态能力，并将这种能力进一步转化为城市竞争力与国家的国际影响力。

具体而言，首先，第 53 问至第 55 问从政策分析的基本框架出发，阐明政府介入开源的时代背景与必要性，明确政府在开源生态中的多重角色定位，并指出制定开源政策时需要避免的典型误区，从而为本篇后续讨论奠定认知与方法论基础。随后，第 56 问至第 58 问转入国内协同治理层面，分别讨论政府如何与企业、开源基金会和教育机构形成制度化合作关系：既通过产业政策与采购机制引导企业从“使用开源”走向“建设生态”，也通过中立平台实现多方共治，避免社区行政化，同时以开源为抓手推进人才培养与科研范式更新，夯实开源生态的长期供给能力。

在此基础上，第 59 问将视角提升到城市尺度，探讨城市如何通过产业机会、公共数字基础设施、开放数据与国际化营商环境，吸引并留住全球开源人才，使城市成为开发者社群的集聚地与创新的发生地。最后，第 60 问进一步面向国际维度，讨论如何将开源作为一种低摩擦、可持续、可复制的国际科技合作机制，服务科技外交与公共外交目标，在风险可控的前提下推动跨国协作、参与数字公共产品输出。通过这一递进结构，本篇试图呈现从“政策设计与治理能力”到“生态协同与制度化合作”，再到“城市与国家层面的外向型能力建设”的完整链条。

第 53 问 政府如何在开源项目中发挥积极作用

53.1 政府介入开源的现实背景与必要性

从全球范围看，开源已经从一种开发模式演变为数字经济时代的基础设施。云计算、大数据、AI、物联网、5G 等关键技术体系，都在不同程度上依赖开源软件和开源社区的持续演进。各国政府越来越意识到：如果在开源领域缺位，就可能在新一轮科技革命和产业竞争中丧失主动权。

对中国而言，政府介入开源的时代背景至少包括 3 层。

(1) 数字经济转型的系统需求。“十四五”规划将“建设数字中国”“发展数字经济”放在前所未有的重要位置，国家层面先后出台《“十四五”软件和信息技术服务业发展规划》《“十四五”数字经济发展规划》等文件，明确提出要发展开源社

区、开源平台和开源项目，把开源视为推动产业升级和数字基础设施建设的关键抓手。

(2) 关键核心技术“卡脖子”的外部压力。中美科技摩擦升级、出口管制趋严，对国外封闭技术路径的高度依赖被视为重大风险。官方文件和政策研究中，越来越多地把“开源算法”“开源社区”“开源生态”写入国家科技和产业发展规划，希望通过参与和建设开源生态，缓冲地缘政治带来的技术封锁压力，增强技术自主可控能力。

(3) 国内开源供给与治理的结构性不足。一方面，根据工业和信息化部体系内智库的调研，2019 年中国约 87% 的企业已经在使用开源软件，但只有约四分之一的企业具备相对规范的开源治理体系，在合规、安全、运维等方面存在明显短板；另一方面，基础软件、底层架构、长期投入的“冷门但关键”项目，往往商业回报不明确、研发周期长，仅靠市场自发与社区自组织难以持续投入。这些领域本身就具有“公共物品”属性，更需要公共部门在资源、规则与协同层面适度介入。

因此，政府参与开源，不再是“要不要”的问题，而是“如何参与、参与到什么程度”的问题。既要看到政府在资源配置、制度供给、跨部门协调上的独特能力，也要看到市场与社区机制在创新活力、开发者动员上的不可替代性。下面将在这样的时代背景下，讨论政府在开源中的角色、工具与边界。

53.2 政府在开源中的多重角色定位

在传统产业政策语境中，政府更多被视为“规则制定者+资金支持者”。但在开源生态中，政府的角色明显更加多元化。

(1) 政策与规则制定者。中央层面通过国家规划、产业政策和技术标准，将开源纳入国家科技和产业发展战略，明确鼓励建设开源社区和开源平台。在法律法规和监管规则中，逐步明确开源许可证的法律地位、合规边界和风险防范要求，为企业、高校、科研机构“放心用开源”提供制度支撑。

(2) 需求提出者与首批用户。政府是数字基础设施和行业应用的重要需求方，政务云、公共数据平台、行业监管系统等项目，本身就可以成为开源软件的“首批用户”和“早期采用者”。欧盟委员会在其 2020~2023 年开源软件战略中，就明确通过内部 IT 系统采用开源软件、建设代码仓库和开源项目办公室（OSPO），把自己定位为“开放政府”的示范用户。美国联邦政府的 M—16—21 文件则要求新开发的联邦源码在一定比例上以开源形式发布，联邦机构既是用户，也是贡献者。

(3) 资金与资源提供者。政府通过科研经费、产业基金、重大专项，为基础软件、关键共性技术和长期开源项目提供“耐心资本”。在算力、数据、测试环境等方面，政府可以建设公共平台，降低社会主体参与开源研发与验证的门槛。

(4) 协调与调节者。开源生态天然跨企业、跨行业、跨地区，往往会涉及知识产权、标准、产业竞争、跨境数据流动等复杂问题，需要有能力在更高层面进行协调。政府可以通过牵头成立开源联盟、产业联盟、标准化组织或通过开源基金会等

平台，引导多方共商治理规则。中国在云计算、AI 开源技术、RISC-V 等领域建立的多个开源联盟与基金会，就是这样的实践。

（5）不同层级政府的分工与侧重。中央政府更侧重顶层设计与方向性引导，在国家战略、法律体系、重大专项等层面把握大局。

地方政府更多承担试点示范与配套服务，通过园区政策、地方基金、应用场景开放等方式，支持本地企业、高校和开源社区。

行业主管部门则在特定行业（金融、能源、交通等）内制定开源应用的合规要求、安全标准和示范项目，实现“行业深耕”。

与传统产业政策相比，政府在开源生态中还多了一层“社群治理参与者”与“数字基础设施建设者”的角色：不仅要“发政策、给资金”，还要理解社区运行逻辑，尊重开发者文化，通过中立平台与透明规则，塑造健康的生态环境。

53.3 支持开源项目的关键方法与工具

在具体实践中，政府支持开源项目可以归纳为 3 类，分别是直接工具、间接工具、制度与合规支持。

（1）直接工具：提供财政与项目支持。

科研与产业项目中嵌入开源要求。在基础软件、操作系统、数据库、中间件等项目的立项与评审中，将“开源社区建设”“代码开放程度”“国际合作贡献”等纳入考核指标，鼓励项目以开源方式组织研发。

专项资金与重点工程。一些国家通过国家级开源平台建设、重点开源项目扶持、共性技术研发专项等形式，直接投入资源。例如，中国在软件和信息技术服务业“十四五”规划中提出，要“扩展开源生态”，并通过支持本土开源托管平台、开源操作系统、开源芯片架构等项目，形成了较为系统的政策组合。

（2）间接工具：采购政策与财政激励。

“开源优先、开放标准优先”的采购规则。政府可以在不排斥商业软件的前提下，对符合安全和质量要求的开源方案给予适度加分或优先考虑，推动开源进入政务云、行业云和基础设施项目。欧盟开源战略提出，要通过内部采购和技术治理，把开源软件视作实现数字主权与互操作性的关键工具。

税收优惠、创新券与产业基金。对开展开源研发、建设开源社区、提供开源服务的企业和机构，通过研发费用加计扣除、创新券报销、产业基金投资等方式，降低其投入成本，激励其在商业模式中纳入开源元素。

（3）制度与合规支持：给生态“铺路”。

明确开源许可证的法律地位与适用边界。通过司法解释、行政规章或标准指南，回应企业在使用 GPL、Apache、MIT 等许可证时，对责任划分、衍生作品界定、专利条款等方面产生的疑虑，降低使用开源的合规成本。

建设开源治理规范与评估体系。例如，鼓励企业和高校建立开源合规流程、组件清单、安全漏洞响应机制等，政府相关部门或行业协会提供参考模型与评估标准。

示范案例与标杆工程。以政务云操作系统、国有企业服务器平台等为载体，选择成熟的国产开源系统（如某些开源服务器操作系统、数据库等）进行规模部署，通过示范效应带动产业链上下游采用开源方案。有研究显示，部分开源操作系统已经在能源、电信、金融等国有企业中广泛部署，用作云基础设施与关键业务支撑，这类实践实际上发挥了“公共部门用户带动产业”的放大效应。

需要强调的是，政府的“支持”更多应体现在“创造条件”而非“亲自下场做甲方爸爸”：重点在于制定公平透明的规则、提供基础性公共资源、营造有利于开源发展的制度环境，而不是用行政命令替代技术和市场选择。

53.4 “积极”与“克制”的边界

政府在开源中“积极作为”是必要的，但“用力过猛”则可能适得其反。把握好“积极”与“克制”的边界，是设计政策和参与治理时必须时刻警惕的问题。

（1）常见“越位”风险。

过度指定“唯一项目/唯一技术”。以行政方式“点名”某个开源项目或技术路线为“国家标准”“唯一推荐方案”，容易压制其他创新路径，削弱竞争活力，甚至诱发“行政垄断式开源”。

将开源社区行政化。如果以科层逻辑去管理社区，把开发者当作“被管理对象”，以行政考核、会议审批、层级报批来替代社区自治与技术讨论，很容易挫伤贡献者积极性，使项目名为“开源”，实为“官办工程”。

指标化、运动式推动开源。把开源贡献简单 KPI 化，例如，要求某一时期必须“新增若干个开源项目”“贡献若干 PR 数量”，或搞一阵风式的“开源改造运动”，往往会催生形式主义，导致项目仓库空心化、代码质量低下、社区无人维护。

（2）“积极”的内涵：不缺位。

在关键基础软件和公共数字基础设施上，政府要敢于通过中长期规划和公共投资“补短板”；在规则和制度层面，尽快回应产业界对许可证、合规、安全等方面的现实需求；在场景与需求侧，为本土开源项目提供真实可用的应用环境，形成良性反馈。

（3）“克制”的内涵：不错位、不越位。

尊重技术社区的自治逻辑，避免用行政命令干预技术路线、项目治理结构、维护者选举等具体事务；避免把开源简单视作“完全可控”的工具，更不能要求开源项目在短期内完成所有政策目标（如安全、自主可控、产业升级、国际话语权等），以免给社区施加过度政治化压力；在短期政绩与长期生态之间保持平衡，避免以短期指标牺牲社区的可持续治理。

（4）通过中立平台间接发挥作用。

在很多国家和地区，政府更倾向于通过开源基金会、行业协会、标准化组织等中立平台来发挥作用，而不是直接成为项目的“管理者”。

欧盟委员会在推进开源战略时，设立开源项目办公室，重点进行协调、赋能和

对接社区工作，而非直接管理具体项目代码。

中国则通过设立开源基金会和多个产业联盟，将企业、高校、科研机构政府部门汇聚在同一平台上，形成“多元共治”的基本格局。

开源生态需要的是“多元主体、共享治理”的结构：政府是关键一方，但不应也不可能成为唯一中心。

第 54 问 如何制定切实可行的开源公共政策

54.1 开源公共政策的内涵与定位

从概念层面看，所谓“开源公共政策”，并不是指某一类孤立存在的政策文件，而是指围绕开源软件、开源硬件、开放数据等新型数字公共资源，政府为实现特定公共目标而作出的制度性安排与政策组合。它既包括直接面向开源社区、项目与平台的政策，也包括嵌入产业、教育、科技、数字政府等领域的具体条款与实施方案。

需要特别强调的是，开源本身并不是一个狭义的“产业门类”。在“十四五”规划以及多地数字经济发展规划中，开源更多是与软件业、数字经济、信息基础设施等统筹部署，被视为数字经济发展的主要支撑和“生态要素”，而不是一个可以简单切割出来，单独统计产值和增加值的独立行业。

如果我们把“开源产业”理解为一组可以直接统计的软件企业、服务机构和平台运营主体，那么可以计算其营业收入、就业规模等指标；但这只是开源价值的一小部分。更大的价值体现在：它作为数字公共产品，为各行各业节约了大量研发与试错成本；作为技术扩散机制，加速了新技术在产业链上下游的传播；作为协作范式与知识公地，改变了创新的组织方式与知识生产模式。

因此，合理的开源公共政策，应当将开源视为“所有产业的催化剂”，而非一个可以简单纳入产业分类体系的板块。在政策体系中，也应当更多采用“嵌入式设计”——在数字经济、软件和信息技术服务业、科技创新、教育、工业互联网、政务信息化等政策中，将开源作为重要手段与制度选项加以考虑，而不是孤立出台几份“开源专项文件”就算完成任务。

54.2 制定开源公共政策的目标是什么

在设计任何公共政策之前，首先要回答的都是同一个问题：“为什么要做？”对开源公共政策而言，这个问题尤为关键。我们是为了“节约软件采购成本”而选择开源，还是为了在新一轮科技革命中形成新的创新范式？我们是把开源当成“短期缓解卡脖子压力的工具”，还是当作“长期建设数字公共基础设施的路径”？我们是只关心“本地国产替代”的目标，还是希望通过开源参与并塑造全球技术生

态？从现实出发，开源公共政策至少承载了以下几个层次的目标。

技术自主与安全可靠。在关键基础软件、基础架构和共性技术领域，通过参与和构建开源项目，降低对单一封闭供应商或专有技术路线的依赖。与《“十四五”软件和信息技术服务业发展规划》中“加快繁荣开源生态，夯实产业发展基础”的要求相衔接。

产业升级与竞争力提升。通过开源提升企业对前沿技术的获取能力，缩短新技术从实验室到产业化的周期。促进产业链上下游的协同创新，形成更具韧性和开放性的产业生态。

人才培养与教育创新。将开源项目作为实践教学平台，使学生和工程师在真实项目与社区中学习，而不仅仅依赖教材和封闭实验环境。推动形成“数字素养—开源基础—项目实训—开源实战”的人才培养路径，为数字经济提供更具协作能力的人才供给。

国际合作与技术影响力。在地缘政治环境更加复杂的背景下，通过开源项目与全球开发者和企业保持技术交流与规则对话。对外输出国产开源技术与平台，通过“开源共建”帮助其他国家提升数字能力，进而在全球建立更广泛的技术合作网络。

在这些目标中，有些是短期可见的政绩目标——例如，某个阶段内开源项目数量的增长、开源替代率的提升、某些自主产品的规模部署；但更多目标具有明显的长期和生态性特征——例如，数字公共产品的积累速度、社区治理能力的提升、国际技术话语权的形成。

开源公共政策若过度追求“短期可量化的数字”，就很容易滑向“补贴驱动”“KPI 导向”的路径，忽视长期生态建设这一核心价值。因此，在明确政策目标时，应当有意识地区分：哪些指标可以作为阶段性考核工具，哪些目标更适合作为中长期方向性约束，避免用短期政绩逻辑去替代生态建设逻辑。

54.3 制定开源政策的基本原则

在明确了“为什么要做”之后，才能进一步讨论“应该如何做”。好的开源公共政策不仅要目标清晰，还应在原则层面形成相对稳定的“底线和方向感”。结合前文关于开源文化、开源治理的讨论，笔者认为，至少有以下 4 条原则值得强调。

（1）开放与中立：避免技术和厂商锁定。

开源政策应当在技术路径、厂商选择和项目推荐方面尽可能保持中立。不宜轻率“指定唯一国民项目、唯一国民技术路线”，以免在行政力量的推动下，压制其他潜在更优方案；政策可以鼓励形成若干具有国际影响力的开源社区和平台，但应避免把这些平台“绝对化”为唯一标准。

（2）鼓励创新与竞争：开源不是“统一标准”，而是“多元探索”。

开源的价值之一在于降低创新门槛、鼓励多样化尝试。政策设计不应过早锁定技术路线，更不应通过行政手段排斥替代性方案，而应鼓励在兼容开放标准的前提下，允许多种开源实现方式展开竞争。

(3) 兼顾安全与开放：在风险可控前提下保持外向性。

自主可控与开放协作并不矛盾。开源政策一方面要重视供应链安全、数据安全和关键信息基础设施安全，另一方面也要鼓励对国际成熟项目、社区的参与和贡献。结合“安全可控、开放合作”的既有政策表述，在安全边界明确的前提下，推动更多项目实现“立足本国、面向全球”。

(4) 尊重社区规律：顺应而非改造开发者文化。

开源社区具有高度自组织特性，治理依赖共识、声誉和协作，而非行政层级和命令。如果政策试图用科层化逻辑直接改造社区治理，那么很可能削弱开源的活力。因此，政策应更多为社区提供基础设施、制度保障和对接渠道，而不是将社区“编制化”“项目化”。

这些原则，与前文关于“开源是数字公共产品”“开源社区是多元主体共治”的论述形成呼应，也为后续具体工具选择提供了价值坐标。

54.4 开源政策的主要工具箱

在具体工具层面，可以借用政策研究中常用的四分法，将开源公共政策的工具划分为法规工具、经济工具、行政工具和信息工具。在此基础上，再结合开源的具体需求进行细化。

(1) 法规工具：为开源活动划定“合法边界”。

许可证与知识产权规则。通过法律法规、司法解释或指导性文件，明确各类开源许可证的法律效力、衍生作品界定、专利授权条款等，减少企业和机构采用开源时的法律不确定性。

数据开放与隐私保护制度。在推进开放数据、数据要素市场化配置过程中，为基于开源工具的数据处理与共享提供制度依据，同时明确隐私、国家安全等红线要求。

(2) 经济工具：通过资源配置形成激励。

财政补贴、奖励与基金。对承担基础性开源项目、构建关键开源平台、提供开源治理与安全服务的主体，给予适当财政支持或引导基金投资，但应重点支持“数字公共产品”和“公共服务能力”，避免简单以代码仓库数量、项目数量为补贴依据。

税收优惠与创新券。对从事开源研发、为开源社区提供服务的企业，给予研发费用加计扣除等政策支持，或通过创新券的方式补贴其使用公共算力、测试平台等资源的费用。

(3) 行政工具：规划、试点与评估。

发展规划与专项行动。在国家 and 地方的“十四五”及后续规划中，将“繁荣开源生态”“打造开源创新高地”等纳入发展目标，明确重点方向和区域布局。

试点示范工程。通过“开源示范城市”“开源示范行业”“开源治理试点企业”等方式，探索不同场景下的政策组合与治理模式。

评估与监管。建立针对开源应用与开源治理的评估标准，将安全合规要求纳入

行业监管体系，避免过度监管抑制创新。

（4）信息工具：知识传播与经验扩散。

白皮书、指南与最佳实践。通过发布开源发展报告、开源治理指南、供应链安全白皮书等，帮助企业、高校和公共部门理解并掌握开源实践经验。

公共数据与公共平台开放。对政府掌握的部分数据资源、基础软件模块、通用工具以开源形式发布，形成“官方开源项目库”与“公共 API”，增强社会创新能力。

从“政策目标—常用工具—适用场景”的角度做一个简要的概括，如表 54-1 所示。

表 54-1 常用政策工具与适用场景

政策目标	常用工具	适用场景
提升技术安全与自主能力	以法规工具和行政工具为主，辅以经济工具	通过规划明确基础软件开源方向，配合专项资金支持关键项目
推动产业升级与企业创新	以经济工具和行政工具为主	产业基金投资、税收优惠、开源示范工程等
扩大数字公共产品供给与社会使用水平	以信息工具和法规工具为主	通过开放数据、开源公共组件、发布治理指南等，引导各类主体广泛采用开源
培养人才与推动教育科研变革	行政工具（教改、学科建设）+信息工具（课程资源、开源教材）	将开源实践纳入教育评价与科研评价体系

在实际政策设计中，上述工具往往需要综合运用，而非单一依赖补贴或奖励。单一的资金刺激很容易造成寻租与投机，只有与法规、规划、评估等手段结合，才能真正形成对开源生态的长期支持。

54.5 自上而下与自下而上的结合

开源天然带有“自下而上”的社区属性，而国家治理体系则具有明显的“自上而下”特点。如何在两者之间实现有机结合，是制定和实施开源公共政策时必须正视的问题。

（1）自上而下：国家层面的战略与顶层设计。

在国家发展规划、数字经济战略、软件和信息技术服务业规划中，明确了开源在技术创新、产业发展和数字治理中的地位与作用。在法律法规层面，应逐步完善开源相关制度，明确知识产权、数据安全、责任承担等问题，为各级政府和社会主体提供统一的规则框架。

（2）自下而上：地方试点、社区实验与行业示范。

地方政府可以结合自身产业基础与人才结构，选择若干重点领域（如智能制造、金融科技、城市治理等）进行开源应用和治理试点。鼓励企业、高校、开源社

区在真实场景中试验新的协作模式和治理机制，让政策在实践中“长出来”，而非完全停留在纸面设计。

(3) 应遵循“试点—评估—推广”的政策迭代路径。

与互联网和科技创新政策类似，开源政策更适合采取“先试点、后推广”的渐进式策略：先在若干城市、园区和行业建立“开源政策试验区”“开源示范城市/行业”；对试点中采取的政策工具、治理模式进行系统评估，识别有效经验与潜在风险；再据此在更大范围内推广，或者进行有针对性的修订。

(4) 构建纵向联动与横向协同机制。

纵向上，通过国家—省级—地市多级联动，形成开源政策目标与工具的分工与衔接：中央重在战略和规则，地方重在实施与服务。横向上，通过开源基金会、产业联盟、教育联盟等平台，把企业、高校、社区和政府部门纳入统一协商框架，推动信息在不同主体之间顺畅流动。

总体而言，切实可行的开源公共政策，既需要自上而下的战略指引和制度供给，也离不开自下而上的实践探索与反馈机制。只有在“顶层设计—基层试点—评估调整”的循环中不断迭代，才能避免政策僵化与一刀切，真正发挥开源在数字经济时代的基础性作用。

第 55 问 制定开源政策时需要避免哪些典型误区

在开源相关政策迅速“遍地开花”的今天，一个不容回避的问题是：很多政策的问题不在于“工具不够多”，而在于“底层想象是错的”。如果政策制定者对开源生态的结构、创新机理、供应链韧性、数字公共产品的特性缺乏真实理解，就很容易在错误的比喻和想象之上“搭楼”，导致方向有偏、执行有害。本文从 5 个方面讨论制定开源政策时常见的典型误区，并尝试从制度逻辑上加以澄清。

55.1 基于错误“供应链想象”的开源断供恐慌

第一类误区，是把开源生态想象成一条线性供应链。仿佛只要某个关键环节被“掐断”，整个开源世界就会瞬间崩塌。由此导出的政策想象是：只要境外某个托管平台“断供”、某个国家议员提案限制，就会导致本国使用开源软件的许可被全面收回。

这种想象至少存在 4 层偏差。

第一，把开源软件的技术传播与许可体系，类比为传统货物贸易或封闭软件的供应链。事实上，开源项目的代码分布在全球无数镜像、分叉、复制与二次分发渠道上，许可本身具有高度“可复制性”，很难通过单点封锁实现“全面断供”。迄今为止，即便在地缘政治冲突最激烈的时期，也没有出现对某一国家实施“整体开源断供”的先例，俄罗斯的情况就是一个生动的反证。

第二，过度夸大单一托管平台的战略地位。开源代码托管平台固然重要，但它

更像是“协作基础设施”和“社区集散地”，而不是可以一键切断的“唯一水龙头”。把 GitHub、Gitee 等平台视作“掐死开源”的关键阀门，既夸大了平台权力，也忽略了开源生态天然的冗余性与自我修复能力。

第三，混淆了“商业软件断供”与“开源供应链断供”的性质。封闭软件的授权可以单方面撤回或不续约，而开源许可一旦合法生效，撤回和限制的空间要小得多。将两者混为一谈，会在舆论和政策层面制造不必要的恐慌与误判。

第四，混淆了“代码资产”与“人力资源”。在 GitHub 与 Hugging Face 这样的平台上，最重要的资产并非代码或模型，而是那些活跃的社区成员。将代码复制到自己的平台上，并不会因此吸引那些开发者一起迁移过来。

基于这种错误供应链想象，一些地区出现了一哄而上的平台工程：纷纷上马本地开源托管平台，将有限资源投入到重复建设上。同时在“防断供”的名义下，强化简单化的“国产替代”叙事与技术民族主义情绪。从政策角度看，更合理的做法，是在充分认识开源生态分布式特性的基础上，提升自身在生态中的“节点能力”与贡献度，而不是试图通过复制平台、封闭生态来“防断供”。

55.2 错误的科技创新想象：把开源当成万能的“突破秘籍”

第二类误区，是对开源在国家科技创新体系中的作用产生了过度浪漫化的期待。在现实语境中，“卡脖子”问题确实存在，国家也确实需要通过集中力量办大事来突破关键技术瓶颈。但在舆论与部分政策话语中，经常可以看到一种未经论证的推理链条：“开源=开放式创新=科技突破的主战场”，仿佛只要“把东西开源”“把项目做成开源社区”，关键技术就会自动从社区里“长出来”。

这一想象忽略了一个重要事实：不同类型的创新，其发生场域和运行机制并不相同。在前述内容中已经明确指出：在开源社区中，主要涌现的是改进型、填补型创新——例如，对现有框架的优化、对不同组件间“缝隙”的填补、对通用模块和开发工具的迭代；而真正的基础性创新和突破性创新，更多仍然产生于专业科研机构以及围绕具体商业场景的企业研发体系。

如果忽视这种分工，就会出现两种极端倾向：一是把“完全自研”与“开源协同”对立起来，认为只有在操作系统、数据库等领域“百分之百关起门来重写一遍”，才算“不受制于人”；二是以为只要“依托开源社区”，就能在没有长期基础研究积累的前提下，“一跃而上”突破复杂技术壁垒。更为合理的理解是：在“新型举国体制”的框架下，开源及其社区处于从属但重要的配合作用，即服务于企业技术创新和国家重大科技任务，在符合科学规律、经济规律与市场规律的前提下，适度开源、合理开源，通过社区力量改进和补全自有产品，而不是试图“把开源当成唯一突破口”。

从政策设计上，若将“开源=科技攻关的唯一主战场”，就容易诱导资源的误配。本该投向长期基础研究的，被派去重复实现已有开源成果；本该通过企业场景

去验证的，被派去“做样子工程”；表面上“开源项目繁荣”，实则削弱了创新体系的整体效率。

55.3 把补贴当主角：试图通过“涓滴效应”扶植开源生态

第三类值得高度警惕的误区，是把各种形式的财政补贴和奖励当作推动开源的“主力工具”。在不少地方的政策文本中，可以看到类似条款：对“首个成功落地的开源解决方案”“在智慧城市建设中推广开源与信创产品的项目”，给予数百万乃至数千万元级别的奖励和资助。

这种做法背后，往往隐含着一种“涓滴效应”式的政策想象：先给上游大企业、大项目“大水漫灌”，再寄希望于这些资源能够自然“滴落”到中小企业、社区和开发者身上。类似于被经济学家所普遍批评的“涓滴经济学”：对高层主体大幅减税或补贴，假定其投资和消费最终将惠及社会底层，但大量研究表明，这种“自动滴落”并不可靠。

现实风险也很清楚。

- 在项目制与补贴驱动的逻辑下，一部分主体倾向于“为拿补贴而开源”，而不是从自身战略出发长期参与开源。
- 为了匹配政策指标，容易产生大量“纸面开源项目”，仓库无人维护、社区难以为继。
- 资源过度集中在少数“政策宠儿”身上，反而挤压了真正以开源为长期战略的中小企业和创业团队的生存空间。

在笔者参加的一次 ChinaOSC 中国开源创新组织与治理模式分论坛上，曾经提出过一个“榨汁效应”的设想。所谓的“榨汁效应”，就是用政策从企业里面榨出钱来。这个策略也是从上往下，但不是浇水，而是施加压力，使上面一层的利润能被挤出来一些，流到下面。这本质上是从开源供应链安全的角度入手，通过施加压力让企业（大型、头部企业）越来越重视开源供应链安全，使企业愿意将钱拿出来投放到开源社区里去，这就是“榨汁”。而不是政府直接给钱或者项目到企业。通过涓滴效应刺激开源，一般会导致投机和寻租，无法产生真正的开源创新生态，应该通过施加压力使企业愿意自己掏钱给开源。

这意味着，开源相关财政与经济工具的设计，应当从“补贴驱动”转向“规则 + 激励的组合”：关键不在于“政府直接给多少钱”，而在于通过合理的监管与激励，让真正受益于开源的主体长期、稳定地回馈这片数字公地。

55.4 用行政与 KPI 逻辑替代社区逻辑

第四类误区，集中体现在用传统行政考核与 KPI 逻辑来取代开源社区的自组织逻辑，在实践中，往往表现为以下几种情况。

- 用短期、量化指标来衡量“开源政绩”：要求在某个时间段内“新建若干个开源项目”“实现若干比例的国产开源替代”“贡献多少 Pull Request”。
- 通过行政方式指定“唯一项目”“唯一技术路线”，试图在开源世界里直接“挑选国家队”“划定正统”。
- 将社区活动全面纳入行政审批和考核体系，用开大会、发文、报批等方式“管理开发者”。

然而，前文关于开源文化与开源治理的讨论已经表明：开源社区的运行，依赖的是共识、声誉、协作与自愿参与，其内在激励结构与科层组织截然不同。行政体系中行之有效的工具——命令、层级、硬性指标，一旦被照搬到社区，很可能产生以下后果。

- 开发者被动配合、表面活跃，实则缺乏内生动力。
- 为完成指标而“开源”的项目，往往缺乏长期维护与治理安排，导致社区空心化。
- 技术路线被行政指定之后，多样性和竞争空间被压缩，难以发挥开源“多种实现并存”的优势。

这并不意味着政府要完全“远离社区”，而是意味着：政府更适合通过中立平台、激励规则和政策导向间接影响社区，而不是直接以行政身份充当“项目经理”或“技术委员会主席”。换言之，开源政策的目标，应是让真正懂技术、懂社区的人在合适的制度环境下发挥作用，而不是用行政力量去代行他们的角色。

55.5 忽视“数字公共产品”本质与制度配套的狭隘视角

最后一类具有根本性影响的误区，是在政策设计中忽视开源的“数字公共产品”本质，把它简单视为一个产业板块或工具选项。在“开源价值论”的框架下，开源生态的总价值，可以被理解为：被共享的数字公共产品，为全社会节约的时间总量。从这个角度看，开源政策的核心任务不只是“扶持几家企业”“打造几个项目”，而是要实现以下目标。

- 扩大数字公共产品的数量与质量。
- 提升其在各类组织中的利用水平。
- 在全社会层面降低 IT 综合成本。
- 在此基础上，为创新与教育提供更肥沃的土壤。

如果忽略以上几点，政策就容易出现两个偏狭倾向：一是只从“产业规模”出发，把开源当作软件业、信息服务业中的一个“子版块”，过度关注“开源企业营业收入”“开源产业园区产值”等指标，而忽视其对其他产业的溢出效应；二是缺乏与知识产权制度、公平竞争环境、人才培养体系、国际合作战略的联动设计，导致企业在参与开源时，既得不到知识产权与市场公平竞争的保障，也缺乏基于开源成果的教育与人才激励机制，更谈不上通过开源形成可持续的国际影响力。在知识产权与“知识落差”问题上尤其如此：如果政策不能保证企业在开源贡献之后，仍然可以通过 License、专利等方式合理管理自身的知识资产预期，就会出现“开源

反受其害”的反弹情绪，从而削弱企业长期参与开源的意愿。

相应地，要想避免陷入上述误区的开源公共政策，应当从一开始就将开源纳入数字公共产品与知识公地建设的宏观框架之中：以建设和维护“数字公共基础设施”为中心目标，而不是只看某一两项项目指标；配套完善知识产权保护、市场竞争和开源商业环境，让基于开源的企业能够可持续经营；通过教育与人才政策，鼓励“数字素养—开源基础—项目实训—开源实战”的培养路径，把开源嵌入教育体系；在对外政策上，将开源视为参与全球技术合作、输出数字公共产品和构建国际影响力的重要途径，而不是仅仅作为“内部国产替代”的工具。

总之，真正需要修正的，并不仅是某一条政策条款，而是政策背后对开源世界的基本想象。只有在认识到开源是一个由多元主体共治的全球数字公共产品体系之后，后续关于工具选择、激励设计与治理结构的讨论，才有可能站在相对稳固的认知基础之上。在接下来的几问中，我们将从正面出发，讨论在公共部门应用、政企合作、政府与基金会及教育机构协同等具体场景下，如何在避免上述误区的前提下，构建更为稳健、务实且具有长期视野的开源公共政策组合。

第 56 问 政府与企业如何通过采购策略与产业政策共建开源生态

56.1 从“项目采购”到“生态共建”的政策逻辑

政府与企业在开源上的合作，若仍停留在“买一套软件/上一套系统”的传统采购逻辑，往往会把开源当作可替换的产品形态；而开源生态的关键价值在于其作为数字基础设施的长期可复用性、可审计性与可协作性。公共部门的真实需求（安全、可靠、可控、可持续）与企业的真实诉求（规模化市场、研发效率、生态位与商业化路径）只有在“生态层面”才可能对齐：政府通过制度与需求侧工具把“可复用、可持续、可贡献”写进规则，企业通过工程化与社区化投入把“可用”升级为“可维护、可演进、可治理”。这样的政府采购，才是面向开源生态的采购。

在政府采购方面，国际上较成熟的做法包括以下几个方面。

- “默认开源/可复用”要求：美国联邦源代码政策要求机构发布至少 20% 的新定制代码为开源，并通过代码清单等机制推动跨机构复用。
- “开放标准+开源实践”纳入政府数字项目评价：英国政府技术实践准则明确提出“Be open and use open source（保持开放，使用开源）”，强调发布代码、使用开源以提升透明度与灵活性，并与采购策略、开放标准要求并列为政府数字项目的关键原则之一。
- “推荐目录/参考清单”降低采购摩擦：法国的 SILL（跨部门自由软件基座）作为推荐开源软件清单，由公务员维护并定期更新，用以推动公共部门标准化采用。

- “软件复用优先”制度化：意大利围绕公共部门软件获取与复用发布了专门指南，并将复用流程与采购技术文件衔接，强化“先复用、后采购、再开发”的秩序。

对中国情境而言，采购条款设计的关键不在于口号式“开源优先”，而在于把可执行条款写清楚：开放标准符合性、源代码与许可证交付、可移植性与退出方案、上游贡献比例与安全响应机制及长期维护责任边界。这样才能实现：政府以“规则与验收”创造生态条件，企业以“工程与开源能力”赢得采购订单。

56.2 自主可控：关键在于保障开源供应链安全

在第 55 问中提出的“榨汁效应”就是从开源供应链安全的角度入手，通过施加压力让企业（大型、头部企业）越来越重视开源供应链安全，使企业愿意将钱拿出来投放到开源社区。这里可以再展开讨论一下。目前的政府采购，往往追求代码自研比例，所谓“自研比例”，其背后的假设是，只有所有的代码都是这个厂家自己的员工写的（100%自研），才是自主可控、安全的。但事实上，这是一种自废武功，且与软件开发规律背道而驰的追求。所谓 100%自研的软件，如果真的存在，那就证明这个软件没有站在巨人的肩膀上，没有善用开源社区已有的成果，重复开发、浪费人力，甚至连质量也难以保障（因为不像开源软件，经过充分的验证）。所以，一家优秀的、有能力的研发企业，应该充分使用开源软件的成果，同时确保能够及时处理开源供应链的各种风险。这才是正确的趋利避害、善用开源的做法。（关于开源供应链风险的详细介绍，参见第 18 问。）

从政策制定的角度，应该对企业的研发过程、供应链管理、风险治理能力，提出明确的要求。例如，在研发过程中全面普及采用 SCA 工具，在内外交付过程中全面采用 SBOM 格式，在签署软件交付相关的协议时明确开源安全与合规义务等。我们需要做的不是“迫使供应商假装 100%自研代码”，而是在供需双方都心知肚明，且基于开源的前提下达成共识。具体的做法，首先是在政府 IT 采购时，提出开源供应链相关的要求，并明确供应商的责任边界与奖惩机制。通过这样的做法，将开源供应链安全的压力，从集成商、供应商，传递到开源企业与开源社区。而且，这样的传递，不应该是无偿的，请求社区给予义务支持的那种形式。而是带着诚意，带着合同，诚心请求支援。

56.3 产业政策：把开源当作“基础设施投资”，而不仅是“应用补贴”

如果产业政策只是补贴“应用落地”，那么往往会加剧“重复造轮子”的现象。更有效的做法是把关键开源组件视为“数字基础设施”，对其维护、安全与演进进行长期投入，并通过产业组织方式把收益外溢到更大范围的企业与行业。关键不在于扶持某个具体开源应用产品，而是降低全社会的数字建设成本。

德国主权科技基金是较具代表性的实践：其定位是投资支撑数字基础设施的开源软件组件，并以安全、稳定与可复用性为核心目标；欧盟互操作门户对其案例总结指出，该基金自 2022 年设立以来已支持 60 多个开源项目，累计资助 2400 多万欧元，并在公共采购法框架下运作，体现了公共部门支持开源的制度化路径。（关于德国与主权科技基金的协作关系，详见 57.2 节。）

对地方政府而言，产业政策还可以更直接地与企业开源实践绑定。以深圳市为例，根据政府部门发布的相关规定，对于基于重大开源项目（如 OpenHarmony）形成的商业发行版软件，资助额度最高达 1000 万元。对于内嵌重大开源项目操作系统的智能终端产品生产企业，资助额度同样最高达 1000 万元，但是资助的方式是按销量分级给予资助，分 3 年兑现，这也体现出“以产业政策引导企业进入开源生态上下游”的思路。

第 57 问 政府与开源基金会如何通过中立平台实现多方共治

要通过开源基金会这一中立平台实现政府、企业、社区和学术界的多方共治，核心在于将传统的“行政协调”逻辑转化为“可持续的生态共治”逻辑。政府可以将基金会视为连接政策目标与技术社区的“翻译器”和“对接桥梁”。Linux 基金会对自身定位的表述可作为中立平台的典型样本：其明确强调自身为开发者提供了一个中立、可信的枢纽，用于编写代码、管理和扩展开源技术项目。通过创建在基金会平台上开放协商的制度，防止项目被任何单一厂商控制。而政府通过支持这种中立平台，可以降低不同利益主体间的摩擦成本。以下从国际经验借鉴与中国本土实践两个层面论述实现这一目标的路径。

57.1 案例研究：欧盟与 Eclipse 基金会的协作关系

欧盟委员会与 Eclipse 基金会之间长期维持着较为深入且多维的伙伴关系。这种关系之所以能够持续推进，一个重要原因在于 Eclipse 基金会本身在欧洲开源生态中的枢纽地位：其总部设在布鲁塞尔，使其在地理空间与制度环境上都更容易与欧盟机构形成高频互动与稳定对接。由此，双方的合作并不仅限于一般意义上的项目支持，而是延伸到科研资助、政策倡议、数字主权建设以及公共部门治理能力提升等多个层面。

在科研合作方面，欧盟委员会通过 Horizon 2020、Horizon Europe 等科研与创新资助计划，为跨国技术协作提供了制度化的资金渠道。而 Eclipse 基金会则以其组织复杂技术倡议与治理大型社区项目的经验，成为这类跨国协作的重要承载方与管理节点。以 2023 年为例，Eclipse 基金会在 Horizon Europe 框架下获得了多个重要研究项目，分别聚焦下一代元操作系统、轻量级软件栈以及计算连续体协同等方

向。对欧盟委员会而言，这类项目不仅是技术研发，更是将学术界、产业界与跨国团队组织起来的系统工程；对 Eclipse 基金会而言，其角色也不仅是“参与者”，还在一定程度上承担起跨国合作组织与治理的功能，帮助把分散的技术力量汇聚成可交付、可持续的工程成果。

在政策倡议与法规制定方面，双方的互动更具现实张力。以欧盟《网络弹性法案》（CRA）为例，相关立法对软件供应链安全与产品责任提出更强约束，在提高安全门槛的同时也可能给开源社区带来合规压力。Eclipse 基金会在这一过程中多次牵头向欧盟机构表达开源界对潜在“寒蝉效应”的担忧，强调开源软件在生产与治理方式上具有特殊性。如果简单地将其套入传统商业软件的责任框架，那么可能抬高社区贡献门槛并影响创新活力。与此同时，为了应对法规的落地要求，基金会还推动与其他国际性基金会合作，尝试形成共同的标准与框架，以便让开源社区能够在更可操作的路径下满足合规目标。这种互动关系并非对立式博弈，而更接近于“政策形成—反馈修正—共同适配”的动态过程：欧盟委员会推动安全与韧性目标，基金会则把社区运行的现实约束与治理经验带入政策讨论，并推动形成更可执行的落地机制。

在数字主权建设方面，欧盟长期强调通过开源增强技术自主性与可控性，Eclipse 基金会在其中扮演了关键平台角色。围绕 Gaia-X 等欧洲数字基础设施相关倡议，基金会接管并托管了部分关键项目，帮助其在开放协作框架下持续演进。更进一步，基金会还与欧洲的标准化机构开展合作，探索如何缩小传统标准化体系与开源软件快速迭代之间的结构性差距，从而提升工业界对开源技术的信任与采用意愿。对欧盟来说，这不仅是“开展一个项目”，更是构建一套能够长期运行的数字基础设施与产业协作机制；而对基金会来说，则是在中立治理框架下，把不同国家、企业和开发者的投入组织起来，降低跨主体协作的摩擦成本。

在治理能力与公共部门开源管理能力建设方面，双方的合作同样具有制度意义。欧盟委员会推动建立自身的开源项目办公室（European Commission Open Source Programme Office, EC OSPO），把它作为执行欧盟开源战略的关键抓手，以提升公共部门的开源治理、复用与合规能力；与此同时，Eclipse 基金会也参与发起或推动 OSPO 相关联盟与实践网络，向各级政府部门提供方法论、工具与最佳实践，帮助公共部门把“支持开源”的政策意图转化为可操作的内部流程和外部协作机制。这一层面的合作，实质上是在把开源从“技术选择”提升为“治理能力”：不仅要开源，更要会管理开源、参与开源、在安全与合规要求下持续贡献。

总体而言，欧盟委员会与 Eclipse 基金会的关系可以理解为一种“战略规划—平台承载—反馈迭代”的协作结构。欧盟委员会依据欧盟顶层战略，制定数字主权与网络安全的具体实施方案和制度框架，推动形成可执行的公共目标；Eclipse 基金会则作为中立平台，把跨国企业与开发者组织起来参与工程协作与治理实践，并将实践中遇到的合规与治理难题反馈到政策与标准讨论中。通过这种机制，原本容易陷入高度摩擦的跨国行政协调，被转化为更可持续的多方共治，使开源在欧洲的制度化发展具备了更强的可执行性与韧性。

57.2 案例研究：德国与主权科技基金的协作关系

德国政府与主权科技基金（Sovereign Tech Fund, STF）之间形成了一种典型的“政府发起、机构化运作”的伙伴关系。STF 并不是一般意义上独立运作的民间组织，而是德国在推进数字主权（Digital Sovereignty）与维护关键数字基础设施背景下，建立的一项具有明确公共政策目标的战略性公共投资安排。其基本逻辑是：把开源软件视为数字时代的公共基础设施，将其安全性、可维护性与可持续演进能力纳入公共治理与公共投资范畴。

在组织与管理架构上，STF 具有清晰的政府背景与制度化执行体系。该基金由德国联邦经济事务和气候行动部（BMWK）发起并提供资金支持，具体实施由主权技术机构（Sovereign Tech Agency, STA）承担。STA 并非临时性项目办公室，而是德国联邦颠覆性创新署（SPRIND）的全资子公司，这意味着其在公共目标约束下具备相对专业化、市场化的项目执行能力。资金方面，STF 的资金完全来自德国联邦预算拨付，年度预算需经联邦议院批准；其资金规模在近年来呈增长态势，例如，预算从 2022 年的约 350 万欧元增长到 2024 年的约 1700 万欧元。在治理层面，STA 在监事会提供战略指导的前提下，由专业高管团队与外部专家网络负责项目筛选、评估与执行，以尽量在公共利益导向与专业判断之间形成平衡。

在协作理念上，德国政府与 STF 的关系建立在一种更“基础设施化”的开源认知之上：开源软件并非单纯的开发方式或商业模式，而是支撑数字经济与公共服务的“道路与桥梁”。因此，STF 把投资重点放在那些位于“应用层之下”、难以直接商业化却被广泛使用的开放数字基础技术上，如各类基础库、协议与开发工具等。其政策意图在于缓解开源生态中的“公地悲剧”与“搭便车”问题：大量关键组件由志愿者维护，长期存在资金不足、维护压力过大与安全风险积累等隐患，公共部门通过制度化资金投入，帮助这些基础组件补齐安全、维护与工程化治理能力。

更具制度特色的是，STF 在德国公共采购法框架下运行，通过合同方式向全球范围内的开源项目提供资助。这意味着政府并非以一次性捐赠方式介入，而是以更可审计、更可问责的方式管理公共资金，把公共目标转化为可执行的项目任务与交付条件。在具体实施上，STF 通常与受资助项目签署合同，依据约定的里程碑分阶段拨付资金；资金规模往往在 5 万至 100 万欧元，里程碑内容可能包括安全审计、技术债治理、文档完善、工程化改造等。与此同时，STF 还通过若干专项计划把“安全韧性”与“维护者支持”制度化。例如，“缺陷韧性计划”（Bug Resilience Program）侧重于技术债管理、安全审计与漏洞赏金平台等手段，以增强关键开源组件的安全性；“维护者团契”（Fellowship for Maintainers）则以直接向关键组件维护者支付报酬的方式，认可维护劳动的公共价值，目前以试点形式运行。

在覆盖范围上，尽管资金来源于德国纳税人，STF 的支持对象具有全球授权与全球外溢效应，已支持包括 cURL、OpenSSH、systemd、FFmpeg、FreeBSD 等在内的 60 多个关键项目。这与其“基础设施投资”的公共政策逻辑一致：基础组件的安全与稳定具有全球外部性，投入即意味着降低供应链风险、提升整体软件生态

韧性。与此同时，德国政府也要求 STF 在选择资助对象时遵循较为严格的评估准则，强调技术的盛行度（是否被广泛集成）、相关性（对关键部门的重要性）、脆弱性（是否存在资金缺口或维护不足导致的风险），并结合公共利益、拟议活动可行性以及贡献者专业能力等因素进行综合判断，以确保公共投入具有战略影响力与可验证产出。

总体而言，德国政府与 STF 的协作模式体现了一种从“偶然使用开源”走向“系统性治理与投资开源”的转型。政府在宏观层面承认开源基础组件的公共品属性，通过财政预算与制度框架提供资源与问责边界；STF 则作为专业化执行主体，把公共目标转译为可落地的工程任务与治理改进，在保持技术专业性与相对中立性的同时，提升关键开源基础设施的安全与可持续性。这种模式为政府如何以更低摩擦、更可持续的方式支持开源生态，提供了具有可操作性的制度参考。

57.3 中国特色的实践

相对于国外的开源基金会，中国的开源基金会在成立之初，就是国家战略导向的体现。2020 年 6 月，中国的第一家开源基金会——开放原子开源基金会（以下简称“开放原子”）成立。2021 年 3 月，开源首次写入《中华人民共和国国民经济和社会发展第十四个五年规划和 2035 年远景目标纲要》，明确提出支持数字技术开源社区等创新联合体发展；国务院印发《“十四五”数字经济发展规划》，提出支持具有自主核心技术的开源社区、开源平台、开源项目发展，推动创新资源共建共享，促进创新模式开放化演进；工业和信息化部印发《“十四五”软件和信息技术服务业发展规划》，系统布局“十四五”开源生态发展。

2023 年 3 月，中国的第二家开源基金会——天工开物开源基金会（以下简称“天工开物”）成立。这标志着中国同时拥有自上而下贯彻国家战略的开源基金会，以及自下而上致力于繁荣生态的开源基金会。这两家开源基金会既有共同之处，也各有特色。

共同点首先体现在“中立治理平台”的制度属性上。开放原子明确通过信息公开与制度体系（项目管理制度、项目群管理制度、孵化毕业标准等）将开源项目的“捐赠—孵化—毕业”过程制度化。其 TOC（技术监督委员会）作为最高技术决策机构，承担项目孵化与毕业审核、技术指导等职责，从机制上确保“技术中立、开放透明、共识合作”的治理逻辑能够落到项目层面。天工开物在制度形态上也呈现出相似路径：基金会在重庆登记成立，业务范围涵盖资助、培训、交流合作、咨询服务及对开源软件/硬件/开放内容相关活动的支持。其工作制度同样将 TOC 定位为基金会“最高技术决策机构”，并将项目孵化、毕业审核纳入技术治理框架。

但两者的特色同样明显，开放原子典型地体现出国家战略导向下的“基础软件与关键数字技术生态平台”定位：例如，OpenHarmony 在其官方项目页说明由开放原子孵化及运营；openEuler 社区亦明确其项目群由开放原子孵化及运营。更值得注意的是，开放原子在 2025 年 11 月宣布 OpenHarmony 与 openEuler 成为其首

批“毕业项目”，这意味着其已形成可操作的项目成熟度评审与生态衡量机制。这类“平台级基础软件项目+全国性生态组织能力”的组合，使其更适合承接跨行业、跨地区的基础软件协同与产业适配任务。

天工开物则更突出“生态繁荣导向”的特征，尤其注重深耕开源教育领域。它更偏向于通过场景牵引、产学研协同与生态活动组织，来汇聚资源、孵化项目与培育人才。这体现在天工开物主导搭建的开源教育联盟推出开源毕业设计项目，以及与 CCF 合作推出的全方位开源教育激励计划。

因此，讨论两家基金会的“共同之处与特色”，关键不在于简单比较规模或名义，而在于识别其背后的治理范式：在国家战略与地方创新并行的条件下，开源基金会正逐步承担起“把公共目标翻译为社区可执行规定、把分散资源组织为可持续协作”的制度功能。这也为后续讨论政府—企业、政府—基金会、政府—高校的协同机制，奠定了更贴近中国现实的分析基座。

第 58 问 政府与教育机构：如何以开源为抓手 培养人才与推动科研

58.1 开源是新型人才培养方式

开源是一种非常有效的人才培养方式，因为它把学习从封闭的课堂推向真实的工程与协作场景。对学生而言，开源社区更像一个长期开放的实战训练场：任务真实、标准清晰、反馈及时，成果还能被外界直接检验。相较只完成作业或课程项目，参与开源更能帮助学生理解软件工程的复杂性，并形成面向产业的能力体系。

首先，开源能够弥合产教之间的鸿沟。学生参与真实项目，会自然接触需求讨论、任务拆解、版本管理、分支协作、代码评审、持续集成、发布迭代与缺陷修复等完整流程。这些能力在课堂里往往只能通过“模拟”来讲解，而开源让学生在真实约束中成长，从知识接受者逐步转变为能够对外部用户与质量负责的工程实践者。

其次，开源同时磨炼硬技能与软素养。持续贡献会迫使学生提升代码质量、加深架构理解、增强测试意识、提高问题定位能力，并学会在既有代码与规则下做增量改进；社区的公开沟通与异步协作机制，也会培养写清问题、解释方案、回应评审、对齐目标、持续推进的能力。这些并非单一课程分数能体现，却是雇主高度看重的主动性、责任感、协作意识与学习敏捷性。

最后，开源贡献天然形成透明、可验证的“技术简历”。提交记录、问题讨论、评审过程与维护痕迹构成公开的信用凭证，显著降低信息不对称，使学生的能力与成长可被直接看到，从而提升其在就业与科研方面的竞争力。

在 AI 驱动、技术快速演进的今天，这种培养方式的重要性反而更高。课堂内容更新周期难以跟上行业变化，而开源社区始终在演进，学生面对的是变化的需

求、迭代的代码与真实用户反馈。AI 工具降低了编码门槛，却抬高了“把事情做成”的要求：能写对、能写稳、能协作、能交付。在开源中，代码要经得起评审与长期维护，这会倒逼学生建立更成熟的工程思维与质量意识，并学会把 AI 当作效率工具而非答案来源。

因此，开源不是课堂教育的替代品，而是把教育延伸到真实世界的训练机制。我们更需要鼓励学生走进开源社区作贡献。在真实项目中学习，在真实协作中成长，在真实标准下快速迭代能力，更早具备面向产业与国际协作的综合竞争力。

58.2 开源教育相关案例介绍

在开源教育与开放式人才培养实践中，不同国家与机构形成了多种可复制的路径。有的通过开放课程资源与国家级在线平台实现教育供给的规模化与普惠化，也有的以开源工程实践贯通“从学习到交付”的全流程训练。以下 3 个案例分别代表了全球开放教育资源运动、国家级在线教育基础设施及开源硬件与芯片人才培养的典型模式。

第一个案例是 MIT OpenCourseWare (OCW)。MIT OCW 于 2001 年启动，被普遍视为开放教育资源 (OER) 运动的重要先驱之一。其核心做法是免费公开 MIT 校内课程的关键教学材料，包括教学大纲、讲义、作业与考试等，并通过知识共享许可协议支持在非商业条件下的传播与再利用，使优质教育资源能够跨越学校边界被全球学习者访问和吸收。从影响力看，截至 2022 年，公开数据显示其访问量超过 1.7 亿人次，课程材料超过 2500 门，覆盖 MIT 全部 33 个学科领域，体现出长期、稳定、系统化的开放供给能力。OCW 的意义不仅在于 MIT 的内容开放，更在于它引领了全球“开放课件运动”的扩散，并推动形成开放课件联盟，吸引全球众多高校加入，共同开放与共享教育资源。这一案例表明，高质量开放资源的持续供给可以塑造一种新的教育公共品形态，为全球范围内的知识传播与人才培养提供基础设施支持。

第二个案例是印度国家技术增强学习计划 (NPTEL) 项目。该项目由印度政府人力资源开发部资助，七所印度理工学院 (IITs) 与印度科学研究院 (IISc) 于 2003 年联合启动，是印度国家级的在线教育基础设施工程。它的特点是供给规模大、学科覆盖广，并形成了相对完整的内容生产与学习认证体系。NPTEL 提供工程、基础科学、人文管理等领域的免费在线课程，根据公开信息，其视频内容时长达到 5.6 万小时，并形成了体量较大的教育内容库与访问量。在运行机制上，课程通常包含视频讲座、讲义与测验，自 2014 年起引入认证考试，通过者可获得由 IITs/IISc 颁发的证书 (考试一般需要付费)，从而在“开放获取”与“可验证能力证明”之间建立了可持续的制度安排。更重要的是，NPTEL 的定位不仅是提供课程，更是面向全国工程教育质量提升的系统工程，通过设立本地章节等方式扩大覆盖面，服务大量学生与自学者。这一案例的启示在于：开放并不等同于零散资源堆积，真正的教育普惠需要“国家级组织能力+高质量内容+标准化学习与认证机

制”的组合。

第三个案例是“一生一芯”计划。该计划由中国科学院计算技术研究所于2019年8月启动。目标指向非常清晰，即面向处理器芯片设计领域培养高水平人才。它的核心价值在于把“开源+敏捷开发”的方法论引入芯片设计教学与实践，强调学生从零开始设计 RISC-V 处理器芯片，并尽可能完成从设计、验证到流片（制造）的全流程实践，形成“带着自己设计的处理器芯片毕业”的标志性培养愿景。与传统课程偏重知识点讲授不同，这种路径把计算机系统、体系结构、数字电路、EDA 工具链、软硬件协同等知识贯通起来，强调用工程结果倒逼知识整合，培养面向计算机系统的全栈式能力结构。从规模与社会影响看，截至2024年，项目已进入第6年，累计报名人数超过8000人，并调整为全年开放报名模式，体现出较强的社区扩散能力与人才吸引力。这一案例说明，开源并不仅适用于软件领域，同样可以成为硬件与芯片人才培养的关键组织方式：通过开放的工具链、可复用的模块与协作式开发流程，让学习过程与产业实践更紧密地对齐。

58.3 受到开源启发的开放科学

开放科学可以理解为受开源理念启发的科研范式升级。它强调透明度、可复用性与协作性，目标是把科研从封闭的“个人成果展示”转向可验证、可接续、可协作的公共知识生产。在数据与计算成为研究基础设施、跨学科合作日益普遍的今天，开放科学不仅是学术理想，更是一套能提升效率与可信度的方法与制度组合。

首先，开放科学致力于打破科研“黑箱”。传统研究往往只公开论文结论与有限方法细节，关键过程如数据清洗、参数设置、迭代记录与负面结果常被隐去，导致难以复现与验证。开放科学倡导公开研究流程，包括实验方法、原始数据、代码、分析脚本与研究笔记，其中“开放研究笔记”强调尽可能披露研究过程，使同行既能看到结论，也能理解结论如何产生，从而提高可复现性与可审计性，增强学术共同体的信任基础。

其次，开放科学加速知识迭代，降低重复成本。通过开放获取与开放数据，研究成果不再受订阅壁垒与机构边界限制，全球研究者可以更快获取前沿成果，在此基础上开展再分析、交叉验证与二次创新。高质量数据与工具一旦开放，往往能被多次复用，形成“复用—新发现—再沉淀”的正循环，避免各自重复采集相似数据、重复搭建相同工具，把资源更多地投入到真正的创新上。

更重要的是，开放科学改变科研协作逻辑，促进群智汇聚。面对气候、公共卫生、能源材料等复杂问题，单一团队难以快速突破。开放科学推动从“垄断知识优先权”转向“开放协作共同攻关”，强调分工协作、共同维护研究资源、在开放平台上持续迭代。总体而言，开放科学不是简单地“免费公开论文”，而是围绕可复现、可复用、可协作重塑科研生产方式，使科学研究更像一项可持续演进的公共工程。

58.4 开放科学的相关案例介绍

开放科学强调把科研从“封闭发表”推进到“过程可检验、数据可复用、协作可追溯”的公共知识生产机制。以下列举的3个案例分别代表了开放科学在AI大模型、药物研发及知识图谱计算研究等不同场景中的典型做法。

第一个案例是 BigScience Workshop 与 BLOOM 模型。BigScience 是由 Hugging Face 等机构协调推动的一年期开放式研究协作（时间跨度约为 2021 年 5 月至 2022 年 5 月），参与规模很大，官方介绍显示，有来自约 60 个国家、250 多个机构的 1000 多名研究者共同参与，其中计算资源由 GENCI（Grand Équipement National de Calcul Intensif，法国国家高性能计算中心）提供，关键训练工作在法国的 Jean Zay 超级计算机上完成。BigScience 项目最知名的成果是 2022 年发布的 BLOOM 模型，公开资料显示，其规模为 1760 亿参数，并以多语言、多学科参与过程透明著称。在资源投入方面，GENCI 的公开信息披露了训练持续数月、计算量达到一百多万计算小时等细节，使其在“算力投入如何被组织为公共科研资产”这一点上具有很强的示范意义。BigScience 这一案例的开放科学价值不只是“开源发布模型”，更在于把大规模协作、训练过程文档化、数据与方法透明化结合起来，让后来者能够在可追溯的基础上复用成果并开展负责任的再研究。

第二个案例是 Open Source Malaria（OSM）。该项目被广泛视为开放科学在药物发现领域的代表性实践之一，其关键特征是采用“开放研究笔记本”的方式，把实验设计、过程记录与数据尽可能实时公开，降低传统药物研发中信息封闭带来的重复成本与协作壁垒。公开资料显示，大约在 2011 年，OSM 由澳大利亚悉尼大学的 Matt Todd 等人发起，并持续以开放协作网络的方式吸引不同背景的研究者与学生参与。学术论文与项目报道也明确强调了其“实时披露数据”的开放式研发路径，说明这种机制不仅是理念倡导，还能够支撑实际的药物发现研究工作流。这一案例的启示在于，在高度复杂、周期漫长的科研场景中，开放并不必然削弱竞争力，反而可能通过扩大协作者规模、加快验证与迭代速度来提高整体产出效率。

第三个案例是 Project Rephetio，它集中体现了开放科学在“计算研究—公共数据整合—开放协作记录”维度的成熟实践。该项目聚焦药物重定位（Drug Repurposing），通过构建并公开异构生物医学知识图谱 Hetionet v1.0，系统整合多种公共数据资源，进而基于图谱结构与路径特征开展预测建模。Zenodo 的公开记录明确标注了 Hetionet v1.0 与 Project Rephetio 的关联，并提供可下载的数据版本与说明文档；同时，项目在开放平台上完整保留了协作与讨论记录。研究论文则对 Hetionet v1.0 的规模和构成给出了可复现的细节描述，突出“数据—代码—分析过程”的整体开放。该案例的价值在于，它将开放协作从“共享最终结果”推进至“共享可复现的研究流水线”，使外部研究者能以更低成本验证、改进并迁移其方法。

58.5 政府如何推动基于开源的产学研协同

在以开源为底座的产学研协同中，政府最关键的作用不是替代市场和高校做技术选择，而是通过制度设计与公共资源配置，降低协作摩擦、扩大创新供给、形成可持续的生态环境。概括而言，政府应当扮演“搭台者”与“连接器”：搭建共享的基础设施与规则框架，同时把高校、科研机构、企业与应用场景连接起来，让开源创新从科研端顺畅走向产业端与公共服务端。

第一是公共资源配置。开源创新需要算力、数据、测试环境和工程化工具链等基础条件，但高校团队与中小微企业常受制于成本与获取渠道。政府可通过发放“算力券”、提供国家级超算中心使用补贴、建设区域算力公共平台等方式降低门槛，并把支持重点放在可复用的公共能力上，如评测基准、数据治理工具、训练与部署工具链、合规与安全组件等，避免重复建设，形成共享型数字基础设施。

第二是采购与应用牵引。政府采购与重大信息化项目能够提供高质量、可持续的真实场景，有助于解决“成果难落地、需求难对接”的问题。在不降低安全合规与性能要求的前提下，可优先采用基于开源标准与开源路线的方案，并在招标条款中强调可维护性、可迁移性、接口开放、社区治理与长期支持，推动从“买产品”升级为“买能力与生态”，减少技术锁定，为校企协同成果提供可验证的落地机会。

第三是资助规则变革，践行“公共资金，公共 AI/软件”的原则。对公共财政资助项目，可明确要求在合理的安全与合规边界内，将关键成果（代码、数据集、模型权重、评测方法与文档）以开源或开放许可发布，并提供复现说明与版本管理机制。这样既能提高科研透明度与可复用性，减少“重复造轮子”，也更有利于企业和后来者在同一基座上持续迭代与转化。

第四是支持中立平台。开源协同的难点往往在治理与信任：高校关注学术产出，企业关注竞争，长期维护容易被忽视。政府可支持建立中立的开源基金会或产业联盟，提供非竞争性的协作空间与治理框架，承接资金与合规管理，组织技术路线与版本治理，协调关键基础软件长期维护，以透明规则降低多方协作顾虑。

为确保政策落地，还需建立配套机制。评价体系上，应建立多元评价，把开源贡献纳入学生学分、奖学金与就业能力证明，并纳入教师与科研人员成果评价，补齐“重论文轻工程”的缺口。资金支持上，可设立专项基金，持续资助开源教材与课程、师资培训、社团与社区活动，以及关键基础软件与基础模型的长期维护，避免短期化。教育推广上，推动高校开设面向全学科的开源通识课，普及协作方法、许可证与合规、安全与治理。实践体系上，建设开源实训基地与校企联合平台，通过开源实习、悬赏、大赛等机制让学生持续接触工业级真实项目。制度约束上，进一步落实开放成果规定，对公共财政资助的软件研发与数字化项目明确开源发布要求与例外边界，确保公共投入沉淀为可复用、可审计、可

持续的社会公共资源。

总体而言，政府推动基于开源的产学研协同，关键在于把“资源、场景、规则、平台”组织起来：用公共资源降门槛，用采购场景牵引落地，用资助规则沉淀公共资源，用中立平台保障长期治理，从而形成持续创新与规模化应用的良性循环。

第 59 问 如何打造能够吸引全球开源人才的城市

开源人才的流动与集聚，表面上是“人才政策”的竞争，实质上是城市综合治理能力与创新生态密度的竞争。尤其在生成式 AI 时代，开源人才、科技人才、创新创业人才与 AI 人才呈现高度重叠：他们共享相近的能力结构（工程化与协作能力）、工作方式（全球协作与远程/混合办公）及价值偏好（开放网络、快速试错、对基础设施的高敏感）。因此，城市要“吸引全球开源人才”，不能只把开源当作一个产业类型，而应将其视为城市创新体系的组织方式与国际连接机制，把政策对象从“个人补贴”转向“生态能力建设”。

59.1 人才的交叠性质：头部人才为何反复涌入少数头部城市

从多份国际研究与指数可以观察到一个稳定事实：不同人才群体所青睐的城市高度重合，呈现“少数头部城市集聚”的趋势。以“人才黏性”（吸引与留存）为例，《全球城市人才黏性指数报告 2025》给出的前十城市依次为纽约、旧金山、波士顿、巴黎、北京、伦敦、悉尼、新加坡、上海、首尔。该报告同时指出，人才黏性与城市的创新潜能、文化开放与公共生活质量等维度关联紧密。

进一步看“创新产出”维度，世界知识产权组织（WIPO）发布的《2025 年全球创新指数报告》中的“科学与技术集群”排名显示，全球最大的集群主要集中在东亚与少数欧美都会区：深圳—香港—广州、东京—横滨、圣何塞—旧金山、北京、首尔、上海—苏州位列前六；美国与欧洲的代表性集群包括圣何塞—旧金山、纽约、伦敦、波士顿—剑桥、巴黎等。换言之，能够持续产出论文与高价值专利的都市圈，本身就是全球技术人才“可持续做事”的主要承载地。

在 AI 人才竞争上，城市集聚趋势更为明显。Counterpoint Research 发布的“2025 全球 AI 城市指数”将新加坡列为全球第一，其后依次是首尔、北京、迪拜和旧金山。这类指数通常同时考察公共部门与私营部门的 AI 活动、政策举措、企业与创业生态等因素，体现出“AI 人才”对城市综合投入与制度供给的敏感性。

此外，Nature 发布的 Global Innovation Hubs Index 2025 给出的排名前 20 城市/都会区也呈现相似名单：旧金山—圣何塞、纽约、北京、波士顿、伦敦、粤港澳大

湾区、上海、巴黎、东京、首尔、新加坡、西雅图等均在列。这些名单与“人才黏性前十城市”“创新集群前列都会区”“AI 城市指数前列城市”之间的交集，恰恰说明：开源人才的“择城”，往往不是择一项优惠政策，而是在择一个能够长期提供机会、资源与国际连接的创新生态系统。

对于开源人才而言，还存在一个常被低估但影响巨大的变量：全球协作网络的“可达性与摩擦成本”。GitHub 的《Octoverse 2025》报告通过全球热力图展示了开发者群体在美国、印度、中国、巴西等地形成高密度集中，并强调开发者增长与 AI 驱动的软件活动持续增强。这意味着城市竞争不仅发生在“本地岗位”，也发生在能否成为全球协作网络的高频节点——包括会议活动、社区组织、跨国企业研发中心、开源基金会与标准化组织的可触达性。

因此，回答“各类人才共同喜欢的城市有哪些”，不宜停留在列举名单，更应抽象出这些城市共同具备的 3 类能力。

（1）机会密度：研发岗位、关键项目、创业与资本、行业场景与试点的丰富程度。

（2）基础设施强度：算力、数据、云与网络、测试验证平台、合规与安全响应能力。

（3）制度与生活可预期性：对国际人才友好的制度接口、稳定透明的治理及能够支持长期生活与家庭安置的公共服务体系。

这些能力叠加，才构成“人才黏性”的本质。

59.2 “吸引”的不同层次：从短期停留到长期定居的分层目标

“吸引全球开源人才”并非单一结果，而是从浅到深的人才吸纳连续谱：旅游（短期访问）—旅居（中期停留）—创业落地—长期定居/移民。城市政策必须认识到：不同阶段的人才决策逻辑不同，工具组合也应分层。

第一层是“短期访问”。对开源人才而言，一座城市的第一次进入往往源于会议、黑客松、开发者社区聚会与技术社群活动。此时城市竞争力表现为国际可达性、活动组织能力与社群开放氛围。政策重点不是补贴个人，而是打造具有全球影响力的长期活动品牌 and 高质量社区基础设施，使城市成为开发者团聚网络的“固定节点”。

第二层是“中期停留”。这一阶段，制度接口的重要性迅速上升：签证、保险、税务、居住便利与公共服务可达性，决定了人才是否愿意“先住下来再观察”。日本外务省对“特定活动（Designated Activities）”签证的材料要求：计划活动与停留期说明、年收入达到一定门槛及覆盖医疗的保险等，且允许最长 6 个月的远程工作型停留。这一类制度设计的政策含义在于：城市（或国家）通过提供可预期的中期停留通道，把“人才接触”从一次性旅游延伸为“可发生合作与落地评

估”的驻留体验。

第三层是“创业落地”。当人才从“生活体验”进入“事业选择”，其关注点会转向融资难度、客户场景、试点与采购机制、合规成本与团队组建便利度。加拿大的 Start-up Visa Program 把“人才落地”与“生态支持”直接绑定：申请需要获得指定机构（如创投基金、天使投资团体或孵化器）的支持，并在规则上明确不同类型机构的支持方式与要求。这反映出一种更具政策理性的路径：不是简单“给个人补贴”，而是通过制度把创业人才导入本地创新体系的关键节点。

第四层是“长期定居/移民”。此时决定性因素往往取决于可持续生活与长期预期：教育医疗、住房成本、社会治理质量、国际化环境与家属安置等。对于开源人才尤其如此，他们的职业资本部分沉淀在全球开源声誉与协作网络中，城市若无法提供开放包容的社会环境、稳定透明的制度与长期成长空间，即便短期吸引，也难以形成“留存与复利”。

因此，城市应将人才战略设计为连续的“人才旅程”：能进来、能停留、能合作、能落地、能扎根，而不是把资源集中在单一环节（例如，只做落户补贴或只做短期大会）。

59.3 人才最重视的机会与基础设施：开源导向人才的“核心需求画像”

从政策供给角度，人才所重视的要素可归纳为“机会+基础设施”。机会提供可能性，基础设施保障可能性得以兑现、可复制、可持续。对开源导向人才而言，这种组合具有更强的“公共品”属性：他们不仅需要岗位，还需要可持续贡献的工程与治理环境。

在“机会”侧，开源人才通常更在意以下几类机会。

- 上游贡献与关键项目机会：城市是否聚集关键开源项目、是否有企业与机构愿意投入上游维护与治理，而非只进行下游使用。
- 跨界场景机会：开源与 AI、数据、城市治理、制造、医疗等结合的真实场景是否充足，是否存在开放的 API 与可插拔的模块机制。
- 创业与融资机会：资本密度、孵化体系、试点场景、供应链完整度、产业客户资源与全球市场触达能力。
- 国际协作机会：参与国际基金会、标准化组织与跨国协作网络的便利程度。

在“基础设施”侧，开源人才尤其关注 5 类城市能力。

- 数字基础设施：高质量网络、稳定云服务及可获得的公共算力与测试验证资源。AI 城市指数的评估框架从侧面反映了“公共与产业活动密度”对 AI 能力的重要性。
- 开放与互操作基础设施：开放标准、开放 API、在合规框架下的数据开放与共享机制。

- 社区与协作基础设施：公共协作空间、长期会议品牌、维护者驻留计划与社群组织能力。
- 制度性基础设施：开源合规、知识产权与商标支持、软件供应链安全响应、争议解决与信用机制。
- 国际化生活基础设施：语言环境、国际学校与医疗服务、面向外籍人才的行政服务可达性。

在城市实践层面，一个可作为“公共部门开放与开源”案例参照的是柏林。柏林开源门户展示并开放柏林行政机构的相关 IT 项目代码，目的之一是提升公共 IT 透明度并增强公众参与性。这类做法对吸引开源人才的启示在于，城市不只把开源当作内部技术选型，还将其纳入公共治理与公共服务的制度框架，从而为开发者形成“可参与、可协作、可复用”的公共创新空间。

59.4 城市人才战略：从愿景、定位到实施

将“吸引全球开源人才”变为可执行的政策，需要从治理上形成清晰链条：愿景定位—目标体系—核心原则—实施路径。

在愿景定位上，建议城市明确自身核心发展定位：是成为开源研发中心、开源产业化中心，还是开源治理与规则的学术中心。三者可以叠加，但必须有主次，否则容易出现资源分散与政策摇摆。

在目标体系上，建议显式区分“短期可见的绩效目标”与“长期生态目标”。前者可包括国际会议与活动密度、开发者来访与驻留规模、开源相关岗位与企业数量、开源创业的融资与试点数量；后者则更应关注上游贡献强度、关键项目的治理参与、城市公共数字基础设施的复用水平及国际协作网络的节点地位。人才黏性指数对“吸引与留存”的分维度评估，可以作为目标体系设计的外部参照，帮助城市避免仅用“引进人才数量”做单一考核。

在核心原则上，应至少坚持以下 4 点。

（1）开放与中立：对技术路线、厂商与具体项目保持中立，避免行政指定“唯一项目/唯一平台”。

（2）鼓励竞争与创新：以规则和接口创造竞争空间，而非用补贴固化路径依赖。

（3）尊重社区规律：避免将社区行政化、指标化，减少对开源贡献的机械式 KPI 考核。

（4）可持续投入：将维护、治理和安全作为长期公共能力建设，而非运动式短期项目。

在实施路径上，城市应采取“组合拳”而非单点工具：以公共服务平台降低合规摩擦，以开放标准与接口释放场景机会，以公共算力与测试资源提高 AI/ 开源项目落地成功率，以试点与采购机制把企业贡献绑定到真实需求，以国际化大会与驻

留计划形成开发者网络的周期性回流。

59.5 度量与保障体系：把战略固化为治理能力

城市对全球开源人才的吸引力，最终要用“可度量、可问责、可迭代”的方式固化为治理能力。建议建立两类机制：度量体系与保障体系。在度量体系上，可采用“流量—留量—贡献量—生态量”的指标结构。

- 流量：国际会议规模、短期来访与驻留人数、跨国协作项目数量。
- 留存量：核心维护者留存、国际团队设点与长期居住比例。
- 贡献量：上游贡献（关键项目 PR/ 问题、治理席位）、安全修复与响应参与。
- 生态量：开源企业密度、开源创业存活率、公共数字基础设施复用率。

与此同时，城市可把前文提到的各种黏性指数、创新指数作为外部对标，形成“对标—诊断—改进”的年度闭环，而非只做宣传。在保障体系上，建议形成“四件套”：组织（跨部门统筹）、预算（可持续投入）、制度（合规与开放规则）、合作（企业—基金会—高校—国际组织的制度化伙伴关系）。

在上述框架基础上，可将本问中介绍的政策建议浓缩为以下清单（供直接落地或改写为行动计划条目）。

- 建立城市级开源生态统筹机制（常设机构、城市 OSPO），负责跨部门协同与年度评估。
- 建设开源公共服务平台，提供合规、知识产权、供应链安全、社区治理与国际协作支持服务。
- 推出面向关键基础设施项目与维护者的中长期资助与资源支持机制，强调可持续维护，鼓励上游贡献。
- 形成公共算力、测试验证与安全响应资源池，在合规框架下向开源与 AI 项目开放。
- 以开放标准、开放数据与可插拔接口重塑政务与公共服务平台，将城市场景打造为创新入口。
- 构建开源友好型招采体系，以开放标准、可移植性为核心条款，建立安全响应机制和明确的退出机制，避免指定唯一技术路线。
- 设计“人才旅程产品”，形成会议活动—驻留计划—创业孵化—长期安置的连续制度接口，并与国家签证政策衔接（数字游民、创业等）。
- 推动“开源+教育”联合工程，在课程、实训与科研评价中认可高质量开源贡献。
- 搭建国际化社区基础设施（空间、活动、双语服务与对外传播），形成全球协作网络的稳定节点。
- 建立公开透明的指标发布与政策迭代机制，把“吸引”转化为长期“留存与复利”。

综上所述，能够吸引全球开源人才的城市，核心不在于单项优惠政策，而在于能否提供持续的机会网络、可靠的基础设施与可预期的制度环境，并通过度量与保障体系把这些能力转化为城市治理的长期优势。

第 60 问 如何通过开源实现科技外交

60.1 科技外交

科技外交（Tech Diplomacy）是指国家、私营部门及民间社会等多元主体，围绕全球数字政策和新兴技术问题展开的国际对话、谈判与实践。其核心是把科学技术作为外交工具来解决全球性挑战、重塑国家间合作与竞争关系，并参与全球数字治理规则的构建。

在分析框架上，科技外交通常可归纳为 3 个相互关联的维度：外交中的科技（Science In Diplomacy）、为了科技的外交（Diplomacy For Science）、为了外交的科技（Science For Diplomacy）。

开源之所以在当代科技外交中“上位”，在于它能同时服务这 3 类目标：既提供可验证的技术证据与公共产品，也为跨国科研协作降低摩擦，更能以共同维护与共治机制形成长期互相信任与互相依赖。

60.2 开源作为科技外交的可能性与特殊性

开源之所以可能成为科技外交，在于其开放式协作的机制。传统科技合作往往停留在“框架协议—项目立项—阶段交付”的路径上，容易受到政局变化、制裁升级、出口管制等不确定性的冲击。而开源可以把合作从“意愿”转化为可持续的“事实”，其关键机制至少包括以下两条主线。

以代码、标准与治理建立“合作事实”，弱化政治不确定性冲击。开源代码可审计、可复现、可分叉；标准与治理规则可公开讨论、可持续迭代。这种“可验证性”天然适合在信任稀薄环境下运作：即便政治关系波动，技术协作仍能通过公开仓库、共同版本、开放接口与合规流程保持最低限度的连续性。开源外交由此成为在高敏感技术领域“建立数字信任”的杠杆。

通过共同维护数字公共产品（DPG）形成长期互赖，比一次性援助更可持续。一次性援助往往“交付即结束”；而数字公共产品（政务、教育、医疗、城市治理等）一旦进入共同维护与共同演进，就会形成跨国协作的长期“制度黏性”：共同的漏洞披露、版本治理、安全响应、人才培养与运维网络，会把关系从“交易型援助”升级为“共建型伙伴关系”。例如，以公共卫生模块开源向全球输出技术能力，可绕过复杂政治障碍并快速形成软实力与国际形象。

开源的特殊性也值得关注，它既是技术供给，也是规则供给。开源不仅输出“可用的代码”，也输出“可用的规则”：许可证、贡献者协议、社区治理、技术路线透明度、供应链安全流程等。这使开源外交天然具有“双层产品结构”：第一层是技术产品（模型、系统、组件、工具链、数据治理模板、运维方案），第二层是制度产品（标准、治理框架、可信开源认证、安全审计与响应机制、跨境协作SOP）。

因此，开源外交的战略目标往往同时包括普惠发展、生态构建、信任塑造、规则与标准话语权提升。

另外，基于开源的IT基础设施建设，特别能够满足日益高涨的数字主权诉求。在《从全球南方视角看数字主权—各国态度和度量体系》一文中，提出了数字主权指数（Digital Sovereignty Index, DSI）框架，该框架以数据所有权自主、数字基建自主、数字空间治理自主、数字化能力自主4个维度构成国家数字主权的结构，并指出这些维度相互支撑、缺一不可。这意味着：对“全球南方”而言，单纯购买闭源系统，通常只能提升“应用可用性”，却难以提升“可控性”；而开源更容易把“可控性”嵌入方案本身（本地可部署、可审计、可定制、可替换、可培养本土维护者）。

60.3 中国的科技外交策略

在全球技术竞争日益激烈，特别是以美国《芯片与科学法案》为代表的技术封锁政策不断升级的背景下，中国采取了“以我为主、开放合作”的双轨战略，体现了高度的战略辩证法：它既是为应对外部技术“脱钩”压力、保障国家安全而采取的防御性举措，也是为打破封锁、重塑全球影响力和产业链格局而进行的建设性布局。中国科技外交正在从“技术引进—消化吸收”转向更加主动的全球议程建构：在全球地缘博弈与技术变革加速背景下，强调以科技进步应对跨国挑战、缩小数字鸿沟，并推动更开放包容的全球技术生态。整体战略可以描述为：防御性自强与建设性开放。

一、防御性支柱，主要通过“自立自强”构建技术韧性

为应对西方技术封锁，中国将科技安全提升至国家安全底线，通过高强度研发投入和精准政策扶持，构建关键核心技术的自主可控能力。

全产业链自主化。中国大力推进关键领域国产化替代，如半导体产业，中芯国际14nm芯片良率超95%，28nm制程自主可控，标志成熟制程取得实质性突破。但在EUV光刻机等先进制程设备和部分EDA工具方面仍面临“卡脖子”问题，是未来攻关核心方向。

关键资源与出口管制。中国利用全球供应链优势建立反制和谈判筹码。一方面，凭借稀土、锂等战略资源定价权与供应稳定性威慑切断供应链行为；另一方面，建立涵盖数百种产品的出口管制体系，精准反制特定对手薄弱环节。

人才与海外利益保护。人才是科技自强根本。中国建立完善人才评价与激励体

解码开源：开源生态问题——228

系，防止高端人才外流，鼓励海外人才归国。同时，通过领事保护、撤侨行动和涉外法治手段，加强海外公民与机构安全保护，完善高风险地区预警机制，维护全球利益。

二、建设性支柱，主要通过“开放合作”拓展全球影响力

中国在巩固技术基础时，积极推行建设性国际科技合作议程，构建多元包容的全球创新网络。

深化科技伙伴关系。中国系统性深化与金砖国家、上合组织、东盟等新兴经济体的科技合作，构建互利共赢创新共同体。在“一带一路”倡议下，通过“数字丝绸之路”建设，用 5G 等技术在中东、北非等地建立数字连接范式，体现“开源普惠”模式，为“全球南方”提供替代西方“闭源垄断”生态的发展路径。

引领国际大科学计划。中国从国际大科学工程参与者变为引领者。牵头或参与中国空间站等重大国际项目，设立全球科学研究基金，提升在全球科技界话语权，为解决全球性科学难题贡献中国智慧和方案。

推动民间与人文交流。官方外交需要民间互信作为基础。中国利用科协、学会等民间渠道，举办国际夏令营等活动，增进国际社会对中国科技发展的理解与认同，为官方科技外交创造条件和契机。

三、战略转型：从“开放合作”到“开源合作”

中国的开源科技外交可以归纳为 4 种战略使命：能力建设与普惠发展、创新与生态、信任构建、标准与话语权。

《从全球南方视角看数字主权—各国态度和度量体系》一文对“金砖五国”的评估显示：中国是除美国外唯一拥有较为完整数字主权的国家，数字主权指数平均得分 4.25，但在国际数字空间规则制定相关指标上相对弱势，这就给出了一个非常清晰的战略含义。从乐观的角度来看，中国更有能力把开源外交做成“可交付、可维护、可扩展”的体系化方案，而不是零散项目。当然，我们也需要意识到：中国必须把“参与国际基金会、标准化组织与多边规则塑造”作为开源外交的主线工程之一，而不能只做技术输出。美国模式与中国模式的对比如表 60-1 所示。

表 60-1 美国模式与中国模式的对比

特征	美国模式：闭源垄断	中国模式：开源普惠
核心逻辑	以资本和专利构筑技术壁垒，通过 API 收费，维持技术领先地位	以开源换取生态系统，通过社区协作吸纳全球开发者，扩大影响力
外交特征	强调“国家安全”，对特定国家和实体实施严格的出口管制与访问限制	提供“第二选择”，为全球特别是发展中国家提供替代西方技术的方案
战略影响	维持短期高额利润，但在发展中国家面临被“绕过”的风险，可能加剧数字鸿沟	以低成本、高兼容性的优势，迅速赢得新兴市场的信任，构建多元化技术联盟

60.4 中国通过开源实现科技外交的区域策略

中国可通过以“技术普惠”和“标准引领”为核心的差异化开源科技外交，将开源技术的透明度与协作性转化为外交软实力与信任资产。政策重点应围绕“全球南方/发展中国家”及共建“一带一路”国家展开，致力于推动超越地缘政治的技术合作，并在国际治理中建立数字信任、构建技术生态及引领全球标准，从而有效实现科技外交目标。

一、面向“全球南方”：开源作为公共产品型供给与能力建设机制

在“全球南方”与发展中国家场景下，开源外交的关键不在于“展示先进”，而在于解决“可及性”与“可持续性”。政策设计应围绕低成本、高可及性、可修改与可本地化的替代方案展开，通过开源的制度型开放对冲技术垄断与地缘不确定性，让更多国家以可负担成本接入数字化与智能化能力，从而共享“智能红利”。

其中，“开源 AI 普惠”构成第一条主线。通过支持并发布高性能、免费商用的开源大模型等关键能力，显著降低模型获取与二次开发门槛，使发展中国家能够以更低的资金与算力成本获取、修改、定制并部署核心技术。这一策略在叙事上对应反对技术政治化与意识形态划线，在行动上则以可用能力形成合作事实，强化中国作为全球数字公平倡导者的定位。

“技术赋权与援助”构成第二条主线，即把开源项目作为可持续、可复制的能力输出载体。通过设立面向全球公共产品的开源基础设施基金，将资源投入公共卫生、气候模型、灾害预警等高外溢性领域，并优先面向“全球南方”，形成“项目资助—社区协作—本地化落地—能力移交”的闭环，使援助从一次性交付转向长期造血。

“替代生态系统构建”构成第三条主线。在高端芯片与闭源模型受到出口管制、技术封锁强化的背景下，发展中市场对“第二选择”的需求会快速上升。中国可通过低成本、高兼容性的开源软件栈与模型体系提供可落地替代方案，短期降低对单一供应链的脆弱性，中长期培育更为多元的技术生态网络，以伙伴关系与生态协作突破地缘政治壁垒。

二、面向“一带一路”：从硬联通升级为数字“软联通”与生态共建

在“一带一路”倡议框架下，科技外交的关键是推动合作形态升级：从以工程交付为主的硬联通，转向以数字化、标准化与人才协作为核心的“软联通”。以开源为载体，将技术能力、产业生态与规则体系进行一体化输出，能够在共建“一带一路”国家形成可持续的数字连接范式与共同利益结构，使合作更具韧性与延续性。

“数字丝绸之路的开源支持”是第一项重点，尤其适用于中东和北非等数字化转型需求突出的地区。开源能够降低云计算、大数据等基础设施部署门槛，并以透明、可验证的方式提升合作的可信度。通过展示开源操作系统社区等底层软件成果，不仅提供可用方案，也提供长期协作接口，增强沿线国家对国产技术路线的信心。

“开源硬件与产业合作”是第二项重点，其价值在于把基础设施合作延伸到产业链与制造能力建设。以“开源硬件+订阅服务”模式降低制造业进入门槛，同时用服务体系保障持续运维、培训与迭代，通过联合实验室、示范工厂与应用试点，面向能源、农业、城市治理等具体场景实现技术转移的可衡量落地。

“绿色与人文科技交流”是第三项重点，用以塑造更高质量的互信基础。开源透明性适合在清洁能源、新能源汽车与气候治理等领域构建可验证协作机制；同时应更多依托科协、学会、行业联盟等民间渠道开展技术路演、国际夏令营与联合科研项目，以共同科学利益降低敏感度，形成“先民间破冰、后官方升级”的合作节奏。

在具体落地层面，面向中东、东南亚等新兴市场，开源策略需要把“可用”升级为“可持续”。在模型、工具链与开发者社区覆盖基础上，进一步提供培训、运维、合规与安全治理支持，使沿线国家能够形成本地开发者群体与产业伙伴网络，并将技术合作沉淀为区域话语权与标准影响力。

三、面向美欧：以原则立场与可验证信任推动规则塑形

在与美国及欧洲发达国家的关系中，开源外交不以单一项目扩张为目标，而以规则与原则层面的长期博弈为主轴：争取全球治理的道德高地，推动标准与治理模式的塑形，并在结构性对立背景下为关系降温提供可操作缓冲带。相较于面向“全球南方”的能力建设叙事，这一方向更强调议程设置、制度设计与多边场域的规则竞争。

其核心起点是“超越地缘政治的共同原则”。在 AI 与数字治理议题上，应持续强调全球治理应超越地缘政治干扰，反对以意识形态划线与泛化国家安全概念、将经贸科技问题政治化，从而把开源置于全球公共利益语境中，保持合作与规则制定的可讨论空间。

同时，开源的透明性与可审计性是修复信任赤字的重要杠杆。通过公开代码库、可复现构建流程、可追溯提交记录与第三方审计路径，将“必须相信”转化为“可以验证”，降低误判概率，为对话与合作保留技术性空间。

在此基础上，应把标准制定与治理模式推广作为关键战场。推动国际组织与多边机制采用开源技术标准、开源治理框架与合规评估方法，本质上是在争夺全球技术规则的默认选项。尤其在与欧洲互动中，将开源与透明监管、风险评估、责任边界、审计机制等制度议题协同推进，有助于形成更强的规则塑造能力。

四、多边机构定位：把开源转化为全球数字治理的制度优势

在国际多边机构与全球治理体系中，开源的透明度、可审计性与普惠性，使其具备在高敏感技术领域重建可验证信任的独特价值。中国需要以更积极的姿态明确政策定位：既展示开放合作，又在全球数字治理议程中形成可持续的制度影响力，增强在技术竞争与封锁压力下的战略回旋空间。

第一层定位是原则倡议。即在 AI 与数字治理中持续确立“超越地缘政治”的叙事支点，反对技术政治化，将合作讨论锚定在共同安全、共同发展与共同责任的框架之中，提升议程合法性与道德感召力。

第二层定位是标准引领。即通过外交与机制性参与，推动国际组织采用开源技术标准与治理模式，使其成为跨境协作的通用规则。标准不仅关乎互操作性，更决定合规要求、审计机制与责任边界；以开源为载体推动规则形成，本质是在全球技术秩序中争取制度性话语权。

第三层定位是信任驱动的公私伙伴关系。即以结构化机制连接政府机构与各类非政府实体，涵盖商业、学术及产业领域，将其技术能力与社会影响力融入多边治理能力，公开寻求、鼓励并评估公众反馈，以更透明、可验证的方式提升政策可信度与社会接受度。

通过上述4个方向的协同推进，开源科技外交可以形成“技术普惠—生态共建—标准引领—数字信任”的闭环，使中国在全球治理与区域合作中以更制度化、可持续的方式实现科技外交目标。

后记 AI时代来临之后，正在剧变中的开源生态

从 2024 年 8 月，与杨海玲老师讨论本书的提纲，到两年后终于写完这本书时，我越来越强烈地感受到：开源世界正在进入一个新的转折点。

如果说过去几十年的开源，是围绕“源代码开放”展开的一场全球协作运动，那么 AI 时代的到来，正在把开源推向一个更复杂、更宽广，也更不确定的新阶段。我们曾经习惯于用代码、许可证、社区、贡献者、基金会、商业模式和供应链来理解开源；但今天，仅仅依靠这些概念，已经不足以完整解释正在发生的变化。

因为 AI 正在改变软件本身。

过去，软件主要被理解为代码产品：开发者把需求转化为代码，代码在特定环境中运行，用户通过界面或接口调用功能。开源软件的核心对象，主要是源码、构建脚本、文档、依赖和许可证。只要代码开放、许可证清晰、社区协作顺畅，一个开源项目的边界通常就比较明确。

但 AI 时代的软件，正在从代码产品转向智能化的能力系统。一个软件系统不再只由代码组成，还包括模型、数据、提示词、智能代理、工具、记忆、评测体系、反馈机制和词元调度。代码仍然重要，但代码不再是软件能力的唯一来源。模型提供推理和生成能力，数据提供知识与上下文，智能代理负责任务理解和行动编排，工具与 API 连接外部世界，词元则成为智能能力调用和成本计量的重要资源。

于是，一个根本问题出现了：当软件的能力不再只存在于代码中，开源还只是开放源码吗？

这是 AI 时代开源生态面临的第一个剧变：开源的对象变了。

AI 系统的核心对象可能是模型结构、训练数据、数据清洗流程、训练代码、模型权重、推理框架、评测基准、对齐方法和运行服务。一个模型开放了权重，但没有说明训练数据来源；一个系统开放了推理代码，但依赖封闭模型 API；一个智能代理框架开放了源码，却把关键工具链和评测体系留在平台内部。面对这些情况，我们很难再简单地说“开源”或“不开源”。

开源的边界正在被重新讨论。未来的开源，不仅要问“源代码是否开放？”还要问“数据是否可追溯？模型是否可研究、可修改、可再分发？训练过程是否足够透明？评测结果是否可复现？推理成本是否可承担？工具链是否可替换？”如果没有这些条件，所谓开放就只是形式上的开放，而不是可能性上的开放。

第二个剧变：开源的协作方式变了。

开源曾经以“人写代码、人审代码、人维护项目”为基本协作模式。AI 辅助

编程出现之后，代码生产成本迅速下降。一个开发者可以借助 AI 更快生成代码、文档、测试和 Demo；一个小团队可以完成过去需要更大团队才能完成的软件原型；一个非专业开发者也可以通过自然语言构建简单工具。这无疑扩大了开源参与的基础。

但这也带来了新的问题：AI 不只提高了生产力，也提高了噪声的生产力。

未来开源社区可能收到更多由 AI 生成的问题报告、PR、文档修改和功能建议。它们中有些会提高效率，有些则只是“看似正确但缺乏理解深度”的低质量贡献。对于维护者来说，真正的压力不一定来自代码数量不足，而可能来自筛选、审查、解释、拒绝和维护的成本上升。个人借助 AI 降低了贡献成本，但社区可能承担更多的验收成本。

这会改变开源社区的治理重心。过去，社区常常鼓励“多贡献、多参与、多提交”。未来，社区还必须学会过滤噪声、识别高质量贡献、标注 AI 生成内容、设计自动化审查流程，并维护人与人之间的信任。开源协作不再只是开发者之间的协作，而是人与人、人与代理、代理与代理共同参与的复杂协作。

第三个剧变：开源项目的价值结构变了。

当代码生成变得容易，普通代码的稀缺性就会下降。样板代码、简单工具、CRUD 逻辑、脚本程序、小型应用，都可能被快速生成、复制和替代。代码不会失去价值，但“仅仅有代码”的价值会下降。过去，一个项目只要开放了有用的代码，就可能获得关注；未来，用户会进一步追问“这个项目是否可靠？是否有人维护？是否经过验证？是否有真实场景？是否能持续演进？是否值得信赖？”

因此，AI 时代的开源价值会从代码本身，转向代码之上的能力：问题定义、场景理解、工程质量、测试验证、安全响应、治理透明度、生态连接和长期信任。

Star 也会随之贬值。Star 仍然代表注意力，但不再足以代表真实价值。一个项目可能因为炫酷 Demo、热点概念、社交传播而迅速爆红，却未必具备持续维护能力；另一个基础项目可能并不耀眼，却支撑着大量真实系统。AI 时代的开源生态，不能只看热度，更要看健康度。真正值得关注的，不是一个项目在短期内获得多少 Star，而是它是否拥有稳定维护者、真实用户、清晰治理、可靠发布、明确安全机制和可持续生态位置。

第四个剧变：开源项目的生命周期变了。

AI 让项目诞生更快，也可能让项目死亡更快。一个想法可以迅速变成 Demo，一个 Demo 可以迅速发布为开源项目，一个开源项目也可能很快被复制、替代或遗忘。开源生态会变得更加繁荣，也更加碎片化。

这并不完全是坏事。更低的试错成本，意味着更多想法可以被验证，更多长尾需求可以被满足，更多个人和小团队可以参与创新。但对于基础软件、关键工具链和公共数字基础设施来说，过快的生命周期也可能带来风险。因为真正支撑社会运行的软件，不能只靠一时热情和短期流量。它需要长期维护、稳定治理、版本演进、安全响应和责任承接。

因此，AI 时代的开源生态建设，不能只追求“更多项目”，而要追求“更可靠的生态”。不能只鼓励发布，也要支持维护；不能只奖励爆款，也要资助基础；不

能只看贡献数量，也要看治理质量。

第五个剧变：开源供应链变了。

传统软件供应链主要关注源码、依赖包、构建系统、容器镜像、制品仓库、许可证和漏洞。AI 时代的软件供应链，则扩展为由代码、模型、数据、提示词、智能代理、插件、API、推理服务、词元服务和平台资源共同构成的复杂网络。

过去，我们问“这个软件用了哪些开源组件？”未来还要问“它调用了哪些模型？依赖了哪些数据？使用了哪些提示词？连接了哪些工具？智能代理具有什么权限？推理服务由谁提供？词元成本如何计算？模型能力变化会不会影响上层软件稳定性？”

软件调用模型，模型依赖数据，智能代理调用工具，工具连接外部服务，RAG 系统调用知识库，词元服务影响成本结构，平台政策影响产品能力，供应链正在从树状依赖走向网络化依赖。任何一个环节的变化，都可能影响整个系统。

这会带来一种新的双重结构：应用层更加碎片化，资源层更加集中化。一方面，AI 让更多小项目、小工具、小的智能代理快速出现；另一方面，模型、算力、数据、推理平台和词元服务可能集中在少数平台手中。开源生态看似更加开放，底层资源却可能更加集中。这种张力，将成为未来开源治理的重要议题。

第六个剧变：开源治理的任务变了。

本书反复强调，开源不是“免费代码”，而是一种开放式协作模式，是数字公共产品生产与治理的重要机制。到了 AI 时代，这一判断不但没有过时，反而更加重要。

因为 AI 时代的开源生态，已经不可能只依赖少数维护者的热情，也不可能只依赖企业的商业投入，更不可能只依赖平台的自我约束。开源作为数字公共基础设施，需要更加成熟的多元共治机制。

政府需要把关键开源项目、开源模型、基础工具链和公共数据资源，视为数字公共基础设施，而不是短期产业指标。企业需要从开源消费者转向开源投资者，不只是使用开源，还要回馈上游、资助维护、开放工具、参与治理。基金会需要继续扮演公共管家的角色，提供中立的法律、治理、商标、合规和社区支持。高校和教育机构需要把开源能力纳入人才培养体系，让学生在真实项目中学习协作、工程、许可证、安全和治理。开发者和用户也需要理解，开源不是无成本资源，而是需要共同维护的知识公地。

AI 时代尤其需要新型开源基金会的能力。过去基金会的核心职能为托管代码项目、开展社区治理、管理商标与许可证。未来，基金会还需为开源项目提供多维度支持，包括处理模型开放、数据合规、评测标准制定、算力供给、智能代理安全管控、供应链信息梳理、跨境协作协调以及公共资助机制建设等相关工作。基金会不只是项目的“代码之家”，也可能成为 AI 时代数字公共产品的制度基础设施。

六大剧变让我们更加确信，开源的精神变得更重要了。

当 AI 让代码更容易生成，开源世界反而更需要重新回答那些最基本的问题：我们为什么要开放？开放什么？为谁开放？如何开放？开放之后谁来维护？谁从中受益？谁承担成本？谁拥有治理权？谁对风险负责？

这些问题并不新。自由软件运动曾经用“四大自由”回答过，开源运动曾经用开放协作和更好的软件回答过，商业开源曾经用效率、复用和生态共赢回答过。AI 时代，我们需要在新的技术条件下再次回答它们。

笔者的理解是，AI 时代的开源不应只是“把模型放出来”，也不应只是“把代码仓库公开”。真正有意义的开放，应该让更多人有能力理解、使用、修改、验证、复现、改进和治理技术系统。它不仅是使用权问题，也是所有权问题；不仅是技术许可问题，也是生态治理问题；不仅是产业竞争问题，也是数字公共产品问题。

这也是本书想表达的一个基本立场：开源从来不只是技术问题，它是知识生产方式，是协作制度，是创新生态，是商业模式，也是公共治理问题。

AI 时代来临之后，开源生态正在剧变。它会更繁荣，也会更混乱；会更开放，也可能更集中；会让更多人参与，也会让维护者承担更大压力；会降低创新门槛，也会提高治理难度。我们不能因为 AI 带来的效率而忽视风险，也不能因为风险而否定开放的价值。

恰恰相反，越是在 AI 深刻改变软件、知识和产业结构的时候，我们越需要开源。因为开源提供的不只是代码，而是一种让复杂技术系统接受公共检验、共同改进和多元治理的可能性。

本书原本试图为读者绘制一份理解开源世界的地图。但在 AI 时代，这份地图还必须不断更新。新的大陆正在浮现，旧的边界正在移动，熟悉的道路正在分叉。模型、数据、智能代理、词元、评测、算力、供应链、公共治理，都将成为开源新地图上的重要地标。

后记不是终点。它更像是提醒：我们刚刚完成的，只是对过去和当下开源生态的一次阶段性梳理。真正的变化还在继续发生。

愿每一位开源世界的探索者、建设者和思考者，都能在 AI 时代继续保持清醒：既拥抱技术变革，也守护开放协作；既追求创新效率，也重视长期维护；既关注项目成功，也关心生态可持续；既看到 AI 的力量，也不忘人的责任。

开源曾经改变软件世界。AI 时代，开源还将继续改变智能世界。但前提是，我们必须共同回答好一个问题：当智能系统越来越强大、越来越复杂、越来越深入社会运行时，我们能否用更加开放、可信、可持续的方式来建设和治理它？

这或许正是 AI 时代开源生态最重要的新命题。

附录 知名开源基金会介绍

一、Linux 基金会

作为全球最大的开源基金会之一，Linux 基金会为 Linux 操作系统及其相关项目提供了大量的支持。它不仅提供资金支持，还通过举办大型技术会议促进开源社区的发展。Linux 基金会的资源支持确保了 Linux 内核的持续改进，并推动了与硬件、云计算、容器等技术的整合。

Linux 基金会的主要工作包括以下几项。

- 认证和培训：截至 2023 年，Linux 基金会颁发了 3.3 万个认证证书，相比 2022 年增长了约 26%。
- 社区支持：Linux 基金会支持的开源项目数量持续增长，2023 年新增 137 个项目，总数达到 1133 个，增幅约为 10%。这些项目涵盖云计算、容器、虚拟化、网络、边缘计算、AI、机器学习等关键领域。
- 活动和会议：2023 年，Linux 基金会共组织了 234 场活动，包括年度开源峰会（Open Source Summit）和 KubeCon+CloudNativeCon，吸引了约 7.5 万名与会者。2023 年 7 月，Linux 基金会还在瑞士日内瓦举办了 Open Source Congress 会议，讨论了开源社区面临的紧迫问题，如网络安全和技术民族主义。
- 合作与创新：Linux 基金会通过与各大科技公司合作，推动开源项目的发展，其董事会成员包括微软、甲骨文、英特尔、VMware、三星、索尼、红帽、华为、Meta、高通、腾讯、日立、富士通和爱立信等知名企业。这种多样化的合作促进了技术的创新和应用。
- 项目孵化与管理：众多不同领域、技术领先的开源项目加入 Linux 基金会，包括 AI/机器学习（如 PyTorch、ONNX）、云原生（如 OpenTelemetry、Crossplane）、区块链（如 Hyperledger Besu）等领域。
- 行业倡议与标准制定：Linux 基金会发起了多个行业倡议，如“绿色软件基金会”，联合微软、英特尔等企业推动可持续软件开发。此外，Linux 基金会通过旗下金融科技开源基金会（Fintech Open Source Foundation, FINOS）合作，推进金融行业的开源标准。2023 年，Linux 基金会将 72% 的资源投入开源软件开发，20% 用于开放标准的制定，体现了其在软件开发和标准化方面的双重关注。
- 安全治理：Linux 基金会通过核心基础设施计划（Core Infrastructure Initiative，简称 CII）与开源安全基金会（Open Source Security Foundation，简称 OpenSSF），推动对处于核心计算功能关键路径上的开源项目提供资助，支持漏洞修复、软件物料清单标准化等安全实践，提升

开源项目的整体安全性。

二、Apache 软件基金会

Apache 软件基金会是另一个重要的开源基金会，它为 Apache HTTP Server、Hadoop、Spark 等项目提供支持，并通过开放的治理结构保证了其支持项目的独立性和社区驱动性。Apache 软件基金会支持的项目涵盖从 Web 服务器到大数据处理等多个领域，深刻影响了现代软件架构的发展。

Apache 软件基金会的主要工作包括以下几项。

- 项目孵化：Apache 软件基金会通过其孵化器项目（Incubator）支持新兴开源项目的发展。截至 2024 年，Apache 软件基金会管理着超过 320 个活跃项目，社区贡献者超过 63 万人，新增提交者 441 人，总计提交者达 8484 人。2020 年以来，共有超过 40 个项目毕业成为顶级项目，包括 Apache Kylin、Apache Linkis、Apache SkyWalking、Apache DolphinScheduler、Apache ECharts、Apache Answer 等众多中国开源项目。
- 基础设施升级：Apache 软件基金会近年来积极拥抱 GitHub 平台，许多项目都使用 GitHub 作为代码托管平台，GitHub Actions 被广泛应用于 Apache 项目的自动化构建、测试和发布环节，实现了高效的 CI/CD 流程。根据 Apache 软件基金会发布的最新数据，其管理的代码仓库托管量已超过 400TB，并且仍在持续增长。
- 社区大会：Apache 软件基金会通过在各地召开的 Apache 峰会（ApacheCon，现改名为 Community Over Code）加强全球开发者协作与交流，每年在美国、欧洲部分国家和中国召开的各个 Community Over Code 大会，总计有超过 1 万名开发者参加。

三、开放原子开源基金会

根据 2024 年度《中国开源年度报告》的开源基金会 OpenRank 排行榜，开放原子开源基金会位列第一，这表明中国在开源领域的影响力正在增强，尤其是在推动大型开源项目方面，成就卓著。

开放原子开源基金会的主要工作包括以下几项。

- 项目孵化：开放原子开源基金会孵化的重点项目包括 XuperCore、OpenHarmony、openEuler 等。
XuperCore：截至 2024 年，该项目在政务、金融、司法、医疗、版权、签约等场景深入结合，在全国 10 余个省份落地应用场景，基于 XuperCore 推出了城市链管平台、区块链政务数据共享平台、一站式版权服务平台、电子签约平台等。
OpenHarmony：截至 2025 年 12 月，已有超过 1 万名开发者参与该项目的社区贡献，累计贡献代码超 1.3 亿行，生态伙伴 523 家，覆盖能源电力、金融、工业等多个关键行业。
openEuler：截至 2025 年，成员单位超过 2100 家，贡献者数量超过 2.3 万人，操作系统装机量超过 1600 万套，该项目已广泛应用于互联网、金

融、运营商等各行业核心场景，实现规模商业落地。在引领产业技术创新方面，openEuler 社区发起的原创子项目累计超过 500 个，覆盖从系统内核到 AI 使能全场景。

- 开发者活动。

举办开放原子全球开源峰会、开放原子开源生态大会、开放原子开发者大会等活动。

举办开放原子开源大赛，截至 2025 年第三届，大赛已累计发布 100 个高质量赛项，吸引 8 万余名选手报名参赛，覆盖 1000 多所国内外高校。